Unity QuadSim Codebase Review

Complete project-authored source and selected textual configuration

Repository: Unity_QuadSim
Branch: test
Commit: f20e44b065c2207a0b4cab8352fb53229ea33d5e
Inspection date: August 23, 2026
Reviewed files: 112

1. Scope

This review covers 112 project-authored source and selected textual configuration files (19,896 displayed source lines) from the repository state identified above. Listings are imported directly from the working tree; each file is preceded by its byte-level SHA-256 digest.

Unity `.meta` files, large scenes and prefabs, materials, terrain data, Shader Graph serialization, models, textures, compiled libraries, vendored dependencies, caches, and build products are outside this code-listing scope. No dedicated third-party section is included.

The external `quadsim-sdk` implementation is not present in this repository. Only the Unity RPC implementation and the repository's project-authored Python clients, examples, and CI utilities are listed.

2. Architecture Overview

`SimulationManager` owns deterministic stepping and drives `DroneManager`; `DroneManager` drives each `QuadPawn`; and each pawn coordinates commands, control, actuation, physics, sensors, camera state, and episode events. `SimCoreApi` and `QuadSimApi` provide global and per-drone facades. The RPC adapter, runtime UI, examples, and RL features consume those facades rather than replacing the underlying lifecycle.

3. Package and Compilation Configuration

Defines the Unity package identity, assembly boundaries, repository filters, and compilation references used by the runtime, editor tools, samples, and tests.

Files in this section

- .gitattributes
- .gitignore
- package.json
- Editor/Nino.QuadSim.Editor.asmdef
- Runtime/Scripts/com.nino.quadsim.asmdef
- Samples~/QuadSim Starter/Scripts/Nino.QuadSim.Samples.asmdef
- Tests/Editor/Nino.QuadSim.Tests.asmdef
- Tests/PlayMode/Nino.QuadSim.PlayModeTests.asmdef

.gitattributes

SHA-256 4736d9c739fb2198e9493478f10d960bcee948d2d4b0205b4817668b4f5e247e | 8 lines | 423 bytes

```
*.fbx filter=lfs diff=lfs merge=lfs -text
*.FBX filter=lfs diff=lfs merge=lfs -text
*.bytes filter=lfs diff=lfs merge=lfs -text

# Keep the small Crazyflie source meshes as ordinary Git files.
Runtime/Prefabs/Models/Crazyflie/ccw_prop.fbx -filter -diff -merge -text
Runtime/Prefabs/Models/Crazyflie/cf2_assembly.fbx -filter -diff -merge -text
Runtime/Prefabs/Models/Crazyflie/cw_prop.fbx -filter -diff -merge -text
```

.gitignore

SHA-256 9f60da9dd46efcac3edb4515d1a57dfcb4413d4e23901023e0656f2cb05200c4 | 47 lines | 602 bytes

```
# Unity
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[Ll]ogs/
[Uu]ser[Ss]ettings/

# Local Unity host projects are not part of this package-root repository.
/Assets/
/Assets.meta
/Packages/
/Packages.meta
/ProjectSettings/
/ProjectSettings.meta
/UIElementsSchema/
/imgui.ini
/imgui.ini.meta
/runs/
/runs.meta
/Logs.meta
/UserSettings.meta
/obj.meta
/*.csproj.meta
/*.sln.meta

# Rider / VS / CLion
.idea/
.vscode/
*.csproj
*.sln
*.user
*.pidb
*.booproj
```

```
# OS
.DS_Store
Thumbs.db
__pycache__/
*.py[cod]

# Crash reports
sysinfo.txt

# Gradle
.gradle/
```

package.json

SHA-256 25a55e3c454515d6f5abd9f09c3c2d21f4de24a3bd87362d6ecc3a1fcd63387d | 39 lines | 1,206 bytes

```
{
  "name": "com.nino.quadsim",
  "version": "0.3.0",
  "displayName": "QuadSim",
  "description": "Deterministic Unity quadrotor simulation with editable starter content, sensors, contr
ollers, RPC, and Python SDK integration.",
  "unity": "6000.0",
  "dependencies": {
    "com.unity.inputsystem": "1.19.0",
    "com.unity.render-pipelines.high-definition": "17.4.0"
  },
  "samples": [
    {
      "displayName": "QuadSim Starter",
      "description": "Land scene, terrain, editable scene configuration, drone catalog, drone prefabs, e
xample script, and YAML overrides.",
      "path": "Samples~/QuadSim Starter"
    },
    {
      "displayName": "PX4 SITL Tools",
      "description": "Optional scripts and documentation for the PX4 SITL bridge workflow.",
      "path": "Samples~/PX4 SITL Tools"
    }
  ],
  "author": {
    "name": "Nicolas Nino Fernandez"
  },
  "license": "MIT",
  "documentationUrl": "https://github.com/ninonick0607/Unity_QuadSim#readme",
  "repository": {
    "type": "git",
    "url": "https://github.com/ninonick0607/Unity_QuadSim.git"
  },
  "keywords": [
    "drone",
    "quadrotor",
    "simulation",
    "robotics",
    "rpc"
  ]
}
```

Editor/Nino.QuadSim.Editor.asmdef

SHA-256 e755cff888ae28b6d5d18398d856fd642a761af220a8a37df9dc77964c020ed8 | 18 lines | 385 bytes

```
{
  "name": "Nino.QuadSim.Editor",
  "rootNamespace": "",
  "references": [
    "Nino.QuadSim"
  ],
  "includePlatforms": [
    "Editor"
  ],
  "excludePlatforms": [],
  "allowUnsafeCode": false,
  "overrideReferences": false,
```

```
  "precompiledReferences": [],
  "autoReferenced": true,
  "defineConstraints": [],
  "versionDefines": [],
  "noEngineReferences": false
}
```

Runtime/Scripts/com.nino.quadsim.asmdef

SHA-256 3ff4f9433455b6bacf01c1357f83a7faa51aaabae846025c4e11e32a0ed00fb8 | 16 lines | 366 bytes

```
{
  "name": "Nino.QuadSim",
  "rootNamespace": "Nino.QuadSim",
  "references": [
    "UImGui"
  ],
  "includePlatforms": [],
  "excludePlatforms": [],
  "allowUnsafeCode": false,
  "overrideReferences": false,
  "precompiledReferences": [],
  "autoReferenced": true,
  "defineConstraints": [],
  "versionDefines": [],
  "noEngineReferences": false
}
```

Samples~/QuadSim Starter/Scripts/Nino.QuadSim.Samples.asmdef

SHA-256 be065d8c822d5457c8e646feacc2c508b96019bbb27e1e2a885785fd97fa2161 | 16 lines | 413 bytes

```
{
  "name": "Nino.QuadSim.Samples",
  "rootNamespace": "Nino.QuadSim.Samples",
  "references": [
    "GUID:953ca8e47f8dbc604b521b39796f23d3"
  ],
  "includePlatforms": [],
  "excludePlatforms": [],
  "allowUnsafeCode": false,
  "overrideReferences": false,
  "precompiledReferences": [],
  "autoReferenced": true,
  "defineConstraints": [],
  "versionDefines": [],
  "noEngineReferences": false
}
```

Tests/Editor/Nino.QuadSim.Tests.asmdef

SHA-256 f058386664c3daa4f05080672c0de9b599623a4c4cbd256dfbd78e1de7a3ad58 | 14 lines | 258 bytes

```
{
  "name": "Nino.QuadSim.Tests",
  "rootNamespace": "Nino.QuadSim.Tests",
  "references": [
    "Nino.QuadSim"
  ],
  "optionalUnityReferences": [
    "TestAssemblies"
  ],
  "includePlatforms": [
    "Editor"
  ],
  "autoReferenced": false
}
```

Tests/PlayMode/Nino.QuadSim.PlayModeTests.asmdef

SHA-256 d32145b55675af21c73dfd4d46019f0707c3d7e35eb898eacd1153d126045c68 | 11 lines | 229 bytes

```
{
  "name": "Nino.QuadSim.PlayModeTests",
  "rootNamespace": "Nino.QuadSim.PlayModeTests",
  "references": [
```

```
      "Nino.QuadSim"
    ],
    "optionalUnityReferences": [
      "TestAssemblies"
    ],
    "autoReferenced": false
  }
```

4. Simulation Kernel and Global APIs

Provides the authoritative clock, fixed-step pipeline, pause and manual-step behavior, source-authority state machine, global API facade, scene configuration, and drone catalog used by the rest of QuadSim.

Files in this section

- Runtime/Scripts/SimCore/ClockFactory.cs
- Runtime/Scripts/SimCore/Common/CommonStructs.cs
- Runtime/Scripts/SimCore/Common/FlightCommandLayout.cs
- Runtime/Scripts/SimCore/ControlAuthorityManager.cs
- Runtime/Scripts/SimCore/DroneCatalog.cs
- Runtime/Scripts/SimCore/FrequencyLimiter.cs
- Runtime/Scripts/SimCore/QuadSimEnvironmentBootstrap.cs
- Runtime/Scripts/SimCore/QuadSimSceneConfig.cs
- Runtime/Scripts/SimCore/SimClock.cs
- Runtime/Scripts/SimCore/SimCoreApi.cs
- Runtime/Scripts/SimCore/SimulationManager.cs
- Runtime/Scripts/SimCore/StepContract.cs

Runtime/Scripts/SimCore/ClockFactory.cs

SHA-256 dd09e59d8fad70bc377bacd954481dd0b1e891a3b9d22990c5cb32fa2c6ecbc1 | 82 lines | 2,655 bytes

```
// Assets/Scripts/SimCore/ClockFactory.cs

using System;

namespace SimCore
{
    /// <summary>
    /// Central clock registry. Everything in the sim should reference time through here.
    /// </summary>
    public static class ClockFactory
    {
        private static SteppableClock _clock;
        private static readonly ScaledClock _scaled = new ScaledClock();
        private static bool _initialized;

        public static bool IsInitialized => _initialized;

        /// <summary>Fixed base dt for simulation (seconds). Default is 250 Hz.</summary>
        public static double BaseDtSec { get; private set; } = 1.0 / 250.0;

        /// <summary>Authoritative sim clock (fixed step).</summary>
        public static SteppableClock Clock
        {
            get
            {
                EnsureInitialized();
                return _clock;
            }
        }

        /// <summary>Pause/scale wrapper used by SimulationManager to decide how much sim-time to advanc
e per frame.</summary>
        public static ScaledClock Scaled => _scaled;

        /// <summary>
        /// Initialize the sim clock system. Safe to call multiple times.
        /// If called again with different params, it will re-init and reset time.
        /// </summary>
        public static void Initialize(double baseHz = 250.0, long startNanos = 0, bool paused = false, d
ouble timeScale = 1.0)
```

```
        {
            if (baseHz <= 0) throw new ArgumentOutOfRangeException(nameof(baseHz), "baseHz must be > 0."
);

            _clock = new SteppableClock(1.0 / baseHz, startNanos);
            BaseDtSec = _clock.StepSizeSec;
            _clock.Reset(startNanos);

            _scaled.SetPaused(paused);
            _scaled.SetTimeScale(timeScale);

            _initialized = true;
        }

        public static void Reset(long? newStartNanos = null)
        {
            EnsureInitialized();
            _clock.Reset(newStartNanos);
        }

        /// <summary>Advance exactly one fixed step (dt = BaseDtSec).</summary>
        public static long StepOnce()
        {
            EnsureInitialized();
            return _clock.Step();
        }

        public static void SetPaused(bool paused)
        {
            _scaled.SetPaused(paused);
        }

        public static void SetTimeScale(double scale)
        {
            _scaled.SetTimeScale(scale);
        }

        private static void EnsureInitialized()
        {
            if (_initialized) return;
            // Default init: 250 Hz, start at 0, running
            Initialize();
        }
    }
}
```

Runtime/Scripts/SimCore/Common/CommonStructs.cs

SHA-256 cb381bdbe889973a966aca5f416e488c49c7f276f990c0f4c6264d008124ca1f | 194 lines | 7,100 bytes

```
// Assets/Scripts/SimCore/Common/CommonStructs.cs
// Phase 1: CommonStructs Verification & Enhancement
//
// CHANGES FROM EXISTING:
//   - Added FlightCommand.SafeHover() factory for authority fallback
//   - Added FlightCommand.IsValid property
//   - Added Motor4 indexer and Zero factory
//   - Added Wrench6 Zero factory
//   - All enums verified complete per API plan
//   - All payload structs verified complete per API plan

using System;
using UnityEngine;

namespace SimCore.Common
{
    // ==================== Enums ==================== //

    /// <summary>
    /// Who is producing commands: the in-sim UI, a user C# script, or an external RPC client.
    /// </summary>
    public enum InputSource : byte { UI, Internal, External }

    /// <summary>
```

```
    /// Lifecycle state of a command source.
    /// UI is never Unavailable. Internal/External can be any state.
    /// Denied is External-only (user clicked "Deny" for the session).
    /// </summary>
    public enum SourceStatus : byte { Unavailable, Available, Connected, Denied }

    /// <summary>
    /// Which controller implementation is active. Independent of GoalMode.
    /// </summary>
    public enum ControllerKind : byte { Cascade, Geometric }

    /// <summary>
    /// Which stage of the cascade to enter at. None = motors off.
    /// Geometric controller ignores this entirely.
    /// </summary>
    public enum GoalMode : byte
    {
        None = 0,
        Position = 1,
        Velocity = 2,
        Angle = 3,
        Rate = 4,
        Passthrough = 5,
        Wrench = 6,
        WrenchBypassed = 7,
        Acceleration = 8
    }

    // ==================== Command Payloads ==================== //

    /// <summary>
    /// 6-DOF wrench for direct force/torque application. Only used in GoalMode.Wrench.
    /// Bypasses the entire cascade and actuation model — applies directly to Rigidbody.
    /// </summary>
    public struct Wrench6
    {
        public Vector3 Force;
        public Vector3 Torque;

        public static Wrench6 Zero => new Wrench6 { Force = Vector3.zero, Torque = Vector3.zero };
    }

    /// <summary>
    /// Raw motor commands [0..1] for Passthrough mode. Skips cascade, goes through actuation model.
    /// </summary>
    public struct Motor4
    {
        public float FL, FR, BL, BR;

        public float this[int index]
        {
            get => index switch { 0 => FL, 1 => FR, 2 => BL, 3 => BR, _ => throw new IndexOutOfRangeExce
ption() };
            set { switch (index) { case 0: FL = value; break; case 1: FR = value; break; case 2: BL = va
lue; break; case 3: BR = value; break; default: throw new IndexOutOfRangeException(); } }
        }

        public static Motor4 Zero => new Motor4 { FL = 0, FR = 0, BL = 0, BR = 0 };
    }

    /// <summary>
    /// 4-axis command payload. Interpretation depends on GoalMode:
    ///   Position:    X=X_m,      Y=Y_m,         Z=Yaw_deg,       W=Alt_m
    ///   Velocity:    X=Vx_mps,   Y=Vy_mps,      Z=YawRate_dps,   W=Vz_mps
    ///   Acceleration:X=Ax_mps2,Y=Ay_mps2,     Z=YawRate_dps,   W=Az_mps2
    ///   Angle:       X=Roll_deg, Y=Pitch_deg,    Z=YawRate_dps,   W=Throttle_01
    ///   Rate:        X=RollRate, Y=PitchRate,    Z=YawRate_dps,   W=Throttle_01
    ///   Passthrough: X=M0,       Y=M1,           Z=M2,            W=M3
    /// </summary>
    [Serializable]
    public struct Axis4
    {
        public float X;
```

```
    public float Y;
    public float Z;
    public float W;

    public Axis4(float x, float y, float z, float w)
    {
        X = x;
        Y = y;
        Z = z;
        W = w;
    }

    // Vector views
    public Vector3 XYZ
    {
        get => new Vector3(X, Y, Z);
        set { X = value.x; Y = value.y; Z = value.z; }
    }

    // Semantic aliases (for Angle/Rate modes)
    public float Roll    { get => X; set => X = value; }
    public float Pitch   { get => Y; set => Y = value; }
    public float Yaw     { get => Z; set => Z = value; }

    // Semantic alias for W depending on mode
    public float Throttle { get => W; set => W = value; }   // Rate mode
    public float AltOrVz  { get => W; set => W = value; }   // Position/Velocity/Angle modes

    public float this[int index]
    {
        get => index switch
        {
            0 => X, 1 => Y, 2 => Z, 3 => W,
            _ => throw new IndexOutOfRangeException($"Axis4 index out of bounds: {index}")
        };
        set
        {
            switch (index)
            {
                case 0: X = value; break;
                case 1: Y = value; break;
                case 2: Z = value; break;
                case 3: W = value; break;
                default: throw new IndexOutOfRangeException($"Axis4 index out of bounds: {index}");
            }
        }
    }

    public void Zero() => X = Y = Z = W = 0f;
    public static Axis4 ZeroValue => new Axis4(0, 0, 0, 0);

    public override string ToString() => $"({X:F3}, {Y:F3}, {Z:F3}, {W:F3})";
}


// ==================== Command Envelope ==================== //

/// <summary>
/// Complete command envelope. Carries the mode, source, timestamp, and payload.
/// Only one payload field is active depending on Mode:
///   - Wrench mode uses Wrench field
///   - Passthrough mode uses Motors field (or Cmd with M0-M3 in XYZW)
///   - All other modes use Cmd (Axis4)
/// </summary>
public struct FlightCommand
{
    public GoalMode Mode;
    public InputSource Source;
    public double TimestampSec;
    public Axis4 Cmd;
    public Wrench6 Wrench;
    public Motor4 Motors;
```

```
        /// <summary>Empty/invalid command.</summary>
        public static FlightCommand None => new FlightCommand { Mode = GoalMode.None };

        /// <summary>Has this command been populated with real data?</summary>
        public bool IsValid => Mode != GoalMode.None && TimestampSec > 0;

        /// <summary>
        /// Safe fallback command — zero rates, zero throttle.
        /// Used when the authority source hasn't sent a command yet or disconnects.
        /// The mode passed in determines the shape of the safe command.
        /// </summary>
        public static FlightCommand SafeFallback(GoalMode currentMode, InputSource source, double timest
amp)
        {
            var cmd = new FlightCommand
            {
                Mode = currentMode,
                Source = source,
                TimestampSec = timestamp,
                Cmd = Axis4.ZeroValue,
                Wrench = Wrench6.Zero,
                Motors = Motor4.Zero
            };
            return cmd;
        }
    }
}
```

Runtime/Scripts/SimCore/Common/FlightCommandLayout.cs

SHA-256 be37741695150556e0c0686032a37edf643ee63507fbbbdd60108d53abe6d1c0 | 37 lines | 1,744 bytes

```
namespace SimCore.Common
{
    public static class FlightAxes
    {
        public const int X   = 0;
        public const int Y   = 1;
        public const int Yaw = 2;
        public const int Z   = 3;
    }
    public static class FlightCommandLayout
    {
        // Position mode expects: (X_m, Y_m, Yaw_deg_or_nan, Z_m)
        public static Axis4 Position(float x_m, float y_m, float z_m, float yaw_deg)
            => new Axis4(x_m, y_m, yaw_deg, z_m);

        // Velocity mode expects: (Vx_mps, Vy_mps, YawRate_deg_s, Vz_mps)
        public static Axis4 Velocity(float vx_mps, float vy_mps, float vz_mps, float yawRate_deg_s)
            => new Axis4(vx_mps, vy_mps, yawRate_deg_s, vz_mps);

        // Angle mode: (Roll_deg, Pitch_deg, YawRate_deg_s, Throttle01)
        public static Axis4 Angle(float roll_deg, float pitch_deg, float yawRate_deg_s, float throttle01
)
            => new Axis4(roll_deg, pitch_deg, yawRate_deg_s, throttle01);

        // Rate mode: (RollRate_deg_s, PitchRate_deg_s, YawRate_deg_s, Throttle01)
        public static Axis4 Rate(float rollRate_deg_s, float pitchRate_deg_s, float yawRate_deg_s, float
 throttle01)
            => new Axis4(rollRate_deg_s, pitchRate_deg_s, yawRate_deg_s, throttle01);

        public static Axis4 Wrench(float tx, float ty, float tz, float fz)
            => new Axis4(tx, ty, tz, fz);

        // yaw lives at index 2 for pos/vel (your UE comment: "yaw lives in Z" meaning the 3rd slot)
        public static bool HasYaw(in Axis4 cmd) => IsFinite(cmd[FlightAxes.Yaw]);

        // .NET 5+ has float.IsFinite; this works everywhere.
        private static bool IsFinite(float v) => !float.IsNaN(v) && !float.IsInfinity(v);
    }
}
```

Runtime/Scripts/SimCore/ControlAuthorityManager.cs

SHA-256 c4ccabafd73218905ced8cc1d9acd8b6881ffd1f4c87134fc74533dd42585242 | 397 lines | 17,538 bytes

```
using System;
using UnityEngine;
using SimCore.Common;

namespace SimCore
{
    /// <summary>
    /// Global authority state machine. Tracks which InputSource has control,
    /// manages source lifecycle, and handles External connection accept/deny flow.
    ///
    /// Authority is global (not per-drone) — all drones share one authority source.
    /// Can be extended to per-drone later if needed.
    /// </summary>
    [DefaultExecutionOrder(-900)] // After SimulationManager (-1000), before DroneManager (-500)
    public sealed class ControlAuthorityManager : MonoBehaviour
    {
        // ================================================================================
        // Configuration
        // ================================================================================

        [Header("Authority Settings")]
        [Tooltip("Which source should have authority on startup. Can be overridden by YAML config.")]
        [SerializeField] private InputSource defaultSource = InputSource.UI;

        [Tooltip("Seconds to wait for default source before showing switch-to-UI banner.")]
        [SerializeField] private float connectionTimeoutSec = 60f;

        [Tooltip("Auto-accept External connections when External is the default source.")]
        [SerializeField] private bool autoAcceptWhenDefault = true;

        // ================================================================================
        // Events
        // ================================================================================

        /// <summary>Fired when authority switches from one source to another.</summary>
        public event Action<InputSource, InputSource> OnAuthorityChanged;

        /// <summary>Fired when any source's lifecycle status changes.</summary>
        public event Action<InputSource, SourceStatus, SourceStatus> OnSourceStatusChanged;

        /// <summary>Fired when an External client wants to connect and needs user approval.</summary>
        public event Action OnExternalConnectionRequest;

        /// <summary>Fired when waiting for the default source to connect (with timeout info).</summary>
        public event Action<InputSource> OnWaitingForSource;

        /// <summary>Fired when waiting timeout expires. UI can show "Switch to UI" option.</summary>
        public event Action OnConnectionTimeout;

        // ================================================================================
        // Runtime State
        // ================================================================================

        private InputSource _effectiveAuthority;
        private InputSource _configuredDefault;
        private bool _sessionOverrideActive;

        private readonly SourceStatus[] _sourceStatuses = new SourceStatus[3]; // indexed by InputSource

        // Connection waiting state
        private bool _waitingForConnection;
        private float _waitTimer;
        private bool _externalPendingApproval;

        // ================================================================================
        // Public Accessors
        // ================================================================================

        /// <summary>Which source currently has authority to send commands.</summary>
```

```
        public InputSource EffectiveAuthority => _effectiveAuthority;

        /// <summary>The configured default source (from YAML or inspector).</summary>
        public InputSource ConfiguredDefault => _configuredDefault;

        /// <summary>True if the user switched away from the default source for this session.</summary>
        public bool IsSessionOverride => _sessionOverrideActive;

        /// <summary>True if waiting for the default source to connect.</summary>
        public bool IsWaitingForConnection => _waitingForConnection;

        /// <summary>True if an External connection is pending user approval.</summary>
        public bool IsExternalPendingApproval => _externalPendingApproval;

        /// <summary>Get the status of a specific source.</summary>
        public SourceStatus GetSourceStatus(InputSource source) => _sourceStatuses[(int)source];

        // ================================================================================
        // Singleton Access (optional, prefer direct references)
        // ================================================================================

        private static ControlAuthorityManager _instance;

        /// <summary>
        /// Get cached instance. Falls back to scene find with warning.
        /// Prefer direct inspector references over this.
        /// </summary>
        public static ControlAuthorityManager Get()
        {
            if (_instance != null) return _instance;
            _instance = FindAnyObjectByType<ControlAuthorityManager>();
            if (_instance != null)
                Debug.LogWarning("[AuthorityManager] Instance resolved via Find. Prefer direct reference
.");
            return _instance;
        }

        // ================================================================================
        // Unity Lifecycle
        // ================================================================================

        private void Awake()
        {
            _instance = this;
            _configuredDefault = defaultSource;

            // Initialize source statuses
            // UI is always Available (never Unavailable)
            SetSourceStatusInternal(InputSource.UI, SourceStatus.Available);
            SetSourceStatusInternal(InputSource.Internal, SourceStatus.Unavailable);
            SetSourceStatusInternal(InputSource.External, SourceStatus.Unavailable);

            // Set initial authority to the default source
            _effectiveAuthority = defaultSource;
            _sessionOverrideActive = false;

            // If default is UI, mark it as Connected immediately
            if (defaultSource == InputSource.UI)
            {
                SetSourceStatusInternal(InputSource.UI, SourceStatus.Connected);
            }
            else
            {
                // Start waiting for the configured default source
                _waitingForConnection = true;
                _waitTimer = 0f;
            }

            Debug.Log($"[AuthorityManager] Initialized: default={defaultSource}, authority={_effectiveAu
thority}");
        }

        private void Start()
```

```
{
    if (_waitingForConnection)
        OnWaitingForSource?.Invoke(_configuredDefault);
}

private void Update()
{
    if (!_waitingForConnection) return;

    _waitTimer += Time.unscaledDeltaTime;

    if (_waitTimer >= connectionTimeoutSec)
    {
        _waitTimer = 0f; // Reset for next timeout cycle
        Debug.Log($"[AuthorityManager] Connection timeout for {_configuredDefault}. " +
                  $"User can switch to UI or keep waiting.");
        OnConnectionTimeout?.Invoke();
    }
}

private void OnDestroy()
{
    if (_instance == this) _instance = null;
}

// ============================================================================
// Authority Control (Public API)
// ============================================================================

/// <summary>
/// Switch authority to a different source. Only succeeds if the target source
/// is in Connected status. Fires OnAuthorityChanged.
/// </summary>
public bool RequestAuthority(InputSource newSource)
{
    if (newSource == _effectiveAuthority)
    {
        Debug.Log($"[AuthorityManager] RequestAuthority({newSource}): already active, no-op.");
        return true;
    }

    var status = GetSourceStatus(newSource);
    if (status != SourceStatus.Connected && status != SourceStatus.Available)
    {
        Debug.LogWarning($"[AuthorityManager] RequestAuthority({newSource}) DENIED: " +
                         $"status={status}. Source must be Connected or Available.");
        return false;
    }

    var old = _effectiveAuthority;
    _effectiveAuthority = newSource;

    // Track if this is a session override
    if (newSource != _configuredDefault)
        _sessionOverrideActive = true;

    Debug.Log($"[AuthorityManager] Authority: {old} -> {newSource}" +
              (_sessionOverrideActive ? " (session override)" : ""));

    OnAuthorityChanged?.Invoke(old, newSource);
    return true;
}

/// <summary>
/// Session override: switch to UI regardless of configured default.
/// Does not change the YAML default — next session starts with configured default again.
/// </summary>
public void SwitchToUIForSession()
{
    _waitingForConnection = false;
    _sessionOverrideActive = true;

    // Ensure UI is Connected
```

```
            SetSourceStatus(InputSource.UI, SourceStatus.Connected);

            var old = _effectiveAuthority;
            _effectiveAuthority = InputSource.UI;

            Debug.Log($"[AuthorityManager] Session override: {old} -> UI");
            OnAuthorityChanged?.Invoke(old, InputSource.UI);
        }

        /// <summary>
        /// Update the configured default source. Takes effect for the current session
        /// (resets session override) and persists if you save YAML.
        /// </summary>
        public void SetDefaultSource(InputSource newDefault)
        {
            _configuredDefault = newDefault;
            defaultSource = newDefault;
            _sessionOverrideActive = false;
            Debug.Log($"[AuthorityManager] Default source changed to: {newDefault}");
        }

        // ========================================================================
        // Source Lifecycle (called by source adapters)
        // ========================================================================

        /// <summary>
        /// Called when a source becomes available (e.g., Internal script registers,
        /// RPC server starts listening). Does not grant authority.
        /// </summary>
        public void SetSourceStatus(InputSource source, SourceStatus newStatus)
        {
            var old = _sourceStatuses[(int)source];
            if (old == newStatus) return;

            // Denied is sticky for External — can't transition out of Denied in same session
            if (old == SourceStatus.Denied && source == InputSource.External)
            {
                Debug.LogWarning($"[AuthorityManager] External is Denied for this session. Ignoring stat
us change to {newStatus}.");
                return;
            }

            SetSourceStatusInternal(source, newStatus);

            Debug.Log($"[AuthorityManager] Source {source}: {old} -> {newStatus}");
            OnSourceStatusChanged?.Invoke(source, old, newStatus);

            // If the default source just connected and we were waiting, stop waiting
            if (source == _configuredDefault && newStatus == SourceStatus.Connected && _waitingForConnec
tion)
            {
                _waitingForConnection = false;
                _waitTimer = 0f;

                // Auto-grant authority if this is the configured default
                if (_effectiveAuthority != source)
                {
                    RequestAuthority(source);
                }

                Debug.Log($"[AuthorityManager] Default source {source} connected. Waiting resolved.");
            }
        }

        /// <summary>
        /// Called when an External client attempts to connect.
        /// If External is default and autoAcceptWhenDefault is true, auto-accepts.
        /// Otherwise, fires OnExternalConnectionRequest for UI to show popup.
        /// </summary>
        public void NotifyExternalConnectionAttempt()
        {
            var currentStatus = GetSourceStatus(InputSource.External);
```

```
            if (currentStatus == SourceStatus.Denied)
            {
                Debug.Log("[AuthorityManager] External connection attempt blocked — Denied for this sess
ion.");
                return;
            }

            if (_configuredDefault == InputSource.External && autoAcceptWhenDefault)
            {
                // Auto-accept when External is the configured default
                AcceptExternalConnection();
                return;
            }

            // Need user approval
            _externalPendingApproval = true;
            Debug.Log("[AuthorityManager] External connection request — awaiting user approval.");
            OnExternalConnectionRequest?.Invoke();
        }

        /// <summary>
        /// User approved the External connection. Resets drone, switches authority.
        /// </summary>
        public void AcceptExternalConnection()
        {
            _externalPendingApproval = false;

            SetSourceStatus(InputSource.External, SourceStatus.Connected);

            var old = _effectiveAuthority;
            _effectiveAuthority = InputSource.External;

            if (old != _effectiveAuthority)
            {
                Debug.Log($"[AuthorityManager] External connection ACCEPTED. Authority: {old} -> Externa
l");
                OnAuthorityChanged?.Invoke(old, InputSource.External);
            }
        }

        /// <summary>
        /// User denied the External connection. Blocks all future attempts for this session.
        /// </summary>
        public void DenyExternalConnection()
        {
            _externalPendingApproval = false;

            SetSourceStatus(InputSource.External, SourceStatus.Denied);
            Debug.Log("[AuthorityManager] External connection DENIED for this session.");
        }

        /// <summary>
        /// Called when an External client disconnects. Does NOT auto-switch authority.
        /// The proxy will use safe fallback commands until authority is explicitly changed.
        /// </summary>
        public void NotifyExternalDisconnected()
        {
            if (GetSourceStatus(InputSource.External) == SourceStatus.Denied) return;

            SetSourceStatus(InputSource.External, SourceStatus.Available);
            Debug.Log("[AuthorityManager] External disconnected. Authority unchanged — proxy uses safe f
allback.");
        }

        /// <summary>
        /// Called when an Internal script registers itself in the scene.
        /// </summary>
        public void NotifyInternalAvailable()
        {
            SetSourceStatus(InputSource.Internal, SourceStatus.Available);
        }

        /// <summary>
```

```
        /// Called when an Internal script starts actively publishing commands.
        /// </summary>
        public void NotifyInternalConnected()
        {
            SetSourceStatus(InputSource.Internal, SourceStatus.Connected);
        }

        /// <summary>
        /// Called when an Internal script is removed or stops publishing.
        /// </summary>
        public void NotifyInternalDisconnected()
        {
            SetSourceStatus(InputSource.Internal, SourceStatus.Unavailable);
        }

        // ================================================================================
        // Query Helpers
        // ================================================================================

        /// <summary>Is the given source the current authority?</summary>
        public bool IsAuthority(InputSource source) => source == _effectiveAuthority;

        /// <summary>Can this source potentially take authority? (Connected or Available)</summary>
        public bool CanRequestAuthority(InputSource source)
        {
            var status = GetSourceStatus(source);
            return status == SourceStatus.Connected || status == SourceStatus.Available;
        }

        /// <summary>Is the External source blocked for this session?</summary>
        public bool IsExternalDenied => GetSourceStatus(InputSource.External) == SourceStatus.Denied;

        // ================================================================================
        // Internal
        // ================================================================================

        private void SetSourceStatusInternal(InputSource source, SourceStatus status)
        {
            _sourceStatuses[(int)source] = status;
        }
    }
}
```

Runtime/Scripts/SimCore/DroneCatalog.cs

SHA-256 8d1f9c6c63cc82dd4abf2bf27cb0c91ffd877bcd86d9c8a2cc1dd501d387b1f6 | 44 lines | 1,506 bytes

```
// Runtime/SimCore/DroneCatalog.cs
using System;
using UnityEngine;
using DroneCore;

namespace QuadSim.SimCore
{
    [CreateAssetMenu(fileName = "DroneCatalog", menuName = "QuadSim/Drone Catalog", order = 1)]
    public sealed class DroneCatalog : ScriptableObject
    {
        [Serializable]
        public struct Entry
        {
            [Tooltip("YAML name (no extension) in StreamingAssets/Configs/Drones or package Resources.")
]
            public string configName;

            [Tooltip("Chassis prefab. Physics comes from the YAML; this is the body, mounts and meshes."
)]
            public QuadPawn pawn;
        }

        [Tooltip("Used when a config has no entry. Should be QuadPawn_Base.")]
        public QuadPawn fallbackPawn;

        public Entry[] entries;
```

```
        public QuadPawn Resolve(string configName)
        {
            if (entries != null && !string.IsNullOrWhiteSpace(configName))
            {
                for (int i = 0; i < entries.Length; i++)
                {
                    if (string.Equals(entries[i].configName, configName,
                            StringComparison.OrdinalIgnoreCase) &&
                        entries[i].pawn != null)
                        return entries[i].pawn;
                }
            }

            if (fallbackPawn == null)
                Debug.LogError($"[DroneCatalog] No pawn for '{configName}' and no fallback set.");
            return fallbackPawn;
        }
    }
}
```

Runtime/Scripts/SimCore/FrequencyLimiter.cs

SHA-256 d227d3238b9f0bd4755bf61bfc2cd9db1230d4f575259020be984819fca156ff | 84 lines | 2,615 bytes

```
// Assets/Scripts/SimCore/FrequencyLimiter.cs

using System;

namespace SimCore
{
    /// <summary>
    /// Clock-driven frequency limiter (sim-time), similar to your UE FrequencyLimiter.
    /// Use this to run sensors/controllers at subrates relative to the base sim dt.
    /// </summary>
    public sealed class FrequencyLimiter
    {
        private readonly double _periodSec;
        private long _nextTickNanos;
        private bool _initialized;

        public double PeriodSec => _periodSec;

        public FrequencyLimiter(double rateHz, long startNowNanos = 0)
        {
            if (rateHz <= 0) throw new ArgumentOutOfRangeException(nameof(rateHz), "rateHz must be > 0."
);
            _periodSec = 1.0 / rateHz;

            if (startNowNanos != 0)
            {
                Reset(startNowNanos);
            }
        }

        /// <summary>
        /// Returns true if it's time to run at the given nowNanos (does not advance schedule).
        /// </summary>
        public bool ShouldRun(long nowNanos)
        {
            if (!_initialized)
            {
                // First call: run immediately, then schedule next tick.
                _nextTickNanos = nowNanos;
                _initialized = true;
            }

            return nowNanos >= _nextTickNanos;
        }

        /// <summary>
        /// Call after you run. Schedules the next tick.
        /// Uses "catch-up" stepping to avoid drift if you miss periods.
        /// </summary>
        public void Consume(long nowNanos)
```

```
        {
            if (!_initialized)
            {
                _nextTickNanos = nowNanos;
                _initialized = true;
            }

            long periodNanos = (long)(_periodSec * 1.0e9);

            // If we're behind, jump forward by as many whole periods as needed (no drift).
            if (nowNanos >= _nextTickNanos)
            {
                long behind = nowNanos - _nextTickNanos;
                long periodsMissed = behind / periodNanos;
                _nextTickNanos += (periodsMissed + 1) * periodNanos;
            }
        }

        public void Reset(long nowNanos)
        {
            _initialized = true;
            _nextTickNanos = nowNanos;
        }

        /// <summary>
        /// Convenience: should run now, and if so consumes.
        /// </summary>
        public bool ShouldRunAndConsume(long nowNanos)
        {
            if (!ShouldRun(nowNanos)) return false;
            Consume(nowNanos);
            return true;
        }
    }
}
```

Runtime/Scripts/SimCore/QuadSimEnvironmentBootstrap.cs

SHA-256 1a6fb945ae54308dde61988ac68ab50634846ece3ccaf0a072d3d1a99b7df1e0 | 150 lines | 7,238 bytes

```
// Assets/Scripts/SimCore/Config/QuadSimEnvironmentBootstrap.cs
//
// Reads a QuadSimSceneConfig asset and pushes each field into the existing
// subsystems at startup. Lives on the QuadSimEnvironment prefab root.
//
// TIMING (why this works without a refactor):
//   SimulationManager  [-1000]  Awake: ClockFactory.Initialize (timing)
//   DroneManager       [-500]   Awake: reads PlayerPrefs config name
//                               OnSimulationStart (fired from SimManager.Start):
//                               loads YAML + SPAWNS the drone
//   ExternalRpcAdapter [0]      Start: ApplyCommandLineOverrides + StartServer (binds)
//   THIS bootstrap     [0]      Awake: writes ports + drone-manager fields
//                               Start: geo origin + wind
//
//   Every Awake completes before any Start, and the spawn happens in Start,
//   so anything written here in Awake is visible to the spawn and to the
//   adapter's StartServer. Default order (0) runs after DroneManager.Awake
//   (-500), so the asset's config name overrides the PlayerPrefs "last used".

using UnityEngine;
using SimCore.Rpc;
using DroneCore.Core;
using DroneCore.Api;
using RobotCore;
using MathUtil;

namespace SimCore
{
    [DefaultExecutionOrder(0)]
    public sealed class QuadSimEnvironmentBootstrap : MonoBehaviour
    {
        [Header("Config")]
        [Tooltip("The scene configuration asset. Set wind/GPS/seeds/ports/drone here — no C#.")]
        [SerializeField] private QuadSimSceneConfig config;
```

```
        [Header("Scene References (auto-resolved if left empty)")]
        [SerializeField] private ExternalRpcAdapter rpcAdapter;
        [SerializeField] private SimulationManager  simManager;
        [SerializeField] private DroneManager droneManager;
        [SerializeField] private QuadSimApi api;
        [SerializeField] private SensorManager sensorManager;
        [Tooltip("Empty child transform marking where the drone spawns.")]
        [SerializeField] private Transform droneSpawnPoint;

        [Tooltip("The WindModule trigger volume on WindZone_Prefab.")]
        [SerializeField] private Experimental.WindModule windZone;


        // ---------------------------------------------------------------------
        // Awake: everything the spawn / socket-bind needs to see.
        // ---------------------------------------------------------------------
        private void Awake()
        {
            if (config == null)
            {
                Debug.LogError("[EnvBootstrap] No QuadSimSceneConfig assigned. " +
                               "Subsystems will use their own inspector defaults.");
                return;
            }

            ResolveReferences();

            // --- RPC ports -> adapter (read later in adapter.Start/StartServer) ---
            if (rpcAdapter != null)
                rpcAdapter.ConfigureEndpoint(
                    config.rpcBindAddress, config.commandPort, config.telemetryPort);

            // --- Drone selection + spawn point -> DroneManager (read in OnSimulationStart) ---
            if (droneManager != null)
            {
                droneManager.SetSelectedConfig(config.droneConfigName);

                if (droneSpawnPoint != null)
                    droneManager.SpawnOrigin = droneSpawnPoint.position;
                // Scene env (seed / noise / PX4 origin) is kept separate from the
                // drone-model DroneConfig. DroneManager forwards it into the drone's
                // SensorManager at spawn, BEFORE SensorManager.OnStart() runs — so the
                // GPS origin and RNG seed are correct from the very first sample.
                // (This is why geo origin is NOT set on Geo directly here: SensorManager
                //  is the single writer of the origin, and we feed it the right value.)
                droneManager.SetSceneEnv(
                    config.masterSeed,
                    config.enableSensorNoise,
                    config.originLatitudeDeg,
                    config.originLongitudeDeg,
                    config.originAltitudeM,
                    config.startupSpawnPolicy,
                    config.droneCatalog != null ? config.droneCatalog.Resolve(config.droneConfigName) :
null,
                    config.payloadPrefab,
                    config.spawnPayload,
                    config.payloadMountPath,
                    config.outputFrame,
                    config.startingMode);
            }

            if (simManager != null)
            {
                simManager.SetMode(config.RunMode);
            }

            if (sensorManager != null)
            {
                sensorManager.SetOutputFrame(config.outputFrame);
            }
            Debug.Log($"[EnvBootstrap] Awake applied: ports=({config.commandPort},{config.telemetryPort}
) " +
```

```
                    $"drone='{config.droneConfigName}' seed={config.masterSeed} noise={config.enableSe
nsorNoise}");
        }

        // ------------------------------------------------------------------
        // Start: scene-global things whose owner is fully initialized by now.
        // (Geo origin is NOT here — it rides the spawn passthrough into
        //  SensorManager so there's no race with GPS.Initialize.)
        // ------------------------------------------------------------------
        private void Start()
        {
            if (config == null) return;

            // --- Wind -> WindModule trigger volume --------------------------
            // WindModule needs no drone reference (OnTriggerStay catches any
            // Rigidbody). Direction = zone orientation; we yaw it from the asset.
            if (windZone != null)
            {
                if (config.windHeadingDeg != 0f)
                    windZone.transform.rotation = Quaternion.Euler(0f, config.windHeadingDeg, 0f);
                windZone.SetWindSpeed(config.windSpeed);
                windZone.SetEnabled(config.windEnabled);
                Debug.Log($"[EnvBootstrap] Wind: enabled={config.windEnabled} " +
                          $"speed={config.windSpeed} headingDeg={config.windHeadingDeg}");
            }
        }

        // ------------------------------------------------------------------
        private void ResolveReferences()
        {
            if (rpcAdapter == null)
                rpcAdapter = FindAnyObjectByType<ExternalRpcAdapter>();
            if (droneManager == null)
                droneManager = FindAnyObjectByType<DroneManager>();
            if (windZone == null)
                windZone = FindAnyObjectByType<Experimental.WindModule>();

            if (rpcAdapter == null)
                Debug.LogWarning("[EnvBootstrap] No ExternalRpcAdapter found; ports not applied.");
            if (droneManager == null)
                Debug.LogWarning("[EnvBootstrap] No DroneManager found; drone/seed not applied.");
            if (droneSpawnPoint == null)
                Debug.LogWarning("[EnvBootstrap] No DroneSpawnPoint set; using DroneManager's default or
igin.");
        }
    }
}
```

Runtime/Scripts/SimCore/QuadSimSceneConfig.cs

SHA-256 32123316f64823696d3c630d2be12f4e37e33deaae78c4b83e02bd8d45ff8cd5 | 138 lines | 6,317 bytes

```
// Assets/Scripts/SimCore/Config/QuadSimSceneConfig.cs
//
// Researcher-facing scene configuration for the QuadSimEnvironment prefab.
//
// Drop the QuadSimEnvironment prefab into an empty scene, point its
// QuadSimEnvironmentBootstrap at one of these assets, hit Play. No C# editing.
//
// Create assets via:  Assets > Create > QuadSim > Scene Config
//

using System;
using UnityEngine;
using DroneCore.Core;
using DroneCore;
using Experimental;
using MathUtil;
using QuadSim.SimCore;
using SimCore.Common;
namespace SimCore
{

    [CreateAssetMenu(
```

```
        fileName = "QuadSimSceneConfig",
        menuName = "QuadSim/Scene Config",
        order = 0)]

    public sealed class QuadSimSceneConfig : ScriptableObject
    {
        // ================================================================
        // Sim and Coordinator
        // ================================================================
        [Header("Run Mode")]
        public SimulationManager.RunMode RunMode = SimulationManager.RunMode.FreeRun;

        // ================================================================
        // Drone
        // ================================================================
        [Header("Drone")]
        [Tooltip("YAML file name (without extension) in StreamingAssets/Configs/ " +
                 "that DroneConfigLoader reads. Selects mass/inertia/motor params.")]
        public string droneConfigName = "flightmare_quad";

        [Header("Drone Prefab")]
        [Tooltip("Drag your Drone Model Prefab here")]
        public DroneCatalog droneCatalog;
        [Header("Spawning")]
        [Tooltip("Choose whether simulation startup may create a default drone when the scene contains n
one.")]
        public StartupSpawnPolicy startupSpawnPolicy = StartupSpawnPolicy.SpawnDefaultIfNone;

        // ================================================================
        // Payload
        // ================================================================
        [Header("Drone Payload")]
        [Tooltip("Attach Payload Prefab")]
        public PayloadAttachment payloadPrefab;

        [Header("Spawn Payload")]
        public bool spawnPayload = false;

        [Header("Payload Mount")]
        public string payloadMountPath = "Bottle";
        // ================================================================
        // Controller
        // ================================================================
        [Header("Frame")]
        public SimFrame outputFrame = SimFrame.FLU;

        [Header("Controller Mode WIP")]
        [Tooltip("Choose which level of the cascade you want to start in")]
        public GoalMode startingMode = GoalMode.Rate;

        // ================================================================
        // RPC (Python SDK boundary)
        // ================================================================
        [Header("RPC Ports")]
        [Tooltip("RPC bind address. Loopback is the secure default; remote exposure must be explicit.")]
        public string rpcBindAddress = "127.0.0.1";

        [Tooltip("ZMQ REQ/REP command port. SDK default 5555.")]
        public int commandPort = 5555;

        [Tooltip("ZMQ PUB/SUB telemetry port. SDK default 5556.")]
        public int telemetryPort = 5556;

        // ================================================================
        // Geodetic origin (WGS84) — where QGC shows the vehicle; EKF2 latches
        // this from the first HIL_GPS fix.
        // ================================================================
        [Header("GPS / Geodetic Origin")]
        [Tooltip("Latitude in degrees. Default: Eglin AFB.")]
        public double originLatitudeDeg = 30.4636;

        [Tooltip("Longitude in degrees. Default: Eglin AFB.")]
        public double originLongitudeDeg = -86.5533;
```

```
        [Tooltip("Ellipsoidal altitude in meters.")]
        public double originAltitudeM = 0.0;

        // ================================================================
        // Sensor noise — keep OFF for clean determinism, ON for PX4 SITL.
        // Per-sensor streams derive from masterSeed + fixed offsets
        // (IMU +1, GPS +2, baro +3, mag +4) inside the drone's SensorManager.
        // ================================================================
        [Header("Sensor Noise")]
        [Tooltip("Master Gaussian RNG seed. Per-sensor offsets are applied downstream " +
                 "so parallel runs stay reproducible.")]
        public int masterSeed = 12345;

        [Tooltip("Inject sensor noise. Default false (deterministic). Set true for PX4 SITL / HIL.")]
        public bool enableSensorNoise = false;

        // ================================================================
        // Wind — applied as a world-frame force directly to the Rigidbody.
        // Human-friendly heading+speed; bootstrap resolves to a world vector.
        // ================================================================
        // WindModule is a trigger volume: it blows along its local +Z (forward) and
        // applies drag to any Rigidbody inside it. Direction = the zone's orientation;
        // the bootstrap yaws the zone transform from windHeadingDeg so you can steer
        // wind from this asset without touching the hierarchy.
        [Header("Wind")]
        [Tooltip("Enable the wind disturbance (drives WindModule.SetEnabled).")]
        public bool windEnabled = false;

        [Tooltip("Fan jet air speed at the source face, m/s (drives WindModule.SetWindSpeed).")]
        public float windSpeed = 0f;

        [Tooltip("Wind heading (yaw degrees) applied to the WindZone transform. " +
                 "0 = the zone's authored forward. Leave at 0 to steer by rotating the zone by hand.")]
        public float windHeadingDeg = 0f;

        // We want to add:
        /* Sim Manager:
         * - Run Mode: Manual/Free Mode
         * Mode Coordinator:
         * - Starting Mode: Position/ etc...
         */
    }
}
```

Runtime/Scripts/SimCore/SimClock.cs

SHA-256 01bb61bfd805de736ba3e44c8ac0a0b7a1e9809fa7cde604e5a1ced47c55be86 | 162 lines | 5,394 bytes

```
// Assets/Scripts/SimCore/SimClock.cs

using System;

namespace SimCore
{
    /// <summary>
    /// Authoritative simulation clock. Use this everywhere (controllers, sensors, limiters, IO).
    /// Do not use UnityEngine.Time for simulation timing.
    /// </summary>
    public interface IClock
    {
        long NowNanos { get; }
        long StartNanos { get; }
        ulong StepCount { get; }

        /// <summary>Base fixed step size in seconds (e.g. 0.004 for 250 Hz).</summary>
        double StepSizeSec { get; }
        long StepSizeNanos { get; }

        /// <summary>Advance exactly one fixed step.</summary>
        long Step();

        /// <summary>Advance by an arbitrary amount (used for scaled clocks / accumulators).</summary>
        long StepBy(double seconds);
```

```
        /// <summary>Reset the clock back to start (and clears step count).</summary>
        void Reset(long? newStartNanos = null);

        double ElapsedSince(long sinceNanos);
    }

    public abstract class ClockBase : IClock
    {
        private ulong _stepCount;

        public abstract long NowNanos { get; }
        public abstract long StartNanos { get; }
        public virtual double StepSizeSec => 0.0;
        public virtual long StepSizeNanos => 0;
        public ulong StepCount => _stepCount;

        public virtual long Step()
        {
            _stepCount++;
            return NowNanos;
        }

        public virtual long StepBy(double seconds)
        {
            // default behavior: count as "a step" even if dt isn't fixed
            _stepCount++;
            return NowNanos;
        }

        public virtual void Reset(long? newStartNanos = null)
        {
            _stepCount = 0;
        }

        public double ElapsedSince(long sinceNanos)
        {
            return ElapsedBetween(NowNanos, sinceNanos);
        }

        public static double ElapsedBetween(long secondNanos, long firstNanos)
        {
            return (secondNanos - firstNanos) / 1.0e9;
        }

        protected static long AddTo(long tNanos, double dtSec)
        {
            // Clamp tiny negatives (can happen with scale/pause edge cases)
            if (dtSec < 0) dtSec = 0;
            return tNanos + (long)(dtSec * 1.0e9);
        }

        protected void AdvanceSteps(ulong count) => _stepCount += count;
        protected void SetStepCount(ulong count) => _stepCount = count;
    }

    /// <summary>
    /// Deterministic fixed-step clock. Default start time = 0 (recommended).
    /// </summary>
    public sealed class SteppableClock : ClockBase
    {
        private long _start;
        private readonly double _stepSec;
        private readonly long _stepNanos;

        public SteppableClock(double stepSec, long startNanos = 0)
        {
            if (stepSec <= 0) throw new ArgumentOutOfRangeException(nameof(stepSec), "Step must be > 0."
);
            _stepNanos = (long)Math.Round(stepSec * 1.0e9);
            if (_stepNanos <= 0) throw new ArgumentOutOfRangeException(nameof(stepSec), "Step is below o
ne nanosecond.");
            _stepSec = _stepNanos / 1.0e9;
```

```
            _start = startNanos;
        }

        public override long NowNanos => _start + (long)StepCount * _stepNanos;
        public override long StartNanos => _start;
        public override double StepSizeSec => _stepSec;
        public override long StepSizeNanos => _stepNanos;

        public override long Step()
        {
            base.Step(); // increments StepCount
            return NowNanos;
        }

        public override long StepBy(double seconds)
        {
            if (seconds <= 0.0) return NowNanos;
            ulong steps = (ulong)Math.Round(seconds * 1.0e9 / _stepNanos);
            AdvanceSteps(steps);
            return NowNanos;
        }

        public override void Reset(long? newStartNanos = null)
        {
            base.Reset(newStartNanos);
            _start = newStartNanos ?? _start;
        }

        public void SetNowNanos(long nowNanos)
        {
            ulong step = nowNanos <= _start
                ? 0
                : (ulong)((nowNanos - _start) / _stepNanos);
            SetStepCount(step);
        }
    }

    /// <summary>
    /// A wrapper that can pause and apply time scaling to an underlying clock.
    /// This does NOT change physics dt; it changes how much sim time you choose to advance
    /// per rendered frame (your SimulationManager will decide how many base steps to run).
    /// </summary>
    public sealed class ScaledClock
    {
        public bool IsPaused { get; private set; }
        public double TimeScale { get; private set; } = 1.0;

        public void SetPaused(bool paused) => IsPaused = paused;

        public void SetTimeScale(double scale)
        {
            // Allow >1 for fast-forward; clamp negatives to 0.
            TimeScale = (scale < 0) ? 0 : scale;
        }

        /// <summary>
        /// Convert "real frame delta" to "sim seconds to advance" (used by SimulationManager accumulato
r).
        /// </summary>
        public double ComputeAdvanceSeconds(double realDeltaSeconds)
        {
            if (IsPaused) return 0.0;
            return realDeltaSeconds * TimeScale;
        }
    }
}
```

Runtime/Scripts/SimCore/SimCoreApi.cs

SHA-256 bd0993c7f38298ee8e07216807be62708e55d239c5940f333df16571e4489d63 | 453 lines | 19,418 bytes

```
// Assets/Scripts/SimCore/SimCoreApi.cs
// Phase 6: Global Simulation API Facade
```

```
//
// PURPOSE:
//   Single entry point for sim-wide operations: time control, authority management,
//   and global events. Lives on SimRoot alongside SimulationManager and ControlAuthorityManager.
//
// LAYER RULE:
//   SimCore is the parent layer. It does NOT reference DroneCore or DroneCore.Api.
//   For per-drone access, users go through QuadSimApi.Selected() or QuadSimApi.Get(i).
//   This keeps dependency direction clean: DroneCore → SimCore, never the reverse.
//
// OWNERSHIP:
//   - Delegates to: SimulationManager, ControlAuthorityManager
//   - Does NOT implement logic itself — pure facade pattern
//   - Singleton MonoBehaviour (one per scene, on SimRoot)
//
// DOES NOT:
//   - Control individual drones (use QuadSimApi)
//   - Access DroneManager or QuadPawn (that's DroneCore's job)
//   - Run physics (SimulationManager)
//   - Manage source state machines (ControlAuthorityManager)

using System;
using RobotCore.Core_Utils;
using UnityEngine;
using SimCore.Common;

namespace SimCore
{
    /// <summary>
    /// Global simulation API facade. Primary interface for sim-wide operations.
    /// Attach to SimRoot GameObject alongside SimulationManager and ControlAuthorityManager.
    ///
    /// Usage:
    /// <code>
    ///   var sim = SimCoreApi.Get();
    ///   sim.Pause();
    ///   sim.SetTimeScale(2.0);
    ///   sim.RequestAuthority(InputSource.Internal);
    /// </code>
    ///
    /// For per-drone access, use QuadSimApi directly:
    /// <code>
    ///   var drone = QuadSimApi.Selected();   // current drone
    ///   var drone0 = QuadSimApi.Get(0);      // by index
    /// </code>
    /// </summary>
    [DefaultExecutionOrder(-950)]
    [DisallowMultipleComponent]
    public sealed class SimCoreApi : MonoBehaviour
    {
        // ================================================================================
        // References (resolved in Awake)
        // ================================================================================

        [Header("Core References (auto-resolved if not set)")]
        [SerializeField] private SimulationManager simManager;
        [SerializeField] private ControlAuthorityManager authorityManager;
        public GeoReferencingSystem Geo { get; private set; }


        private bool _initialized;

        // ================================================================================
        // Singleton
        // ================================================================================

        private static SimCoreApi _instance;

        /// <summary>
        /// Get cached SimCoreApi instance. Falls back to scene find with warning.
        /// Prefer direct inspector references for performance-critical code.
        /// </summary>
        public static SimCoreApi Get()
```

```
        {
            if (_instance != null) return _instance;
            _instance = FindAnyObjectByType<SimCoreApi>();
            if (_instance != null)
                Debug.LogWarning("[SimCoreApi] Instance resolved via Find. Prefer direct reference.");
            return _instance;
        }

        // ============================================================================
        // Public Accessors
        // ============================================================================

        /// <summary>Has this API resolved all required references?</summary>
        public bool IsInitialized => _initialized;

        /// <summary>Direct access to SimulationManager (advanced use).</summary>
        public SimulationManager SimManager => simManager;

        /// <summary>Direct access to ControlAuthorityManager (advanced use).</summary>
        public ControlAuthorityManager AuthorityManager => authorityManager;

        // ============================================================================
        // Events (forwarded from ControlAuthorityManager)
        // ============================================================================

        /// <summary>Fired when authority changes. Args: (oldSource, newSource).</summary>
        public event Action<InputSource, InputSource> OnAuthorityChanged;

        /// <summary>Fired when a source's status changes. Args: (source, oldStatus, newStatus).</summar
y>
        public event Action<InputSource, SourceStatus, SourceStatus> OnSourceStatusChanged;

        /// <summary>Fired when an External client requests connection.</summary>
        public event Action OnExternalConnectionRequest;

        /// <summary>Fired when waiting for a source to connect.</summary>
        public event Action<InputSource> OnWaitingForSource;

        /// <summary>Fired when a source connection times out.</summary>
        public event Action OnConnectionTimeout;

        // ============================================================================
        // Unity Lifecycle
        // ============================================================================

        private void Awake()
        {
            _instance = this;
            Geo = new GeoReferencingSystem(
                47.397742,
                8.545594,
                488.0);
            ResolveReferences();
        }

        private void OnDestroy()
        {
            UnsubscribeEvents();
            if (_instance == this) _instance = null;
        }

        private void ResolveReferences()
        {
            if (simManager == null)
            {
                simManager = GetComponent<SimulationManager>();
                if (simManager == null)
                    simManager = FindAnyObjectByType<SimulationManager>();
            }

            if (authorityManager == null)
            {
                authorityManager = GetComponent<ControlAuthorityManager>();
```

```
                if (authorityManager == null)
                    authorityManager = ControlAuthorityManager.Get();
            }

            if (simManager == null)
                Debug.LogError("[SimCoreApi] No SimulationManager found! Time control will not work.");
            if (authorityManager == null)
                Debug.LogError("[SimCoreApi] No ControlAuthorityManager found! Authority control will no
t work.");

            SubscribeEvents();

            _initialized = (simManager != null && authorityManager != null);

            Debug.Log($"[SimCoreApi] Initialized (sim={simManager != null}, authority={authorityManager
!= null})");
        }

        private void SubscribeEvents()
        {
            if (authorityManager != null)
            {
                authorityManager.OnAuthorityChanged += HandleAuthorityChanged;
                authorityManager.OnSourceStatusChanged += HandleSourceStatusChanged;
                authorityManager.OnExternalConnectionRequest += HandleExternalConnectionRequest;
                authorityManager.OnWaitingForSource += HandleWaitingForSource;
                authorityManager.OnConnectionTimeout += HandleConnectionTimeout;
            }
        }

        private void UnsubscribeEvents()
        {
            if (authorityManager != null)
            {
                authorityManager.OnAuthorityChanged -= HandleAuthorityChanged;
                authorityManager.OnSourceStatusChanged -= HandleSourceStatusChanged;
                authorityManager.OnExternalConnectionRequest -= HandleExternalConnectionRequest;
                authorityManager.OnWaitingForSource -= HandleWaitingForSource;
                authorityManager.OnConnectionTimeout -= HandleConnectionTimeout;
            }
        }

        // ================================================================================
        // Time Control
        // ================================================================================

        /// <summary>Pause the simulation. Physics stops advancing.</summary>
        public void Pause()
        {
            if (simManager == null) { WarnMissing("SimulationManager", "Pause"); return; }
            simManager.SetPaused(true);
        }

        /// <summary>Resume the simulation from a paused state.</summary>
        public void Resume()
        {
            if (simManager == null) { WarnMissing("SimulationManager", "Resume"); return; }
            simManager.SetPaused(false);
        }

        /// <summary>Toggle pause state.</summary>
        public void TogglePause()
        {
            if (simManager == null) { WarnMissing("SimulationManager", "TogglePause"); return; }
            simManager.TogglePaused();
        }

        /// <summary>Whether the simulation is currently paused.</summary>
        public bool IsPaused => simManager != null && simManager.IsPaused;

        /// <summary>
        /// Advance exactly one physics step. Works even when paused.
        /// Useful for debugging, lockstep integration, and RL stepping.
```

```
/// </summary>
public void StepOnce()
{
    if (simManager == null) { WarnMissing("SimulationManager", "StepOnce"); return; }
    simManager.StepOnce();
}

/// <summary>
/// Queue N physics steps. Useful for RL fast-forward or lockstep bursts.
/// </summary>
public void StepMany(int steps)
{
    if (simManager == null) { WarnMissing("SimulationManager", "StepMany"); return; }
    simManager.StepMany(steps);
}

/// <summary>
/// Set the simulation time scale. 1.0 = real-time, 2.0 = 2x speed, 0.5 = half speed.
/// Does not affect physics dt — only how many steps are taken per rendered frame.
/// </summary>
public void SetTimeScale(double scale)
{
    if (simManager == null) { WarnMissing("SimulationManager", "SetTimeScale"); return; }
    simManager.SetTimeScale(scale);
}

/// <summary>Current simulation time scale.</summary>
public double TimeScale => simManager != null ? ClockFactory.Scaled.TimeScale : 1.0;

/// <summary>
/// Switch between FreeRun (real-time) and Manual (step-by-step) modes.
/// </summary>
public void SetRunMode(SimulationManager.RunMode mode)
{
    if (simManager == null) { WarnMissing("SimulationManager", "SetRunMode"); return; }
    simManager.SetRunMode(mode);
}

/// <summary>Current run mode.</summary>
public SimulationManager.RunMode RunMode =>
    simManager != null ? simManager.Mode : SimulationManager.RunMode.FreeRun;

/// <summary>The fixed physics timestep in seconds (e.g., 0.004 at 250Hz).</summary>
public double FixedDtSec => simManager != null ? simManager.BaseDtSec : 0.004;

/// <summary>Current simulation time in nanoseconds.</summary>
public long SimTimeNanos => ClockFactory.IsInitialized ? ClockFactory.Clock.NowNanos : 0;

/// <summary>Current simulation time in seconds.</summary>
public double SimTimeSec => SimTimeNanos / 1_000_000_000.0;

/// <summary>
/// Reset the entire simulation: time resets to 0, all systems notified.
/// </summary>
public void ResetSimulation()
{
    if (simManager == null) { WarnMissing("SimulationManager", "ResetSimulation"); return; }
    simManager.ResetSimulation();
}

// ================================================================================
// Authority Control
// ================================================================================

/// <summary>Who currently has authority to send commands.</summary>
public InputSource CurrentAuthority =>
    authorityManager != null ? authorityManager.EffectiveAuthority : InputSource.UI;

/// <summary>
/// Request authority for a specific source. Returns true if granted.
/// The source must be in Connected status to receive authority.
/// </summary>
public bool RequestAuthority(InputSource source)
```

```
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "RequestAuthority");
return false; }
            return authorityManager.RequestAuthority(source);
        }

        /// <summary>
        /// Set the default authority source (what the system waits for on startup).
        /// Does not immediately switch authority — call RequestAuthority for that.
        /// </summary>
        public void SetDefaultAuthority(InputSource source)
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "SetDefaultAuthority"
); return; }
            authorityManager.SetDefaultSource(source);
        }

        /// <summary>Immediately switch authority to UI for this session.</summary>
        public void SwitchToUI()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "SwitchToUI"); return
; }
            authorityManager.SwitchToUIForSession();
        }

        /// <summary>Get the current status of a specific input source.</summary>
        public SourceStatus GetSourceStatus(InputSource source)
        {
            if (authorityManager == null) return SourceStatus.Unavailable;
            return authorityManager.GetSourceStatus(source);
        }

        /// <summary>Check if a specific source currently has authority.</summary>
        public bool IsAuthority(InputSource source)
        {
            if (authorityManager == null) return source == InputSource.UI;
            return authorityManager.IsAuthority(source);
        }

        /// <summary>Whether External connections have been denied for this session.</summary>
        public bool IsExternalDenied =>
            authorityManager != null && authorityManager.IsExternalDenied;

        // --- Internal Source Lifecycle ---

        /// <summary>Notify that an Internal source is available (loaded but not commanding).</summary>
        public void NotifyInternalAvailable()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "NotifyInternalAvaila
ble"); return; }
            authorityManager.NotifyInternalAvailable();
        }

        /// <summary>Notify that an Internal source is connected and ready to command.</summary>
        public void NotifyInternalConnected()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "NotifyInternalConnec
ted"); return; }
            authorityManager.NotifyInternalConnected();
        }

        /// <summary>Notify that an Internal source has disconnected.</summary>
        public void NotifyInternalDisconnected()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "NotifyInternalDiscon
nected"); return; }
            authorityManager.NotifyInternalDisconnected();
        }

        // --- External Source Lifecycle ---

        /// <summary>Notify that an External client is attempting to connect.</summary>
        public void NotifyExternalConnectionAttempt()
```

```
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "NotifyExternalConnec
tionAttempt"); return; }
            authorityManager.NotifyExternalConnectionAttempt();
        }

        /// <summary>Accept a pending External connection.</summary>
        public void AcceptExternalConnection()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "AcceptExternalConnec
tion"); return; }
            authorityManager.AcceptExternalConnection();
        }

        /// <summary>Deny a pending External connection (sticky for session).</summary>
        public void DenyExternalConnection()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "DenyExternalConnecti
on"); return; }
            authorityManager.DenyExternalConnection();
        }

        /// <summary>Notify that the External client has disconnected.</summary>
        public void NotifyExternalDisconnected()
        {
            if (authorityManager == null) { WarnMissing("ControlAuthorityManager", "NotifyExternalDiscon
nected"); return; }
            authorityManager.NotifyExternalDisconnected();
        }
        // ============================================================================
        // Status Snapshot
        // ============================================================================

        /// <summary>
        /// Get a combined status snapshot of the simulation.
        /// Useful for external monitoring, logging, or RPC status endpoints.
        /// </summary>
        public SimStatus GetStatus()
        {
            return new SimStatus
            {
                IsPaused = IsPaused,
                TimeScale = TimeScale,
                RunMode = RunMode,
                SimTimeSec = SimTimeSec,
                FixedDtSec = FixedDtSec,
                CurrentAuthority = CurrentAuthority,
                UIStatus = GetSourceStatus(InputSource.UI),
                InternalStatus = GetSourceStatus(InputSource.Internal),
                ExternalStatus = GetSourceStatus(InputSource.External)
            };
        }

        // ============================================================================
        // Internal Helpers
        // ============================================================================

        private void WarnMissing(string component, string caller)
        {
            Debug.LogWarning($"[SimCoreApi] {caller}() — No {component} available.");
        }

        private void HandleAuthorityChanged(InputSource old, InputSource @new) =>
            OnAuthorityChanged?.Invoke(old, @new);

        private void HandleSourceStatusChanged(InputSource src, SourceStatus old, SourceStatus @new) =>
            OnSourceStatusChanged?.Invoke(src, old, @new);

        private void HandleExternalConnectionRequest() =>
            OnExternalConnectionRequest?.Invoke();

        private void HandleWaitingForSource(InputSource src) =>
            OnWaitingForSource?.Invoke(src);
```

```
        private void HandleConnectionTimeout() =>
            OnConnectionTimeout?.Invoke();
    }

    // ============================================================================
    // Sim Status Snapshot
    // ============================================================================

    /// <summary>
    /// Combined simulation status for monitoring, logging, or RPC.
    /// Does not include drone count — that's DroneCore's concern.
    /// </summary>
    public struct SimStatus
    {
        public bool IsPaused;
        public double TimeScale;
        public SimulationManager.RunMode RunMode;
        public double SimTimeSec;
        public double FixedDtSec;
        public InputSource CurrentAuthority;
        public SourceStatus UIStatus;
        public SourceStatus InternalStatus;
        public SourceStatus ExternalStatus;
    }
}
```

Runtime/Scripts/SimCore/SimulationManager.cs

SHA-256 b0778b124698ea9e5b4e8f17dc2a39c094121034e5091725c863d94a5db5770e | 318 lines | 11,267 bytes

```
// Assets/Scripts/SimCore/SimulationManager.cs

using System;
using System.Collections.Generic;
using UnityEngine;

namespace SimCore
{

    /// <summary>
    /// Unity equivalent to your UE SimulationManager/TimeController backbone:
    /// - Owns authoritative stepping (manual Physics.Simulate).
    /// - Owns pause/play/single-step/time-scale.
    /// - Drives a deterministic fixed-step pipeline: PrePhysics -> Physics -> PostPhysics.
    /// </summary>
    [DefaultExecutionOrder(-1000)] // run as early as possible
    public sealed class SimulationManager : MonoBehaviour
    {
        public enum RunMode
        {
            FreeRun,   // advance based on real time * timeScale
            Manual     // only advance when StepOnce() is requested (API-driven / lockstep-ready)
        }

        [Header("Timing")]
        [Tooltip("Base simulation rate (Hz). 250 Hz = dt 0.004s.")]
        [SerializeField] private double baseHz = 250.0;

        [Tooltip("How sim time advances each rendered frame (FreeRun) or only via explicit steps (Manual
).")]
        [SerializeField] private RunMode runMode = RunMode.FreeRun;

        [Tooltip("When in FreeRun: simAdvance = realDelta * timeScale.")]
        [SerializeField] private double timeScale = 1.0;

        [Tooltip("Safety cap: max fixed steps per rendered frame (prevents spiral-of-death).")]
        [SerializeField] private int maxStepsPerFrame = 1000;

        [Header("Control")]
        [SerializeField] private bool startPaused = false;

        [Header("Debug")]
        [SerializeField] private bool logEffectiveHz = true;
```

```
        [SerializeField] private float logIntervalSeconds = 1.0f;

        private List<ISimulatable> RegisteredManagers = new List<ISimulatable>(128);
        private bool _startFired = false;

        // FreeRun accumulator in seconds (sim-time to execute, converted into fixed dt steps).
        private double _accumulatorSec;

        // Manual stepping queue count (for API-driven stepping / future lockstep).
        private int _pendingSteps;

        // Debug
        private float _logTimer;
        private int _stepsThisInterval;

        public double BaseDtSec => ClockFactory.BaseDtSec;
        public ulong StepIndex => ClockFactory.Clock.StepCount;
        public long SimTimeNanos => ClockFactory.Clock.NowNanos;
        public RunMode Mode => runMode;

        public bool IsPaused => ClockFactory.Scaled.IsPaused;

        public void SetTimeScale(double ts)
        {
            timeScale = ts;
            ClockFactory.SetTimeScale(timeScale);
        }
        /// <summary>
        /// Run N fixed steps immediately on the main thread.
        /// Use this for lockstep RPC calls that must return post-step sensors.
        /// </summary>
        public void StepImmediate(int steps)
        {
            if (steps <= 0) return;
            RunFixedSteps(steps);
        }
        private void Awake()
        {
            Application.runInBackground = true;        // CRITICAL: Unity throttles to ~single-digit
            QualitySettings.vSyncCount = 0;            // fps when unfocused — and you're always
            Application.targetFrameRate = -1;          // focused on the terminal running Optuna

            Physics.simulationMode = SimulationMode.Script;
            ClockFactory.Initialize(baseHz: baseHz, startNanos: 0, paused: startPaused, timeScale: timeS
cale);
        }

        private void Start()
        {
            _startFired = true;
            foreach (var s in RegisteredManagers)
                s.OnSimulationStart(this);
        }

        private void OnDestroy()
        {
            // Restore default so exiting play mode doesn't leave editor in weird state.
            Physics.simulationMode = SimulationMode.FixedUpdate;
        }

        private void Update()
        {
            // Keep timeScale in sync if edited during play mode.
            ClockFactory.SetTimeScale(timeScale);

            // Decide how much sim-time to advance this frame.
            if (ClockFactory.Scaled.IsPaused)
            {
                // No stepping unless manual steps are queued.
            }
            else if (runMode == RunMode.FreeRun)
            {
                // Advance based on wall-frame delta * timeScale; then consume into fixed steps.
```

```
                double advance = ClockFactory.Scaled.ComputeAdvanceSeconds(Time.unscaledDeltaTime);
                _accumulatorSec += advance;
            }

            // Manual mode ignores accumulator; only StepOnce/StepMany enqueues steps.
            int stepsToRun = 0;

            if (runMode == RunMode.FreeRun)
            {
                double dt = BaseDtSec;
                if (dt <= 0) return;

                // Convert accumulated sim time into fixed steps.
                stepsToRun = (int)Math.Floor(_accumulatorSec / dt);
                if (stepsToRun > 0)
                {
                    _accumulatorSec -= stepsToRun * dt;
                }
            }

            // Add pending manual steps (works in both modes; useful for single-step while paused).
            if (_pendingSteps > 0)
            {
                stepsToRun += _pendingSteps;
                _pendingSteps = 0;
            }

            if (stepsToRun <= 0)
            {
                TickDebug(0);
                return;
            }

            if (stepsToRun > maxStepsPerFrame)
            {
                // Drop extra steps; keep sim responsive. This is a deliberate non-realtime behavior gua
rd.
                stepsToRun = maxStepsPerFrame;
                _accumulatorSec = 0; // avoid runaway accumulation
            }

            // Run fixed-step pipeline.
            RunFixedSteps(stepsToRun);

            TickDebug(stepsToRun);
        }

        private void RunFixedSteps(int steps)
        {
            double dt = BaseDtSec;
            float dtF = (float)dt;

            for (int i = 0; i < steps; i++)
            {
                long now = ClockFactory.StepOnce();

                // Pre-physics: controllers read commands, compute actuator targets, etc.
                for (int k = 0; k < RegisteredManagers.Count; k++)
                {
                    RegisteredManagers[k].PrePhysicsStep(dt, now);
                }

                // Physics step (deterministic fixed dt)
                Physics.Simulate(dtF);

                // Post-physics: sensors sample, logging, IO publish, etc.
                for (int k = 0; k < RegisteredManagers.Count; k++)
                {
                    RegisteredManagers[k].PostPhysicsStep(dt, now);
                }

                _stepsThisInterval++;
            }
```

```
        }

        private void TickDebug(int stepsRanThisFrame)
        {
            if (!logEffectiveHz) return;

            _logTimer += Time.unscaledDeltaTime;
            if (_logTimer < logIntervalSeconds) return;

            float interval = _logTimer;
            _logTimer = 0;

            double effectiveHz = _stepsThisInterval / interval;
            _stepsThisInterval = 0;

            Debug.Log($"[SimulationManager] mode={runMode} paused={ClockFactory.Scaled.IsPaused} dt={Bas
eDtSec:F6}s " +
                      $"timeScale={timeScale:F2} effHz={effectiveHz:F1} steps/frame(last)={stepsRanThisF
rame}");
        }

        /// <summary>
        /// Rebuilds the registry of simulatable systems (drones, sensors, IO).
        /// Call this if you dynamically spawn/despawn drones.
        /// </summary>
        public void RegisterManagers(ISimulatable manager)
        {
            if (!RegisteredManagers.Contains(manager))
            {
                RegisteredManagers.Add(manager);
                RegisteredManagers.Sort(CompareSimulatables);
                // If Start already ran, catch up the late registrant
                if (_startFired)
                    manager.OnSimulationStart(this);
            }
        }
        public void UnRegisterManager(ISimulatable manager)
        {
            RegisteredManagers.Remove(manager);
        }

        // ---- External control surface (API / UI / future PX4 lockstep) ----

        public void SetPaused(bool paused)
        {
            ClockFactory.SetPaused(paused);
        }

        public void TogglePaused()
        {
            ClockFactory.SetPaused(!ClockFactory.Scaled.IsPaused);
        }

        /// <summary>
        /// Queue exactly one fixed step. Works even if paused. Intended for UI "single step" and lockst
ep.
        /// </summary>
        public void StepOnce()
        {
            _pendingSteps += 1;
        }

        /// <summary>
        /// Queue N fixed steps. Useful for RL fast-forward or lockstep bursts.
        /// </summary>
        public void StepMany(int steps)
        {
            if (steps <= 0) return;
            _pendingSteps += steps;
        }

        public void SetRunMode(RunMode mode)
        {
```

```
            runMode = mode;
            // When entering Manual mode, clear accumulator so you don't "catch up" unexpectedly later.
            if (runMode == RunMode.Manual)
            {
                _accumulatorSec = 0;
            }
        }

        /// <summary>
        /// Reset sim-time to 0 and notify systems to reset their internal state.
        /// You will later also reset rigidbodies here (or via QuadPawn.Reset()).
        /// </summary>
        public void ResetSimulation()
        {
            ResetClock();

            foreach (var s in RegisteredManagers)
            {
                s.OnSimulationReset(this);
            }
        }

        public void ResetClock()
        {
            ClockFactory.Reset(newStartNanos: 0);
            _accumulatorSec = 0;
            _pendingSteps = 0;
        }

        public void SetMode(RunMode mode)
        {
            runMode = mode;
        }

        private static int CompareSimulatables(ISimulatable a, ISimulatable b)
        {
            int order = a.ExecutionOrder.CompareTo(b.ExecutionOrder);
            return order != 0 ? order : string.CompareOrdinal(a.DebugName, b.DebugName);
        }
    }

    /// <summary>
    /// Unity equivalent of your ISimulatable/loop callbacks used by SimulationManager.
    /// Pure contract: no Unity time usage inside implementations.
    /// </summary>
    public interface ISimulatable
    {
        /// <summary>Lower runs earlier. Use this to impose deterministic ordering.</summary>
        int ExecutionOrder { get; }

        string DebugName { get; }

        void OnSimulationStart(SimulationManager sim);
        void OnSimulationReset(SimulationManager sim);

        /// <param name="dtSec">Fixed dt seconds (0.004 at 250Hz)</param>
        /// <param name="nowNanos">ClockFactory.Clock.NowNanos after stepping</param>
        void PrePhysicsStep(double dtSec, long nowNanos);

        void PostPhysicsStep(double dtSec, long nowNanos);
    }
}
```

Runtime/Scripts/SimCore/StepContract.cs

SHA-256 751da8ccc622a883e8c3bc642a2648446b1892d7692c329e8a125ff6a07b2703 | 28 lines | 886 bytes

```
using System;

namespace SimCore
{
    public enum StepTargetStatus
    {
        Accepted,
```

```
        Missing,
        LateOrDuplicate,
        CountMismatch,
        Skipped,
    }

    public static class StepContract
    {
        public static StepTargetStatus Validate(
            long currentStep, int count, bool strict, bool hasTarget,
            long targetStep, out long expectedStep)
        {
            expectedStep = currentStep + Math.Max(1, count);
            if (strict && !hasTarget) return StepTargetStatus.Missing;
            if (!hasTarget || targetStep == expectedStep) return StepTargetStatus.Accepted;
            if (targetStep <= currentStep) return StepTargetStatus.LateOrDuplicate;
            if (targetStep < expectedStep) return StepTargetStatus.CountMismatch;
            return StepTargetStatus.Skipped;
        }
    }
}
```

5. External RPC and Protocol

Implements the MessagePack protocol and the ZeroMQ request and telemetry endpoints. The background server owns socket I/O, while the Unity-side adapter dispatches queued requests to the simulation and per-drone APIs on the main thread.

Files in this section

Runtime/Scripts/SimCore/Rpc/ExternalRpcAdapter.cs
Runtime/Scripts/SimCore/Rpc/RpcProtocol.cs
Runtime/Scripts/SimCore/Rpc/RpcServerThread.cs

Runtime/Scripts/SimCore/Rpc/ExternalRpcAdapter.cs

SHA-256 fe9ac3f5db45c2c02f7f433bb49fd5c3feb9d7fd86f18119041f5d6d31f640f6 | 1,964 lines | 84,196 bytes

```
// Assets/Scripts/SimCore/Rpc/ExternalRpcAdapter.cs
// Phase 9a + 9b + 9c: External RPC Adapter
//
// PURPOSE:
//   MonoBehaviour that lives on SimRoot. Bridges the background ZMQ server thread
//   to the Unity main thread where SimCoreApi and QuadSimApi calls must happen.
//
// RESPONSIBILITIES:
//   - Start/stop the RpcServerThread
//   - Drain incoming requests in Update() and dispatch to method handlers
//   - Manage External source lifecycle through SimCoreApi
//   - Track connection state and heartbeat timeout
//   - Publish telemetry on PUB socket at configurable rate (Phase 9c)
//
// PHASE 9a: connect / disconnect / heartbeat / get_status
// PHASE 9b: set_mode, send_command, get_sensor_data, get_telemetry,
//           set_controller, reset_all, reset_pose, reset_rotation,
//           reset_physics, reset_controller, pause, resume, step,
//           set_time_scale, reset_simulation
// PHASE 9c: subscribe / unsubscribe, PUB telemetry streaming

using System;
using System.Collections.Generic;
using UnityEngine;
using SimCore.Common;
using DroneCore;
using DroneCore.Api;
using DroneCore.CameraSystem;
using DroneCore.Core;
using MathUtil;
using RobotCore;
using NetMQ;
using QuadSim.RL;

namespace SimCore.Rpc
{
    /// <summary>
    /// Bridges External Python clients to the simulation via ZeroMQ.
    /// Attach to SimRoot alongside SimCoreApi.
    ///
    /// The adapter starts a background thread with REP + PUB sockets.
    /// Requests arrive on the background thread, get queued to main thread,
    /// processed against SimCoreApi/QuadSimApi, and responses are sent back.
    /// </summary>
    [DisallowMultipleComponent]
    public sealed class ExternalRpcAdapter : MonoBehaviour
    {
        // ================================================================================
        // Configuration
        // ================================================================================

        [Header("Server Settings")]
        [Tooltip("Port for the REQ/REP command socket.")]
        [SerializeField] private int commandPort = 5555;
```

```
        [Tooltip("Port for the PUB telemetry socket.")]
        [SerializeField] private int telemetryPort = 5556;

        [Tooltip("Address exposed by the RPC sockets. Use 0.0.0.0 only for an explicitly secured remote
deployment.")]
        [SerializeField] private string bindAddress = "127.0.0.1";

        [Tooltip("Start the RPC server automatically on Awake.")]
        [SerializeField] private bool autoStart = true;

        [Header("Connection Settings")]
        [Tooltip("Seconds without a heartbeat before considering the client disconnected.")]
        [SerializeField] private float heartbeatTimeoutSec = 5.0f;

        [Tooltip("Maximum requests to process per frame (prevents stalling).")]
        [SerializeField] private int maxRequestsPerFrame = 10;

        [Header("Telemetry Streaming (Phase 9c)")]
        [Tooltip("Default publish rate in Hz. Client can override via subscribe. 0 = disabled.")]
        [SerializeField] private float defaultPublishHz = 50f;

        // ================================================================================
        // Runtime State
        // ================================================================================

        private RpcServerThread _server;
        private SimCoreApi _sim;
        private bool _netMqCleanedUp;

        // Connection tracking
        private bool _clientConnected;
        private string _clientName;
        private float _lastHeartbeatTime;

        // Pending accept/deny (when External is not auto-accepted)
        private bool _pendingConnectionApproval;
        private QueuedRequest _pendingConnectRequest;

        // Method dispatch table
        private Dictionary<string, Func<RpcRequest, RpcResponse>> _handlers;

        // Telemetry streaming state (Phase 9c)
        private bool _streamingActive;
        private float _publishHz;
        private float _publishTimer;
        private HashSet<string> _subscribedTopics = new() { "sensors", "telemetry" };
        private long _rlEpisodeId = 0;
        private ulong _telemetrySequence;
        private ulong _imageSequence;
        private ulong _stateSequence;
        private ulong _eventSequence;

        private readonly struct PreparedMultiAction
        {
            public readonly QuadPawn Pawn;
            public readonly QuadSimApi Api;
            public readonly GoalMode Mode;
            public readonly Axis4 Command;

            public PreparedMultiAction(QuadPawn pawn, QuadSimApi api, GoalMode mode, Axis4 command)
            {
                Pawn = pawn;
                Api = api;
                Mode = mode;
                Command = command;
            }
        }

        private readonly struct PreparedEpisodeReset
        {
            public readonly QuadPawn Pawn;
            public readonly QuadSimApi Api;
```

```
    public readonly string AgentId;
    public readonly string TeamId;
    public readonly int DroneIndex;
    public readonly Vector3 Position;
    public readonly Quaternion Rotation;

    public PreparedEpisodeReset(QuadPawn pawn, QuadSimApi api, string agentId,
        string teamId, int droneIndex, Vector3 position, Quaternion rotation)
    {
        Pawn = pawn;
        Api = api;
        AgentId = agentId;
        TeamId = teamId;
        DroneIndex = droneIndex;
        Position = position;
        Rotation = rotation;
    }
}

// ============================================================================
// String ↔ Enum Maps (Phase 9b)
// ============================================================================

private static readonly Dictionary<string, GoalMode> GoalModeMap = new()
{
    ["none"]        = GoalMode.None,
    ["position"]    = GoalMode.Position,
    ["velocity"]    = GoalMode.Velocity,
    ["acceleration"] = GoalMode.Acceleration,
    ["accel"]       = GoalMode.Acceleration,
    ["angle"]       = GoalMode.Angle,
    ["rate"]        = GoalMode.Rate,
    ["passthrough"] = GoalMode.Passthrough,
    ["wrench"]      = GoalMode.Wrench,
    ["wrench_allocated"] = GoalMode.Wrench,
    ["wrench_bypassed"] = GoalMode.WrenchBypassed,
    ["wrench_bypass"] = GoalMode.WrenchBypassed,

};

private static readonly Dictionary<GoalMode, string> GoalModeReverseMap = new()
{
    [GoalMode.None]        = "none",
    [GoalMode.Position]    = "position",
    [GoalMode.Velocity]    = "velocity",
    [GoalMode.Acceleration] = "acceleration",
    [GoalMode.Angle]       = "angle",
    [GoalMode.Rate]        = "rate",
    [GoalMode.Passthrough] = "passthrough",
    [GoalMode.Wrench]      = "wrench",
    [GoalMode.WrenchBypassed] = "wrench_bypassed",
};

private static readonly Dictionary<string, ControllerKind> ControllerKindMap = new()
{
    ["cascade"]   = ControllerKind.Cascade,
    ["geometric"] = ControllerKind.Geometric,
};

private static readonly Dictionary<ControllerKind, string> ControllerKindReverseMap = new()
{
    [ControllerKind.Cascade]   = "cascade",
    [ControllerKind.Geometric] = "geometric",
};

// ============================================================================
// Public Accessors
// ============================================================================

/// <summary>True if the ZMQ server thread is running.</summary>
public bool IsServerRunning => _server?.IsRunning ?? false;

/// <summary>True if an External client is currently connected.</summary>
```

```
        public bool IsClientConnected => _clientConnected;

        /// <summary>Name of the connected client (from connect message), or null.</summary>
        public string ClientName => _clientName;

        private SimFrame _clientFrame = SimFrame.FLU;

        /// <summary>Command port number.</summary>
        public int CommandPort => commandPort;

        /// <summary>Telemetry port number.</summary>
        public int TelemetryPort => telemetryPort;

        public string BindAddress => bindAddress;

        /// <summary>Set ports before StartServer(). CLI overrides still win (applied in Start).</summar
y>
        public void ConfigurePorts(int cmd, int tel)
        {
            commandPort = cmd;
            telemetryPort = tel;
        }

        public void ConfigureEndpoint(string address, int cmd, int tel)
        {
            bindAddress = string.IsNullOrWhiteSpace(address) ? "127.0.0.1" : address.Trim();
            ConfigurePorts(cmd, tel);
        }
        public event Action<Dictionary<string, object>> OnVizPayload;

        // ================================================================================
        // Unity Lifecycle
        // ================================================================================

        private void Awake()
        {
            BuildHandlerTable();
        }

        private void Start()
        {
            ApplyCommandLineOverrides();
            _sim = SimCoreApi.Get();
            if (_sim == null)
            {
                Debug.LogError("[RpcAdapter] No SimCoreApi found. ExternalRpcAdapter requires SimCoreApi
 on SimRoot.");
                enabled = false;
                return;
            }

            // Subscribe to authority events for accept/deny flow
            _sim.OnAuthorityChanged += HandleAuthorityChanged;

            if (autoStart)
            {
                StartServer();
            }
        }

        private void Update()
        {
            if (_server == null) return;

            // Drain log messages from background thread
            DrainLogs();

            // Process incoming requests
            ProcessRequests();

            // Check heartbeat timeout
            CheckHeartbeatTimeout();
```

```
    // Publish telemetry stream (Phase 9c)
    PublishTick();
}

private void OnDestroy()
{
    ShutdownNetMq();

    if (_sim != null)
    {
        _sim.OnAuthorityChanged -= HandleAuthorityChanged;
    }
}

private void OnApplicationQuit()
{
    ShutdownNetMq();
}

// ============================================================================
// Server Lifecycle
// ============================================================================

/// <summary>Start the ZMQ server thread.</summary>
///
private void ApplyCommandLineOverrides()
{
    var args = Environment.GetCommandLineArgs();
    for (int i = 0; i < args.Length - 1; i++)
    {
        if (args[i] == "-rpcPort" && int.TryParse(args[i + 1], out int rp))
            commandPort = rp;
        if (args[i] == "-telemetryPort" && int.TryParse(args[i + 1], out int tp))
            telemetryPort = tp;
        if (args[i] == "-rpcBindAddress")
            bindAddress = args[i + 1];
    }
    if (string.IsNullOrWhiteSpace(bindAddress)) bindAddress = "127.0.0.1";
    if (bindAddress != "127.0.0.1" && bindAddress != "localhost" && bindAddress != "::1")
        Debug.LogWarning($"[RpcAdapter] RPC is remotely exposed on '{bindAddress}'.");
    Debug.Log($"[RpcAdapter] Endpoint: {bindAddress} REP={commandPort} PUB={telemetryPort}");
}
public void StartServer()
{
    // Non-null also covers the startup window before IsRunning becomes true.
    // Never replace a server whose background thread is still starting.
    if (_server != null)
    {
        Debug.LogWarning("[RpcAdapter] Server already starting or running.");
        return;
    }

    _netMqCleanedUp = false;
    _server = new RpcServerThread(commandPort, telemetryPort, bindAddress);
    _server.Start();

    Debug.Log($"[RpcAdapter] Starting RPC server — REP:{commandPort} PUB:{telemetryPort}");
}

/// <summary>Stop the ZMQ server thread and clean up.</summary>
public void StopServer()
{
    if (_server == null) return;

    // If a client was connected, notify disconnect.
    if (_clientConnected)
    {
        HandleClientDisconnect("server_shutdown");
    }

    // Dispose calls Stop(). RpcServerThread.Stop() also handles the
    // startup window before its sockets have finished binding.
    RpcServerThread server = _server;
```

```
    server.Dispose();

    if (server.IsThreadAlive)
    {
        // Keep the reference so StartServer cannot launch a second worker.
        // More importantly, ShutdownNetMq will not terminate the global
        // context while this thread may still own a socket.
        Debug.LogError("[RpcAdapter] RPC worker is still alive after shutdown timeout; " +
                       "skipping NetMQ cleanup to avoid a context race.");
        return;
    }

    _server = null;
    Debug.Log("[RpcAdapter] RPC server stopped.");
}

/// <summary>
/// NetMQ cleanup is process-global. Run it exactly once, and only after
/// the socket thread has fully stopped and joined. The sockets use zero
/// linger, so there are no pending messages that justify blocking cleanup.
/// In particular, Cleanup(true) can wait forever for NetMQ's reaper on
/// macOS and freeze Unity's main thread while exiting Play mode.
/// </summary>
private void ShutdownNetMq()
{
    StopServer();

    // StopServer deliberately retains _server when its worker failed to
    // join. Global cleanup would be unsafe while that thread owns sockets.
    if (_server != null) return;

    if (_netMqCleanedUp) return;
    _netMqCleanedUp = true;

    try
    {
        NetMQConfig.Cleanup(false);
        Debug.Log("[RpcAdapter] NetMQ cleanup completed.");
    }
    catch (Exception ex)
    {
        Debug.LogWarning($"[RpcAdapter] NetMQ cleanup warning: {ex.Message}");
    }
}

// ============================================================================
// Request Processing (Main Thread)
// ============================================================================

private void ProcessRequests()
{
    int processed = 0;

    while (processed < maxRequestsPerFrame && _server.TryDequeue(out var queued))
    {
        processed++;

        RpcResponse response;

        try
        {
            if (_handlers.TryGetValue(queued.Request.Method, out var handler))
            {
                // Any valid request from a connected client proves liveness —
                // refresh heartbeat so active clients don't time out.
                if (_clientConnected)
                {
                    _lastHeartbeatTime = Time.unscaledTime;
                }

                response = handler(queued.Request);
            }
            else
```

```
                {
                    response = RpcResponse.Error(
                        $"unknown_method: {queued.Request.Method}",
                        queued.Request.RequestId);
                }
            }
            catch (Exception ex)
            {
                Debug.LogError($"[RpcAdapter] Exception handling '{queued.Request.Method}': {ex}");
                response = RpcResponse.Error("internal_error", queued.Request.RequestId);
            }

            // Serialize and signal the background thread
            byte[] responseBytes = RpcSerializer.SerializeResponse(response);
            queued.SetResponse(responseBytes);
        }
    }

    private void DrainLogs()
    {
        while (_server.TryDequeueLog(out var level, out var message))
        {
            switch (level)
            {
                case RpcServerThread.LogLevel.Info:
                    Debug.Log(message);
                    break;
                case RpcServerThread.LogLevel.Warning:
                    Debug.LogWarning(message);
                    break;
                case RpcServerThread.LogLevel.Error:
                    Debug.LogError(message);
                    break;
            }
        }
    }

    // ================================================================================
    // Heartbeat Monitoring
    // ================================================================================

    private void CheckHeartbeatTimeout()
    {
        if (!_clientConnected) return;
        if (heartbeatTimeoutSec <= 0f) return;

        float elapsed = Time.unscaledTime - _lastHeartbeatTime;

        if (elapsed > heartbeatTimeoutSec)
        {
            Debug.LogWarning($"[RpcAdapter] Heartbeat timeout ({elapsed:F1}s > {heartbeatTimeoutSec}
s). " +
                             $"Disconnecting client '{_clientName}'.");
            HandleClientDisconnect("heartbeat_timeout");
        }
    }

    // ================================================================================
    // Handler Table
    // ================================================================================

    private void BuildHandlerTable()
    {
        _handlers = new Dictionary<string, Func<RpcRequest, RpcResponse>>
        {
            // --- Phase 9a: Connection lifecycle ---
            ["connect"]          = HandleConnect,
            ["disconnect"]       = HandleDisconnect,
            ["heartbeat"]        = HandleHeartbeat,
            ["get_status"]       = HandleGetStatus,
            ["list_drones"]      = HandleListDrones,
            ["set_frame"] = HandleSetFrame,
```

```
                // --- Phase 9b: Drone control ---
                ["set_mode"]         = HandleSetMode,
                ["send_command"]     = HandleSendCommand,
                ["get_sensor_data"]  = HandleGetSensorData,
                ["capture_images"]   = HandleCaptureImages,
                ["get_telemetry"]    = HandleGetTelemetry,
                ["set_controller"]   = HandleSetController,

                // --- Phase 9b: Reset operations ---
                ["reset_all"]        = HandleResetAll,
                ["reset_pose"]       = HandleResetPose,
                ["reset_rotation"]   = HandleResetRotation,
                ["reset_physics"]    = HandleResetPhysics,
                ["reset_controller"] = HandleResetController,

                // --- Phase 9b: Simulation control ---
                ["pause"]            = HandlePause,
                ["resume"]           = HandleResume,
                ["step"]             = HandleStep,
                ["step_with_command"] = HandleStepWithCommand,
                ["step_multi"]       = HandleStepMulti,
                ["reset_episode"]    = HandleResetEpisode,
                ["set_time_scale"]   = HandleSetTimeScale,
                ["reset_simulation"] = HandleResetSimulation,
                ["set_wind"]         = HandleSetWind,
                ["set_geo_origin"] = HandleSetGeoOrigin,
                ["update_viz"] = HandleUpdateViz,
                // --- Phase 9c: Telemetry streaming ---
                ["subscribe"]        = HandleSubscribe,
                ["unsubscribe"]      = HandleUnsubscribe,
            };
        }

        /// <summary>
        /// Register additional method handlers (for Phase 9c, 9d extensions).
        /// Call this before or after server start — handlers are on the main thread.
        /// </summary>
        public void RegisterHandler(string method, Func<RpcRequest, RpcResponse> handler)
        {
            _handlers[method] = handler;
        }

        // ============================================================================
        // Phase 9a Handlers: Connection Lifecycle
        // ============================================================================

        // --- connect ---

        private RpcResponse HandleConnect(RpcRequest req)
        {
            if (_clientConnected)
            {
                return RpcResponse.Error("already_connected", req.RequestId);
            }

            string clientName = req.GetString("client_name") ?? "unknown";

            if (req.TryGetInt("protocol_major", out int clientProtocolMajor)
                && clientProtocolMajor != RpcCapabilities.ProtocolMajor)
            {
                return RpcResponse.Error(
                    $"incompatible_protocol: server={RpcCapabilities.ProtocolMajor}, client={clientProto
colMajor}",
                    req.RequestId);
            }

            Debug.Log($"[RpcAdapter] Connection request from '{clientName}'");

            // Check if External is denied for this session
            if (_sim.IsExternalDenied)
            {
                Debug.Log($"[RpcAdapter] Rejecting '{clientName}' — External is denied for this session.
");
```

```
            return RpcResponse.Error("denied", req.RequestId);
        }

        // Notify the authority manager
        _sim.NotifyExternalConnectionAttempt();

        // Check if it was auto-accepted (External is default + autoAccept)
        var externalStatus = _sim.GetSourceStatus(InputSource.External);
        if (externalStatus == SourceStatus.Connected)
        {
            // Auto-accepted
            CompleteConnection(clientName);
            return RpcResponse.Ok(req.RequestId,
                RpcCapabilities.ConnectData(commandPort, telemetryPort, bindAddress));
        }

        if (externalStatus == SourceStatus.Denied)
        {
            return RpcResponse.Error("denied", req.RequestId);
        }


        _sim.AcceptExternalConnection();
        CompleteConnection(clientName);

        return RpcResponse.Ok(req.RequestId,
            RpcCapabilities.ConnectData(commandPort, telemetryPort, bindAddress));
    }

    private void CompleteConnection(string clientName)
    {
        _clientConnected = true;
        _clientName = clientName;
        _lastHeartbeatTime = Time.unscaledTime;

        Debug.Log($"[RpcAdapter] Client '{clientName}' connected. External source active.");
    }

    // --- disconnect ---

    private RpcResponse HandleDisconnect(RpcRequest req)
    {
        if (!_clientConnected)
        {
            return RpcResponse.Ok(req.RequestId); // Idempotent
        }

        Debug.Log($"[RpcAdapter] Client '{_clientName}' requested disconnect.");
        HandleClientDisconnect("client_request");

        return RpcResponse.Ok(req.RequestId);
    }

    // --- heartbeat ---

    private RpcResponse HandleHeartbeat(RpcRequest req)
    {
        if (!_clientConnected)
        {
            return RpcResponse.Error("not_connected", req.RequestId);
        }

        _lastHeartbeatTime = Time.unscaledTime;

        return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
        {
            ["sim_time"] = _sim.SimTimeSec,
            ["is_paused"] = _sim.IsPaused,
        });
    }


    private RpcResponse HandleSetFrame(RpcRequest req)
```

```
{
    if (!RequireConnected(req, out var err)) return err;

    string frameStr = req.GetString("frame");
    if (frameStr == null)
        return RpcResponse.Error("missing_param: frame", req.RequestId);

    switch (frameStr.ToLowerInvariant())
    {
        case "flu":
            _clientFrame = SimFrame.FLU;
            break;
        case "frd":
            _clientFrame = SimFrame.FRD;
            break;
        default:
            return RpcResponse.Error($"invalid_frame: {frameStr}", req.RequestId);
    }

    if (!TryGetSelectedDrone(req, out var drone, out var droneErr))
        return droneErr;

    if (!drone.SetSensorOutputFrame(_clientFrame))
        return RpcResponse.Error($"unsupported_sensor_frame: {frameStr}", req.RequestId);

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["frame"] = _clientFrame == SimFrame.FRD ? "frd" : "flu"
    });
}

// --- get_status ---

private RpcResponse HandleGetStatus(RpcRequest req)
{
    var status = _sim.GetStatus();

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["is_paused"] = status.IsPaused,
        ["time_scale"] = status.TimeScale,
        ["sim_time"] = status.SimTimeSec,
        ["sim_time_ns"] = _sim.SimTimeNanos,
        ["step_index"] = ClockFactory.Clock.StepCount,
        ["episode_id"] = _rlEpisodeId,
        ["fixed_dt"] = status.FixedDtSec,
        ["authority"] = status.CurrentAuthority.ToString(),
        ["ui_status"] = status.UIStatus.ToString(),
        ["internal_status"] = status.InternalStatus.ToString(),
        ["external_status"] = status.ExternalStatus.ToString(),
        ["client_connected"] = _clientConnected,
        ["client_name"] = _clientName ?? "",
    });
}

private RpcResponse HandleListDrones(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;

    DroneManager manager = DroneManager.Get();
    if (manager == null)
        return RpcResponse.Error("no_drone_manager", req.RequestId);

    var drones = manager.RegisteredDrones;
    object[] entries = new object[drones.Count];

    for (int i = 0; i < drones.Count; i++)
    {
        var drone = drones[i];
        entries[i] = new Dictionary<string, object>
        {
            ["agent_id"] = drone.AgentId,
            ["team_id"] = drone.TeamId,
```

```
            ["drone_index"] = drone.DroneIndex,
            ["drone_id"] = drone.DroneID,
            ["is_initialized"] = drone.IsInitialized,
        };
    }

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["drones"] = entries,
    });
}

// ============================================================================
// Phase 9b Handlers: Drone Control
// ============================================================================

// --- set_mode ---

private RpcResponse HandleSetMode(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;

    string modeStr = req.GetString("mode");
    if (modeStr == null)
    {
        return RpcResponse.Error("missing_param: mode", req.RequestId);
    }

    if (!GoalModeMap.TryGetValue(modeStr.ToLowerInvariant(), out var goalMode))
    {
        return RpcResponse.Error($"invalid_mode: {modeStr}", req.RequestId);
    }

    if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

    drone.SetMode(goalMode);

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["mode"] = GoalModeReverseMap[drone.CurrentMode],
    });
}

// --- send_command ---

private RpcResponse HandleSendCommand(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

    if (_sim.CurrentAuthority != InputSource.External)
    {
        return RpcResponse.Error("authority_rejected", req.RequestId);
    }

    // Extract raw values from client (in client's frame convention)
    float x = req.GetFloat("x");
    float y = req.GetFloat("y");
    float z = req.GetFloat("z");
    float w = req.GetFloat("w");

    // Optional mode override
    string modeStr = req.GetString("mode");
    GoalMode? goalMode = null;

    if (modeStr != null)
    {
        if (!GoalModeMap.TryGetValue(modeStr.ToLowerInvariant(), out var gm))
            return RpcResponse.Error($"invalid_mode: {modeStr}", req.RequestId);
        goalMode = gm;
    }

    // Determine the active mode (either from override or current)
```

```
        GoalMode activeMode = goalMode ?? drone.CurrentMode;

        // Remap client frame → internal Axis4 layout
        Axis4 axis4 = RemapClientCommand(x, y, z, w, activeMode, _clientFrame);

        bool success;
        if (goalMode.HasValue)
            success = drone.SendCommand(InputSource.External, axis4, goalMode.Value);
        else
            success = drone.SendCommand(InputSource.External, axis4);

        if (!success)
            return RpcResponse.Error("command_failed", req.RequestId);

        return RpcResponse.Ok(req.RequestId);
        }
    private static Axis4 RemapClientCommand(float x, float y, float z, float w,
                                            GoalMode mode, SimFrame clientFrame)
    {
        if (clientFrame == SimFrame.UnityBody)
        {
            // No remapping — client is sending raw Axis4 layout
            return new Axis4(x, y, z, w);
        }

        // For FLU/FRD clients, the meaning of (x,y,z,w) depends on mode
        switch (mode)
        {
            case GoalMode.Position:
                // Client sends: (X, Y, Z, Yaw_deg) in the client WORLD frame:
                //   clientFrame FLU → ENU (E, N, Up)   [the SDK default]
                //   clientFrame FRD → NED (N, E, Down)
                // Internal Axis4 layout: (X_world, Y_world, Yaw_deg, Alt_world)
                //
                // Convert position from client world frame → Unity world:
                Vector3 posClient = new Vector3(x, y, z);
                Vector3 posUnity = Frames.InverseTransformWorldPosition(posClient, clientFrame);
                // Map into Axis4 layout: (X, Z, Yaw, Y_alt)
                return new Axis4(posUnity.x, posUnity.z, w, posUnity.y);

            case GoalMode.Velocity:
                // Client sends: (Vx, Vy, Vz, YawRate_dps) in client body frame
                // Internal Axis4 layout: (Vx, Vy, YawRate, Vz)
                //
                // Convert velocity from client frame → Unity body:
                Vector3 velClient = new Vector3(x, y, z);
                Vector3 velUnity = Frames.InverseTransformLinear(velClient, clientFrame);
                // Map into Axis4 layout: (Vx, Vz, YawRate, Vy_alt)
                return new Axis4(velUnity.x, velUnity.z, w, velUnity.y);

            case GoalMode.Acceleration:
                // Client sends: (Ax, Ay, Az, YawRate_dps) in client body/output frame.
                // Internal Axis4 layout: (Ax, Ay, YawRate, Az).
                // Acceleration is a true vector, so it uses the same linear transform
                // as body-frame velocity.
                Vector3 accelClient = new Vector3(x, y, z);
                Vector3 accelUnity = Frames.InverseTransformLinear(accelClient, clientFrame);
                return new Axis4(accelUnity.x, accelUnity.z, w, accelUnity.y);

            case GoalMode.Angle:
            case GoalMode.Rate:
                // Client sends: (Roll, Pitch, YawRate, Throttle)
                // These are already in the correct layout — no reorder needed.
                // Frame signs for attitude commands would come from Frames.TransformAttitude
                // but for now we assume the client sends in the frame the controller expects.
                return new Axis4(x, y, z, w);

            case GoalMode.WrenchBypassed:
                // Client sends: (tx, ty, tz, thrust) — torques in client body frame
                // Thrust is a scalar along body vertical — no frame conversion needed.
                //
                // Torques are pseudovectors (same transform as angular velocity).
                // Use Frames.InverseTransformAngularVelocity to convert client→Unity,
```

```
                    // which handles both the axis swap AND chirality correction.
                    Vector3 torqueClient = new Vector3(x, y, z);
                    Vector3 torqueUnity = Frames.InverseTransformAngularVelocity(torqueClient, clientFra
me);
                    return new Axis4(torqueUnity.x, torqueUnity.y, torqueUnity.z, w);
                case GoalMode.Wrench:
                    // External wrench routed through the effectiveness-matrix
                    // allocator (built in the FLU internal frame). The FLU->Unity
                    // mapping happens implicitly inside the allocate->actuate round
                    // trip, so — unlike GoalMode.Wrench — do NOT convert here.
                    // Assumes clientFrame == allocator build frame (FLU). If you add
                    // FRD clients, convert torque client->FLU at this point.
                    return new Axis4(x, y, z, w);   // (Mx, My, Mz, f), FLU passthrough
                case GoalMode.Passthrough:
                case GoalMode.None:
                default:
                    // No transformation — motor values and None don't need remapping
                    return new Axis4(x, y, z, w);
            }
        }
        // --- get_sensor_data ---

        private RpcResponse HandleGetSensorData(RpcRequest req)
        {
            if (!RequireConnected(req, out var err)) return err;
            if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

            var s = drone.GetSensorData();
            return RpcResponse.Ok(req.RequestId, SensorDataToDict(s));
        }

        private RpcResponse HandleCaptureImages(RpcRequest req)
        {
            if (!RequireConnected(req, out var err)) return err;
            if (!req.TryGetArray("requests", out var rawRequests) || rawRequests.Count == 0)
                return RpcResponse.Error("missing_or_invalid_param: requests", req.RequestId);

            DroneManager manager = DroneManager.Get();
            if (manager == null)
                return RpcResponse.Error("no_drone_manager", req.RequestId);

            var prepared = new List<(QuadPawn pawn, string type, int width, int height,
                string encoding, int quality, float fov, float maxDepth)>(rawRequests.Count);

            for (int i = 0; i < rawRequests.Count; i++)
            {
                if (!RpcRequest.TryGetMap(rawRequests[i], out var requestMap))
                    return RpcResponse.Error($"invalid_image_request: {i}", req.RequestId);

                var image = new RpcRequest("capture_image", requestMap);
                string agentId = image.GetString("agent_id");
                QuadPawn pawn = manager.GetDroneByAgentId(agentId);
                if (pawn == null)
                    return RpcResponse.Error($"unknown_agent: {agentId ?? "missing"}", req.RequestId);

                string type = (image.GetString("image_type") ?? "rgb").ToLowerInvariant();
                string encoding = (image.GetString("encoding") ?? "raw").ToLowerInvariant();
                int width = image.GetInt("width", 64);
                int height = image.GetInt("height", 64);
                int quality = image.GetInt("quality", 90);
                float fov = image.GetFloat("field_of_view_deg", 90f);
                float maxDepth = image.GetFloat("max_depth_m", 100f);

                if (type != "rgb" && type != "gray" && type != "depth")
                    return RpcResponse.Error($"invalid_image_type: {type}", req.RequestId);
                if (encoding != "raw" && encoding != "png" && encoding != "jpeg")
                    return RpcResponse.Error($"invalid_image_encoding: {encoding}", req.RequestId);
                if (type == "depth" && encoding != "raw")
                    return RpcResponse.Error("depth_requires_raw_encoding", req.RequestId);
                if (width < 16 || width > 512 || height < 16 || height > 512)
                    return RpcResponse.Error("invalid_image_size: expected 16..512", req.RequestId);
                if (!IsFinite(fov) || fov < 10f || fov > 160f)
                    return RpcResponse.Error("invalid_field_of_view", req.RequestId);
```

```
            if (!IsFinite(maxDepth) || maxDepth <= 0f || maxDepth > 1000f)
                return RpcResponse.Error("invalid_max_depth", req.RequestId);

            prepared.Add((pawn, type, width, height, encoding,
                Mathf.Clamp(quality, 1, 100), fov, maxDepth));
        }

        object[] frames = new object[prepared.Count];
        for (int i = 0; i < prepared.Count; i++)
        {
            var capture = prepared[i];
            DroneImageSensor sensor = capture.pawn.ImageSensor;
            frames[i] = sensor.Capture(
                capture.type, capture.width, capture.height, capture.encoding,
                capture.quality, capture.fov, capture.maxDepth);
        }

        return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
        {
            ["episode_id"] = _rlEpisodeId,
            ["step_index"] = ClockFactory.Clock.StepCount,
            ["sim_time_ns"] = _sim.SimTimeNanos,
            ["sim_time"] = _sim.SimTimeSec,
            ["sequence"] = _imageSequence++,
            ["source_frame"] = "camera_optical/body_flu/world_enu",
            ["frames"] = frames,
        });
    }

    // --- get_telemetry ---

    private RpcResponse HandleGetTelemetry(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;
        if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

        var t = drone.GetTelemetry();
        return RpcResponse.Ok(req.RequestId, TelemetryToDict(t, _telemetrySequence++));
    }

    // --- set_controller ---

    private RpcResponse HandleSetController(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;

        string kindStr = req.GetString("controller");
        if (kindStr == null)
        {
            return RpcResponse.Error("missing_param: controller", req.RequestId);
        }

        if (!ControllerKindMap.TryGetValue(kindStr.ToLowerInvariant(), out var kind))
        {
            return RpcResponse.Error($"invalid_controller: {kindStr}", req.RequestId);
        }

        if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

        drone.SetController(kind);

        return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
        {
            ["controller"] = ControllerKindReverseMap[drone.CurrentController],
        });
    }

    // ============================================================================
    // Environment Handlers: Plant / Disturbance
    // ============================================================================

    // --- set_wind ---
    // Toggles (and optionally re-speeds) every WindModule in the scene. Wind is
```

```
// plant configuration, not vehicle control, so it requires a connected client
// but NOT External drone authority — same tier as pause / set_time_scale.
private RpcResponse HandleSetWind(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;

    bool enabled = req.GetBool("enabled", true);

    // wind_speed is optional — only overridden if the client sent it.
    bool hasSpeed = req.Params.ContainsKey("wind_speed");
    float windSpeed = req.GetFloat("wind_speed", 0f);

    int n = 0;
    foreach (var w in Experimental.WindModule.Active)
    {
        if (w == null) continue;
        w.SetEnabled(enabled);
        if (hasSpeed) w.SetWindSpeed(windSpeed);
        n++;
    }

    if (n == 0)
        Debug.LogWarning("[RpcAdapter] set_wind: no WindModule in scene — nothing toggled.");
    else
        Debug.Log($"[RpcAdapter] set_wind: enabled={enabled}" +
                  (hasSpeed ? $", wind_speed={windSpeed}" : "") + $" -> {n} module(s).");

    var data = new Dictionary<string, object> { ["enabled"] = enabled, ["modules"] = n };
    if (hasSpeed) data["wind_speed"] = windSpeed;
    return RpcResponse.Ok(req.RequestId, data);
}


private RpcResponse HandleSetGeoOrigin(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    float lat = req.GetFloat("lat", 30.4636f);
    float lon = req.GetFloat("lon", -86.5533f);
    float alt = req.GetFloat("alt", 0.0f);

    _sim.Geo.SetOrigin(lat, lon, alt);

    return RpcResponse.Ok(req.RequestId);
}

// ============================================================================
// Phase 9b Handlers: Reset Operations
// ============================================================================

// --- reset_all ---

private RpcResponse HandleResetAll(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

    drone.ResetAll();

    return RpcResponse.Ok(req.RequestId);
}

// --- reset_pose ---

private RpcResponse HandleResetPose(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

    // Read client-frame position
    float px = req.GetFloat("x", 0f);
    float py = req.GetFloat("y", 0f);
    float pz = req.GetFloat("z", 0f);
```

```
        // Read client-frame quaternion (defaults to identity)
        float qx = req.GetFloat("qx", 0f);
        float qy = req.GetFloat("qy", 0f);
        float qz = req.GetFloat("qz", 0f);
        float qw = req.GetFloat("qw", 1f);

        // Convert from client WORLD frame → Unity.
        // clientFrame FLU → position interpreted as ENU (SDK default);
        // clientFrame FRD → interpreted as NED.
        Vector3 position = Frames.InverseTransformWorldPosition(
            new Vector3(px, py, pz), _clientFrame);
        Quaternion rotation = Frames.InverseTransformQuaternion(
            new Quaternion(qx, qy, qz, qw), _clientFrame);

        drone.ResetPose(position, rotation);

        return RpcResponse.Ok(req.RequestId);
    }

    // --- reset_rotation ---

    private RpcResponse HandleResetRotation(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;
        if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

        drone.ResetRotation();

        return RpcResponse.Ok(req.RequestId);
    }

    // --- reset_physics ---

    private RpcResponse HandleResetPhysics(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;
        if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

        drone.ResetPhysics();

        return RpcResponse.Ok(req.RequestId);
    }

    // --- reset_controller ---

    private RpcResponse HandleResetController(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;
        if (!TryGetSelectedDrone(req, out var drone, out var droneErr)) return droneErr;

        drone.ResetController();

        return RpcResponse.Ok(req.RequestId);
    }

    // ========================================================================
    // Phase 9b Handlers: Simulation Control
    // ========================================================================

    // --- pause ---

    private RpcResponse HandlePause(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;

        _sim.Pause();

        return RpcResponse.Ok(req.RequestId);
    }

    // --- resume ---

    private RpcResponse HandleResume(RpcRequest req)
```

```
{
    if (!RequireConnected(req, out var err)) return err;

    _sim.Resume();

    return RpcResponse.Ok(req.RequestId);
}

// --- step ---

private RpcResponse HandleStep(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    if (!ValidateTargetStep(req, out err)) return err;

    int count = req.GetInt("count", 1);
    if (count <= 0)
        return RpcResponse.Error("invalid_param: count must be > 0", req.RequestId);

    if (_sim == null || _sim.SimManager == null)
        return RpcResponse.Error("simulation_manager_missing", req.RequestId);

    _sim.SimManager.StepImmediate(count);

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["sim_time"] = _sim.SimTimeSec,
        ["sim_time_ns"] = _sim.SimTimeNanos,
        ["step_index"] = ClockFactory.Clock.StepCount,
        ["steps_ran"] = count,
    });
}

private int _px4ProfileCount;
private double _px4CmdMs;
private double _px4StepMs;
private double _px4VizMs;
private double _px4SensorMs;
private double _px4TotalMs;

private static double TicksToMs(long ticks)
{
    return ticks * 1000.0 / System.Diagnostics.Stopwatch.Frequency;
}

private RpcResponse HandleStepWithCommand(RpcRequest req)
{
    if (!RequireConnected(req, out RpcResponse connectionError)) return connectionError;
    if (!ValidateTargetStep(req, out RpcResponse targetError)) return targetError;
    if (req.GetInt("count", 1) <= 0)
        return RpcResponse.Error("invalid_param: count must be > 0", req.RequestId);
    var cmdResp = HandleSendCommand(req);
    if (!cmdResp.Success) return cmdResp;

    var stepResp = HandleStep(req);
    if (!stepResp.Success) return stepResp;

    return HandleGetSensorData(req);
}

private RpcResponse HandleStepMulti(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;
    if (!ValidateTargetStep(req, out err)) return err;
    if (_sim.CurrentAuthority != InputSource.External)
        return RpcResponse.Error("authority_rejected", req.RequestId);
    if (_sim.SimManager == null)
        return RpcResponse.Error("simulation_manager_missing", req.RequestId);

    int count = req.GetInt("count", 1);
    if (count <= 0)
        return RpcResponse.Error("invalid_param: count must be > 0", req.RequestId);
    if (!req.TryGetArray("actions", out var rawActions))
```

```
                return RpcResponse.Error("missing_or_invalid_param: actions", req.RequestId);

            DroneManager manager = DroneManager.Get();
            if (manager == null)
                return RpcResponse.Error("no_drone_manager", req.RequestId);
            if (rawActions.Count != manager.DroneCount)
                return RpcResponse.Error("action_count_mismatch", req.RequestId);

            var prepared = new List<PreparedMultiAction>(rawActions.Count);
            var seenAgents = new HashSet<string>(StringComparer.Ordinal);

            for (int i = 0; i < rawActions.Count; i++)
            {
                if (!RpcRequest.TryGetMap(rawActions[i], out var actionMap))
                    return RpcResponse.Error($"invalid_action: {i}", req.RequestId);

                var action = new RpcRequest("step_multi_action", actionMap);
                string agentId = action.GetString("agent_id");
                if (string.IsNullOrEmpty(agentId))
                    return RpcResponse.Error($"missing_agent_id: {i}", req.RequestId);
                if (!seenAgents.Add(agentId))
                    return RpcResponse.Error($"duplicate_agent: {agentId}", req.RequestId);

                QuadPawn pawn = manager.GetDroneByAgentId(agentId);
                if (pawn == null)
                    return RpcResponse.Error($"unknown_agent: {agentId}", req.RequestId);
                QuadSimApi api = pawn.Api;
                if (api == null || !api.IsInitialized)
                    return RpcResponse.Error($"agent_not_initialized: {agentId}", req.RequestId);

                string modeName = action.GetString("mode");
                if (modeName == null || !GoalModeMap.TryGetValue(modeName.ToLowerInvariant(), out var mo
de))
                    return RpcResponse.Error($"invalid_mode: {modeName ?? "missing"}", req.RequestId);

                if (!action.TryGetFloat("x", out float x)
                    || !action.TryGetFloat("y", out float y)
                    || !action.TryGetFloat("z", out float z)
                    || !action.TryGetFloat("w", out float w))
                {
                    return RpcResponse.Error($"invalid_action_dimensions: {agentId}", req.RequestId);
                }
                if (!IsFinite(x) || !IsFinite(y) || !IsFinite(z) || !IsFinite(w))
                    return RpcResponse.Error($"non_finite_action: {agentId}", req.RequestId);

                Axis4 command = RemapClientCommand(x, y, z, w, mode, _clientFrame);
                prepared.Add(new PreparedMultiAction(pawn, api, mode, command));
            }

            for (int i = 0; i < manager.RegisteredDrones.Count; i++)
            {
                string requiredAgent = manager.RegisteredDrones[i].AgentId;
                if (!seenAgents.Contains(requiredAgent))
                    return RpcResponse.Error($"missing_action: {requiredAgent}", req.RequestId);
            }

            prepared.Sort((a, b) => a.Pawn.DroneIndex.CompareTo(b.Pawn.DroneIndex));
            for (int i = 0; i < prepared.Count; i++)
            {
                PreparedMultiAction action = prepared[i];
                if (!action.Api.SendCommand(InputSource.External, action.Command, action.Mode))
                    return RpcResponse.Error($"command_failed: {action.Pawn.AgentId}", req.RequestId);
            }

            _sim.SimManager.StepImmediate(count);

            Dictionary<string, object> states = CaptureRLStates(
                manager, _sim.SimTimeSec, out object[] events);

            return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
            {
                ["episode_id"] = _rlEpisodeId,
                ["step_index"] = ClockFactory.Clock.StepCount,
```

```
            ["sim_time_ns"] = _sim.SimTimeNanos,
            ["sim_time"] = _sim.SimTimeSec,
            ["steps_ran"] = count,
            ["agents"] = states,
            ["events"] = events,
        });
    }

    private RpcResponse HandleResetEpisode(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;
        if (_sim.CurrentAuthority != InputSource.External)
            return RpcResponse.Error("authority_rejected", req.RequestId);
        if (_sim.SimManager == null)
            return RpcResponse.Error("simulation_manager_missing", req.RequestId);
        if (!req.TryGetInt("seed", out int seed))
            return RpcResponse.Error("missing_or_invalid_param: seed", req.RequestId);

        int settleSteps = req.GetInt("settle_steps", 2);
        if (settleSteps < 0)
            return RpcResponse.Error("invalid_param: settle_steps must be >= 0", req.RequestId);
        if (!req.TryGetArray("agents", out var rawAgents))
            return RpcResponse.Error("missing_or_invalid_param: agents", req.RequestId);

        DroneManager manager = DroneManager.Get();
        if (manager == null)
            return RpcResponse.Error("no_drone_manager", req.RequestId);
        if (rawAgents.Count != manager.DroneCount)
            return RpcResponse.Error("agent_count_mismatch", req.RequestId);

        var prepared = new List<PreparedEpisodeReset>(rawAgents.Count);
        var seenAgentIds = new HashSet<string>(StringComparer.Ordinal);
        var seenIndices = new HashSet<int>();

        for (int i = 0; i < rawAgents.Count; i++)
        {
            if (!RpcRequest.TryGetMap(rawAgents[i], out var agentMap))
                return RpcResponse.Error($"invalid_agent: {i}", req.RequestId);

            var agent = new RpcRequest("reset_episode_agent", agentMap);
            string agentId = agent.GetString("agent_id");
            string teamId = agent.GetString("team_id");
            if (string.IsNullOrWhiteSpace(agentId))
                return RpcResponse.Error($"missing_agent_id: {i}", req.RequestId);
            if (string.IsNullOrWhiteSpace(teamId))
                return RpcResponse.Error($"missing_team_id: {i}", req.RequestId);
            if (!agent.TryGetInt("drone_index", out int droneIndex) || droneIndex < 0)
                return RpcResponse.Error($"invalid_drone_index: {i}", req.RequestId);
            if (!seenAgentIds.Add(agentId))
                return RpcResponse.Error($"duplicate_agent: {agentId}", req.RequestId);
            if (!seenIndices.Add(droneIndex))
                return RpcResponse.Error($"duplicate_drone_index: {droneIndex}", req.RequestId);

            QuadPawn pawn = manager.GetDroneByPersistentIndex(droneIndex);
            if (pawn == null)
                return RpcResponse.Error($"unknown_drone_index: {droneIndex}", req.RequestId);
            QuadSimApi api = pawn.Api;
            if (api == null || !api.IsInitialized)
                return RpcResponse.Error($"agent_not_initialized: {agentId}", req.RequestId);

            if (!TryGetVector3(agent, "position_enu", out Vector3 positionEnu))
                return RpcResponse.Error($"invalid_position_enu: {agentId}", req.RequestId);
            if (!agent.TryGetFloat("yaw_deg", out float yawDeg) || !IsFinite(yawDeg))
                return RpcResponse.Error($"invalid_yaw_deg: {agentId}", req.RequestId);

            Vector3 position = Frames.InverseTransformWorldPosition(positionEnu, SimFrame.FLU);
            Quaternion rotationEnu = Quaternion.AngleAxis(yawDeg, Vector3.forward);
            Quaternion rotation = Frames.InverseTransformQuaternion(rotationEnu, SimFrame.FLU);
            prepared.Add(new PreparedEpisodeReset(
                pawn, api, agentId, teamId, droneIndex, position, rotation));
        }

        var preparedObjects = new List<ScenarioObjectSpec>();
```

```
var seenObjectIds = new HashSet<string>(StringComparer.Ordinal);
if (req.Params.ContainsKey("objects"))
{
    if (!req.TryGetArray("objects", out var rawObjects))
        return RpcResponse.Error("missing_or_invalid_param: objects", req.RequestId);

    for (int i = 0; i < rawObjects.Count; i++)
    {
        if (!RpcRequest.TryGetMap(rawObjects[i], out var objectMap))
            return RpcResponse.Error($"invalid_object: {i}", req.RequestId);

        var sceneObject = new RpcRequest("reset_episode_object", objectMap);
        string objectId = sceneObject.GetString("object_id");
        string shape = sceneObject.GetString("shape")?.ToLowerInvariant();
        if (string.IsNullOrWhiteSpace(objectId))
            return RpcResponse.Error($"missing_object_id: {i}", req.RequestId);
        if (!seenObjectIds.Add(objectId))
            return RpcResponse.Error($"duplicate_object: {objectId}", req.RequestId);
        if (shape != "box" && shape != "sphere"
            && shape != "cylinder" && shape != "goal")
        {
            return RpcResponse.Error($"invalid_shape: {objectId}", req.RequestId);
        }
        if (!TryGetVector3(sceneObject, "position_enu", out Vector3 positionEnu))
            return RpcResponse.Error($"invalid_position_enu: {objectId}", req.RequestId);
        if (!TryGetVector3(sceneObject, "size_enu", out Vector3 sizeEnu)
            || sizeEnu.x <= 0f || sizeEnu.y <= 0f || sizeEnu.z <= 0f)
        {
            return RpcResponse.Error($"invalid_size_enu: {objectId}", req.RequestId);
        }

        float yawDeg = sceneObject.GetFloat("yaw_deg", 0f);
        if (!IsFinite(yawDeg))
            return RpcResponse.Error($"invalid_yaw_deg: {objectId}", req.RequestId);
        if (objectMap.TryGetValue("collision_enabled", out object collisionValue)
            && collisionValue is not bool)
        {
            return RpcResponse.Error(
                $"invalid_collision_enabled: {objectId}", req.RequestId);
        }

        bool collisionEnabled = sceneObject.GetBool(
            "collision_enabled", shape != "goal");
        Color defaultColor = shape == "goal"
            ? new Color(0.2f, 1f, 0.25f, 0.7f)
            : new Color(0.55f, 0.55f, 0.6f, 1f);
        if (!TryGetColor(sceneObject, "color_rgba", defaultColor, out Color color))
            return RpcResponse.Error($"invalid_color_rgba: {objectId}", req.RequestId);

        preparedObjects.Add(new ScenarioObjectSpec(
            objectId, shape, positionEnu, yawDeg, sizeEnu,
            collisionEnabled, color));
    }
}

prepared.Sort((a, b) => a.DroneIndex.CompareTo(b.DroneIndex));
preparedObjects.Sort((a, b) =>
    string.CompareOrdinal(a.ObjectId, b.ObjectId));

_sim.Pause();
_sim.SimManager.ResetClock();
_rlEpisodeId++;
_telemetrySequence = 0;
_imageSequence = 0;
_stateSequence = 0;
_eventSequence = 0;

ScenarioObjectManager objectManager =
    ScenarioObjectManager.GetOrCreate(gameObject);
objectManager.Apply(preparedObjects);

for (int i = 0; i < prepared.Count; i++)
{
```

```
            PreparedEpisodeReset reset = prepared[i];
            reset.Pawn.SetEpisodeIdentity(reset.AgentId, reset.TeamId, reset.DroneIndex);
        }
        manager.RefreshIdentityOrder();

        for (int i = 0; i < prepared.Count; i++)
        {
            PreparedEpisodeReset reset = prepared[i];
            reset.Api.ResetEpisodeState(
                reset.Position, reset.Rotation, _rlEpisodeId, seed, reset.DroneIndex);
        }
        Physics.SyncTransforms();

        if (settleSteps > 0)
            _sim.SimManager.StepImmediate(settleSteps);

        return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
        {
            ["episode_id"] = _rlEpisodeId,
            ["step_index"] = ClockFactory.Clock.StepCount,
            ["sim_time_ns"] = _sim.SimTimeNanos,
            ["sim_time"] = _sim.SimTimeSec,
            ["seed"] = seed,
            ["settle_steps"] = settleSteps,
            ["agents"] = CaptureRLStates(manager, _sim.SimTimeSec, out object[] events),
            ["events"] = events,
            ["objects"] = objectManager.Snapshot(),
        });
    }

    private RpcResponse HandleUpdateViz(RpcRequest req)
    {
        OnVizPayload?.Invoke(req.Params);
        return RpcResponse.Ok(req.RequestId);
    }

    // --- set_time_scale ---

    private RpcResponse HandleSetTimeScale(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;

        float scale = req.GetFloat("scale", 1.0f);

        if (scale <= 0f)
        {
            return RpcResponse.Error("invalid_param: scale must be > 0", req.RequestId);
        }

        _sim.SetTimeScale(scale);

        return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
        {
            ["time_scale"] = _sim.TimeScale,
        });
    }

    // --- reset_simulation ---

    private RpcResponse HandleResetSimulation(RpcRequest req)
    {
        if (!RequireConnected(req, out var err)) return err;

        _sim.ResetSimulation();

        return RpcResponse.Ok(req.RequestId);
    }

    // ================================================================================
    // Phase 9c Handlers: Telemetry Streaming
    // ================================================================================

    // --- subscribe ---
```

```
        private static Dictionary<string, object> SensorDataToDict(SensorData s) => new Dictionary<strin
g, object>
        {
            ["episode_id"]     = s.EpisodeId,
            ["step_index"]     = s.StepIndex,
            ["sim_time_ns"]    = s.SimTimeNanos,
            ["sim_time"]       = s.SimTimeNanos * 1e-9,
            ["sequence"]       = s.Sequence,
            ["source_frame"]   = s.SourceFrame ?? "body_flu/world_enu",
            // Body-axis convention of the imu_*/mag_* fields ("flu" or "frd").
            // World-frame fields are unaffected: gps_position is always ENU,
            // gps_vel_ned is always NED, gps_lat/lon/alt is WGS84.
            ["frame"]           = s.BodyFrame == SimFrame.FRD ? "frd" : "flu",
            ["imu_ang_vel"]     = Vec3ToArray(s.ImuAngVel),
            ["imu_attitude"]    = Vec3ToArray(s.ImuAttitude),
            ["imu_accel"]       = Vec3ToArray(s.ImuAccel),
            ["imu_vel"]         = Vec3ToArray(s.ImuVel),
            ["imu_orientation"] = QuatToArray(s.ImuOrientation),
            ["imu_timestamp"]   = s.ImuTimestampSec,
            ["imu_valid"]       = s.ImuValid,
            ["gps_position"]    = Vec3ToArray(s.GpsPosition),
            ["gps_timestamp"]   = s.GpsTimestampSec,
            ["gps_valid"]       = s.GpsValid,
            ["mag_field"]            = Vec3ToArray(s.MagField),
            ["mag_heading_deg"]      = s.MagHeadingDeg,
            ["mag_declination_deg"]  = s.MagDeclinationDeg,
            ["mag_timestamp"]        = s.MagTimestampSec,
            ["mag_valid"]            = s.MagValid,
            ["baro_pressure_pa"]       = s.BaroPressurePa,
            ["baro_pressure_hpa"]      = s.BaroPressureHPa,
            ["baro_temperature_c"]     = s.BaroTemperatureC,
            ["baro_altitude_msl"]      = s.BaroAltitudeMSL,
            ["baro_pressure_altitude"] = s.BaroPressureAltitude,
            ["baro_timestamp"]         = s.BaroTimestampSec,
            ["baro_valid"]             = s.BaroValid,
            ["gps_lat"]     = s.GpsLatDeg,
            ["gps_lon"]     = s.GpsLonDeg,
            ["gps_alt"]     = s.GpsAltM,
            ["gps_vel_ned"] = Vec3ToArray(s.GpsVelNED),
        };

        private Dictionary<string, object> TelemetryToDict(DroneTelemetry t, ulong sequence)
        {
            var allocation = t.Allocation;
            return new Dictionary<string, object>
            {
                ["episode_id"] = _rlEpisodeId,
                ["step_index"] = ClockFactory.Clock.StepCount,
                ["sim_time_ns"] = _sim.SimTimeNanos,
                ["sim_time"] = _sim.SimTimeSec,
                ["sequence"] = sequence,
                ["source_frame"] = "body_unity",
                ["drone_id"] = t.DroneID ?? "",
                ["mode"] = GoalModeReverseMap.GetValueOrDefault(t.Mode, "unknown"),
                ["controller"] = ControllerKindReverseMap.GetValueOrDefault(t.Controller, "unknown"),
                ["motors"] = new object[] { t.MotorOutputs.FL, t.MotorOutputs.FR,
                    t.MotorOutputs.BL, t.MotorOutputs.BR },
                ["motor_thrusts"] = new object[] { t.MotorThrusts.FL, t.MotorThrusts.FR,
                    t.MotorThrusts.BL, t.MotorThrusts.BR },
                ["desired_rates_deg"] = Vec3ToArray(t.ControllerTelemetry.DesiredRatesDeg),
                ["desired_angles_deg"] = Vec3ToArray(t.ControllerTelemetry.DesiredAnglesDeg),
                ["desired_vel"] = Vec3ToArray(t.ControllerTelemetry.DesiredVel),
                ["desired_accel"] = Vec3ToArray(t.ControllerTelemetry.DesiredAccel),
                ["external_cmd"] = new object[] { t.ControllerTelemetry.ExternalCmd.X,
                    t.ControllerTelemetry.ExternalCmd.Y,
                    t.ControllerTelemetry.ExternalCmd.Z,
                    t.ControllerTelemetry.ExternalCmd.W },
                ["achieved_force_body"] = Vec3ToArray(t.AchievedForceBody),
                ["achieved_torque_body"] = Vec3ToArray(t.AchievedTorqueBody),
                ["allocator_rank"] = t.AllocatorRank,
                ["allocator_condition_number"] = t.AllocatorConditionNumber,
                ["allocation_requested_flu"] = allocation == null
```

```
            ? Vec4ToArray(Vector4.zero) : Vec4ToArray(allocation.Requested),
        ["allocation_achieved_flu"] = allocation == null
            ? Vec4ToArray(Vector4.zero) : Vec4ToArray(allocation.Achieved),
        ["allocation_residual"] = allocation?.Residual ?? 0f,
        ["roll_pitch_scale"] = allocation?.RollPitchScale ?? 1f,
        ["yaw_scale"] = allocation?.YawScale ?? 1f,
        ["saturated_motors"] = allocation?.SaturatedMotors ?? Array.Empty<bool>(),
    };
}


private RpcResponse HandleSubscribe(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;

    float hz = req.GetFloat("hz", defaultPublishHz);
    if (hz <= 0f) hz = defaultPublishHz;
    if (hz > 200f) hz = 200f; // Cap at 200 Hz to prevent flooding

    _publishHz = hz;

    // Parse optional topic list — default to both
    _subscribedTopics.Clear();
    string topicsStr = req.GetString("topics");
    if (topicsStr != null)
    {
        // Accept comma-separated or single topic
        foreach (var t in topicsStr.Split(','))
        {
            string trimmed = t.Trim().ToLowerInvariant();
            if (trimmed == "sensors" || trimmed == "telemetry")
            {
                _subscribedTopics.Add(trimmed);
            }
        }
    }

    // If no valid topics parsed, default to both
    if (_subscribedTopics.Count == 0)
    {
        _subscribedTopics.Add("sensors");
        _subscribedTopics.Add("telemetry");
    }

    _streamingActive = true;
    _publishTimer = 0f;

    Debug.Log($"[RpcAdapter] Streaming started: {_publishHz} Hz, " +
              $"topics=[{string.Join(", ", _subscribedTopics)}]");

    return RpcResponse.Ok(req.RequestId, new Dictionary<string, object>
    {
        ["telemetry_port"] = telemetryPort,
        ["hz"] = _publishHz,
        ["topics"] = new List<object>(_subscribedTopics),
    });
}

// --- unsubscribe ---

private RpcResponse HandleUnsubscribe(RpcRequest req)
{
    if (!RequireConnected(req, out var err)) return err;

    StopStreaming();

    return RpcResponse.Ok(req.RequestId);
}

// ============================================================================
// Telemetry Publishing (Phase 9c)
// ============================================================================
```

```
private void PublishTick()
{
    if (!_streamingActive || !_clientConnected) return;
    if (_publishHz <= 0f) return;

    _publishTimer += Time.deltaTime;
    float interval = 1f / _publishHz;

    if (_publishTimer < interval) return;
    _publishTimer -= interval;

    // Clamp to prevent spiral if frame rate drops below publish rate
    if (_publishTimer > interval) _publishTimer = 0f;

    PublishTelemetry();
}

private void PublishTelemetry()
{
    var drone = QuadSimApi.Selected();
    if (drone == null) return;

    if (_subscribedTopics.Contains("sensors"))
    {
        var s = drone.GetSensorData();
        byte[] frame = RpcSerializer.SerializePublish("sensors", SensorDataToDict(s));
        _server.EnqueuePublish(frame);
    }

    if (_subscribedTopics.Contains("telemetry"))
    {
        var t = drone.GetTelemetry();
        var telemData = TelemetryToDict(t, _telemetrySequence++);

        byte[] frame = RpcSerializer.SerializePublish("telemetry", telemData);
        _server.EnqueuePublish(frame);
    }
}

private void StopStreaming()
{
    if (_streamingActive)
    {
        Debug.Log("[RpcAdapter] Streaming stopped.");
    }

    _streamingActive = false;
    _publishTimer = 0f;
}

// ============================================================================
// Disconnect Handling
// ============================================================================

private void HandleClientDisconnect(string reason)
{
    string name = _clientName ?? "unknown";

    // Stop streaming on disconnect
    StopStreaming();

    _clientConnected = false;
    _clientName = null;
    _lastHeartbeatTime = 0f;

    // Notify authority manager
    if (_sim != null)
    {
        _sim.NotifyExternalDisconnected();
    }

    Debug.Log($"[RpcAdapter] Client '{name}' disconnected. Reason: {reason}");
}
```

```
// ============================================================================
// Event Handlers
// ============================================================================

private void HandleAuthorityChanged(InputSource oldAuth, InputSource newAuth)
{
    // If authority moved away from External while a client is connected,
    // the client's commands will be rejected by the proxy. We don't
    // disconnect them — they can still read telemetry and get_status.
    if (_clientConnected && oldAuth == InputSource.External && newAuth != InputSource.External)
    {
        Debug.Log($"[RpcAdapter] Authority changed from External to {newAuth}. " +
                  "Client remains connected but commands will be rejected.");
    }
}

// ============================================================================
// Shared Helpers (Phase 9b)
// ============================================================================

/// <summary>
/// Guard: require the client to be connected. Returns false and sets the
/// error response if not connected.
/// </summary>
private bool RequireConnected(RpcRequest req, out RpcResponse errorResponse)
{
    if (_clientConnected)
    {
        errorResponse = default;
        return true;
    }

    errorResponse = RpcResponse.Error("not_connected", req.RequestId);
    return false;
}

private bool ValidateTargetStep(RpcRequest req, out RpcResponse errorResponse)
{
    bool strict = req.GetBool("strict_lockstep", false);
    if (strict && !_sim.IsPaused)
    {
        errorResponse = RpcResponse.Error(
            "strict_lockstep_requires_pause", req.RequestId);
        return false;
    }
    bool hasTarget = req.TryGetLong("target_step", out long targetStep);
    StepTargetStatus status = StepContract.Validate(
        (long)ClockFactory.Clock.StepCount,
        req.GetInt("count", 1), strict, hasTarget, targetStep,
        out long expected);
    if (status == StepTargetStatus.Accepted)
    {
        errorResponse = default;
        return true;
    }

    string reason = status switch
    {
        StepTargetStatus.Missing => "missing_param: target_step",
        StepTargetStatus.LateOrDuplicate => "late_or_duplicate_step",
        StepTargetStatus.CountMismatch => "target_step_count_mismatch",
        _ => "skipped_step",
    };
    if (status == StepTargetStatus.Missing)
    {
        errorResponse = RpcResponse.Error(reason, req.RequestId);
        return false;
    }
    errorResponse = RpcResponse.Error(
        $"{reason}: expected {expected}, got {targetStep}", req.RequestId);
    return false;
}
```

```
        /// <summary>
        /// Guard: resolve the selected drone via QuadSimApi.Selected().
        /// Returns false and sets the error response if no drone is available.
        /// Phase 9b is single-drone — multi-drone index parameter is a future addition.
        /// </summary>
        private bool TryGetSelectedDrone(RpcRequest req, out QuadSimApi drone, out RpcResponse errorResp
onse)
        {
            drone = QuadSimApi.Selected();

            if (drone != null)
            {
                errorResponse = default;
                return true;
            }

            errorResponse = RpcResponse.Error("no_drone", req.RequestId);
            return false;
        }

        /// <summary>Convert Vector3 to float[] for MessagePack serialization.</summary>
        private static object[] Vec3ToArray(Vector3 v)
        {
            return new object[] { v.x, v.y, v.z };
        }

        private static object[] Vec4ToArray(Vector4 v)
        {
            return new object[] { v.x, v.y, v.z, v.w };
        }

        /// <summary>Convert Quaternion to float[] for MessagePack serialization.</summary>
        private static object[] QuatToArray(Quaternion q)
        {
            return new object[] { q.x, q.y, q.z, q.w };
        }

        private static bool IsFinite(float value) =>
            !float.IsNaN(value) && !float.IsInfinity(value);

        private static bool TryGetVector3(RpcRequest req, string key, out Vector3 value)
        {
            value = Vector3.zero;
            if (!req.TryGetArray(key, out var raw) || raw.Count != 3)
                return false;
            if (!RpcRequest.TryConvertFloat(raw[0], out float x)
                || !RpcRequest.TryConvertFloat(raw[1], out float y)
                || !RpcRequest.TryConvertFloat(raw[2], out float z)
                || !IsFinite(x) || !IsFinite(y) || !IsFinite(z))
            {
                return false;
            }

            value = new Vector3(x, y, z);
            return true;
        }

        private static bool TryGetColor(
            RpcRequest req, string key, Color defaultValue, out Color value)
        {
            value = defaultValue;
            if (!req.Params.ContainsKey(key)) return true;
            if (!req.TryGetArray(key, out var raw)
                || (raw.Count != 3 && raw.Count != 4))
            {
                return false;
            }
            if (!RpcRequest.TryConvertFloat(raw[0], out float r)
                || !RpcRequest.TryConvertFloat(raw[1], out float g)
                || !RpcRequest.TryConvertFloat(raw[2], out float b)
                || !IsFinite(r) || !IsFinite(g) || !IsFinite(b))
            {
```

```
            return false;
        }

        float a = defaultValue.a;
        if (raw.Count == 4
            && (!RpcRequest.TryConvertFloat(raw[3], out a) || !IsFinite(a)))
        {
            return false;
        }
        value = new Color(r, g, b, a);
        return true;
    }

    private Dictionary<string, object> CaptureRLStates(
        DroneManager manager, double simTime, out object[] eventPayloads)
    {
        var states = new Dictionary<string, object>(manager.DroneCount);
        var events = new List<RLAgentEvent>();
        for (int i = 0; i < manager.RegisteredDrones.Count; i++)
        {
            QuadPawn pawn = manager.RegisteredDrones[i];
            RLAgentState state = pawn.Api.GetRLState();
            states[state.AgentId] = RLAgentStateToDict(state, _stateSequence++);
            pawn.DrainRLEvents(events);
            if (!state.IsValid)
                events.Add(new RLAgentEvent("invalid_state", state.AgentId, null, simTime));
        }

        events.Sort((a, b) =>
        {
            int order = string.CompareOrdinal(a.AgentId, b.AgentId);
            if (order != 0) return order;
            order = string.CompareOrdinal(a.Type, b.Type);
            if (order != 0) return order;
            order = string.CompareOrdinal(a.OtherAgentId, b.OtherAgentId);
            return order != 0
                ? order
                : string.CompareOrdinal(a.ObjectId, b.ObjectId);
        });

        eventPayloads = new object[events.Count];
        for (int i = 0; i < events.Count; i++)
            eventPayloads[i] = RLAgentEventToDict(events[i], _eventSequence++);
        return states;
    }

    private Dictionary<string, object> RLAgentEventToDict(
        RLAgentEvent agentEvent, ulong sequence)
    {
        var payload = new Dictionary<string, object>
        {
            ["episode_id"] = _rlEpisodeId,
            ["step_index"] = ClockFactory.Clock.StepCount,
            ["sim_time_ns"] = (long)Math.Round(agentEvent.SimTime * 1e9),
            ["type"] = agentEvent.Type,
            ["agent_id"] = agentEvent.AgentId,
            ["sim_time"] = agentEvent.SimTime,
            ["sequence"] = sequence,
            ["source_frame"] = "world_enu",
        };
        if (!string.IsNullOrEmpty(agentEvent.OtherAgentId))
            payload["other_agent_id"] = agentEvent.OtherAgentId;
        if (!string.IsNullOrEmpty(agentEvent.ObjectId))
            payload["object_id"] = agentEvent.ObjectId;
        return payload;
    }

    private Dictionary<string, object> RLAgentStateToDict(
        RLAgentState state, ulong sequence)
    {
        return new Dictionary<string, object>
        {
            ["episode_id"] = _rlEpisodeId,
```

```
                ["step_index"] = ClockFactory.Clock.StepCount,
                ["sim_time_ns"] = _sim.SimTimeNanos,
                ["sim_time"] = _sim.SimTimeSec,
                ["sequence"] = sequence,
                ["source_frame"] = "body_flu/world_enu",
                ["agent_id"] = state.AgentId,
                ["team_id"] = state.TeamId,
                ["drone_index"] = state.DroneIndex,
                ["position_enu"] = Vec3ToArray(state.PositionEnu),
                ["velocity_enu"] = Vec3ToArray(state.VelocityEnu),
                ["orientation_flu_to_enu"] = QuatToArray(state.OrientationFluToEnu),
                ["body_velocity_flu"] = Vec3ToArray(state.BodyVelocityFlu),
                ["body_angular_velocity_flu"] = Vec3ToArray(state.BodyAngularVelocityFlu),
                ["forward_enu"] = Vec3ToArray(state.ForwardEnu),
                ["up_enu"] = Vec3ToArray(state.UpEnu),
                ["is_valid"] = state.IsValid,
                ["has_collision"] = state.HasCollision,
                ["is_grounded"] = state.IsGrounded,
            };
        }
    }
}
```

Runtime/Scripts/SimCore/Rpc/RpcProtocol.cs

SHA-256 31610e315d804e85639de22f7de20ab4e8f6260b5e2aa40ed5cae0ceb064911a | 376 lines | 13,876 bytes

```
// Assets/Scripts/SimCore/Rpc/RpcProtocol.cs
// Phase 9a + 9b: Message Protocol Definitions
//
// PURPOSE:
//   Defines the wire protocol for External RPC communication.
//   All messages are MessagePack-encoded dictionaries.
//   This file provides serialization/deserialization helpers and
//   strongly-typed request/response wrappers.
//
// PROTOCOL:
//   Request:  { "method": "...", ...params }
//   Response: { "status": "ok"|"error", "data": {...}, "error_msg": "..." }
//
// NOTE ON TYPE COERCION (Phase 9b fix):
//   MessagePack-CSharp with ContractlessStandardResolver deserializes integers
//   into the smallest fitting C# type:
//     0..127      → byte
//     -32..-1     → sbyte
//     128..32767  → short (or int/long for larger)
//   The GetInt/GetFloat helpers must handle ALL numeric types, not just int/long.
//
// DEPENDENCIES:
//   - MessagePack-CSharp (NuGet: MessagePack)

using System;
using System.Collections.Generic;
using MessagePack;
using UnityEngine;

namespace SimCore.Rpc
{
    public static class RpcCapabilities
    {
        public const int ProtocolMajor = 1;
        public const int ProtocolMinor = 0;
        public const string UnityPackageVersion = "0.3.0";
        public const int FrameSchemaVersion = 1;
        public const int TimeSchemaVersion = 1;

        public static readonly object[] Features =
        {
            "protocol_negotiation",
            "strict_lockstep",
            "post_step_observations",
            "episode_reset",
            "multi_agent_step",
            "allocation_diagnostics",
```

```
            "seeded_sensor_noise",
            "collision_events",
            "yaml_schema_v1",
            "rpc_request_limits",
        };

        public static Dictionary<string, object> ConnectData(
            int commandPort, int telemetryPort, string bindAddress)
        {
            return new Dictionary<string, object>
            {
                ["message"] = "connected",
                ["server"] = "QuadSim",
                ["protocol_major"] = ProtocolMajor,
                ["protocol_minor"] = ProtocolMinor,
                ["unity_package_version"] = UnityPackageVersion,
                ["features"] = Features,
                ["frame_schema_version"] = FrameSchemaVersion,
                ["time_schema_version"] = TimeSchemaVersion,
                ["command_port"] = commandPort,
                ["telemetry_port"] = telemetryPort,
                ["bind_address"] = bindAddress,
            };
        }
    }

    // ============================================================================
    // Wire-Level Types (MessagePack dictionaries)
    // ============================================================================

    /// <summary>
    /// Parsed request from the Python client.
    /// All fields are extracted from a MessagePack map.
    /// </summary>
    public readonly struct RpcRequest
    {
        /// <summary>The method name (e.g., "connect", "heartbeat", "set_mode").</summary>
        public readonly string Method;

        /// <summary>Raw parameters dictionary. Method handlers extract what they need.</summary>
        public readonly Dictionary<string, object> Params;

        /// <summary>Unique request ID for correlation (optional, client-assigned).</summary>
        public readonly long RequestId;

        public RpcRequest(string method, Dictionary<string, object> parameters, long requestId = 0)
        {
            Method = method;
            Params = parameters ?? new Dictionary<string, object>();
            RequestId = requestId;
        }

        /// <summary>Get a string parameter, or null if missing.</summary>
        public string GetString(string key)
        {
            if (Params.TryGetValue(key, out var val) && val is string s)
                return s;
            return null;
        }

        /// <summary>
        /// Get a float parameter, with default.
        /// Handles all numeric types that MessagePack may deserialize into.
        /// </summary>
        public float GetFloat(string key, float defaultValue = 0f)
        {
            return TryGetFloat(key, out float value) ? value : defaultValue;
        }

        public bool TryGetFloat(string key, out float value)
        {
            value = 0f;
            if (!Params.TryGetValue(key, out var val)) return false;
```

```
    return TryConvertFloat(val, out value);
}

public static bool TryConvertFloat(object val, out float value)
{
    value = 0f;
    switch (val)
    {
        case float f: value = f; return true;
        case double d: value = (float)d; return true;
        case byte b: value = b; return true;
        case sbyte sb: value = sb; return true;
        case short s: value = s; return true;
        case ushort us: value = us; return true;
        case int i: value = i; return true;
        case uint ui: value = ui; return true;
        case long l: value = l; return true;
        case ulong ul: value = ul; return true;
        default: return false;
    }
}

/// <summary>
/// Get an int parameter, with default.
/// Handles all numeric types that MessagePack may deserialize into.
/// </summary>
public int GetInt(string key, int defaultValue = 0)
{
    return TryGetInt(key, out int value) ? value : defaultValue;
}

public bool TryGetInt(string key, out int value)
{
    value = 0;
    if (!Params.TryGetValue(key, out var raw)) return false;

    switch (raw)
    {
        case byte b: value = b; return true;
        case sbyte sb: value = sb; return true;
        case short s: value = s; return true;
        case ushort us: value = us; return true;
        case int i: value = i; return true;
        case uint ui when ui <= int.MaxValue: value = (int)ui; return true;
        case long l when l >= int.MinValue && l <= int.MaxValue: value = (int)l; return true;
        case ulong ul when ul <= int.MaxValue: value = (int)ul; return true;
        default: return false;
    }
}

public bool TryGetLong(string key, out long value)
{
    value = 0;
    if (!Params.TryGetValue(key, out var raw)) return false;

    switch (raw)
    {
        case byte b: value = b; return true;
        case sbyte sb: value = sb; return true;
        case short s: value = s; return true;
        case ushort us: value = us; return true;
        case int i: value = i; return true;
        case uint ui: value = ui; return true;
        case long l: value = l; return true;
        case ulong ul when ul <= long.MaxValue: value = (long)ul; return true;
        default: return false;
    }
}

/// <summary>Get a bool parameter, with default.</summary>
public bool GetBool(string key, bool defaultValue = false)
{
```

```
            if (Params.TryGetValue(key, out var val) && val is bool b)
                return b;
            return defaultValue;
        }

        public bool TryGetArray(string key, out IReadOnlyList<object> values)
        {
            values = null;
            if (!Params.TryGetValue(key, out var raw)) return false;

            if (raw is object[] array)
            {
                values = array;
                return true;
            }

            if (raw is List<object> list)
            {
                values = list;
                return true;
            }

            return false;
        }

        public static bool TryGetMap(object raw, out Dictionary<string, object> values)
        {
            if (raw is Dictionary<string, object> stringMap)
            {
                values = stringMap;
                return true;
            }

            if (raw is IDictionary<object, object> objectMap)
            {
                values = new Dictionary<string, object>(objectMap.Count);
                foreach (var pair in objectMap)
                {
                    if (pair.Key is not string key)
                    {
                        values = null;
                        return false;
                    }
                    values[key] = pair.Value;
                }
                return true;
            }

            values = null;
            return false;
        }
    }

    /// <summary>
    /// Response to send back to the Python client.
    /// Serialized as a MessagePack map.
    /// </summary>
    public readonly struct RpcResponse
    {
        public readonly bool Success;
        public readonly string ErrorMessage;
        public readonly Dictionary<string, object> Data;
        public readonly long RequestId;

        private RpcResponse(bool success, string errorMessage,
                            Dictionary<string, object> data, long requestId)
        {
            Success = success;
            ErrorMessage = errorMessage;
            Data = data;
            RequestId = requestId;
        }
```

```
        public static RpcResponse Ok(long requestId = 0, Dictionary<string, object> data = null)
            => new RpcResponse(true, null, data, requestId);

        public static RpcResponse Error(string message, long requestId = 0)
            => new RpcResponse(false, message, null, requestId);
    }

    // ============================================================================
    // Serialization Helpers
    // ============================================================================

    /// <summary>
    /// Serialization utilities for the RPC wire protocol.
    /// Uses MessagePack with the typeless resolver for dictionary-based messages.
    /// </summary>
    public static class RpcSerializer
    {
        /// <summary>
        /// Deserialize a raw MessagePack byte[] into an RpcRequest.
        /// Expects a map with at least a "method" key.
        /// Returns null if deserialization fails.
        /// </summary>
        public static RpcRequest? DeserializeRequest(byte[] data)
        {
            try
            {
                var dict = MessagePackSerializer.Deserialize<Dictionary<string, object>>(
                    data, MessagePack.Resolvers.ContractlessStandardResolver.Options);

                if (dict == null || !dict.TryGetValue("method", out var methodObj) || methodObj is not s
tring method)
                {
                    Debug.LogWarning("[RpcProtocol] Request missing 'method' field.");
                    return null;
                }
                if (dict.Count > 64 || string.IsNullOrWhiteSpace(method) || method.Length > 64)
                {
                    Debug.LogWarning("[RpcProtocol] Request shape exceeds protocol limits.");
                    return null;
                }

                long requestId = 0;
                if (dict.TryGetValue("request_id", out var idObj))
                {
                    requestId = idObj switch
                    {
                        byte b    => b,
                        sbyte sb  => sb,
                        short s   => s,
                        ushort us => us,
                        int i     => i,
                        uint ui   => ui,
                        long l    => l,
                        ulong ul  => (long)ul,
                        _ => 0
                    };
                }

                // Remove protocol fields, leave params
                dict.Remove("method");
                dict.Remove("request_id");

                return new RpcRequest(method, dict, requestId);
            }
            catch (Exception ex)
            {
                Debug.LogWarning($"[RpcProtocol] Failed to deserialize request: {ex.Message}");
                return null;
            }
        }

        /// <summary>
        /// Serialize an RpcResponse to MessagePack bytes for sending over ZMQ.
```

```
        /// </summary>
        public static byte[] SerializeResponse(RpcResponse response)
        {
            var dict = new Dictionary<string, object>
            {
                ["status"] = response.Success ? "ok" : "error"
            };

            if (response.RequestId != 0)
                dict["request_id"] = response.RequestId;

            if (!response.Success && response.ErrorMessage != null)
                dict["error_msg"] = response.ErrorMessage;

            if (response.Data != null)
            {
                foreach (var kvp in response.Data)
                    dict[kvp.Key] = kvp.Value;
            }

            return MessagePackSerializer.Serialize(dict,
                MessagePack.Resolvers.ContractlessStandardResolver.Options);
        }

        /// <summary>
        /// Serialize a telemetry/sensor publish message.
        /// Topic prefix is prepended as a separate ZMQ frame by the caller.
        /// </summary>
        public static byte[] SerializePublish(string topic, Dictionary<string, object> data)
        {
            data["topic"] = topic;
            data["timestamp"] = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();

            return MessagePackSerializer.Serialize(data,
                MessagePack.Resolvers.ContractlessStandardResolver.Options);
        }
    }
}
```

Runtime/Scripts/SimCore/Rpc/RpcServerThread.cs

SHA-256 9673e021d0f8e355f89aeeedec25fdc92f580ec500b516775784bdb80daeec57 | 395 lines | 15,788 bytes

```
// Assets/Scripts/SimCore/Rpc/RpcServerThread.cs
// Phase 9a + 9c: Background ZMQ Server Thread
//
// PURPOSE:
//   Runs on a background thread. Owns the NetMQ REP and PUB sockets.
//   Polls REP for incoming requests, enqueues them for the main thread,
//   and waits for the main thread to provide responses.
//   Drains the publish queue and sends frames on the PUB socket (Phase 9c).
//
// THREADING CONTRACT:
//   - This class runs entirely on the background thread.
//   - Communication with Unity main thread via ConcurrentQueues.
//   - Main thread signals responses via ManualResetEventSlim.
//   - Shutdown is signaled via _shutdownToken.
//
// DOES NOT:
//   - Call any Unity API (no MonoBehaviour, no Debug.Log from bg thread)
//   - Process requests — just relays them
//   - Know about SimCoreApi, QuadSimApi, or any game logic

using System;
using System.Collections.Concurrent;
using System.Threading;
using NetMQ;
using NetMQ.Sockets;

namespace SimCore.Rpc
{
    /// <summary>
    /// Queued request from background thread to main thread.
    /// </summary>
```

```
public sealed class QueuedRequest
{
    public RpcRequest Request;
    public readonly ManualResetEventSlim ResponseReady = new ManualResetEventSlim(false);
    public byte[] ResponseBytes;

    public void SetResponse(byte[] bytes)
    {
        ResponseBytes = bytes;
        ResponseReady.Set();
    }
}

/// <summary>
/// Background thread that manages ZMQ sockets for the External RPC adapter.
///
/// Lifecycle:
///   1. Constructed with config (ports, timeouts)
///   2. Start() spins up the thread
///   3. Main thread calls TryDequeue() in Update() to get requests
///   4. Main thread processes request, calls SetResponse() on the QueuedRequest
///   5. Background thread wakes up and sends the response over ZMQ
///   6. Main thread calls EnqueuePublish() to push telemetry frames (Phase 9c)
///   7. Background thread drains publish queue onto PUB socket each cycle
///   8. Stop() signals shutdown, joins the thread
/// </summary>
public sealed class RpcServerThread : IDisposable
{
    // ================================================================================
    // Configuration
    // ================================================================================

    private readonly int _repPort;
    private readonly int _pubPort;
    private readonly int _pollTimeoutMs;
    private readonly string _bindAddress;
    private readonly int _maxRequestBytes;
    private readonly int _maxRequestsPerSecond;
    private long _rateWindowStartedMs;
    private int _requestsInWindow;

    // ================================================================================
    // Thread Communication
    // ================================================================================

    /// <summary>
    /// Requests waiting for the main thread to process.
    /// Background thread enqueues, main thread dequeues.
    /// </summary>
    private readonly ConcurrentQueue<QueuedRequest> _pendingRequests = new();

    /// <summary>
    /// Log messages from the background thread (since we can't use Debug.Log).
    /// Main thread drains these in Update().
    /// </summary>
    private readonly ConcurrentQueue<(LogLevel, string)> _logQueue = new();

    /// <summary>
    /// Publish frames queued by the main thread for the PUB socket (Phase 9c).
    /// Main thread enqueues serialized MessagePack frames, background thread sends them.
    /// </summary>
    private readonly ConcurrentQueue<byte[]> _publishQueue = new();

    public enum LogLevel { Info, Warning, Error }

    // ================================================================================
    // Thread State
    // ================================================================================

    private Thread _thread;
    private volatile bool _shutdownRequested;
    private volatile bool _isRunning;
```

```
        /// <summary>True if the background thread is actively running and sockets are bound.</summary>
        public bool IsRunning => _isRunning;

        /// <summary>True during startup, normal operation, or a delayed shutdown.</summary>
        public bool IsThreadAlive => _thread != null && _thread.IsAlive;

        // ================================================================================
        // Construction
        // ================================================================================

        /// <param name="repPort">Port for REQ/REP command socket (default 5555).</param>
        /// <param name="pubPort">Port for PUB telemetry socket (default 5556).</param>
        /// <param name="pollTimeoutMs">ZMQ poll interval in ms. Lower = more responsive, higher = less
CPU (default 10).</param>
        public RpcServerThread(
            int repPort = 5555, int pubPort = 5556,
            string bindAddress = "127.0.0.1", int pollTimeoutMs = 10,
            int maxRequestBytes = 1024 * 1024, int maxRequestsPerSecond = 1000)
        {
            _repPort = repPort;
            _pubPort = pubPort;
            _bindAddress = string.IsNullOrWhiteSpace(bindAddress) ? "127.0.0.1" : bindAddress;
            _pollTimeoutMs = pollTimeoutMs;
            _maxRequestBytes = maxRequestBytes;
            _maxRequestsPerSecond = maxRequestsPerSecond;
            _rateWindowStartedMs = MonotonicMilliseconds();
        }

        // ================================================================================
        // Lifecycle
        // ================================================================================

        /// <summary>
        /// Start the background thread. Binds ZMQ sockets and begins polling.
        /// </summary>
        public void Start()
        {
            // IsRunning becomes true only after both sockets bind. Guard on the
            // actual thread so another Start cannot enter during that window.
            if (_thread != null && _thread.IsAlive)
            {
                Log(LogLevel.Warning, "Server thread already starting or running.");
                return;
            }

            _shutdownRequested = false;

            _thread = new Thread(Run)
            {
                Name = "QuadSim-RPC-Server",
                IsBackground = true
            };
            _thread.Start();
        }

        /// <summary>
        /// Signal the background thread to shut down and wait for it to finish.
        /// </summary>
        public void Stop()
        {
            // Always stop and join, including while the thread is still creating
            // or binding sockets. The old IsRunning guard leaked that startup case.
            _shutdownRequested = true;

            Thread thread = _thread;
            if (thread != null && thread.IsAlive && Thread.CurrentThread != thread)
            {
                if (!thread.Join(TimeSpan.FromSeconds(3)))
                {
                    // Keep the reference alive. The adapter must not run global
                    // NetMQ cleanup while this worker may still own sockets.
                    Log(LogLevel.Warning, "Server thread did not exit within 3s.");
                    return;
```

```
        }
    }

    if (_thread == thread && (thread == null || !thread.IsAlive))
        _thread = null;
}

/// <summary>
/// Try to dequeue a pending request for main-thread processing.
/// Returns true if a request was available.
/// </summary>
public bool TryDequeue(out QueuedRequest request)
{
    return _pendingRequests.TryDequeue(out request);
}

/// <summary>
/// Drain log messages from the background thread.
/// Call from main thread (e.g., in Update).
/// </summary>
public bool TryDequeueLog(out LogLevel level, out string message)
{
    if (_logQueue.TryDequeue(out var entry))
    {
        level = entry.Item1;
        message = entry.Item2;
        return true;
    }

    level = default;
    message = null;
    return false;
}

// ============================================================================
// PUB Socket — Phase 9c
// ============================================================================

/// <summary>
/// Enqueue a serialized MessagePack frame for publishing on the PUB socket.
/// Called from the main thread (ExternalRpcAdapter.PublishTelemetry).
/// The background thread drains this queue each poll cycle.
/// </summary>
public void EnqueuePublish(byte[] frame)
{
    _publishQueue.Enqueue(frame);
}

// ============================================================================
// Background Thread Entry Point
// ============================================================================

private void Run()
{
    ResponseSocket repSocket = null;
    PublisherSocket pubSocket = null;

    try
    {
        // Select the managed AsyncIO backend before creating sockets.
        AsyncIO.ForceDotNet.Force();

        if (_shutdownRequested) return;
        repSocket = new ResponseSocket();

        if (_shutdownRequested) return;
        pubSocket = new PublisherSocket();

        if (_shutdownRequested) return;

        string repAddress = $"tcp://{_bindAddress}:{_repPort}";
        string pubAddress = $"tcp://{_bindAddress}:{_pubPort}";
```

```
            repSocket.Bind(repAddress);
            pubSocket.Bind(pubAddress);

            Log(LogLevel.Info, $"REP socket bound to {repAddress}");
            Log(LogLevel.Info, $"PUB socket bound to {pubAddress}");

            _isRunning = true;

            // Main poll loop
            while (!_shutdownRequested)
            {
                PollIteration(repSocket, pubSocket);
            }
        }
        catch (Exception ex)
        {
            if (_shutdownRequested)
                Log(LogLevel.Info, $"Server stopped during startup/shutdown: {ex.Message}");
            else
                Log(LogLevel.Error, $"Server thread exception: {ex.Message}\n{ex.StackTrace}");
        }
        finally
        {
            _isRunning = false;

            try { repSocket?.Unbind($"tcp://{_bindAddress}:{_repPort}"); } catch { }
            try { pubSocket?.Unbind($"tcp://{_bindAddress}:{_pubPort}"); } catch { }

            repSocket?.Dispose();
            pubSocket?.Dispose();

            // NetMQConfig.Cleanup is process-global, not socket-local. The
            // adapter performs one blocking cleanup after this thread joins.
            Log(LogLevel.Info, "Server thread stopped. Sockets closed.");
        }
    }

    private void PollIteration(ResponseSocket repSocket, PublisherSocket pubSocket)
    {
        // --- 1. Handle REP requests ---

        if (repSocket.TryReceiveFrameBytes(TimeSpan.FromMilliseconds(_pollTimeoutMs),
                out byte[] requestBytes))
        {
            if (requestBytes.Length > _maxRequestBytes)
            {
                repSocket.SendFrame(RpcSerializer.SerializeResponse(
                    RpcResponse.Error("request_too_large")));
                return;
            }
            if (!AcceptRequestRate())
            {
                repSocket.SendFrame(RpcSerializer.SerializeResponse(
                    RpcResponse.Error("rate_limited")));
                return;
            }

            // Deserialize
            var parsed = RpcSerializer.DeserializeRequest(requestBytes);
            if (parsed == null)
            {
                // Bad message — send error response immediately (must reply to maintain REQ/REP)
                var errorResponse = RpcSerializer.SerializeResponse(
                    RpcResponse.Error("invalid_message"));
                repSocket.SendFrame(errorResponse);
            }
            else
            {
                // Enqueue for main thread processing
                var queued = new QueuedRequest { Request = parsed.Value };
                _pendingRequests.Enqueue(queued);

                // Wait in short slices so shutdown can interrupt this wait and
```

```
                    // Stop() can reliably join the worker instead of waiting 5 seconds.
                    bool gotResponse = false;
                    const int responseTimeoutMs = 5000;
                    const int waitSliceMs = 10;
                    for (int waitedMs = 0;
                         waitedMs < responseTimeoutMs && !_shutdownRequested;
                         waitedMs += waitSliceMs)
                    {
                        if (queued.ResponseReady.Wait(waitSliceMs))
                        {
                            gotResponse = true;
                            break;
                        }
                    }

                    if (_shutdownRequested) return;

                    if (gotResponse && queued.ResponseBytes != null)
                    {
                        repSocket.SendFrame(queued.ResponseBytes);
                    }
                    else
                    {
                        // Timeout — main thread didn't respond in time
                        Log(LogLevel.Warning, $"Response timeout for method '{parsed.Value.Method}'");
                        var timeoutResponse = RpcSerializer.SerializeResponse(
                            RpcResponse.Error("server_timeout"));
                        repSocket.SendFrame(timeoutResponse);
                    }
                }
            }

            // --- 2. Drain PUB queue (Phase 9c) ---

            while (_publishQueue.TryDequeue(out byte[] pubFrame))
            {
                pubSocket.SendFrame(pubFrame);
            }
        }

        // ================================================================================
        // Helpers
        // ================================================================================

        private void Log(LogLevel level, string message)
        {
            _logQueue.Enqueue((level, $"[RpcServer] {message}"));
        }

        private bool AcceptRequestRate()
        {
            long now = MonotonicMilliseconds();
            if (now - _rateWindowStartedMs >= 1000)
            {
                _rateWindowStartedMs = now;
                _requestsInWindow = 0;
            }
            _requestsInWindow++;
            return _requestsInWindow <= _maxRequestsPerSecond;
        }

        private static long MonotonicMilliseconds()
        {
            return (long)(System.Diagnostics.Stopwatch.GetTimestamp() * 1000.0
                / System.Diagnostics.Stopwatch.Frequency);
        }

        public void Dispose()
        {
            Stop();
        }
    }
}
```

6. Drone Entity, Lifecycle, and Internal API

Defines the physical drone entity and the manager that registers, spawns, selects, resets, and steps drones. The per-drone API, command proxy, and mode coordinator provide the controlled interface used by UI, internal scripts, and RPC handlers.

Files in this section

- Runtime/Scripts/DroneCore/QuadPawn.cs
- Runtime/Scripts/DroneCore/DroneRootBootstrap.cs
- Runtime/Scripts/DroneCore/Api/QuadSimApi.cs
- Runtime/Scripts/DroneCore/Core/DroneManager.cs
- Runtime/Scripts/DroneCore/Core/ModeCoordinator.cs
- Runtime/Scripts/DroneCore/Interfaces/FlightCommandProxy.cs
- Runtime/Scripts/DroneCore/Interfaces/ICommandSource.cs

Runtime/Scripts/DroneCore/QuadPawn.cs

SHA-256 fd1664bda52caf1f50da8e0cf810cb8e928b7e78596a5544f1dbc71917ceee7d | 868 lines | 34,218 bytes

```
using System;
using System.Collections.Generic;
using DroneCore.Controllers;
using DroneCore.Controls;
using DroneCore.CameraSystem;
using DroneCore.Core;
using DroneCore.Interfaces;
using RobotCore;
using SimCore;
using SimCore.Common;
using UnityEngine;
using Yaml.Drone;
using DroneCore.Api;
using QuadSim.RL;

namespace DroneCore
{
    /// <summary>
    /// QuadPawn is the physical drone entity — the "vehicle" that exists in the world.
    /// It owns references to all drone subsystems and orchestrates the physics step:
    ///   Controller → Apply Wrench → Physics → Sensors
    ///
    /// DroneManager iterates QuadPawns in the pre- and post-physics phases.
    /// QuadPawn is NOT an ISimulatable — DroneManager drives it.
    ///
    /// Mirrors UE's AQuadPawn: owns the CustomPhysicsTick sequence,
    /// buffered force application, and controller routing.
    /// </summary>
    [RequireComponent(typeof(Rigidbody))]
    [RequireComponent(typeof(DroneCameraRig))]
    [DisallowMultipleComponent]
    public sealed class QuadPawn : MonoBehaviour
    {
        // ================================================================================
        // Serialized Fields — Set in Inspector / Prefab
        // ================================================================================

        [Header("Agent Identity")]
        [Tooltip("Stable RPC identity. Leave empty to assign drone_<index> when registered.")]
        [SerializeField] private string agentId = "";

        [Tooltip("Team identity used by multi-agent tasks.")]
        [SerializeField] private string teamId = "neutral";

        [Tooltip("Persistent RPC index. Leave at -1 to assign the next available index.")]
        [SerializeField] private int droneIndex = -1;

        [Header("Core Physics")]
```

```
        [SerializeField] private Rigidbody rb;

        [Header("Motor Transforms")]
        [SerializeField] private Transform motorFL;
        [SerializeField] private Transform motorFR;
        [SerializeField] private Transform motorBL;
        [SerializeField] private Transform motorBR;

        [Header("Subsystem References")]
        [SerializeField] private CascadedController controller;
        [SerializeField] private SensorManager sensorManager;
        [SerializeField] private FlightCommandProxy commandProxy;
        [SerializeField] private ModeCoordinator modeCoordinator;
        [SerializeField] private QuadSimApi quadSimApi;

        [Header("Camera")]
        [SerializeField] private Transform fpvCameraPoint;
        [SerializeField] private DroneCameraRig cameraRig;
        [SerializeField] private DroneImageSensor imageSensor;

        [Header("Visuals")]
        [SerializeField] private Transform[] propellerMeshes = new Transform[4];

        // ================================================================================
        // Runtime State
        // ================================================================================

        // Canonical motor order: [0]=FL, [1]=FR, [2]=BL, [3]=BR
        private readonly Transform[] _motors = new Transform[4];
        private readonly float[] _propellerRPMs = new float[4];

        // Config
        private DroneConfig _config;
        private bool _configApplied;

        // Rotor physics derived from config (matches UE's RotorPhysics member)
        private RotorPhysicsDerived _rotorPhysics;

        public string DroneID => gameObject.name;

        // Initialization tracking
        private bool _sensorsReady;
        private bool _controllerReady;
        private bool _initialized;
        public bool HasConfigApplied => _configApplied;
        /// <summary>Read-only view of the applied YAML config. UI reads limits/ranges from here.</summa
ry>
        public Yaml.Drone.DroneConfig Config => _config;
        // Active controller type (matches UE's currentControllerType)
        [Header("Controller Selection")]
        [SerializeField] private ControllerKind currentControllerKind = ControllerKind.Cascade;

        // Buffered wrench (matches UE's PhysicsState pattern)
        // In Unity we don't need atomics since PrePhysicsStep and Physics.Simulate
        // run on the same thread, but we keep the generation pattern for clarity.
        private Vector3 _bufferedForce;
        private Vector3 _bufferedTorque;
        private uint _wrenchGeneration;
        private uint _lastAppliedGeneration;
        private readonly HashSet<Collider> _activeColliders = new HashSet<Collider>();
        private readonly HashSet<Collider> _groundColliders = new HashSet<Collider>();
        private readonly List<RLAgentEvent> _pendingRLEvents = new List<RLAgentEvent>();

        // Manager reference (set explicitly by DroneManager, not searched)
        private DroneManager _droneManager;

        // ================================================================================
        // Public Accessors
        // ================================================================================

        public void SetDroneManager(DroneManager manager) => _droneManager = manager;

        public Rigidbody Rigidbody => rb;
```

```
        public Transform[] Motors => _motors;
        public Transform MotorFL => _motors[0];
        public Transform MotorFR => _motors[1];
        public Transform MotorBL => _motors[2];
        public Transform MotorBR => _motors[3];

        public CascadedController Controller => controller;
        public SensorManager Sensors => sensorManager;
        public FlightCommandProxy CommandProxy => commandProxy;
        public ModeCoordinator ModeCoordinator => modeCoordinator;
        public QuadSimApi Api => quadSimApi;
        public RotorPhysicsDerived RotorPhysics => _rotorPhysics;
        public bool IsInitialized => _initialized;
        public ControllerKind CurrentControllerKind => currentControllerKind;
        public string AgentId => agentId;
        public string TeamId => teamId;
        public int DroneIndex => droneIndex;
        public DroneImageSensor ImageSensor { get { EnsureCameraRig(); return imageSensor; } }
        public bool HasCollision => _activeColliders.Count > 0;
        public bool IsGrounded => _groundColliders.Count > 0;

        internal void EnsureAgentIdentity(int fallbackIndex)
        {
            if (droneIndex < 0)
                droneIndex = fallbackIndex;
            if (string.IsNullOrWhiteSpace(agentId))
                agentId = $"drone_{droneIndex}";
            if (string.IsNullOrWhiteSpace(teamId))
                teamId = "neutral";
        }

        internal void SetEpisodeIdentity(string newAgentId, string newTeamId, int newDroneIndex)
        {
            agentId = newAgentId;
            teamId = newTeamId;
            droneIndex = newDroneIndex;
        }

        // ================================================================================
        // Unity Lifecycle
        // ================================================================================

        private void Reset()
        {
            // Editor helper: auto-grab rigidbody when component is added
            rb = GetComponent<Rigidbody>();
        }

        private void Awake()
        {
            // Auto-bind subsystem references if not set in prefab
            if (rb == null) rb = GetComponent<Rigidbody>();
            if (controller == null) controller = GetComponent<CascadedController>();
            if (sensorManager == null) sensorManager = GetComponent<SensorManager>();
            if (commandProxy == null) commandProxy = GetComponent<FlightCommandProxy>();
            if (modeCoordinator == null) modeCoordinator = GetComponent<ModeCoordinator>();
            if (quadSimApi == null) quadSimApi = GetComponent<QuadSimApi>();
            EnsureCameraRig();

            // Build motor array from serialized references
            _motors[0] = motorFL;
            _motors[1] = motorFR;
            _motors[2] = motorBL;
            _motors[3] = motorBR;

            // Safety fallback: if motor refs weren't assigned, try find by name
            if (_motors[0] == null) _motors[0] = transform.Find("MotorFL");
            if (_motors[1] == null) _motors[1] = transform.Find("MotorFR");
            if (_motors[2] == null) _motors[2] = transform.Find("MotorBL");
            if (_motors[3] == null) _motors[3] = transform.Find("MotorBR");

            if (controller == null) Debug.LogError($"[QuadPawn] {DroneID}: Missing CascadedController!")
;
```

```
            if (sensorManager == null) Debug.LogError($"[QuadPawn] {DroneID}: Missing SensorManager!");
            if (commandProxy == null) Debug.LogError($"[QuadPawn] {DroneID}: Missing FlightCommandProxy!
");

            // Register during Awake so scene-placed drones are known before
            // SimulationManager evaluates DroneManager's startup spawn policy.
            if (_droneManager == null)
                _droneManager = DroneManager.Get();

            if (_droneManager != null)
            {
                _droneManager.RegisterDrone(this);
                Debug.Log($"[QuadPawn] Registered with DroneManager: {DroneID}");
            }
            else
            {
                Debug.LogWarning($"[QuadPawn] {DroneID}: No DroneManager — drone will not be managed.");
            }
        }

        private void Start()
        {
            // If config was applied before Start(), initialize now
            if (_configApplied)
            {
                InitializeSubsystems();
            }
        }

        private void OnDestroy()
        {
            if (_droneManager != null)
            {
                _droneManager.UnregisterDrone(this);
            }
        }

        private void OnCollisionEnter(Collision collision)
        {
            bool wasGrounded = IsGrounded;
            bool isGroundContact = TrackCollision(collision);

            QuadPawn otherPawn = collision.collider.GetComponentInParent<QuadPawn>();
            if (otherPawn != null && otherPawn != this)
                AddRLEvent("agent_collision", otherPawn.AgentId);
            RLSceneObject sceneObject = collision.collider.GetComponentInParent<RLSceneObject>();
            if (sceneObject != null)
                AddRLEvent("object_collision", objectId: sceneObject.ObjectId);
            if (isGroundContact && !wasGrounded)
                AddRLEvent("ground_contact");
        }

        private void OnCollisionStay(Collision collision)
        {
            bool wasGrounded = IsGrounded;
            if (TrackCollision(collision) && !wasGrounded)
                AddRLEvent("ground_contact");
        }

        private void OnCollisionExit(Collision collision)
        {
            _activeColliders.Remove(collision.collider);
            _groundColliders.Remove(collision.collider);
        }

        private bool TrackCollision(Collision collision)
        {
            Collider other = collision.collider;
            _activeColliders.Add(other);

            bool isGroundContact = false;
            for (int i = 0; i < collision.contactCount; i++)
            {
```

```
                if (Vector3.Dot(collision.GetContact(i).normal, Vector3.up) >= 0.5f)
                {
                    isGroundContact = true;
                    break;
                }
            }

            if (isGroundContact)
                _groundColliders.Add(other);
            else
                _groundColliders.Remove(other);

            return isGroundContact;
        }

        private void AddRLEvent(
            string type,
            string otherAgentId = null,
            string objectId = null)
        {
            double simTime = ClockFactory.IsInitialized
                ? ClockFactory.Clock.NowNanos / 1_000_000_000.0
                : 0.0;
            _pendingRLEvents.Add(new RLAgentEvent(
                type, AgentId, otherAgentId, simTime, objectId));
        }

        public void DrainRLEvents(List<RLAgentEvent> destination)
        {
            if (destination == null || _pendingRLEvents.Count == 0) return;
            destination.AddRange(_pendingRLEvents);
            _pendingRLEvents.Clear();
        }

        // ================================================================================
        // Configuration (called by DroneManager after spawning)
        // ================================================================================

        /// <summary>
        /// Apply a drone configuration. Mirrors UE's AQuadPawn::ApplyConfig.
        /// </summary>
        public void ApplyConfig(DroneConfig config)
        {
            _config = config;
            _configApplied = true;

            EnsureCameraRig();
            CameraDirector director = CameraDirector.Get();
            if (director != null && director.TargetDrone == this)
                director.ApplyCameraSettings(_config.CamParams);

            Debug.Log($"[QuadPawn] {DroneID}: Applying config...");

            ApplyPhysicalConfig();

            if (_config.DroneParams.Arm_Length > 0f)
            {
                _rotorPhysics = new RotorPhysicsDerived();
                Debug.Log($"[QuadPawn] {DroneID}: RotorPhysics computed (arm={_config.DroneParams.Arm_Le
ngth}m)");
            }
            InitializeSubsystems();


            ApplyControllerConfig();

            Debug.Log("==== Rotor Geometry Check ====");

            for (int i = 0; i < _motors.Length; i++)
            {
                if (_motors[i] == null) continue;

                Vector3 local = _motors[i].localPosition;
```

```
            Vector3 world = _motors[i].position;

            float armLocalXZ = new Vector2(local.x, local.z).magnitude;
            float armWorldXZ = new Vector2(
                world.x - transform.position.x,
                world.z - transform.position.z
            ).magnitude;

            Debug.Log(
                $"Motor {i}: " +
                $"local={local} " +
                $"world={world} " +
                $"armLocal={armLocalXZ:F3}m " +
                $"armWorld={armWorldXZ:F3}m");
        }
    }

    private void ApplyPhysicalConfig()
    {
        if (rb == null) return;

        // Mass
        if (_config.DroneParams.Mass > 0.01f)
            rb.mass = _config.DroneParams.Mass;
        else
            rb.mass = 1.28f; // Default

        rb.centerOfMass = _config.DroneParams.CenterOfMass;

        if (_config.DroneParams.Inertia != Vector3.zero)
             rb.inertiaTensor = _config.DroneParams.Inertia;
        if (_config.DroneParams.LinearDrag_Coeff >= 0)
            rb.linearDamping = _config.DroneParams.LinearDrag_Coeff;
        if (_config.DroneParams.AngularDrag_Coeff >= 0)
            rb.angularDamping = _config.DroneParams.AngularDrag_Coeff;

        // Rigidbody setup for simulation
        rb.useGravity = true;
        rb.isKinematic = false;
        // Smooth the rendered pose between fixed physics ticks. This does not
        // alter the Rigidbody's simulated position, velocity, or applied forces.
        rb.interpolation = RigidbodyInterpolation.Interpolate;
        // rb.collisionDetectionMode = CollisionDetectionMode.Discrete;
        rb.maxAngularVelocity = 100f;
        Debug.Log($"[QuadPawn] {DroneID}: Inertia tensor = {rb.inertiaTensor}");
        Debug.Log($"[QuadPawn] {DroneID}: Physical config applied (mass={rb.mass}kg)");

        var src = _config.RotorParams.Rotors;
        if (src != null && src.Length > 0)
        {
            Vector3 com = rb.centerOfMass;
            var rebased = new RotorInstance[src.Length];
            for (int i = 0; i < src.Length; i++)
            {
                rebased[i] = src[i];                                // struct copy
                rebased[i].Location_Body = src[i].Location_Body - com;
            }
            _config.RotorParams.Rotors = rebased;

            Vector3 resid = Vector3.zero;
            for (int i = 0; i < rebased.Length; i++) resid += rebased[i].Location_Body;
            resid /= rebased.Length;
            Debug.Log($"[QuadPawn] {DroneID}: rotors rebased on CoM={com:F4}; " +
                      $"residual centroid={resid:F4} (perp component couples thrust→torque)");
        }
    }

    private void ApplyControllerConfig()
    {
        // Cascaded controller
        if (controller != null)
        {
```

```
            _rotorPhysics.ComputeRotor(_config.RotorParams.Model, _config.DroneParams.Arm_Length);
            controller.SetRotorPhysics(_rotorPhysics, _config);
            Debug.Log($"[QuadPawn] {DroneID}: CascadedController config applied");
        }

        // Future: geometric controller
        // if (geoController != null)
        //     geoController.ApplyConfig(_config, _rotorPhysics);
    }

    // ================================================================================
    // Initialization
    // ================================================================================

    private void InitializeSubsystems()
    {
        if (_initialized) return;

        Debug.Log($"[QuadPawn] {DroneID}: Initializing subsystems...");

        // 1. Sensors
        if (sensorManager != null)
        {
            sensorManager.OnStart();
            _sensorsReady = true;
            Debug.Log($"[QuadPawn] {DroneID}: SensorManager initialized");
        }

        // 2. Controller — only ready if sensors are
        if (controller != null && _sensorsReady)
        {
            // Controller handles its own internal init (allocator, gains, etc.)
            // QuadPawn just needs to know it exists and sensors are available.
            controller.Initialize(this, _config, sensorManager);
            _controllerReady = true;
            Debug.Log($"[QuadPawn] {DroneID}: Controller initialized");
        }

        if (modeCoordinator != null && commandProxy != null)
        {
            modeCoordinator.Initialize(this, commandProxy);
            Debug.Log($"[QuadPawn] {DroneID}: ModeCoordinator initialized");
        }

        if (quadSimApi != null)
        {
            quadSimApi.Initialize(this, commandProxy, modeCoordinator, sensorManager);
            Debug.Log($"[QuadPawn] {DroneID}: QuadSimApi initialized");
        }
        _initialized = true;
        Debug.Log($"[QuadPawn] {DroneID}: Initialization complete");
    }

    public void PrePhysicsStep(double dtSec, long nowNanos)
    {
        if (!_initialized) return;

        float dt = (float)dtSec;

        if (_controllerReady)
        {
            RunController(dt);
        }

        ApplyBufferedForces();
    }

    public void PostPhysicsStep(double dtSec, long nowNanos)
    {
        if (!_initialized) return;
        if (sensorManager != null && _sensorsReady)
            sensorManager.UpdateSensors(dtSec, nowNanos);
    }
```

```
private void RunController(float dt)
{
    // Use ModeCoordinator as source of truth if available, else fall back to local field
    ControllerKind kind = modeCoordinator != null
        ? modeCoordinator.ActiveController
        : currentControllerKind;

    switch (kind)
    {
        case ControllerKind.Cascade:
            if (controller != null)
                controller.StepController(dt);
            break;

        case ControllerKind.Geometric:
            // Future: geoController.Update(sensorManager.Latest, dt);
            break;
    }
}

// ================================================================================
// Buffered Force Application (mirrors UE's SetBodyWrench / ApplyBufferedForces)
// ================================================================================

/// <summary>
/// Set the net wrench to apply at next physics step.
/// Called by controllers after computing control output.
/// Force is body-frame Z thrust, Torque is body-frame (X,Y,Z).
/// </summary>
public void SetBodyWrench(Vector3 netForce, Vector3 netTorque)
{
    _bufferedForce = netForce;
    _bufferedTorque = netTorque;
    _wrenchGeneration++;

    // Wake body if significant force
    if (rb != null && (netForce.sqrMagnitude > 1f || netTorque.sqrMagnitude > 0.1f))
    {
        rb.WakeUp();
    }
}

/// <summary>
/// Apply buffered wrench to rigidbody. Body-local to world transform.
/// Called at end of PhysicsStep, before Physics.Simulate().
/// </summary>
private void ApplyBufferedForces()
{
    if (rb == null) return;
    if (_wrenchGeneration == _lastAppliedGeneration) return;

    QuadActuationModel.BodyToWorldWrench(
        transform.rotation, _bufferedForce, _bufferedTorque,
        out Vector3 worldForce, out Vector3 worldTorque);

    rb.AddForce(worldForce, ForceMode.Force);
    rb.AddTorque(worldTorque, ForceMode.Force);

    _lastAppliedGeneration = _wrenchGeneration;
}

// ================================================================================
// Reset Functions
// ================================================================================

/// <summary>
/// Full reset — position, physics, controller state.
/// </summary>
public void ResetDrone()
{
    ResetPhysics();
    ResetControllerState();
```

```
            Debug.Log($"[QuadPawn] {DroneID}: Full reset complete");
        }

        /// <summary>
        /// Reset to specific position and rotation.
        /// </summary>
        public void ResetPosition(Vector3? targetPosition = null, Quaternion? targetRotation = null)
        {
            Vector3 pos = targetPosition ?? Vector3.up;
            Quaternion rot = targetRotation ?? Quaternion.identity;

            if (rb == null)
                rb = GetComponent<Rigidbody>();

            if (rb != null)
            {
                // Rigidbody is authoritative for physics + sensors.
                rb.position = pos;
                rb.rotation = rot;
                rb.linearVelocity = Vector3.zero;
                rb.angularVelocity = Vector3.zero;

                // Clear sleeping/contact weirdness after teleport.
                rb.Sleep();
                rb.WakeUp();
            }

            // Keep Transform synchronized too.
            transform.SetPositionAndRotation(pos, rot);
            Physics.SyncTransforms();

            // Clear buffered wrench/controller state.
            ResetPhysics();
            ResetControllerState();

            if (sensorManager != null)
                sensorManager.OnReset();

            Debug.Log($"[QuadPawn] {DroneID}: Position reset to {pos}, rb.position={(rb != null ? rb.pos
ition.ToString() : "NULL")}");
        }
        /// <summary>
        /// Level the drone (zero roll/pitch, keep yaw), zero velocities.
        /// </summary>
        public void ResetRotation()
        {
            if (rb == null) return;

            Vector3 euler = transform.eulerAngles;
            transform.rotation = Quaternion.Euler(0f, euler.y, 0f);

            rb.angularVelocity = Vector3.zero;
            rb.linearVelocity = Vector3.zero;
        }

        /// <summary>
        /// Zero all velocities without moving.
        /// </summary>
        public void ResetPhysics()
        {
            _activeColliders.Clear();
            _groundColliders.Clear();
            _pendingRLEvents.Clear();

            if (rb == null) return;
            rb.linearVelocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;

            // Clear buffered wrench
            _bufferedForce = Vector3.zero;
            _bufferedTorque = Vector3.zero;
            _wrenchGeneration = 0;
            _lastAppliedGeneration = 0;
```

```
    rb.WakeUp();
}

/// <summary>
/// Reset all controller and command state.
/// </summary>
public void ResetControllerState()
{
    // Opaque reset — controller handles its own internals (PIDs, allocator, etc.)
    if (controller != null)
    {
        controller.ResetControllerState();
    }

    // if (commandProxy != null)
    // {
    //     commandProxy.ResetToSafe();
    // }

    // Clear any pending wrench
    _bufferedForce = Vector3.zero;
    _bufferedTorque = Vector3.zero;
}

// ================================================================================
// Config Reload (mirrors UE)
// ================================================================================

public void ReapplyConfig()
{
    if (_configApplied)
        ApplyConfig(_config);
}

public void ReloadYaml()
{
    if (_droneManager != null)
        _droneManager.ReloadCurrentDrone(false);
}

// ================================================================================
// Visual Updates (called by DroneManager from Update)
// ================================================================================

public void UpdateVisuals(float deltaTime)
{
    UpdatePropellerVisuals(deltaTime);
    // Future: debug draw, effects
}

private void UpdatePropellerVisuals(float deltaTime)
{
    if (propellerMeshes == null || propellerMeshes.Length != 4) return;
    if (controller == null) return;

    var motorOutputs = controller.LastMotor01;
    if (motorOutputs == null || motorOutputs.Count != 4) return;

    // Thrust model params from config or defaults
    float thrustCoef = 1e-5f;
    float propDiameter = 0.127f;
    float airDensity = 1.225f;
    float maxThrustPerMotor = 4.12f;

    if (_configApplied)
    {
        if (_config.RotorParams.Model.ThrustCoeff > 0)
            thrustCoef = _config.RotorParams.Model.ThrustCoeff;
        if (_config.RotorParams.Model.Prop_Diameter > 0)
            propDiameter = _config.RotorParams.Model.Prop_Diameter;
        if (_config.FlightParams.MaxThrust > 0)
            maxThrustPerMotor = _config.FlightParams.MaxThrust / 4f;
```

```
            }

            float denom = thrustCoef * airDensity * Mathf.Pow(propDiameter, 4);
            if (denom < 1e-10f) return;

            // Spin direction from config or default X-config
            int[] spinDir = { 1, -1, -1, 1 };
            if (_configApplied && _config.RotorParams.Rotors != null && _config.RotorParams.Rotors.Lengt
h == 4)
            {
                for (int i = 0; i < 4; i++)
                    spinDir[i] = _config.RotorParams.Rotors[i].Spin_Dir;
            }

            for (int i = 0; i < 4; i++)
            {
                if (propellerMeshes[i] == null) continue;

                float thrust01 = Mathf.Max(0f, motorOutputs[i]);
                float thrustN = thrust01 * maxThrustPerMotor;
                float rpm = 60f * Mathf.Sqrt(thrustN / denom);
                _propellerRPMs[i] = rpm;

                float direction = spinDir[i];
                float frameRotation = (rpm / 60f) * 360f * deltaTime * direction;

                propellerMeshes[i].Rotate(0f, 0f, frameRotation, Space.Self);            }
        }

        // ================================================================================
        // Camera
        // ================================================================================

        public enum CameraMode { ThirdPerson, FPV }

        [Header("Camera")]
        [Tooltip("Drone view camera. Auto-found in children if unset.")]
        [SerializeField] private Camera droneCamera;

        public CameraMode CamMode { get; private set; } = CameraMode.ThirdPerson;

        private Transform _tpParent;
        private Vector3 _tpLocalPos;
        private Quaternion _tpLocalRot;
        private bool _camInit;

        private void EnsureCameraRig()
        {
            if (cameraRig == null)
                cameraRig = GetComponent<DroneCameraRig>();
            if (cameraRig == null)
                cameraRig = gameObject.AddComponent<DroneCameraRig>();
            cameraRig.Initialize(this, fpvCameraPoint);

            if (imageSensor == null)
                imageSensor = GetComponent<DroneImageSensor>();
            if (imageSensor == null)
                imageSensor = gameObject.AddComponent<DroneImageSensor>();
            imageSensor.Initialize(this, cameraRig);
        }

        private void InitCameraRefs()
        {
            if (_camInit) return;

            if (droneCamera == null)
            {
                // Prefer the canonical ThirdPerson camera by name; fall back to any child camera.
                Transform tp = transform.Find("ThirdPerson") ?? FindDeepChild(transform, "ThirdPerson");
                if (tp != null) droneCamera = tp.GetComponent<Camera>();
                if (droneCamera == null) droneCamera = GetComponentInChildren<Camera>(true);
            }
            if (droneCamera == null) return;
```

```
            _tpParent   = droneCamera.transform.parent;
            _tpLocalPos = droneCamera.transform.localPosition;
            _tpLocalRot = droneCamera.transform.localRotation;
            _camInit = true;
        }

        private static Transform FindDeepChild(Transform root, string name)
        {
            foreach (Transform t in root.GetComponentsInChildren<Transform>(true))
                if (t.name == name) return t;
            return null;
        }

        /// <summary>This drone's camera (may be null if prefab has none).</summary>
        public Camera DroneCamera { get { InitCameraRefs(); return droneCamera; } }

        /// <summary>Mount-only camera data for the central CameraDirector.</summary>
        public DroneCameraRig CameraRig { get { EnsureCameraRig(); return cameraRig; } }

        /// <summary>Enable/disable this drone's camera (+AudioListener). DroneManager
        /// calls this on selection so only the selected drone renders.</summary>
        public void SetCameraActive(bool active)
        {
            CameraDirector director = CameraDirector.Get();
            if (director != null)
            {
                if (active) director.Bind(this);
                return;
            }

            InitCameraRefs();
            if (droneCamera == null) return;
            droneCamera.enabled = active;
            var listener = droneCamera.GetComponent<AudioListener>();
            if (listener != null) listener.enabled = active;
        }

        public void SetCameraMode(CameraMode mode)
        {
            CameraDirector director = CameraDirector.Get();
            if (director != null)
            {
                director.Bind(this, snap: false);
                director.SetMode(mode == CameraMode.FPV ? CameraViewMode.FPV : CameraViewMode.Follow);
                CamMode = mode;
                return;
            }

            InitCameraRefs();
            if (droneCamera == null) return;

            var camT = droneCamera.transform;
            if (mode == CameraMode.FPV)
            {
                Transform mount = fpvCameraPoint != null ? fpvCameraPoint : transform;
                camT.SetParent(mount, worldPositionStays: false);
                camT.localPosition = Vector3.zero;
                camT.localRotation = Quaternion.identity;
            }
            else
            {
                camT.SetParent(_tpParent, worldPositionStays: false);
                camT.localPosition = _tpLocalPos;
                camT.localRotation = _tpLocalRot;
            }
            CamMode = mode;
        }

        public void SwitchCamera()
            => SetCameraMode(CamMode == CameraMode.ThirdPerson ? CameraMode.FPV : CameraMode.ThirdPerson
);
```

```
        public void ForceFPVCameraActive()
        {
            SetCameraActive(true);
            SetCameraMode(CameraMode.FPV);
        }

        public Transform GetFPVCameraPoint()
        {
            return CameraRig.FpvMount;
        }

        // ================================================================================
        // Physics Helpers
        // ================================================================================

        /// <summary>
        /// Body inertia tensor in SI (kg·m²). Unity already uses SI.
        /// </summary>
        public Vector3 GetBodyInertiaSI()
        {
            return rb != null ? rb.inertiaTensor : Vector3.zero;
        }
    }
}
```

Runtime/Scripts/DroneCore/DroneRootBootstrap.cs

SHA-256 02cce7adcd113825a3fc7d729f00b341a3917241d3d3f22eb5978ed7a8850bf6 | 108 lines | 3,753 bytes

```
#if UNITY_EDITOR
using UnityEditor;
#endif
using DroneCore.Common;
using DroneCore.Controllers;
using DroneCore.Core;
using DroneCore.Interfaces;
using DroneCore.Api;
using UnityEngine;

namespace DroneCore
{
    /// <summary>
    /// Editor-time and runtime component assembly helper.
    /// Ensures all required components are present on a drone prefab/GameObject.
    ///
    /// This is a "prefab builder" - it doesn't handle runtime lifecycle,
    /// that's QuadPawn's job.
    /// </summary>
    [DisallowMultipleComponent]
    public sealed class DroneRootBootstrap : MonoBehaviour
    {
#if UNITY_EDITOR
        private bool _buildQueued;
#endif

        private void Reset() => Build();

        private void Awake()
        {
            // Runtime only; OnValidate handles editor-time.
            if (Application.isPlaying) Build();
        }

#if UNITY_EDITOR
        private void OnValidate()
        {
            if (Application.isPlaying) return;
            if (_buildQueued) return;

            _buildQueued = true;
            EditorApplication.delayCall += () =>
            {
                _buildQueued = false;
                if (this == null) return;
```

```
                if (Application.isPlaying) return;
                Build();
            };
        }
#endif

        [ContextMenu("Build DroneRoot")]
        public void Build()
        {
            // DEDUPE FIRST (important - prevents multiple components)
            RemoveDuplicates<Rigidbody>(gameObject);
            RemoveDuplicates<QuadPawn>(gameObject);
            RemoveDuplicates<CascadedController>(gameObject);
            RemoveDuplicates<RobotCore.SensorManager>(gameObject);
            RemoveDuplicates<RobotCore.RobotCore>(gameObject);
            RemoveDuplicates<FlightCommandProxy>(gameObject);
            RemoveDuplicates<ModeCoordinator>(gameObject);
            RemoveDuplicates<QuadSimApi>(gameObject);
            // Add required components
            var rb       = GetOrAdd<Rigidbody>(gameObject);
            var body     = GetOrAdd<QuadPawn>(gameObject);
            var ctrl     = GetOrAdd<CascadedController>(gameObject);
            var sensors  = GetOrAdd<RobotCore.SensorManager>(gameObject);
            var core     = GetOrAdd<RobotCore.RobotCore>(gameObject);
            var cmd      = GetOrAdd<FlightCommandProxy>(gameObject);
            var modeCoord = GetOrAdd<ModeCoordinator>(gameObject);
            var api = GetOrAdd<QuadSimApi>(gameObject);

            // Configure Rigidbody defaults
            rb.mass = 1.28f;
            rb.useGravity = true;
            rb.isKinematic = false;
            rb.interpolation = RigidbodyInterpolation.Interpolate;
            rb.collisionDetectionMode = CollisionDetectionMode.Discrete;
            rb.maxAngularVelocity = 100f; // Prevent Unity from clamping angular velocity

            Debug.Log($"[DroneRootBootstrap] Built drone components on {gameObject.name}");
        }

        private static void RemoveDuplicates<T>(GameObject go) where T : Component
        {
            var comps = go.GetComponents<T>();
            for (int i = 1; i < comps.Length; i++)
            {
#if UNITY_EDITOR
                if (!Application.isPlaying)
                    Object.DestroyImmediate(comps[i]);
                else
                    Object.Destroy(comps[i]);
#else
                Object.Destroy(comps[i]);
#endif
            }
        }

        private static T GetOrAdd<T>(GameObject go) where T : Component
        {
            if (!go.TryGetComponent<T>(out var c))
                c = go.AddComponent<T>();
            return c;
        }
    }
}
```

Runtime/Scripts/DroneCore/Api/QuadSimApi.cs

SHA-256 1f1ffadf2a2d1dc9090e039c715782ff6f70387ac994e2b6b5dfeb8122e4e4ad | 507 lines | 20,091 bytes

```
// Assets/Scripts/DroneCore/Api/QuadSimApi.cs
// Phase 5: Per-Drone API Facade
//
// PURPOSE:
//   Single entry point for controlling a specific drone.
//   Internal scripts, External RPC, and (eventually) UI all interact
```

```
//   with the drone through this facade rather than touching subsystems directly.
//
// OWNERSHIP:
//   - Delegates to: QuadPawn, FlightCommandProxy, ModeCoordinator, SensorManager
//   - Does NOT implement logic itself — pure facade pattern
//   - Lives on the drone prefab (added by DroneRootBootstrap)
//   - Initialized by QuadPawn.InitializeSubsystems()
//
// ACCESS PATTERN:
//   SimCoreApi handles sim-wide concerns (time, authority) — no drone references.
//   QuadSimApi handles per-drone concerns — and provides static accessors for convenience:
//     QuadSimApi.Selected()  → currently selected drone
//     QuadSimApi.Get(0)      → drone by index
//   This keeps dependency direction clean: DroneCore → SimCore, never the reverse.
//
// TELEMETRY DESIGN:
//   - GetSensorData() → measured state (IMU + GPS). Use for control algorithms.
//   - GetTelemetry()  → operational state + controller outputs. Use for logging/monitoring.
//   - No Rigidbody ground truth in either path.

using System;
using UnityEngine;
using SimCore;
using SimCore.Common;
using DroneCore.Controllers;
using DroneCore.Core;
using DroneCore.Interfaces;
using MathUtil;
using RobotCore;
using QuadSim.RL;

namespace DroneCore.Api
{
    /// <summary>
    /// Per-drone API facade. Primary interface for Internal scripts and External RPC.
    /// Attach to the same GameObject as QuadPawn (added automatically by DroneRootBootstrap).
    ///
    /// Access patterns:
    /// <code>
    ///   // Get the currently selected drone
    ///   var drone = QuadSimApi.Selected();
    ///
    ///   // Get drone by index (multi-drone)
    ///   var drone0 = QuadSimApi.Get(0);
    ///   var drone1 = QuadSimApi.Get(1);
    ///
    ///   // Use the API
    ///   drone.SetMode(GoalMode.Rate);
    ///   drone.SendCommand(InputSource.Internal, new Axis4(0, 0, 5, 0.41f));
    ///   var sensors = drone.GetSensorData();
    /// </code>
    /// </summary>
    [DisallowMultipleComponent]
    public sealed class QuadSimApi : MonoBehaviour
    {
        // ================================================================================
        // Static Convenience Accessors
        // ================================================================================

        /// <summary>
        /// Get the QuadSimApi for the currently selected drone.
        /// Returns null if no drone is selected or not yet initialized.
        /// </summary>
        public static QuadSimApi Selected()
        {
            var dm = DroneManager.Get();
            return dm?.SelectedDrone?.Api;
        }

        /// <summary>
        /// Get the QuadSimApi for a drone by index.
        /// Returns null if index is out of range or drone hasn't initialized.
        /// </summary>
```

```
public static QuadSimApi Get(int index)
{
    var dm = DroneManager.Get();
    if (dm == null) return null;

    var drones = dm.RegisteredDrones;
    if (index < 0 || index >= drones.Count) return null;

    return drones[index]?.Api;
}

/// <summary>
/// Number of registered drones. Convenience wrapper around DroneManager.
/// </summary>
public static int DroneCount
{
    get
    {
        var dm = DroneManager.Get();
        return dm?.DroneCount ?? 0;
    }
}

// ================================================================================
// References (set during Initialize)
// ================================================================================

private QuadPawn _pawn;
private FlightCommandProxy _proxy;
private ModeCoordinator _modeCoordinator;
private SensorManager _sensorManager;
private CascadedController _controller;

private bool _initialized;

// ================================================================================
// Public Accessors
// ================================================================================

/// <summary>Has this API been initialized with valid references?</summary>
public bool IsInitialized => _initialized;

/// <summary>The drone's unique identifier.</summary>
public string DroneID => _pawn != null ? _pawn.DroneID : name;

/// <summary>Current GoalMode from ModeCoordinator.</summary>
public GoalMode CurrentMode =>
    _modeCoordinator != null ? _modeCoordinator.ActiveMode : GoalMode.None;

/// <summary>Current ControllerKind from ModeCoordinator.</summary>
public ControllerKind CurrentController =>
    _modeCoordinator != null ? _modeCoordinator.ActiveController : ControllerKind.Cascade;

/// <summary>Direct access to the underlying QuadPawn (advanced use only).</summary>
public QuadPawn Pawn => _pawn;

/// <summary>Direct access to the command proxy (advanced use only).</summary>
public FlightCommandProxy CommandProxy => _proxy;

/// <summary>Direct access to the mode coordinator (advanced use only).</summary>
public ModeCoordinator ModeCoord => _modeCoordinator;

// ================================================================================
// Events (forwarded from subsystems)
// ================================================================================

/// <summary>Fired when GoalMode changes on this drone.</summary>
public event Action<GoalMode, GoalMode> OnModeChanged;

/// <summary>Fired when ControllerKind changes on this drone.</summary>
public event Action<ControllerKind, ControllerKind> OnControllerChanged;

// ================================================================================
```

```
// Initialization
// ============================================================================

/// <summary>
/// Initialize with references to the drone's subsystems.
/// Called by QuadPawn during InitializeSubsystems().
/// </summary>
public void Initialize(QuadPawn pawn, FlightCommandProxy proxy,
                       ModeCoordinator modeCoordinator, SensorManager sensorManager)
{
    if (_initialized)
    {
        Debug.LogWarning($"[QuadSimApi] {name}: Already initialized. Ignoring duplicate init.");
        return;
    }

    _pawn = pawn;
    _proxy = proxy;
    _modeCoordinator = modeCoordinator;
    _sensorManager = sensorManager;

    _controller = _pawn != null ? _pawn.Controller : null;

    if (_modeCoordinator != null)
    {
        _modeCoordinator.OnModeChanged += HandleModeChanged;
        _modeCoordinator.OnControllerChanged += HandleControllerChanged;
    }

    _initialized = true;

    Debug.Log($"[QuadSimApi] {DroneID}: Initialized " +
              $"(pawn={_pawn != null}, proxy={_proxy != null}, " +
              $"modeCoord={_modeCoordinator != null}, sensors={_sensorManager != null})");
}

private void OnDestroy()
{
    if (_modeCoordinator != null)
    {
        _modeCoordinator.OnModeChanged -= HandleModeChanged;
        _modeCoordinator.OnControllerChanged -= HandleControllerChanged;
    }
}

// ============================================================================
// Mode & Controller Control
// ============================================================================

/// <summary>
/// Set the active GoalMode for this drone.
/// Triggers ModeCoordinator reset policies (PID integrator reset, goal clear).
/// </summary>
public bool SetMode(GoalMode mode)
{
    if (!AssertInitialized("SetMode")) return false;

    if (_modeCoordinator == null)
    {
        Debug.LogError($"[QuadSimApi] {DroneID}: No ModeCoordinator — cannot set mode.");
        return false;
    }

    if (_modeCoordinator.ActiveMode == mode)
        return false;

    _modeCoordinator.SetMode(mode);
    return true;
}

/// <summary>
/// Set the active controller type for this drone.
/// Triggers a full reset (integrators + dynamics) per ModeCoordinator reset policy.
```

```
/// </summary>
public bool SetController(ControllerKind kind)
{
    if (!AssertInitialized("SetController")) return false;

    if (_modeCoordinator == null)
    {
        Debug.LogError($"[QuadSimApi] {DroneID}: No ModeCoordinator — cannot set controller.");
        return false;
    }

    if (_modeCoordinator.ActiveController == kind)
        return false;

    _modeCoordinator.SetController(kind);
    return true;
}

// ================================================================================
// Command Sending
// ================================================================================

/// <summary>
/// Send a command to this drone. Uses the current ActiveMode from ModeCoordinator.
/// The proxy validates the caller against the current authority.
/// </summary>
public bool SendCommand(InputSource source, Axis4 value)
{
    if (!AssertInitialized("SendCommand")) return false;

    if (_proxy == null)
    {
        Debug.LogError($"[QuadSimApi] {DroneID}: No FlightCommandProxy — cannot send command.");
        return false;
    }

    GoalMode mode = _modeCoordinator != null ? _modeCoordinator.ActiveMode : GoalMode.Rate;
    _proxy.SetCommand(value, mode, source);
    return true;
}

/// <summary>
/// Send a command with an explicit GoalMode override.
/// Sets the mode via ModeCoordinator first, then sends the command.
/// </summary>
public bool SendCommand(InputSource source, Axis4 value, GoalMode mode)
{
    if (!AssertInitialized("SendCommand")) return false;

    if (_modeCoordinator != null && _modeCoordinator.ActiveMode != mode)
    {
        _modeCoordinator.SetMode(mode);
    }

    if (_proxy == null)
    {
        Debug.LogError($"[QuadSimApi] {DroneID}: No FlightCommandProxy — cannot send command.");
        return false;
    }

    _proxy.SetCommand(value, mode, source);
    return true;
}

// ================================================================================
// Reset Operations
// ================================================================================

/// <summary>Full drone reset: physics, controller state, command proxy.</summary>
public void ResetAll()
{
    if (!AssertInitialized("ResetAll")) return;
```

```
    _pawn?.ResetDrone();

    if (_proxy != null && _modeCoordinator != null)
    {
        _proxy.ResetToSafe(_modeCoordinator.ActiveMode);
    }

    Debug.Log($"[QuadSimApi] {DroneID}: Full reset");
}

/// <summary>Reset drone to a specific position and rotation.</summary>
public void ResetPose(Vector3 position, Quaternion rotation)
{
    if (!AssertInitialized("ResetPose")) return;

    _pawn?.ResetPosition(position, rotation);

    if (_proxy != null && _modeCoordinator != null)
    {
        _proxy.ResetToSafe(_modeCoordinator.ActiveMode);
    }

    Debug.Log($"[QuadSimApi] {DroneID}: Pose reset to {position}");
}

public void ResetEpisodeState(
    Vector3 position, Quaternion rotation, long episodeId, int seed, int droneIndex)
{
    if (!AssertInitialized("ResetEpisodeState")) return;

    _sensorManager?.BeginEpisode(episodeId, seed, droneIndex);
    _pawn?.ImageSensor?.ResetSequence();
    _pawn?.ResetPosition(position, rotation);
    _modeCoordinator?.SetMode(GoalMode.None);
    _proxy?.ResetToSafe(GoalMode.None);
}

/// <summary>Level the drone (zero roll/pitch, keep yaw) and zero velocities.</summary>
public void ResetRotation()
{
    if (!AssertInitialized("ResetRotation")) return;
    _pawn?.ResetRotation();
}

/// <summary>Zero all velocities without moving the drone.</summary>
public void ResetPhysics()
{
    if (!AssertInitialized("ResetPhysics")) return;
    _pawn?.ResetPhysics();
}

/// <summary>Reset only controller state (PID integrators, allocator).</summary>
public void ResetController()
{
    if (!AssertInitialized("ResetController")) return;
    _pawn?.ResetControllerState();
}

// ================================================================================
// Sensor Data (Measured State)
// ================================================================================

/// <summary>Set the body frame used by IMU and magnetometer outputs.</summary>
public bool SetSensorOutputFrame(SimFrame frame)
{
    if (!AssertInitialized("SetSensorOutputFrame")) return false;
    return _sensorManager != null && _sensorManager.SetOutputFrame(frame);
}

/// <summary>
/// Get the latest sensor readings (IMU + GPS).
/// This is the measured state — what a real flight controller would see.
/// Use this for control algorithms and sim-to-real workflows.
```

```
/// </summary>
public SensorData GetSensorData()
{
    if (_sensorManager == null) return default;
    return _sensorManager.Latest;
}

public RLAgentState GetRLState()
{
    return RLAgentState.Capture(_pawn);
}

// ============================================================================
// Telemetry (Operational State + Controller Outputs)
// ============================================================================

/// <summary>
/// Get the cascaded controller's internal telemetry (desired values from cascade).
/// </summary>
public CascadedController.CascadeTelemetrySnapshot GetControllerTelemetry()
{
    if (_controller == null) return default;
    return _controller.TelemetrySnapshot;
}

/// <summary>
/// Get a combined telemetry snapshot: operational state + controller outputs.
/// Does NOT include measured state — use GetSensorData() for that.
/// </summary>
public DroneTelemetry GetTelemetry()
{
    var telem = new DroneTelemetry
    {
        DroneID = DroneID,
        Mode = CurrentMode,
        Controller = CurrentController,
        ControllerTelemetry = GetControllerTelemetry(),
        AchievedForceBody = _controller != null
            ? _controller.LastAchievedForceBody : Vector3.zero,
        AchievedTorqueBody = _controller != null
            ? _controller.LastAchievedTorqueBody : Vector3.zero,
        AllocatorRank = _controller != null ? _controller.AllocatorRank : 0,
        AllocatorConditionNumber = _controller != null
            ? _controller.AllocatorConditionNumber : float.PositiveInfinity,
        Allocation = _controller != null ? _controller.LastAllocation : null,
    };

    if (_controller != null && _controller.LastMotor01 != null)
    {
        telem.MotorOutputs = new Motor4
        {
            FL = _controller.LastMotor01[0],
            FR = _controller.LastMotor01[1],
            BL = _controller.LastMotor01[2],
            BR = _controller.LastMotor01[3]
        };
    }
    if (_controller != null && _controller.Thrusts != null)
    {
        telem.MotorThrusts = new Motor4
        {
            FL = _controller.Thrusts[0],
            FR = _controller.Thrusts[1],
            BL = _controller.Thrusts[2],
            BR = _controller.Thrusts[3]
        };
    }

    return telem;
}

// ============================================================================
// Configuration
```

```
        // ================================================================================

        /// <summary>Whether this drone has had its YAML config applied.</summary>
        public bool HasConfig => _pawn != null && _pawn.HasConfigApplied;

        /// <summary>Force a reload of the drone's YAML configuration.</summary>
        public void ReloadConfig()
        {
            if (!AssertInitialized("ReloadConfig")) return;
            _pawn?.ReloadYaml();
        }

        /// <summary>Re-apply the current config without reloading from disk.</summary>
        public void ReapplyConfig()
        {
            if (!AssertInitialized("ReapplyConfig")) return;
            _pawn?.ReapplyConfig();
        }

        // ================================================================================
        // Internal Helpers
        // ================================================================================

        private bool AssertInitialized(string caller)
        {
            if (_initialized) return true;

            Debug.LogWarning($"[QuadSimApi] {name}: {caller}() called before initialization. " +
                             "Ensure QuadPawn.InitializeSubsystems() has run.");
            return false;
        }

        private void HandleModeChanged(GoalMode oldMode, GoalMode newMode) =>
            OnModeChanged?.Invoke(oldMode, newMode);

        private void HandleControllerChanged(ControllerKind oldKind, ControllerKind newKind) =>
            OnControllerChanged?.Invoke(oldKind, newKind);
    }

    // ================================================================================
    // Telemetry Data Struct
    // ================================================================================

    /// <summary>
    /// Combined telemetry snapshot for a single drone.
    /// Captures operational state and controller outputs — NOT measured state.
    /// For measured state (IMU, GPS), use QuadSimApi.GetSensorData().
    /// </summary>
    public struct DroneTelemetry
    {
        public string DroneID;
        public GoalMode Mode;
        public ControllerKind Controller;
        public Motor4 MotorOutputs;
        public Motor4 MotorThrusts;
        public CascadedController.CascadeTelemetrySnapshot ControllerTelemetry;
        public Vector3 AchievedForceBody;
        public Vector3 AchievedTorqueBody;
        public int AllocatorRank;
        public float AllocatorConditionNumber;
        public DroneCore.Controls.AllocationDiagnostics Allocation;
    }
}
```

Runtime/Scripts/DroneCore/Core/DroneManager.cs

SHA-256 b7b80cfb80fc747b9c62bbe8c98c15beabe0703e14816d0fffe263069f73de9c | 670 lines | 25,924 bytes

```
    using System.Collections.Generic;
    using DroneCore.CameraSystem;
using Experimental;
using MathUtil;
using SimCore;
using SimCore.Common;
```

```
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Core
{
    public enum StartupSpawnPolicy
    {
        SpawnDefaultIfNone = 0,
        SceneDronesOnly = 1,
    }

    /// <summary>
    /// DroneManager is the central registry and orchestrator for all drones in the simulation.
    /// It handles spawning, selection, configuration, and drives the physics step pipeline.
    ///
    /// Implements ISimulatable so SimulationManager drives the loop:
    ///   PrePhysicsStep  → controllers and body wrenches
    ///   PostPhysicsStep → post-step sensor samples
    ///   Update()        → visual updates (prop spin, effects)
    /// </summary>
    ///
    [DefaultExecutionOrder(-500)]
    public sealed class DroneManager : MonoBehaviour, ISimulatable
    {
        // ================================================================================
        // Serialized Fields
        // ================================================================================

        [Header("Spawning")]
        [Tooltip("Choose whether simulation startup may create a default drone when the scene contains n
one.")]
        [SerializeField] private StartupSpawnPolicy startupSpawnPolicy = StartupSpawnPolicy.SpawnDefault
IfNone;

        [Tooltip("Prefab to instantiate when spawning drones.")]
        [SerializeField] private QuadPawn quadPawnPrefab;

        [Tooltip("Default spawn location. Set in inspector or override via SpawnOrigin property.")]
        [SerializeField] private Vector3 defaultSpawnOrigin = new Vector3(0f, 1f, 0f);

        [Tooltip("Offset between multiple spawned drones.")]
        [SerializeField] private float spawnOffset = 1.5f;

        [Header("Configuration")]
        [Tooltip("Default config name to load.")]
        [SerializeField] private string selectedConfigName = "standard_quad";

        [Header("Core References")]
        [Tooltip("SimulationManager reference. Set in inspector, fallback to scene find.")]
        [SerializeField] private SimulationManager simManager;

        [Header("Swarm")]
        [Tooltip("Enable swarm mode (all drones receive same commands).")]
        [SerializeField] private bool swarmMode = false;

        [Header("Payload")]
        [SerializeField] private PayloadAttachment payloadPrefab;
        [SerializeField] private bool spawnPayload = false;
        [SerializeField] private string payloadMountPath = "Bottle";

        [Header("Scene Env")]
        [SerializeField] private int     masterSeed       = 12345;
        [SerializeField] private bool    enableSensorNoise = false;
        [SerializeField] private double  originLat        = 30.4636;
        [SerializeField] private double  originLon        = -86.5533;
        [SerializeField] private double  originAlt        = 0.0;
        [SerializeField] private SimFrame frame = SimFrame.FLU;
        [SerializeField] private GoalMode startingMode = GoalMode.Rate;
        // ================================================================================
        // Runtime State
        // ================================================================================

        private readonly List<QuadPawn> _registeredDrones = new List<QuadPawn>();
```

```
        private int _selectedDroneIndex = 0;
        private DroneConfig _currentConfig;
        private bool _hasCurrentConfig;

        // ================================================================================
        // ISimulatable Implementation
        // ================================================================================

        public int ExecutionOrder => 10;
        public string DebugName => "DroneManager";

        // ================================================================================
        // Public Accessors
        // ================================================================================

        public IReadOnlyList<QuadPawn> RegisteredDrones => _registeredDrones;
        public int DroneCount => _registeredDrones.Count;
        public int SelectedDroneIndex => _selectedDroneIndex;
        public string SelectedConfigName => selectedConfigName;
        public bool SwarmMode { get => swarmMode; set => swarmMode = value; }
        public Vector3 SpawnOrigin { get => defaultSpawnOrigin; set => defaultSpawnOrigin = value; }
        public StartupSpawnPolicy SpawnPolicy => startupSpawnPolicy;

        /// <summary>
        /// Currently selected drone, or null.
        /// </summary>
        public QuadPawn SelectedDrone
        {
            get
            {
                if (_selectedDroneIndex >= 0 && _selectedDroneIndex < _registeredDrones.Count)
                    return _registeredDrones[_selectedDroneIndex];
                return null;
            }
        }

        // ================================================================================
        // Static Accessor
        // ================================================================================

        private static DroneManager _instance;

        /// <summary>
        /// Get cached DroneManager instance. Falls back to scene find with warning.
        /// </summary>
        public static DroneManager Get()
        {
            if (_instance != null) return _instance;

            _instance = FindAnyObjectByType<DroneManager>();
            if (_instance != null)
                Debug.LogWarning("[DroneManager] Instance resolved via FindAnyObjectByType. Prefer direc
t reference.");

            return _instance;
        }

        // ================================================================================
        // Unity Lifecycle
        // ================================================================================

        private void Awake()
        {
            _instance = this;

            string savedConfig = PlayerPrefs.GetString("QuadSim.DefaultConfig", "");
            if (!string.IsNullOrEmpty(savedConfig))
                selectedConfigName = savedConfig;

            Debug.Log($"[DroneManager] Awake: selectedConfig={selectedConfigName}");

            // Register with SimManager during Awake so we're in the list
            // before SimulationManager.Start() fires OnSimulationStart
```

```
            if (simManager == null)
            {
                simManager = FindAnyObjectByType<SimulationManager>();
                if (simManager != null)
                    Debug.LogWarning("[DroneManager] SimulationManager resolved via find. Prefer inspect
or assignment.");
            }
            if (simManager != null)
            {
                simManager.RegisterManagers(this);
                Debug.Log("[DroneManager] Registered with SimulationManager");
            }
        }

        private void Start()
        {
        }

        private void OnDestroy()
        {
            if (simManager != null)
            {
                simManager.UnRegisterManager(this);
            }

            if (_instance == this)
            {
                _instance = null;
            }
        }

        // ================================================================================
        // ISimulatable — Simulation Lifecycle
        // ================================================================================

        public void OnSimulationStart(SimulationManager sim)
        {
            simManager = sim;

            _hasCurrentConfig = DroneConfigLoader.LoadConfig(selectedConfigName, out _currentConfig);
            if (_hasCurrentConfig)
                Debug.Log($"[DroneManager] Loaded config: {selectedConfigName}");
            else
                Debug.LogWarning($"[DroneManager] Failed to load config: {selectedConfigName}");

            if (_registeredDrones.Count == 0 && startupSpawnPolicy == StartupSpawnPolicy.SpawnDefaultIfN
one)
            {
                Debug.Log("[DroneManager] Startup policy is SpawnDefaultIfNone; spawning default drone."
);
                SpawnDroneWithConfig(selectedConfigName, defaultSpawnOrigin, Quaternion.identity);
            }
            else if (_registeredDrones.Count == 0)
            {
                Debug.Log("[DroneManager] Startup policy is SceneDronesOnly; starting without a drone.")
;
            }
            else
            {
                foreach (var d in _registeredDrones)
                {
                    if (d == null) continue;

                    if (d.Sensors != null)
                        d.Sensors.Configure(masterSeed, enableSensorNoise, originLat, originLon, originA
lt, frame);

                    if (_hasCurrentConfig && !d.HasConfigApplied)
                        d.ApplyConfig(_currentConfig);

                    if (d.Api != null && d.Api.IsInitialized)
                        d.Api.SetMode(startingMode);
                }
```

```
    }
}

public void OnSimulationReset(SimulationManager sim)
{
    Debug.Log("[DroneManager] OnSimulationReset");
    ResetAllDrones();
}

/// <summary>
/// Drive the physics step for all drones.
/// SimulationManager calls this BEFORE Physics.Simulate().
/// Each QuadPawn runs its controller and applies the next body wrench.
/// </summary>
public void PrePhysicsStep(double dtSec, long nowNanos)
{
    CleanupInvalidDrones();

    for (int i = 0; i < _registeredDrones.Count; i++)
    {
        QuadPawn drone = _registeredDrones[i];
        if (drone != null)
        {
            drone.PrePhysicsStep(dtSec, nowNanos);
        }
    }
}

public void PostPhysicsStep(double dtSec, long nowNanos)
{
    for (int i = 0; i < _registeredDrones.Count; i++)
    {
        QuadPawn drone = _registeredDrones[i];
        if (drone != null)
            drone.PostPhysicsStep(dtSec, nowNanos);
    }
}

// ============================================================================
// Visual Updates — Called by Unity Update (rendering frame)
// ============================================================================

private void Update()
{
    float dt = Time.deltaTime;
    for (int i = 0; i < _registeredDrones.Count; i++)
    {
        QuadPawn drone = _registeredDrones[i];
        if (drone != null)
        {
            drone.UpdateVisuals(dt);
        }
    }
}

// ============================================================================
// Spawning
// ============================================================================

/// <summary>
/// Spawn a drone using the currently selected config.
/// </summary>
public QuadPawn SpawnDrone(Vector3 position, Quaternion rotation)
{
    return SpawnDroneWithConfig(selectedConfigName, position, rotation);
}

/// <summary>
/// Spawn a drone with a specific config name.
/// </summary>
public QuadPawn SpawnDroneWithConfig(string configName, Vector3 position, Quaternion rotation)
{
    if (quadPawnPrefab == null)
```

```
            {
                Debug.LogError("[DroneManager] Cannot spawn: QuadPawnPrefab not set!");
                return null;
            }

            // Load config
            if (!DroneConfigLoader.LoadConfig(configName, out DroneConfig config))
            {
                Debug.LogError($"[DroneManager] Failed to load config: {configName}");
                return null;
            }

            // Determine spawn position
            Vector3 spawnPos = DetermineSpawnPosition(position);
            Quaternion spawnRot = rotation == Quaternion.identity ? DetermineSpawnRotation() : rotation;

            int droneNumber = _registeredDrones.Count;

            // Instantiate
            QuadPawn newDrone = Instantiate(quadPawnPrefab, spawnPos, spawnRot);
            if (newDrone == null)
            {
                Debug.LogError("[DroneManager] Failed to instantiate QuadPawn prefab");
                return null;
            }

            newDrone.name = $"Drone_{droneNumber:D2}";

            if (newDrone.Sensors != null)
                newDrone.Sensors.Configure(masterSeed, enableSensorNoise, originLat, originLon, originAl
t,frame);

            // Inject manager reference BEFORE Start() runs
            newDrone.SetDroneManager(this);

            // Apply config (triggers QuadPawn initialization)
            newDrone.ApplyConfig(config);

            // The scene config owns the initial cascade entry point. Apply it
            // to the instantiated pawn, after ModeCoordinator/QuadSimApi init.
            if (newDrone.Api != null && newDrone.Api.IsInitialized)
                newDrone.Api.SetMode(startingMode);

            if (spawnPayload && payloadPrefab != null)
            {
                Transform mount = newDrone.transform.Find(payloadMountPath);

                if (mount == null)
                {
                    Debug.LogError(
                        $"[DroneManager] Payload mount '{payloadMountPath}' " +
                        $"not found under {newDrone.name}.");
                }
                else
                {
                    PayloadAttachment payload = Instantiate(
                        payloadPrefab,
                        mount.position,
                        mount.rotation);

                    payload.name = $"{newDrone.name}_Payload";
                    payload.AttachTo(newDrone.Rigidbody, mount);
                }
            }

            // Select the new drone
            int newIndex = GetDroneIndex(newDrone);
            if (newIndex >= 0)
            {
                SelectDroneByIndex(newIndex, true);
            }

            Debug.Log($"[DroneManager] Spawned drone: {newDrone.name} at {spawnPos}");
```

```
    return newDrone;
}

private Vector3 DetermineSpawnPosition(Vector3 requestedPosition)
{
    if (requestedPosition != Vector3.zero)
        return requestedPosition;

    // Stack next to last drone
    if (_registeredDrones.Count > 0)
    {
        QuadPawn last = _registeredDrones[_registeredDrones.Count - 1];
        if (last != null)
            return last.transform.position + last.transform.right * spawnOffset;
    }

    return defaultSpawnOrigin;
}

private Quaternion DetermineSpawnRotation()
{
    if (_registeredDrones.Count > 0)
    {
        QuadPawn last = _registeredDrones[_registeredDrones.Count - 1];
        if (last != null)
            return last.transform.rotation;
    }
    return Quaternion.identity;
}

// ============================================================================
// Registration
// ============================================================================

public void RegisterDrone(QuadPawn drone)
{
    if (drone == null) return;

    if (_registeredDrones.Contains(drone))
    {
        Debug.LogWarning($"[DroneManager] Drone already registered: {drone.name}");
        return;
    }

    drone.EnsureAgentIdentity(GetNextAvailableDroneIndex());

    for (int i = 0; i < _registeredDrones.Count; i++)
    {
        QuadPawn existing = _registeredDrones[i];
        if (existing == null) continue;

        if (existing.DroneIndex == drone.DroneIndex)
        {
            Debug.LogError($"[DroneManager] Duplicate drone_index {drone.DroneIndex}; " +
                           $"cannot register {drone.name}.");
            return;
        }

        if (string.Equals(existing.AgentId, drone.AgentId, System.StringComparison.Ordinal))
        {
            Debug.LogError($"[DroneManager] Duplicate agent_id '{drone.AgentId}'; " +
                           $"cannot register {drone.name}.");
            return;
        }
    }

    bool wasEmpty = _registeredDrones.Count == 0;
    QuadPawn selectedBefore = SelectedDrone;
    _registeredDrones.Add(drone);
    _registeredDrones.Sort(CompareDroneIdentity);

    Debug.Log($"[DroneManager] Registered {drone.AgentId} " +
              $"(index={drone.DroneIndex}, team={drone.TeamId}, total={_registeredDrones.Count})
```

```
");

            if (wasEmpty)
            {
                SelectDroneByIndex(0, true);
            }
            else if (selectedBefore != null)
            {
                _selectedDroneIndex = _registeredDrones.IndexOf(selectedBefore);
            }
        }

        public void UnregisterDrone(QuadPawn drone)
        {
            if (drone == null) return;

            int index = _registeredDrones.IndexOf(drone);
            if (index < 0) return;

            _registeredDrones.RemoveAt(index);

            if (_selectedDroneIndex >= _registeredDrones.Count)
            {
                _selectedDroneIndex = Mathf.Max(0, _registeredDrones.Count - 1);
            }

            Debug.Log($"[DroneManager] Unregistered drone: {drone.name} (remaining: {_registeredDrones.C
ount})");
        }

        // ================================================================================
        // Selection
        // ================================================================================

        public int GetDroneIndex(QuadPawn drone) => _registeredDrones.IndexOf(drone);

        public QuadPawn GetDroneByAgentId(string agentId)
        {
            if (string.IsNullOrEmpty(agentId)) return null;

            for (int i = 0; i < _registeredDrones.Count; i++)
            {
                QuadPawn drone = _registeredDrones[i];
                if (drone != null && string.Equals(drone.AgentId, agentId, System.StringComparison.Ordin
al))
                    return drone;
            }
            return null;
        }

        public QuadPawn GetDroneByPersistentIndex(int droneIndex)
        {
            for (int i = 0; i < _registeredDrones.Count; i++)
            {
                QuadPawn drone = _registeredDrones[i];
                if (drone != null && drone.DroneIndex == droneIndex)
                    return drone;
            }
            return null;
        }

        internal void RefreshIdentityOrder()
        {
            QuadPawn selected = SelectedDrone;
            _registeredDrones.Sort(CompareDroneIdentity);
            if (selected != null)
                _selectedDroneIndex = _registeredDrones.IndexOf(selected);
        }

        public void SelectDroneByIndex(int index, bool alsoPossess = false)
        {
            if (index < 0 || index >= _registeredDrones.Count) return;
```

```
        QuadPawn target = _registeredDrones[index];
        if (target == null) return;

        _selectedDroneIndex = index;

        // Activate only the selected drone's camera; keep its current camera mode.
        if (alsoPossess)
        {
            CameraDirector director = CameraDirector.Get();
            if (director != null && director.ViewCamera != null)
            {
                director.Bind(target);
            }
            else
            {
                for (int i = 0; i < _registeredDrones.Count; i++)
                {
                    if (_registeredDrones[i] != null)
                        _registeredDrones[i].SetCameraActive(i == _selectedDroneIndex);
                }
            }
        }

        Debug.Log($"[DroneManager] Selected drone: {target.name} (index {index})");
    }

    public void SelectNextDrone()
    {
        if (_registeredDrones.Count == 0) return;
        int next = (_selectedDroneIndex + 1) % _registeredDrones.Count;
        SelectDroneByIndex(next, true);
    }

    public void SelectPreviousDrone()
    {
        if (_registeredDrones.Count == 0) return;
        int prev = (_selectedDroneIndex - 1 + _registeredDrones.Count) % _registeredDrones.Count;
        SelectDroneByIndex(prev, true);
    }

    // ================================================================================
    // Configuration
    // ================================================================================

    public void SetSelectedConfig(string configName)
    {
        if (string.IsNullOrEmpty(configName)) return;
        selectedConfigName = configName;
        Debug.Log($"[DroneManager] Selected config: {configName}");
    }

    public void SetSceneEnv(int seed, bool noise, double lat, double lon, double alt,
        StartupSpawnPolicy _spawnPolicy, QuadPawn _quadSimPrefab,
        PayloadAttachment _payloadPrefab, bool spawnPayloads,
        string payloadMount, SimFrame _frame, GoalMode _startingMode)
    {
        masterSeed = seed;
        enableSensorNoise = noise;
        originLat = lat;
        originLon = lon;
        originAlt = alt;
        startupSpawnPolicy = _spawnPolicy;
        quadPawnPrefab = _quadSimPrefab;
        payloadPrefab = _payloadPrefab;
        spawnPayload =  spawnPayloads;
        payloadMountPath = payloadMount;
        frame = _frame;
        startingMode = _startingMode;
    }

    public List<string> GetAvailableConfigs()
    {
        return DroneConfigLoader.GetAvailableConfigs();
```

```
}

public void SaveCurrentConfigAsDefault()
{
    if (string.IsNullOrEmpty(selectedConfigName)) return;
    PlayerPrefs.SetString("QuadSim.DefaultConfig", selectedConfigName);
    PlayerPrefs.Save();
    Debug.Log($"[DroneManager] Saved default config: {selectedConfigName}");
}

public void ReloadCurrentDrone(bool resetPosition = true)
{
    QuadPawn oldDrone = SelectedDrone;
    if (oldDrone == null)
    {
        Debug.LogWarning("[DroneManager] No drone selected to reload");
        return;
    }

    Vector3 pos = resetPosition ? defaultSpawnOrigin : oldDrone.transform.position;
    Quaternion rot = resetPosition ? Quaternion.identity : oldDrone.transform.rotation;

    Destroy(oldDrone.gameObject);
    CleanupInvalidDrones();

    SpawnDroneWithConfig(selectedConfigName, pos, rot);
}

// ============================================================================
// Reset
// ============================================================================

public void ResetAllDrones()
{
    for (int i = 0; i < _registeredDrones.Count; i++)
    {
        QuadPawn drone = _registeredDrones[i];
        if (drone == null) continue;

        Vector3 pos = defaultSpawnOrigin + Vector3.right * (i * spawnOffset);
        drone.ResetPosition(pos, Quaternion.identity);
    }

    Debug.Log($"[DroneManager] Reset {_registeredDrones.Count} drones");
}

public void ResetSelectedDrone()
{
    QuadPawn drone = SelectedDrone;
    if (drone == null) return;
    drone.ResetPosition(defaultSpawnOrigin, Quaternion.identity);
}

// ============================================================================
// Utility
// ============================================================================

private int GetNextAvailableDroneIndex()
{
    int candidate = 0;
    while (GetDroneByPersistentIndex(candidate) != null)
        candidate++;
    return candidate;
}

private static int CompareDroneIdentity(QuadPawn a, QuadPawn b)
{
    if (ReferenceEquals(a, b)) return 0;
    if (a == null) return 1;
    if (b == null) return -1;

    int indexOrder = a.DroneIndex.CompareTo(b.DroneIndex);
    return indexOrder != 0
```

```
                ? indexOrder
                : string.CompareOrdinal(a.AgentId, b.AgentId);
        }

        private void CleanupInvalidDrones()
        {
            for (int i = _registeredDrones.Count - 1; i >= 0; i--)
            {
                if (_registeredDrones[i] == null)
                    _registeredDrones.RemoveAt(i);
            }
        }

        public string GetStateJson()
        {
            return $"{{\"drone_count\": {_registeredDrones.Count}, \"selected_index\": {_selectedDroneIn
dex}, \"swarm_mode\": {swarmMode.ToString().ToLower()}, \"config\": \"{selectedConfigName}\"}}";
        }
    }
}
```

Runtime/Scripts/DroneCore/Core/ModeCoordinator.cs

SHA-256 ffc376851b44b0c639069426144949a974e4fe9e0dfb4e662d841eb7aae8a58e | 323 lines | 13,549 bytes

```
// Assets/Scripts/DroneCore/Core/ModeCoordinator.cs
// Phase 4: Per-Drone Mode & Controller Switching
//
// PURPOSE:
//   Single source of truth for which GoalMode and ControllerKind are active on this drone.
//   Handles reset policies when mode, controller, or authority changes.
//   Emits events so UI and other systems can react.
//
// OWNERSHIP:
//   - ActiveMode (GoalMode): what stage of the cascade to enter at
//   - ActiveController (ControllerKind): which controller implementation runs
//   - Reset policy execution on transitions
//
// DOES NOT:
//   - Run controllers (that's QuadPawn.RunController)
//   - Store or validate commands (that's FlightCommandProxy)
//   - Manage authority (that's ControlAuthorityManager)
//
// RESET POLICIES (from API plan):
//   Mode switch    → Reset PID integrators + clear goals. Do NOT reset dynamics.
//   Authority switch → Reset PID integrators + clear goals. Optionally reset dynamics (default: yes for
 External).
//   Controller switch → Reset everything (integrators + dynamics).

using System;
using UnityEngine;
using SimCore;
using SimCore.Common;
using DroneCore.Interfaces;

namespace DroneCore.Core
{
    /// <summary>
    /// Per-drone component that owns mode/controller state and executes reset policies.
    /// Attach to the same GameObject as QuadPawn.
    ///
    /// ModeCoordinator writes the GoalMode into the FlightCommandProxy so the controller
    /// reads it each tick. This replaces the previous pattern where ControlDeckController
    /// or external callers set the mode directly on the proxy alongside commands.
    /// </summary>
    [DisallowMultipleComponent]
    public sealed class ModeCoordinator : MonoBehaviour
    {
        // ================================================================================
        // Configuration
        // ================================================================================

        [Header("Defaults")]
        [Tooltip("Starting GoalMode for this drone.")]
```

```
    [SerializeField] private GoalMode startingMode = GoalMode.Rate;

    [Tooltip("Starting controller for this drone.")]
    [SerializeField] private ControllerKind startingController = ControllerKind.Cascade;

    [Header("Reset Policy")]
    [Tooltip("Reset drone dynamics (position/velocity) on authority switch to External.")]
    [SerializeField] private bool resetDynamicsOnExternalAuthority = true;

    [Tooltip("Reset drone dynamics on any authority switch (not just External).")]
    [SerializeField] private bool resetDynamicsOnAnyAuthoritySwitch = false;

    // ================================================================================
    // Events
    // ================================================================================

    /// <summary>Fired when GoalMode changes. Includes old and new mode.</summary>
    public event Action<GoalMode, GoalMode> OnModeChanged;

    /// <summary>Fired when ControllerKind changes. Includes old and new kind.</summary>
    public event Action<ControllerKind, ControllerKind> OnControllerChanged;

    // ================================================================================
    // Runtime State
    // ================================================================================

    private GoalMode _activeMode;
    private ControllerKind _activeController;
    private bool _initialized;

    // References (set during Initialize)
    private QuadPawn _pawn;
    private FlightCommandProxy _proxy;
    private ControlAuthorityManager _authorityManager;

    // ================================================================================
    // Public Accessors
    // ================================================================================

    /// <summary>Current GoalMode for this drone.</summary>
    public GoalMode ActiveMode => _activeMode;

    /// <summary>Current ControllerKind for this drone.</summary>
    public ControllerKind ActiveController => _activeController;

    /// <summary>Has this coordinator been initialized?</summary>
    public bool IsInitialized => _initialized;

    // ================================================================================
    // Initialization
    // ================================================================================

    /// <summary>
    /// Initialize with references to the drone's subsystems.
    /// Called by QuadPawn during InitializeSubsystems().
    /// </summary>
    public void Initialize(QuadPawn pawn, FlightCommandProxy proxy)
    {
        _pawn = pawn;
        _proxy = proxy;

        // Resolve authority manager
        _authorityManager = ControlAuthorityManager.Get();

        // Subscribe to authority changes
        if (_authorityManager != null)
        {
            _authorityManager.OnAuthorityChanged += HandleAuthorityChanged;
        }
        else
        {
            Debug.LogWarning($"[ModeCoordinator] {name}: No ControlAuthorityManager found. " +
                             "Authority-triggered resets will not work.");
```

```
            }

            // Set initial state
            _activeMode = startingMode;
            _activeController = startingController;

            // Push initial mode to proxy so controller sees it on first tick
            if (_proxy != null)
            {
                _proxy.ResetToSafe(_activeMode);
            }

            _initialized = true;
            Debug.Log($"[ModeCoordinator] {name}: Initialized (mode={_activeMode}, controller={_activeCo
ntroller})");
        }

        // ========================================================================
        // Mode Switching
        // ========================================================================

        /// <summary>
        /// Switch GoalMode. No-op if already in the requested mode.
        /// Resets PID integrators and clears goals. Does NOT reset dynamics.
        /// </summary>
        public bool SetMode(GoalMode newMode)
        {
            if (!_initialized)
            {
                Debug.LogWarning($"[ModeCoordinator] {name}: SetMode called before initialization.");
                return false;
            }

            if (newMode == _activeMode)
            {
                return true; // No-op, already in this mode
            }

            // Validate: Geometric controller ignores GoalModes
            if (_activeController == ControllerKind.Geometric &&
                newMode != GoalMode.None)
            {
                Debug.LogWarning($"[ModeCoordinator] {name}: GoalMode switching not applicable " +
                                 $"for Geometric controller. Ignoring SetMode({newMode}).");
                return false;
            }

            var oldMode = _activeMode;
            _activeMode = newMode;

            // Execute reset policy: integrators + goals, NOT dynamics
            ResetControllersOnly();

            // Push new mode to proxy
            if (_proxy != null)
            {
                // Use ForceSet because this is a mode change, not a command from a source.
                // The proxy's mode field must reflect ModeCoordinator's decision.
                var currentAuthority = _authorityManager != null
                    ? _authorityManager.EffectiveAuthority
                    : InputSource.UI;
                _proxy.ForceSetCommand(Axis4.ZeroValue, _activeMode, currentAuthority);
            }

            Debug.Log($"[ModeCoordinator] {name}: Mode {oldMode} -> {newMode}");
            OnModeChanged?.Invoke(oldMode, newMode);

            return true;
        }

        // ========================================================================
        // Controller Switching
        // ========================================================================
```

```
        /// <summary>
        /// Switch ControllerKind. No-op if already using the requested controller.
        /// Resets everything: integrators, goals, AND dynamics.
        /// </summary>
        public bool SetController(ControllerKind newController)
        {
            if (!_initialized)
            {
                Debug.LogWarning($"[ModeCoordinator] {name}: SetController called before initialization.
");
                return false;
            }

            if (newController == _activeController)
            {
                return true; // No-op
            }

            var oldController = _activeController;
            _activeController = newController;

            // Controller switch: full reset (integrators + dynamics)
            ResetAll();

            // If switching to Geometric, mode becomes irrelevant
            // If switching to Cascade, keep current mode
            if (newController == ControllerKind.Geometric)
            {
                _activeMode = GoalMode.None;
            }

            Debug.Log($"[ModeCoordinator] {name}: Controller {oldController} -> {newController}");
            OnControllerChanged?.Invoke(oldController, newController);

            return true;
        }

        // ================================================================================
        // Authority Change Handler
        // ================================================================================

        /// <summary>
        /// Called when global authority changes. Executes reset policy.
        /// </summary>
        private void HandleAuthorityChanged(InputSource oldSource, InputSource newSource)
        {
            if (!_initialized) return;

            Debug.Log($"[ModeCoordinator] {name}: Authority changed {oldSource} -> {newSource}. Executin
g reset policy.");

            // Always reset integrators and goals on authority switch
            ResetControllersOnly();

            // Optionally reset dynamics
            bool shouldResetDynamics = resetDynamicsOnAnyAuthoritySwitch ||
                                       (resetDynamicsOnExternalAuthority && newSource == InputSource.Ext
ernal);

            if (shouldResetDynamics)
            {
                if (_pawn != null)
                {
                    _pawn.ResetPhysics();
                    Debug.Log($"[ModeCoordinator] {name}: Dynamics reset (authority -> {newSource})");
                }
            }

            // Update proxy's active source to match new authority
            if (_proxy != null)
            {
                _proxy.SetActiveSource(newSource);
```

```
            }
        }

        // ================================================================================
        // Reset Helpers
        // ================================================================================

        /// <summary>
        /// Reset PID integrators and clear goals. Does NOT reset drone dynamics.
        /// Used for mode switches and authority switches.
        /// </summary>
        private void ResetControllersOnly()
        {
            if (_pawn != null)
            {
                _pawn.ResetControllerState();
            }
        }

        /// <summary>
        /// Full reset: integrators, goals, AND dynamics.
        /// Used for controller switches.
        /// </summary>
        private void ResetAll()
        {
            if (_pawn != null)
            {
                _pawn.ResetDrone();
            }
        }

        // ================================================================================
        // Query Helpers
        // ================================================================================

        /// <summary>Is the cascade controller active?</summary>
        public bool IsCascade => _activeController == ControllerKind.Cascade;

        /// <summary>Is the geometric controller active?</summary>
        public bool IsGeometric => _activeController == ControllerKind.Geometric;

        /// <summary>Is the drone in a mode where it should be actively flying?</summary>
        public bool IsFlightActive => _activeMode != GoalMode.None;

        /// <summary>Is the drone in a direct control mode (Rate or Passthrough)?</summary>
        public bool IsDirectControl => _activeMode == GoalMode.Rate || _activeMode == GoalMode.Passthrou
gh;

        // ================================================================================
        // Cleanup
        // ================================================================================

        private void OnDestroy()
        {
            if (_authorityManager != null)
            {
                _authorityManager.OnAuthorityChanged -= HandleAuthorityChanged;
            }
        }
    }
}
```

Runtime/Scripts/DroneCore/Interfaces/FlightCommandProxy.cs

SHA-256 3608891a64774340725009c3765ee3adef7f1e440e68ad0325addb76cde64496 | 257 lines | 11,135 bytes

```
// Assets/Scripts/DroneCore/Interfaces/FlightCommandProxy.cs
// Phase 10 Polish — Stale Command Log Throttle
//
// PURPOSE:
//   Stores the winning command for the current tick.
//   Validates that only the authority source can write.
//   Falls back to safe commands when the authority source goes silent.
//
```

```
// PHASE 10 CHANGE (Step 2a):
//   Added _staleFallbackLogged flag to suppress repeated "Stale command detected"
//   warnings. The warning fires once on first detection, then goes quiet until a
//   fresh command arrives via SetCommand(). This prevents log spam after reset()
//   where ~0.5s of stale ticks fire before the next Python command arrives.

using SimCore;
using SimCore.Common;
using UnityEngine;

namespace DroneCore.Interfaces
{
    /// <summary>
    /// Stores the winning command for the current tick.
    /// Validates that only the authority source can write.
    /// Falls back to safe commands when the authority source goes silent.
    ///
    /// CascadedController reads from this via ICommandSource — that interface is unchanged.
    /// </summary>
    public sealed class FlightCommandProxy : MonoBehaviour, ICommandSource
    {
        // ================================================================================
        // Serialized State (visible in Inspector for debugging)
        // ================================================================================

        [Header("Current Command")]
        [SerializeField] private Axis4 _command;
        [SerializeField] private GoalMode _mode = GoalMode.None;
        [SerializeField] private InputSource _lastWriter = InputSource.UI;

        [Header("Authority")]
        [SerializeField] private InputSource _activeSource = InputSource.UI;

        [Header("Diagnostics")]
        [SerializeField] private double _lastCommandTimeSec;
        [SerializeField] private int _rejectedCount;
        [SerializeField] private bool _usingFallback;

        // ================================================================================
        // Authority Reference
        // ================================================================================

        private ControlAuthorityManager _authorityManager;

        /// <summary>
        /// Inject reference to the authority manager. Called during initialization.
        /// If not set, falls back to ControlAuthorityManager.Get() on first use.
        /// </summary>
        public void SetAuthorityManager(ControlAuthorityManager manager)
        {
            _authorityManager = manager;
        }

        // ================================================================================
        // Stale Command Detection
        // ================================================================================

        [Tooltip("Seconds without a command before falling back to safe. 0 = disabled.")]
        [SerializeField] private float staleCommandThresholdSec = 0.5f;

        /// <summary>
        /// Phase 10 Polish (Step 2a): Tracks whether we've already logged the stale
        /// fallback warning. Prevents log spam when the proxy stays stale for many
        /// ticks (e.g. after reset, before the next Python command arrives).
        /// Reset to false in SetCommand() when a fresh command arrives.
        /// </summary>
        private bool _staleFallbackLogged;

        // ================================================================================
        // ICommandSource Implementation (read interface — unchanged for controllers)
        // ================================================================================

        public Axis4 GetCommandValue()
```

```
        {
            CheckForStaleCommand();
            return _command;
        }

        public GoalMode GetGoalMode() => _mode;
        public InputSource GetActiveSource() => _activeSource;

        // ================================================================================
        // Extended Read Interface
        // ================================================================================

        /// <summary>Who last successfully wrote a command.</summary>
        public InputSource GetLastWriter() => _lastWriter;

        /// <summary>Timestamp of the last valid command (sim seconds).</summary>
        public double GetLastCommandTime() => _lastCommandTimeSec;

        /// <summary>How many commands have been rejected since last successful write.</summary>
        public int GetRejectedCount() => _rejectedCount;

        /// <summary>True if the proxy is currently returning safe fallback commands.</summary>
        public bool IsUsingFallback() => _usingFallback;

        // ================================================================================
        // Write Interface (Source Validated)
        // ================================================================================

        /// <summary>
        /// Set the current command. Validates that the caller matches the effective authority.
        /// Returns false if rejected (caller is not the authority source).
        /// </summary>
        public bool SetCommand(Axis4 inCmd, GoalMode inMode, InputSource caller)
        {
            // Resolve authority — prefer injected reference, fallback to singleton
            var authority = GetEffectiveAuthority();

            if (caller != authority)
            {
                _rejectedCount++;

                // Throttle warnings to avoid log spam
                if (_rejectedCount <= 3 || _rejectedCount % 100 == 0)
                {
                    Debug.LogWarning($"[Proxy] REJECT #{_rejectedCount} Caller={caller} Authority={autho
rity} " +
                                     $"Mode={inMode} Cmd={inCmd}");
                }
                return false;
            }

            _command = inCmd;
            _mode = inMode;
            _lastWriter = caller;
            _lastCommandTimeSec = GetSimTime();
            _rejectedCount = 0;
            _usingFallback = false;
            _staleFallbackLogged = false;  // Phase 10: Reset log throttle on fresh command

            return true;
        }

        /// <summary>
        /// Bypass authority check. Used internally for:
        ///   - Reset flows (QuadPawn.ResetControllerState)
        ///   - Safe fallback injection
        ///   - Initialization
        /// Should NOT be called by command sources.
        /// </summary>
        public void ForceSetCommand(Axis4 inCmd, GoalMode inMode, InputSource caller)
        {
            _command = inCmd;
            _mode = inMode;
```

```
            _lastWriter = caller;
            _lastCommandTimeSec = GetSimTime();
            _usingFallback = false;
            _staleFallbackLogged = false;  // Phase 10: Reset log throttle
        }

        // ================================================================================
        // Authority Sync
        // ================================================================================

        /// <summary>
        /// Explicitly set the active source. Called by ControlAuthorityManager when
        /// authority changes. Clears rejected count.
        /// </summary>
        public void SetActiveSource(InputSource inActive)
        {
            if (_activeSource == inActive) return;

            Debug.Log($"[Proxy] ActiveSource: {_activeSource} -> {inActive}");
            _activeSource = inActive;
            _rejectedCount = 0;
        }

        // ================================================================================
        // Safe Fallback
        // ================================================================================

        /// <summary>
        /// Reset the proxy to a safe state. Zero commands, given mode, clear diagnostics.
        /// Used during drone reset and initialization.
        /// </summary>
        public void ResetToSafe(GoalMode safeMode = GoalMode.None)
        {
            _command = Axis4.ZeroValue;
            _mode = safeMode;
            _lastCommandTimeSec = GetSimTime();
            _rejectedCount = 0;
            _usingFallback = false;
            _staleFallbackLogged = false;  // Phase 10: Reset log throttle on reset
        }

        /// <summary>
        /// Check if the current command is stale and inject safe fallback if needed.
        /// Called automatically on read (GetCommandValue).
        ///
        /// Phase 10 Polish (Step 2a): Now only logs the warning ONCE per stale episode.
        /// Subsequent ticks while still stale are silent. The _staleFallbackLogged flag
        /// resets when SetCommand() receives a fresh command.
        /// </summary>
        private void CheckForStaleCommand()
        {
            if (staleCommandThresholdSec <= 0f) return;
            if (_mode == GoalMode.None) return;

            double now = GetSimTime();
            double age = now - _lastCommandTimeSec;

            if (age > staleCommandThresholdSec && !_usingFallback)
            {
                _usingFallback = true;
                _command = Axis4.ZeroValue; // Zero rates / zero throttle

                // Phase 10: Only log once per stale episode
                if (!_staleFallbackLogged)
                {
                    _staleFallbackLogged = true;
                    Debug.LogWarning($"[Proxy] Stale command detected (age={age:F2}s, threshold={staleCo
mmandThresholdSec}s). " +
                                     $"Falling back to safe command. Authority={_activeSource}, LastWrit
er={_lastWriter}");
                }
            }
        }
```

```
        // ================================================================================
        // Helpers
        // ================================================================================

        private InputSource GetEffectiveAuthority()
        {
            if (_authorityManager != null)
                return _authorityManager.EffectiveAuthority;

            // Lazy resolve
            _authorityManager = ControlAuthorityManager.Get();
            if (_authorityManager != null)
                return _authorityManager.EffectiveAuthority;

            // No authority manager in scene — fall back to local _activeSource
            return _activeSource;
        }

        private double GetSimTime()
        {
            // Use ClockFactory if available, otherwise fallback to Unity time
            try
            {
                return ClockFactory.Clock.NowNanos / 1_000_000_000.0;
            }
            catch
            {
                return Time.timeAsDouble;
            }
        }
    }
}
```

Runtime/Scripts/DroneCore/Interfaces/ICommandSource.cs

SHA-256 8487a0052df77d7d7e4dafc0bb3f35c7c9e50236b11f111e447cb5ca97aefe4f | 11 lines | 219 bytes

```
using SimCore.Common;

namespace DroneCore.Interfaces
{
    public interface ICommandSource
    {
        Axis4 GetCommandValue();
        GoalMode GetGoalMode();
        InputSource GetActiveSource();
    }
}
```

7. Flight Control and Actuation

Converts high-level position, velocity, attitude, and rate goals into rotor commands. It contains the cascaded PID stages, actuator lag model, rotor wrench calculation, and effectiveness-matrix allocation with desaturation diagnostics.

Files in this section

- Runtime/Scripts/DroneCore/Common/QuadPIDController.cs
- Runtime/Scripts/DroneCore/Controllers/CascadedController.cs
- Runtime/Scripts/DroneCore/Controllers/AxisControllers/AcroController.cs
- Runtime/Scripts/DroneCore/Controllers/AxisControllers/AngleController.cs
- Runtime/Scripts/DroneCore/Controllers/AxisControllers/PositionController.cs
- Runtime/Scripts/DroneCore/Controllers/AxisControllers/VelocityController.cs
- Runtime/Scripts/DroneCore/Controls/ControlAllocator.cs
- Runtime/Scripts/DroneCore/Controls/Quadactuationmodel.cs

Runtime/Scripts/DroneCore/Common/QuadPIDController.cs

SHA-256 0c1c5d2f5612d89ccc7fd12f6a2ea95b3ca40aeb13ce58be876d4e4f8c8dfca9 | 116 lines | 3,982 bytes

```
using UnityEngine;

namespace DroneCore.Common
{
    [System.Serializable]
    public sealed class QuadPIDController
    {
        public float Kp = 0f;
        public float Ki = 0f;
        public float Kd = 0f;

        public float MinOutput = -1f;
        public float MaxOutput = 1f;

        public float IntegralLimit = 10f;
        public float DecayFactor = 0.99f;

        private float _integralSum = 0f;
        private float _prevError = 0f;
        private bool _hasLastError = false;
        private bool _positiveOutputSaturated;
        private bool _negativeOutputSaturated;

        public float IntegralState => _integralSum;

        public void SetGains(float p, float i, float d) { Kp = p; Ki = i; Kd = d; }
        public void SetLimits(float min, float max) { MinOutput = min; MaxOutput = max; }
        //public void SetIntegralLimits(float limit) { IntegralLimit = Mathf.Abs(limit); }
        public void SetDecayFactor(float decay) { DecayFactor = Mathf.Clamp01(decay); }

        public void Reset()
        {
            _integralSum = 0f;
            _prevError = 0f;
            _hasLastError = false;
            ClearSaturation();
        }

        /// <summary>
        /// Feed downstream actuator tracking back into the integrator. This is
        /// evaluated on the next controller update, which matches the one-step
        /// delay between PID calculation and motor allocation.
        /// </summary>
        public void SetSaturationFromTracking(float requested, float achieved, float tolerance = 1e-4f)
        {
            _positiveOutputSaturated = requested > achieved + tolerance;
```

```
            _negativeOutputSaturated = requested < achieved - tolerance;
        }

        public void ClearSaturation()
        {
            _positiveOutputSaturated = false;
            _negativeOutputSaturated = false;
        }

        public float Calculate(float desired, float measured, float dt)
        {
            if (dt <= Mathf.Epsilon) return 0f;

            float error = desired - measured;

            // P
            float pTerm = Kp * error;

            // I (leak + conditional integration). Do not retain hidden state
            // while Ki is disabled, and do not integrate farther into a
            // downstream actuator limit reported by the mixer/allocator.
            if (Ki > Mathf.Epsilon)
            {
                _integralSum *= DecayFactor;
                bool pushesFurtherIntoSaturation =
                    (error > 0f && _positiveOutputSaturated) ||
                    (error < 0f && _negativeOutputSaturated);
                if (!pushesFurtherIntoSaturation)
                    _integralSum += error * dt;
                _integralSum = Mathf.Clamp(_integralSum, -IntegralLimit, IntegralLimit);
            }
            else
            {
                _integralSum = 0f;
            }
            float iTerm = Ki * _integralSum;

            // D
            float dTerm = 0f;
            if (_hasLastError)
            {
                float derivative = (error - _prevError) / dt;
                dTerm = Kd * derivative;
            }
            else
            {
                _hasLastError = true;
            }

            _prevError = error;

            // Combine + clamp
            float output = pTerm + iTerm + dTerm;
            output = Mathf.Clamp(output, MinOutput, MaxOutput);

            // Back-calculation anti-windup
            if (Ki > Mathf.Epsilon)
            {
                float maxITerm = MaxOutput - pTerm - dTerm;
                float minITerm = MinOutput - pTerm - dTerm;
                float clampedITerm = Mathf.Clamp(iTerm, minITerm, maxITerm);

                if (!Mathf.Approximately(iTerm, clampedITerm))
                    _integralSum = clampedITerm / Ki;
            }

            return output;
        }
    }
}
```

Runtime/Scripts/DroneCore/Controllers/CascadedController.cs

SHA-256 3d9b2c3bb01ce584c4848411fa6571658908264c13ee2fb48ca10daf145055fd | 1,122 lines | 47,213 bytes

```
using System;
using System.Collections.Generic;
using MathUtil;
using DroneCore.Common;
using DroneCore.Controllers.AxisControllers;
using DroneCore.Controls;
using DroneCore.Interfaces;
using RobotCore;
using SimCore.Common;
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Controllers
{
    /// <summary>
    /// Cascaded flight controller implementing full Position -> Velocity -> Angle -> Rate cascade.
    /// Mirrors UE's UQuadDroneController.
    ///
    /// NOT an ISimulatable — QuadPawn drives this via Initialize(), Update(), ResetControllerState().
    /// </summary>
    [RequireComponent(typeof(QuadPawn))]
    [DisallowMultipleComponent]
    public sealed class CascadedController : MonoBehaviour
    {
        // ================================================================================
        // Enums & Structs
        // ================================================================================

        public enum EPIDStage
        {
            Position,
            Velocity,
            Angle,
            Rate
        }

        [Serializable]
        public struct PIDKey : IEquatable<PIDKey>
        {
            public EPIDStage Stage;
            public byte Axis;

            public bool Equals(PIDKey other) => Stage == other.Stage && Axis == other.Axis;
            public override bool Equals(object obj) => obj is PIDKey other && Equals(other);
            public override int GetHashCode() => HashCode.Combine(Stage, Axis);
        }

        [Serializable]
        public class PIDInfo
        {
            public string Label;
            public QuadPIDController PID; // non-owning reference
        }

        public class CascadeAxis
        {
            public PositionController Position;
            public VelocityController Velocity;
            public AngleController Angle;
            public AcroController Rate;
        }

        [Serializable]
        public struct CascadeTelemetrySnapshot
        {
            public GoalMode Mode;
            public Axis4 ExternalCmd;
            public Vector3 DesiredVel;
            public Vector3 DesiredAccel;
            public Vector3 DesiredAnglesDeg;
```

```
            public Vector3 DesiredRatesDeg;
            public bool HasVelSetpoint;
            public bool HasAccelSetpoint;
            public bool HasAngleSetpoint;
            public bool HasRateSetpoint;
        }

        // ================================================================================
        // Runtime References (set via Initialize, not inspector)
        // ================================================================================

        private QuadPawn _body;
        private SensorManager _sensorManager;
        private ICommandSource _goalSource;

        // ================================================================================
        // Configuration & Physics
        // ================================================================================

        private RotorPhysicsDerived _rotorPhysics;
        private DroneConfig _cachedConfig;
        private bool _configApplied;

        // ================================================================================
        // Runtime State
        // ================================================================================

        private long _lastControllerUpdateTime;
        private SensorData _lastSensorData;

        // Derived Controls
        private float _desiredBodyUpCos = 1f;
        private Vector3 _desiredSpecificForceHeading = new Vector3(0f, 0f, 9.81f);
        private float _requestedCollective01;
        private bool _verticalVelocityLoopActive;
        private readonly CascadeAxis[] _axes = new CascadeAxis[4];
        private ControlAllocator _controlAllocator = new ControlAllocator();

        // Limits & Constants (Cached for speed)
        private float _maxAngleRateDeg;
        private float _maxYawRateDeg;
        private float _hoverThrottle01;

        // Telemetry Data
        private CascadeTelemetrySnapshot _telemetry;
        private Dictionary<PIDKey, PIDInfo> _pidMap = new Dictionary<PIDKey, PIDInfo>();

        // Mode Transition
        private GoalMode _lastMode = GoalMode.None;
        private bool _lastModeValid;

        // Motor outputs
        private readonly float[] _motor = new float[4];
        private readonly float[] _thrusts = new float[4];
        private Vector3 _lastAchievedForceBody;
        private Vector3 _lastAchievedTorqueBody;
        private int _waDebugTick;

        // ── Plant / actuation (external-wrench allocated path) ──
        [Header("Plant / Actuation")]
        [SerializeField] private bool  enableMotorLag      = false;   // inspector toggle
        [SerializeField] private float motorLagTauSeconds  = 0.020f;  // legacy symmetric / spin-up valu
e
        [SerializeField] private float motorLagTauDownSeconds = 0.020f;
        [SerializeField] private float allocatedMinIdle01  = 0.0f;    // 0 lets motors reach 0 at loop a
pex
        private readonly MotorLagFilter _motorLag = new MotorLagFilter();

        // ================================================================================
        // Public Accessors
        // ================================================================================

        public IReadOnlyList<float> LastMotor01 => _motor;
```

```
        public IReadOnlyList<float> Thrusts => _thrusts;
        public CascadeTelemetrySnapshot TelemetrySnapshot => _telemetry;
        public IReadOnlyDictionary<PIDKey, PIDInfo> PIDMap => _pidMap;
        public EffectivenessMatrix Effectiveness => _controlAllocator.Effectiveness;
        public AllocationDiagnostics LastAllocation => _controlAllocator.LastAllocation;
        public int AllocatorRank => _controlAllocator.Rank;
        public float AllocatorConditionNumber => _controlAllocator.ConditionNumber;
        public Vector3 LastAchievedForceBody => _lastAchievedForceBody;
        public Vector3 LastAchievedTorqueBody => _lastAchievedTorqueBody;

        /// <summary>Debug visuals hook. RuntimeUI writes this; consume in a future debug-draw pass. TOD
O.</summary>
        public bool DebugVisualsEnabled { get; set; }

        /// <summary>
        /// Rebuild the effectiveness matrix for the CURRENT output frame.
        /// Must be called after SensorManager.outputFrame changes at runtime,
        /// otherwise the allocator torque signs are stale (FLU-built matrix under FRD).
        /// </summary>
        public void RebuildAllocator()
        {
            if (!_configApplied || !_axesBuilt) return;
            _controlAllocator.RebuildEffectiveness(
                _cachedConfig.RotorParams.Rotors, _rotorPhysics, GetCurrentFrame());
            Debug.Log($"[CascadedController] {name}: allocator rebuilt for frame {GetCurrentFrame()}");
        }
        // ================================================================================
        // Initialization (called by QuadPawn)
        // ================================================================================

        public void Initialize(QuadPawn body, DroneConfig config, SensorManager sensors)
        {
            _body = body;
            _sensorManager = sensors;
            _goalSource = body.CommandProxy;
            //_thrusters = body.Thrusters;

            if (_body == null)
            {
                Debug.LogError($"[CascadedController] {name}: QuadPawn reference is null!");
                return;
            }

            // Prevent Unity angular velocity clamping
            if (_body.Rigidbody != null)
            {
                _body.Rigidbody.maxAngularVelocity = 100f;
            }

            // Initialize axis array
            for (int i = 0; i < 4; i++)
            {
                _axes[i] = new CascadeAxis();
            }

            ResetControllerState();

            Debug.Log($"[CascadedController] {name}: Initialized");
        }

        /// <summary>
        /// Set rotor physics and rebuild allocator. Mirrors UE's SetRotorPhysics.
        /// Called by QuadPawn.ApplyControllerConfig after RotorPhysicsDerived is computed.
        /// </summary>

        private bool _axesBuilt;

        public void SetRotorPhysics(RotorPhysicsDerived rotorPhysics, DroneConfig config)
        {
            _rotorPhysics = rotorPhysics;
            _cachedConfig = config;
            _configApplied = true;
```

```
            _maxAngleRateDeg = config.FlightParams.MaxRateRollPitch;
            _maxYawRateDeg = config.FlightParams.MaxRateYaw;

            float configuredTauUp = config.RotorParams.Model.MotorTimeConstantUp;
            float configuredTauDown = config.RotorParams.Model.MotorTimeConstantDown;
            if (configuredTauUp > 0f && configuredTauDown > 0f)
            {
                enableMotorLag = true;
                motorLagTauSeconds = configuredTauUp;
                motorLagTauDownSeconds = configuredTauDown;
                _motorLag.Reset(4);
            }
            else
            {
                enableMotorLag = false;
            }

            // Compute hover throttle from mass
            if (_body != null && _body.Rigidbody != null)
            {
                float currentMass = _body.Rigidbody.mass;
                _hoverThrottle01 = _rotorPhysics.ComputeHoverThrottle(currentMass, 4);

                // // --- FIX 1: Math/Physics Alignment ---
                // Vector3 centroid = Vector3.zero;
                // var rotors = _cachedConfig.RotorParams.Rotors;
                //
                // for (int i = 0; i < rotors.Length; i++)
                //     centroid += rotors[i].Location_Body;
                //
                // if (rotors.Length > 0) centroid /= rotors.Length;
                //
                // // Shift Unity's physics pivot
                // _body.Rigidbody.centerOfMass = centroid;
                //
                // // Shift mathematical pivot. This satisfies the Allocator's
                // // decoupled pseudo-inverse requirement (Sum of positions = 0)
                // for (int i = 0; i < rotors.Length; i++)
                // {
                //     rotors[i].Location_Body -= centroid;
                // }

                float totalMaxThrust = _rotorPhysics.MaxThrust * 4.0f;
                float hoverForce = currentMass * 9.81f;
                float twr = (hoverForce > 0.0f) ? totalMaxThrust / hoverForce : 0.0f;

                Debug.Log($"[CascadedController] {name}:");
                Debug.Log($"  - Mass: {currentMass:F3} kg");
                Debug.Log($"  - Max Thrust/Motor: {_rotorPhysics.MaxThrust:F2} N");
                Debug.Log($"  - Hover Throttle: {_hoverThrottle01:F2}");
                Debug.Log($"  - TWR: {twr:F2}");
            }

            if (!_configApplied) return;
            if (_goalSource == null || _sensorManager == null) return;
            if (!_axesBuilt)
            {
                AxisSetup();
                BuildPIDTuningMap();
                _axesBuilt = true;
            }
            _controlAllocator.RebuildEffectiveness(_cachedConfig.RotorParams.Rotors, _rotorPhysics, GetC
urrentFrame());

            Debug.Log($"[CascadedController] {name}: RotorConfig applied");
        }


        // ================================================================================
        // Axis Setup
        // ================================================================================

        private void AxisSetup()
```

```
{
    for (int axis = 0; axis < 4; axis++)
    {
        _axes[axis].Velocity = new VelocityController(_cachedConfig);
        _axes[axis].Rate = new AcroController(_cachedConfig);

        if (axis != 2) // No position controller for yaw
            _axes[axis].Position = new PositionController(_cachedConfig);
        if (axis < 3) // Roll, Pitch, Yaw have angle controllers
            _axes[axis].Angle = new AngleController(_cachedConfig);
    }

    // Initialize all controllers
    for (int axis = 0; axis < 4; axis++)
    {
        if (_axes[axis].Position != null)
            _axes[axis].Position.Initialize(axis, _goalSource, _sensorManager);

        _axes[axis].Velocity.Initialize(axis, _goalSource, _sensorManager);
        _axes[axis].Rate.Initialize(axis, _goalSource, _sensorManager);

        if (_axes[axis].Angle != null)
            _axes[axis].Angle.Initialize(axis, _goalSource, _sensorManager);
    }
}

// ============================================================================
// Reset (called by QuadPawn)
// ============================================================================

public void ResetControllerState()
{
    // Reset all cascade controllers
    for (int axis = 0; axis < 4; axis++)
    {
        if (_axes[axis].Position != null) _axes[axis].Position.Reset();
        if (_axes[axis].Velocity != null) _axes[axis].Velocity.Reset();
        if (_axes[axis].Angle != null) _axes[axis].Angle.Reset();
        if (_axes[axis].Rate != null) _axes[axis].Rate.Reset();
        if (_axes[axis].Angle != null) _axes[axis].Angle.ClearExternalGoal();
    }

    _desiredBodyUpCos = 1f;
    _desiredSpecificForceHeading = new Vector3(0f, 0f, 9.81f);
    _requestedCollective01 = 0f;
    _verticalVelocityLoopActive = false;
    _lastAchievedForceBody = Vector3.zero;
    _lastAchievedTorqueBody = Vector3.zero;
    _controlAllocator.ResetDiagnostics();

    for (int axis = 0; axis < 4; axis++)
    {
        if (_axes[axis].Velocity != null) _axes[axis].Velocity.ClearExternalGoal();
        if (_axes[axis].Rate != null) _axes[axis].Rate.ClearExternalGoal();
    }

    // Zero thrusts and motors
    for (int i = 0; i < 4; i++)
    {
        _thrusts[i] = 0f;
        _motor[i] = 0f;
    }
    _motorLag.Reset(4);

    // Preserve mode/source, just zero the command
    if (_body != null && _body.CommandProxy != null)
    {
        var keepMode = _body.CommandProxy.GetGoalMode();
        var owner = _body.CommandProxy.GetActiveSource();
        _body.CommandProxy.ForceSetCommand(new Axis4(), keepMode, owner);
    }

    Debug.Log($"[CascadedController] {name}: State reset");
```

```
        }

        // ================================================================================
        // Update (called by QuadPawn.RunController)
        // ================================================================================

        public void StepController(float deltaTime)
        {
            if (!_configApplied) return;

            float actualDeltaTime = Mathf.Clamp(deltaTime, 0.001f, 0.1f);

            if (_goalSource != null)
            {
                GoalMode newMode = _goalSource.GetGoalMode();
                if (!_lastModeValid)
                {
                    _lastMode = newMode;
                    _lastModeValid = true;
                }
                else if (newMode != _lastMode)
                {
                    OnGoalModeChanged(newMode, _lastMode);
                    _lastMode = newMode;
                }
            }

            FlightController(actualDeltaTime);
        }


        // ================================================================================
        // Flight Controller Core
        // ================================================================================

        private void FlightController(float deltaTime)
        {
            if (_body == null || _goalSource == null) return;

            // Single source of truth for Mode
            GoalMode mode = _goalSource.GetGoalMode();
            Axis4 cmd = _goalSource.GetCommandValue();

            InputSource source = (_body != null && _body.CommandProxy != null) ? _body.CommandProxy.GetA
ctiveSource() : InputSource.UI;
            bool bGamepadAngle = (mode == GoalMode.Angle && source == InputSource.UI);

            TelemetryInit(mode, cmd);

            // --- WRENCH MODE ---
            // This is to bypass directly to the rigid body
            if (mode == GoalMode.WrenchBypassed)
            {
                Vector3 torque = new Vector3(cmd.X, cmd.Y, cmd.Z);
                Vector3 force = new Vector3(0, cmd.W, 0);
                _body.SetBodyWrench(force, torque);
                _lastAchievedForceBody = force;
                _lastAchievedTorqueBody = torque;
                return;
            }

            if (mode == GoalMode.Wrench)
            {
                float[] motor01 = new float[4];
                Vector3 mFlu = new Vector3(cmd.X, cmd.Y, cmd.Z);

                float sMax = _controlAllocator.AllocateWrench(
                    cmd.W,
                    mFlu.x,
                    mFlu.y,
                    mFlu.z,
                    _rotorPhysics.MaxThrust,
                    allocatedMinIdle01,
```

```
            motor01
        );

        // Plant: optional first-order motor lag (live toggle), then actuate.
        _motorLag.Enabled        = enableMotorLag;
        _motorLag.TauUpSeconds   = motorLagTauSeconds;
        _motorLag.TauDownSeconds = motorLagTauDownSeconds;
        _motorLag.Apply(motor01, deltaTime);

        QuadActuationModel.ComputeBodyWrench(
            motor01,
            _cachedConfig.RotorParams.Rotors,
            _rotorPhysics,
            out Vector3 netForce,
            out Vector3 netTorque
        );

        _body.SetBodyWrench(netForce, netTorque);
        _lastAchievedForceBody = netForce;
        _lastAchievedTorqueBody = netTorque;

        for (int i = 0; i < 4; i++)
        {
            _motor[i]   = motor01[i];
            _thrusts[i] = motor01[i] * _rotorPhysics.MaxThrust;
        }

        return;
    }

    // --- NONE MODE ---
    if (mode == GoalMode.None)
    {
        _body.SetBodyWrench(Vector3.zero, Vector3.zero);
        _lastAchievedForceBody = Vector3.zero;
        _lastAchievedTorqueBody = Vector3.zero;
        for (int i = 0; i < 4; i++)
        {
            _motor[i] = 0f;
            _thrusts[i] = 0f;
        }
        return;
    }

    Axis4 output = new Axis4();
    _desiredBodyUpCos = 1f;
    _desiredSpecificForceHeading = new Vector3(0f, 0f, 9.81f);
    _requestedCollective01 = 0f;
    _verticalVelocityLoopActive = false;

    // 1. Clear external goals
    for (int axis = 0; axis < 4; axis++)
    {
        if (_axes[axis].Velocity != null) _axes[axis].Velocity.ClearExternalGoal();
        if (_axes[axis].Angle != null) _axes[axis].Angle.ClearExternalGoal();
        if (_axes[axis].Rate != null) _axes[axis].Rate.ClearExternalGoal();
    }

    // 2. POSITION STAGE
    if (mode == GoalMode.Position)
    {
        // Position PID outputs are velocity setpoints in the output frame.
        // Output frame convention:
        //   .x = forward, .y = lateral, .z = altitude
        // This holds for FLU (y=left, z=up) and FRD (y=right, z=down).
        Vector3 vSpOutputFrame = Vector3.zero;

        if (_axes[0].Position != null)
        {
            _axes[0].Position.Update(deltaTime);
            vSpOutputFrame.x = _axes[0].Position.GetOutput();  // forward
        }
        if (_axes[1].Position != null)
```

```
                {
                    _axes[1].Position.Update(deltaTime);
                    vSpOutputFrame.y = _axes[1].Position.GetOutput();  // lateral
                }
                if (_axes[3].Position != null)
                {
                    _axes[3].Position.Update(deltaTime);
                    vSpOutputFrame.z = _axes[3].Position.GetOutput();  // altitude
                }

                // Rotate horizontal components into heading frame
                Quaternion bodyRot = _body.transform.rotation;
                Vector3 vSpHeading = OutputFrameVelToHeadingVel(vSpOutputFrame, bodyRot);

                // Constrain velocity: XY = horizontal, Z = vertical (in output frame)
                // TODO: DEFINE THESE IN CONFIG
                Vector3 vSpConstrained = ConstrainVelSetpoint_HeadingFrame(vSpHeading, 4.0f, 10f, 8f);

                // The velocity setpoint is already in the output frame —
                // no Frames.TransformLinear needed.
                if (_axes[0].Velocity != null) _axes[0].Velocity.SetExternalGoal(vSpConstrained.x);
                if (_axes[1].Velocity != null) _axes[1].Velocity.SetExternalGoal(vSpConstrained.y);
                if (_axes[3].Velocity != null) _axes[3].Velocity.SetExternalGoal(vSpConstrained.z);

                _telemetry.DesiredVel = vSpConstrained;
                _telemetry.HasVelSetpoint = true;
            }


            // 3. ACCELERATION STAGE
            // Acceleration mode bypasses Position and Velocity PID. The external
            // command is a desired acceleration setpoint in the output/body frame:
            //   X = ax [m/s^2], Y = ay [m/s^2], Z = yaw-rate [deg/s], W = az [m/s^2].
            Vector3 accelCmd = Vector3.zero;
            if (mode == GoalMode.Acceleration)
            {
                accelCmd = new Vector3(cmd.X, cmd.Y, cmd.W);
                _telemetry.DesiredAccel = accelCmd;
                _telemetry.HasAccelSetpoint = true;
            }

            // 4. VELOCITY STAGE
            if (mode == GoalMode.Position || mode == GoalMode.Velocity || mode == GoalMode.Angle)
            {
                if (mode == GoalMode.Velocity)
                {
                    _telemetry.DesiredVel = new Vector3(cmd[FlightAxes.X], cmd[FlightAxes.Y], cmd[Flight
Axes.Z]);
                    _telemetry.HasVelSetpoint = true;
                }

                for (int axis = 0; axis < 4; axis++)
                {
                    // Angle mode: only Z velocity matters
                    if (mode == GoalMode.Angle && axis != 3)
                        continue;

                    // Gamepad Angle mode handles throttle manually
                    if (bGamepadAngle && axis == 3)
                        continue;

                    if (_axes[axis].Velocity != null)
                        _axes[axis].Velocity.Update(deltaTime);
                }
            }

            // 5. ANGLE STAGE
            if (mode != GoalMode.Rate)
            {
                if (mode == GoalMode.Angle)
                {
                    _telemetry.DesiredAnglesDeg = new Vector3(cmd.X, cmd.Y, cmd.Z);
                    _telemetry.HasAngleSetpoint = true;
```

```
        _desiredBodyUpCos = AttitudeVerticalProjection(cmd.X, cmd.Y);

        if (bGamepadAngle)
        {
            _telemetry.DesiredRatesDeg.z = cmd.Z; // Yaw is rate-controlled on Gamepad
            _telemetry.HasRateSetpoint = true;
        }
    }

    // Roll/pitch source
    if (mode == GoalMode.Position || mode == GoalMode.Velocity)
    {
        float velocityOutputX = _axes[0].Velocity.GetOutput();
        float velocityOutputY = _axes[1].Velocity.GetOutput();
        float velocityOutputZ = _axes[3].Velocity.GetOutput();
        Vector2 attCmd = AccelerationToAttitude(
            velocityOutputX, velocityOutputY, velocityOutputZ,
            out _desiredBodyUpCos, out _desiredSpecificForceHeading);

        _axes[1].Angle.SetExternalGoal(attCmd.x);
        _axes[0].Angle.SetExternalGoal(attCmd.y);

        // Feed force-cone saturation back to the horizontal
        // velocity integrators on the next control update.
        _axes[0].Velocity.Pid.SetSaturationFromTracking(
            velocityOutputX, _desiredSpecificForceHeading.x);
        _axes[1].Velocity.Pid.SetSaturationFromTracking(
            velocityOutputY, _desiredSpecificForceHeading.y);

        _telemetry.DesiredAnglesDeg.x = attCmd.x;
        _telemetry.DesiredAnglesDeg.y = attCmd.y;
        _telemetry.HasAngleSetpoint = true;
        _telemetry.DesiredAccel = new Vector3(
            velocityOutputX,
            velocityOutputY,
            velocityOutputZ);
        _telemetry.HasAccelSetpoint = true;
        _verticalVelocityLoopActive = true;
    }
    else if (mode == GoalMode.Acceleration)
    {
        Vector2 attCmd = AccelerationToAttitude(
            accelCmd.x, accelCmd.y, accelCmd.z,
            out _desiredBodyUpCos, out _desiredSpecificForceHeading);

        _axes[1].Angle.SetExternalGoal(attCmd.x);
        _axes[0].Angle.SetExternalGoal(attCmd.y);

        _telemetry.DesiredAnglesDeg.x = attCmd.x;
        _telemetry.DesiredAnglesDeg.y = attCmd.y;
        _telemetry.HasAngleSetpoint = true;
    }

    for (int axis = 0; axis < 3; axis++)
    {
        // Only Position mode owns a yaw-angle setpoint. Velocity,
        // Acceleration, and Angle modes command yaw rate directly.
        if (axis == 2 && mode != GoalMode.Position)
            continue;

        if (_axes[axis].Angle != null)
            _axes[axis].Angle.Update(deltaTime);
    }

    // Wire Angle -> Rate
    for (int axis = 0; axis < 3; axis++)
    {
        float angleOutput = _axes[axis].Angle != null ? _axes[axis].Angle.GetOutput() : 0f;

        // Yaw is rate-controlled in every stabilized mode except
        // Position, where the wrapped yaw-angle PID feeds this rate loop.
        if (axis == 2 && mode != GoalMode.Position)
        {
```

```
            angleOutput = cmd.Z;
        }

        if (_axes[axis].Rate != null)
            _axes[axis].Rate.SetExternalGoal(angleOutput);

        if (axis == 0)
        {
            _telemetry.DesiredRatesDeg.x = angleOutput;
            _telemetry.HasRateSetpoint = true;
        }
        if (axis == 1)
        {
            _telemetry.DesiredRatesDeg.y = angleOutput;
            _telemetry.HasRateSetpoint = true;
        }
        if (axis == 2)
        {
            _telemetry.DesiredRatesDeg.z = angleOutput;
            _telemetry.HasRateSetpoint = true;
        }
    }
}

// 6. RATE STAGE
if (mode == GoalMode.Rate)
{
    _telemetry.DesiredRatesDeg = new Vector3(cmd.X, cmd.Y, cmd.Z);
    _telemetry.HasRateSetpoint = true;
}

for (int axis = 0; axis < 3; axis++)
{
    if (_axes[axis].Rate != null)
    {
        _axes[axis].Rate.Update(deltaTime);
        output[axis] = _axes[axis].Rate.GetOutput();
    }
}

// 7. THROTTLE
const float MinIdleThrottle = 0.04f;
if (mode == GoalMode.Rate || bGamepadAngle)
{
    output[3] = Mathf.Max(cmd.W, MinIdleThrottle);
}
else
{
    float zAccel = (mode == GoalMode.Acceleration)
        ? accelCmd.z
        : _axes[3].Velocity.GetOutput();

    if (mode == GoalMode.Position || mode == GoalMode.Velocity ||
        mode == GoalMode.Acceleration)
    {
        // Geometric force projection: command only the component
        // of the desired force that the current body-up axis can
        // actually produce. As attitude converges this naturally
        // reaches |F_des| without a planned-cosine transient boost.
        output[3] = DesiredForceToCollective(_desiredSpecificForceHeading);
    }
    else
    {
        // Non-gamepad Angle mode retains altitude-velocity control.
        output[3] = _hoverThrottle01 * (1.0f + zAccel / 9.81f)
                    / Mathf.Max(0.1f, _desiredBodyUpCos);
        _verticalVelocityLoopActive = true;
    }

    _requestedCollective01 = output[3];
}

if (mode == GoalMode.Passthrough)
```

```
            {
                // Direct motor passthrough — bypass cascade and allocator entirely.
                // cmd packing: X=FL, Y=FR, Z=BL, W=BR (matches Motor4 order)
                float[] motor01 = new float[4];
                motor01[0] = Mathf.Clamp01(cmd.X);  // FL
                motor01[1] = Mathf.Clamp01(cmd.Y);  // FR
                motor01[2] = Mathf.Clamp01(cmd.Z);  // BL
                motor01[3] = Mathf.Clamp01(cmd.W);  // BR

                _motorLag.Enabled        = enableMotorLag;
                _motorLag.TauUpSeconds   = motorLagTauSeconds;
                _motorLag.TauDownSeconds = motorLagTauDownSeconds;
                _motorLag.Apply(motor01, deltaTime);

                // Compute body wrench from motor commands through the actuation model
                Vector3 netForce, netTorque;
                QuadActuationModel.ComputeBodyWrench(
                    motor01,
                    _cachedConfig.RotorParams.Rotors,
                    _rotorPhysics,
                    out netForce,
                    out netTorque
                );
                _body.SetBodyWrench(netForce, netTorque);
                _lastAchievedForceBody = netForce;
                _lastAchievedTorqueBody = netTorque;

                // Store for telemetry / UI
                for (int i = 0; i < 4; i++)
                {
                    _motor[i] = motor01[i];
                    _thrusts[i] = motor01[i] * _rotorPhysics.MaxThrust;
                }
                return;
            }


            ThrustMixer(output, deltaTime);
        }

        // ================================================================================
        // Thrust Mixer
        // ================================================================================

        private void ThrustMixer(Axis4 output, float deltaTime)
        {
            float rollOut = output[0];
            float pitchOut = output[1];
            float yawOut = output[2];
            float thrustOut = output[3];
            float yawAuthorityScale = 1.0f; // Tune this

            // Normalize rate PID outputs to torque demands (-1 to 1)
            float rollTorque = Mathf.Clamp(rollOut / _maxAngleRateDeg, -1f, 1f);
            float pitchTorque = Mathf.Clamp(pitchOut / _maxAngleRateDeg, -1f, 1f);
            float yawTorque = Mathf.Clamp(yawOut / _maxYawRateDeg, -1f, 1f) * yawAuthorityScale;
            // Calculate throttle
            float throttle01 = Mathf.Clamp(thrustOut, 0f, 1f);

            const float MinIdleThrottle = 0.04f;
            throttle01 = Mathf.Clamp(throttle01, MinIdleThrottle, 1f);

            // Use effectiveness-based allocation
            float[] motor01 = new float[4];
            //float yawScale = 1.0f;

            _controlAllocator.AllocateYawLimited(rollTorque,pitchTorque,yawTorque,throttle01,MinIdleThro
ttle,motor01);

            AllocationDiagnostics allocation = _controlAllocator.LastAllocation;
            _axes[0].Rate.Pid.SetSaturationFromTracking(rollTorque, allocation.Achieved.x);
            _axes[1].Rate.Pid.SetSaturationFromTracking(pitchTorque, allocation.Achieved.y);
            _axes[2].Rate.Pid.SetSaturationFromTracking(yawTorque, allocation.Achieved.z);
```

```
            if (_verticalVelocityLoopActive)
            {
                _axes[3].Velocity.Pid.SetSaturationFromTracking(
                    _requestedCollective01, allocation.Achieved.w);
            }

            // Fallback if allocator fails
            if (motor01 == null || motor01.Length != 4)
            {
                motor01 = new float[4];
                for (int i = 0; i < 4; i++) motor01[i] = throttle01;
            }

            _motorLag.Enabled        = enableMotorLag;
            _motorLag.TauUpSeconds   = motorLagTauSeconds;
            _motorLag.TauDownSeconds = motorLagTauDownSeconds;
            _motorLag.Apply(motor01, deltaTime);

            // Compute body wrench
            Vector3 netForce, netTorque;
            QuadActuationModel.ComputeBodyWrench(motor01,_cachedConfig.RotorParams.Rotors,_rotorPhysics,
out netForce,out netTorque);
            _body.SetBodyWrench(netForce, netTorque);
            _lastAchievedForceBody = netForce;
            _lastAchievedTorqueBody = netTorque;

            // Store for UI/debugging
            for (int i = 0; i < 4; i++)
            {
                _motor[i] = motor01[i];
                _thrusts[i] = motor01[i] * _rotorPhysics.MaxThrust;
            }
        }

        // ================================================================================
        // Utilities
        // ================================================================================

        private Vector2 AccelerationToAttitude(
            float accelX, float accelY, float accelZ,
            out float bodyUpCos, out Vector3 desiredSpecificForceHeading)
        {
            const float g = 9.81f;

            float maxSpecificThrust = _hoverThrottle01 > 1e-5f
                ? g / _hoverThrottle01
                : float.PositiveInfinity;
            desiredSpecificForceHeading = ConstrainDesiredSpecificForce(
                accelX, accelY, accelZ,
                _cachedConfig.FlightParams.MaxAngle, maxSpecificThrust);

            if (desiredSpecificForceHeading.sqrMagnitude <= 1e-8f)
            {
                bodyUpCos = 1f;
                return Vector2.zero;
            }

            Vector3 bodyZ = desiredSpecificForceHeading.normalized;
            bodyUpCos = bodyZ.z;

            float pitchRad = Mathf.Atan2(bodyZ.x, bodyZ.z);
            float rollRad  = Mathf.Atan2(bodyZ.y, bodyZ.z);

            return new Vector2(Mathf.Rad2Deg * pitchRad, Mathf.Rad2Deg * -rollRad);
        }

        /// <summary>
        /// Construct and constrain F_des / mass in the yaw-heading FLU frame.
        /// Horizontal authority shrinks with available vertical force, so a
        /// request near free-fall cannot simultaneously demand an impossible tilt.
        /// </summary>
        public static Vector3 ConstrainDesiredSpecificForce(
            float accelX, float accelY, float accelZ,
```

```
    float maxTiltDeg, float maxSpecificThrust)
{
    const float g = 9.81f;
    float vertical = Mathf.Max(0f, g + accelZ);
    float maxHorizontal = vertical * Mathf.Tan(
        Mathf.Clamp(maxTiltDeg, 0f, 89f) * Mathf.Deg2Rad);
    Vector2 horizontal = Vector2.ClampMagnitude(
        new Vector2(accelX, accelY), maxHorizontal);

    Vector3 force = new Vector3(horizontal.x, horizontal.y, vertical);
    if (maxSpecificThrust > 0f && !float.IsInfinity(maxSpecificThrust))
        force = Vector3.ClampMagnitude(force, maxSpecificThrust);
    return force;
}

private float DesiredForceToCollective(Vector3 desiredSpecificForceHeading)
{
    const float g = 9.81f;
    Vector3 bodyUpWorld = _body.transform.up;
    float yawRad = _body.transform.eulerAngles.y * Mathf.Deg2Rad;
    float cosYaw = Mathf.Cos(yawRad);
    float sinYaw = Mathf.Sin(yawRad);
    Vector3 bodyUpHeading = new Vector3(
        cosYaw * bodyUpWorld.x + sinYaw * bodyUpWorld.z,
        -sinYaw * bodyUpWorld.x + cosYaw * bodyUpWorld.z,
        bodyUpWorld.y);

    float producibleSpecificForce = Mathf.Max(
        0f, Vector3.Dot(desiredSpecificForceHeading, bodyUpHeading));
    return _hoverThrottle01 * producibleSpecificForce / g;
}

private static float AttitudeVerticalProjection(float rollDeg, float pitchDeg)
{
    float rollRad = rollDeg * Mathf.Deg2Rad;
    float pitchRad = pitchDeg * Mathf.Deg2Rad;
    return Mathf.Max(0f, Mathf.Cos(rollRad) * Mathf.Cos(pitchRad));
}
private SimFrame GetCurrentFrame()
{
    if (_body != null && _sensorManager != null)
    {
        return _sensorManager.outputFrame;
    }
    return SimFrame.FLU; // Default fallback
}

private Vector3 OutputFrameVelToHeadingVel(Vector3 vOutputFrame, Quaternion bodyRot)
{
    // Extract yaw angle from Unity's rotation (always around Unity Y axis)
    float yawDeg = bodyRot.eulerAngles.y;
    float yawRad = yawDeg * Mathf.Deg2Rad;

    // In the output frame (FLU/FRD), horizontal plane is XY.
    // Rotate the horizontal components by -yaw to get heading-aligned frame.
    float cosY = Mathf.Cos(yawRad);
    float sinY = Mathf.Sin(yawRad);

    // 2D rotation of (x, y) by -yaw:
    //   x_heading =  cos(yaw) * x + sin(yaw) * y
    //   y_heading = -sin(yaw) * x + cos(yaw) * y
    float hx =  cosY * vOutputFrame.x + sinY * vOutputFrame.y;
    float hy = -sinY * vOutputFrame.x + cosY * vOutputFrame.y;

    // Z (altitude) passes through unchanged
    return new Vector3(hx, hy, vOutputFrame.z);
}

private Vector3 ConstrainVelSetpoint_HeadingFrame(
    Vector3 vSp,
    float vXyMax,
    float vUpMax,
    float vDownMax)
```

```
        {
            Vector3 output = vSp;

            // XY magnitude clamp
            Vector2 vxy = new Vector2(output.x, output.y);
            float n = vxy.magnitude;
            if (n > vXyMax && n > 1e-6f)
            {
                Vector2 clamped = vxy * (vXyMax / n);
                output.x = clamped.x;
                output.y = clamped.y;
            }

            // Z clamp
            output.z = Mathf.Clamp(output.z, -vUpMax, vDownMax);

            return output;
        }

        // ================================================================================
        // PID Tuning Interface
        // ================================================================================

        public bool GetPIDGains(EPIDStage stage, byte axis, out float p, out float i, out float d)
        {
            p = i = d = 0f;

            PIDKey key = new PIDKey { Stage = stage, Axis = axis };
            if (!_pidMap.TryGetValue(key, out PIDInfo info) || info.PID == null)
                return false;

            p = info.PID.Kp;
            i = info.PID.Ki;
            d = info.PID.Kd;
            return true;
        }

        public bool SetPIDGainTerm(EPIDStage stage, byte axis, int term, float value, bool resetIntegral
)
        {
            PIDKey key = new PIDKey { Stage = stage, Axis = axis };
            if (!_pidMap.TryGetValue(key, out PIDInfo info) || info.PID == null)
                return false;

            float p = info.PID.Kp;
            float i = info.PID.Ki;
            float d = info.PID.Kd;

            if (term == 0) p = value;
            else if (term == 1) i = value;
            else if (term == 2) d = value;
            else return false;

            info.PID.SetGains(p, i, d);

            if (resetIntegral)
                info.PID.Reset();

            return true;
        }

        public void GetPIDTuningList(out List<PIDKey> outKeys, out List<string> outLabels)
        {
            outKeys = new List<PIDKey>();
            outLabels = new List<string>();

            foreach (var pair in _pidMap)
            {
                outKeys.Add(pair.Key);
                outLabels.Add(pair.Value.Label);
            }
        }
```

```
        private void BuildPIDTuningMap()
        {
            _pidMap.Clear();

            void Add(EPIDStage stage, byte axis, string label, QuadPIDController pid)
            {
                if (pid == null) return;
                _pidMap[new PIDKey { Stage = stage, Axis = axis }] = new PIDInfo { Label = label, PID =
pid };
            }

            // POSITION
            if (_axes[0].Position != null) Add(EPIDStage.Position, 0, "Position X", _axes[0].Position.Ge
tPID());
            if (_axes[1].Position != null) Add(EPIDStage.Position, 1, "Position Y", _axes[1].Position.Ge
tPID());
            if (_axes[3].Position != null) Add(EPIDStage.Position, 3, "Position Z", _axes[3].Position.Ge
tPID());

            // VELOCITY
            if (_axes[0].Velocity != null) Add(EPIDStage.Velocity, 0, "Velocity X", _axes[0].Velocity.Ge
tPID());
            if (_axes[1].Velocity != null) Add(EPIDStage.Velocity, 1, "Velocity Y", _axes[1].Velocity.Ge
tPID());
            if (_axes[3].Velocity != null) Add(EPIDStage.Velocity, 3, "Velocity Z", _axes[3].Velocity.Ge
tPID());

            // ANGLE
            if (_axes[0].Angle != null) Add(EPIDStage.Angle, 0, "Angle Roll", _axes[0].Angle.GetPID());
            if (_axes[1].Angle != null) Add(EPIDStage.Angle, 1, "Angle Pitch", _axes[1].Angle.GetPID());
            if (_axes[2].Angle != null) Add(EPIDStage.Angle, 2, "Angle Yaw", _axes[2].Angle.GetPID());

            // RATE
            if (_axes[0].Rate != null) Add(EPIDStage.Rate, 0, "Rate Roll", _axes[0].Rate.GetPID());
            if (_axes[1].Rate != null) Add(EPIDStage.Rate, 1, "Rate Pitch", _axes[1].Rate.GetPID());
            if (_axes[2].Rate != null) Add(EPIDStage.Rate, 2, "Rate Yaw", _axes[2].Rate.GetPID());
        }

        public bool IsPIDStageActive(EPIDStage stage, byte axis)
        {
            GoalMode mode = _goalSource != null ? _goalSource.GetGoalMode() : GoalMode.None;

            switch (stage)
            {
                case EPIDStage.Position:
                    return mode == GoalMode.Position;

                case EPIDStage.Velocity:
                    if (mode == GoalMode.Angle)
                        return axis == 3; // Z velocity still runs for throttle
                    return mode == GoalMode.Position || mode == GoalMode.Velocity;

                case EPIDStage.Angle:
                    return mode != GoalMode.Rate;

                case EPIDStage.Rate:
                    return true;

                default:
                    return false;
            }
        }

        // ============================================================================
        // Telemetry
        // ============================================================================

        private void TelemetryInit(GoalMode mode, Axis4 cmd)
        {
            _telemetry.Mode = mode;
            _telemetry.ExternalCmd = cmd;
            _telemetry.DesiredVel = Vector3.zero;
            _telemetry.DesiredAccel = Vector3.zero;
```

```
            _telemetry.DesiredAnglesDeg = Vector3.zero;
            _telemetry.DesiredRatesDeg = Vector3.zero;
            _telemetry.HasVelSetpoint = false;
            _telemetry.HasAccelSetpoint = false;
            _telemetry.HasAngleSetpoint = false;
            _telemetry.HasRateSetpoint = false;
        }

        // ================================================================================
        // Mode Transition
        // ================================================================================

        private void OnGoalModeChanged(GoalMode newMode, GoalMode oldMode)
        {
            for (int axis = 0; axis < 4; axis++)
            {
                if (_axes[axis].Position != null) _axes[axis].Position.Reset();
                if (_axes[axis].Velocity != null) _axes[axis].Velocity.Reset();
                if (_axes[axis].Angle != null) _axes[axis].Angle.Reset();
                if (_axes[axis].Rate != null) _axes[axis].Rate.Reset();
                if (_axes[axis].Angle != null) _axes[axis].Angle.ClearExternalGoal();
            }

            for (int axis = 0; axis < 4; axis++)
            {
                if (_axes[axis].Velocity != null) _axes[axis].Velocity.ClearExternalGoal();
                if (_axes[axis].Rate != null) _axes[axis].Rate.ClearExternalGoal();
            }

            Debug.Log($"[CascadedController] GoalMode changed {oldMode} -> {newMode}");
        }

        // ================================================================================
        // Public Accessors
        // ================================================================================

        public float GetCurrentThrustOutput(int thrusterIndex)
        {
            if (thrusterIndex >= 0 && thrusterIndex < _thrusts.Length)
                return _thrusts[thrusterIndex];
            return 0.0f;
        }

        public bool IsConfigApplied() => _configApplied;
    }
}
```

Runtime/Scripts/DroneCore/Controllers/AxisControllers/AcroController.cs

SHA-256 9a2c1e2de94fed5120a1795d49ea6aeb10fdd6fe5810697d22b34d995b8274a3 | 100 lines | 3,339 bytes

```
using DroneCore.Common;
using DroneCore.Interfaces;
using RobotCore;
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Controllers.AxisControllers
{
    [System.Serializable]
    public sealed class AcroController
    {
        [SerializeField] private QuadPIDController _pidSet = new QuadPIDController();
        public QuadPIDController Pid => _pidSet;

        // Axis index: 0=roll, 1=pitch, 2=yaw, 3=throttle (pass-through)
        public int _axis { get; private set; }
        private ICommandSource _cmdGoal;
        private float _externalGoal;
        private SensorManager _sensorManager;
        private DroneConfig _config;

        private bool _bInitialized;
        private bool _bUseExternalGoal;
```

```
        // TODO: Setting but private? We dont need outside variables setting it at all
        private float _output;

        public AcroController(DroneConfig inConfig)
        {
            _config = inConfig;
        }

        public void Initialize(int axis, ICommandSource cmd, SensorManager sensors)
        {
            _axis = axis;
            _cmdGoal = cmd;
            _sensorManager = sensors;
            AcroPID PID = _config.Acro;
            if (axis != 2 && axis != 3)
                _pidSet.SetLimits(-_config.FlightParams.MaxRateRollPitch, _config.FlightParams.MaxRateRo
llPitch);
            else if (axis == 2)
                _pidSet.SetLimits(-_config.FlightParams.MaxRateYaw, _config.FlightParams.MaxRateYaw);

            _pidSet.SetGains(PID.GetPGains()[_axis], PID.GetIGains()[_axis], PID.GetDGains()[_axis]);

            _bInitialized = true;
            Debug.Log("[AcroController] Initialized");

            // if (iLimit > 0f) pid.SetIntegralLimits(iLimit);
        }

        public void Update(float deltaTime)
        {
            if (!_bInitialized || _sensorManager == null || _pidSet == null || (_cmdGoal == null && !_bU
seExternalGoal))
            {
                _output = 0.0f;
                return;
            }

            float StateRate = _sensorManager.Latest.ImuAngVel[_axis];
            float Goal = _bUseExternalGoal ? _externalGoal : _cmdGoal.GetCommandValue()[_axis];
            float StateRateDegS = Mathf.Rad2Deg * StateRate;

            if (_axis == 3)
            {
                _output = Goal;
                return;
            }

            _output = _pidSet.Calculate(Goal, StateRateDegS, deltaTime);
        }

        public void Reset()
        {
            if(_pidSet!=null) _pidSet.Reset();
            _output = 0.0f;
            _bUseExternalGoal = false;
        }

        public void SetExternalGoal(float Value)
        {
            _externalGoal = Value;
            _bUseExternalGoal = true;
        }

        public void ClearExternalGoal()
        {
            // TODO: Double check using this is actually needed or if its updated during runtime
            _externalGoal = 0.0f;
            _bUseExternalGoal = false;
        }
        public float GetOutput() { return _output; }
        public float GetExternalGoal() { return _externalGoal; }
        public QuadPIDController GetPID()
```

```
        {
            return _pidSet;
        }

    }
}
```

Runtime/Scripts/DroneCore/Controllers/AxisControllers/AngleController.cs

SHA-256 1e0ccac8048bd5c1cdf2062220e6882d184b534cd73cc9d04207f693c35e1f61 | 103 lines | 3,548 bytes

```
using DroneCore.Common;
using DroneCore.Interfaces;
using RobotCore; // SensorManager, SensorData
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Controllers.AxisControllers
{
    [System.Serializable]
    public sealed class AngleController // per-axis, UE-style
    {
        [SerializeField] private QuadPIDController _pidSet = new QuadPIDController();
        public QuadPIDController Pid => _pidSet;

        private int _axis;
        private ICommandSource _cmdGoal;
        private SensorManager _sensorManager;
        private DroneConfig _config;
        private float _output;
        private float _externalGoal;
        private bool _bInitialized;
        private bool _bUseExternalGoal;

        public AngleController(DroneConfig inConfig)
        {
            _config = inConfig;
        }

        public void Initialize(int axis, ICommandSource cmd, SensorManager sensors)
        {
            _axis = axis;
            _cmdGoal = cmd;
            _sensorManager = sensors;
            AnglePID PID = _config.Angle;

            // Angle PID output is a rate setpoint (deg/s)
            if (axis == 2)
                _pidSet.SetLimits(-_config.FlightParams.MaxRateYaw, _config.FlightParams.MaxRateYaw);
            else
                _pidSet.SetLimits(-_config.FlightParams.MaxRateRollPitch, _config.FlightParams.MaxRateRo
llPitch);

            _pidSet.SetGains(PID.GetPGains()[_axis], PID.GetIGains()[_axis], PID.GetDGains()[_axis]);

            _bInitialized = true;
        }

        public void Update(float deltaTime)
        {
            if (!_bInitialized || _sensorManager == null || _pidSet == null || (_cmdGoal == null && !_bU
seExternalGoal))
            {
                _output = 0.0f;
                return;
            }

            SensorData StateData = _sensorManager.Latest;
            float StateAngle = StateData.ImuAttitude[_axis];
            float Goal = _bUseExternalGoal ? _externalGoal :  _cmdGoal.GetCommandValue()[_axis];

            if (_axis == 3)
            {
                _output = Goal;
```

```
                return;
            }

            // Euler yaw wraps at +/-180 degrees. Move the goal to the
            // equivalent angle nearest the measurement before applying PID,
            // matching AirSim's shortest-arc yaw handling.
            if (_axis == 2)
                Goal = NearestEquivalentAngle(StateAngle, Goal);

            _output = _pidSet.Calculate(Goal, StateAngle, deltaTime);
        }

        public static float NearestEquivalentAngle(float measuredDeg, float goalDeg)
        {
            return measuredDeg + Mathf.DeltaAngle(measuredDeg, goalDeg);
        }
        public void Reset()
        {
            if(_pidSet!=null) _pidSet.Reset();
            _output = 0.0f;
            _bUseExternalGoal = false;
        }
        public void SetExternalGoal(float Value)
        {
            _externalGoal = Value;
            _bUseExternalGoal = true;
        }
        public void ClearExternalGoal()
        {
            // TODO: Double check using this is actually needed or if its updated during runtime
            _externalGoal = 0.0f;
            _bUseExternalGoal = false;
        }
        public float GetOutput() { return _output; }
        public float GetExternalGoal() { return _externalGoal; }
        public QuadPIDController GetPID()
        {
            return _pidSet;
        }

    }
}
```

Runtime/Scripts/DroneCore/Controllers/AxisControllers/PositionController.cs

SHA-256 165d2c7aeed521d172551313901da87d3a15397b431cf870beed57d6a0b638d0 | 106 lines | 3,499 bytes

```
using DroneCore.Common;
using DroneCore.Interfaces;
using RobotCore;
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Controllers.AxisControllers
{
    public class PositionController
    {
        [SerializeField] private QuadPIDController _pidSet = new QuadPIDController();
        public QuadPIDController Pid => _pidSet;

        private int _axis;
        private ICommandSource _cmdGoal;
        private SensorManager _sensorManager;
        private DroneConfig _config;
        private float _output;
        private float _externalGoal;
        private bool _bInitialized;

        public PositionController(DroneConfig inConfig)
        {
            _config = inConfig;
        }

        public void Initialize(int axis, ICommandSource cmd, SensorManager sensors)
```

```
{
    _axis = axis;
    _cmdGoal = cmd;
    _sensorManager = sensors;
    PositionPID PID = _config.Position;

    if (axis == 3)
        _pidSet.SetLimits(-_config.FlightParams.MaxVelZ, _config.FlightParams.MaxVelZ);
    else
        _pidSet.SetLimits(-_config.FlightParams.MaxVelXY, _config.FlightParams.MaxVelXY);

    _pidSet.SetGains(PID.GetPGains()[_axis], PID.GetIGains()[_axis], PID.GetDGains()[_axis]);
    _bInitialized = true;
}

public void Update(float deltaTime)
{
    if (!_bInitialized || _cmdGoal == null || _sensorManager == null || _pidSet == null)
    {
        _output = 0f;
        return;
    }

    SensorData StateData = _sensorManager.Latest;
    float Goal = _cmdGoal.GetCommandValue()[_axis];

    // Axis 2 (yaw) passes through — no position PID for yaw
    if (_axis == 2)
    {
        _output = Goal;
        return;
    }

    // GPS position is in the output frame (e.g. FLU).
    // Cascade axis mapping:
    //   Axis 0 (forward)  → output frame .x
    //   Axis 1 (lateral)  → output frame .y
    //   Axis 3 (altitude) → output frame .z
    //
    // This works for FLU (x=fwd, y=left, z=up) and FRD (x=fwd, y=right, z=down).
    // The PID sign is correct in both cases because the command frame matches
    // the measurement frame — both are in the output frame.
    float State;
    switch (_axis)
    {
        case 0: State = StateData.GpsPosition.x; break;  // forward
        case 1: State = StateData.GpsPosition.y; break;  // lateral
        case 3: State = StateData.GpsPosition.z; break;  // altitude
        default: State = 0f; break;
    }

    _output = _pidSet.Calculate(Goal, State, deltaTime);
}

public void Reset()
{
    if (_pidSet != null) _pidSet.Reset();
    _output = 0.0f;
}

public void SetExternalGoal(float Value)
{
    _externalGoal = Value;
}

public void ClearExternalGoal()
{
    _externalGoal = 0.0f;
}

public float GetOutput() { return _output; }
public float GetExternalGoal() { return _externalGoal; }
```

```
        public QuadPIDController GetPID()
        {
            return _pidSet;
        }
    }
}
```

Runtime/Scripts/DroneCore/Controllers/AxisControllers/VelocityController.cs

SHA-256 98f7d9121041d87e4df94845179596f70b2c1e8dbfbe63535a762d9d73546344 | 109 lines | 3,829 bytes

```
using System;
using DroneCore.Common;
using DroneCore.Interfaces;
using RobotCore; // SensorManager, SensorData
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.Controllers.AxisControllers
{
    public class VelocityController
    {
        private const float Gravity = 9.81f;

        // Keep half of hover lift available while braking an ascent. A full
        // -g request turns the motors effectively off until the vehicle slows,
        // which presents as a stall after a large vertical velocity step.
        public const float ClosedLoopMinLiftFraction = 0.5f;

        [SerializeField] private QuadPIDController _pidSet = new QuadPIDController();
        public QuadPIDController Pid => _pidSet;

        private int _axis;
        private ICommandSource _cmdGoal;
        private SensorManager _sensorManager;
        private DroneConfig _config;
        private float _output;
        private float _externalGoal;
        private bool _bInitialized;
        private bool _bUseExternalGoal;
        private float maxAcceleration;
        public VelocityController(DroneConfig inConfig)
        {
            _config = inConfig;
        }

        public void Initialize(int axis, ICommandSource cmd, SensorManager sensors)
        {
            _axis = axis;
            _cmdGoal = cmd;
            _sensorManager = sensors;
            VelocityPID PID = _config.Velocity;

            float maxAccelLimit = Mathf.Tan(_config.FlightParams.MaxAngle * Mathf.Deg2Rad) * Gravity;
            if (axis == 3)
            {
                float maxDownwardAccel = Gravity * (1f - ClosedLoopMinLiftFraction);
                _pidSet.SetLimits(-maxDownwardAccel, Gravity);
            }
            else
            {
                _pidSet.SetLimits(-maxAccelLimit, maxAccelLimit);
            }
            _pidSet.SetGains(PID.GetPGains()[_axis], PID.GetIGains()[_axis], PID.GetDGains()[_axis]);

            _bInitialized = true;
        }

        public void Update(float deltaTime)
        {
            if (!_bInitialized || _sensorManager == null || _pidSet == null || (_cmdGoal == null && !_bU
seExternalGoal))
            {
                _output = 0.0f;
```

```
                return;
            }

            SensorData StateData = _sensorManager.Latest;
            float Goal = _bUseExternalGoal ? _externalGoal :  _cmdGoal.GetCommandValue()[_axis];

            if (_axis == 2)
            {
                _output = Goal;
                return;
            }

            // Preserve the original body-local semantics for forward/left.
            // Vertical control is the exception: use world-up velocity so pitch
            // and roll cannot project horizontal flight into the altitude loop.
            float StateVel = _axis == 3
                ? StateData.HeadingVel.z
                : StateData.ImuVel[_axis];
            _output = _pidSet.Calculate(Goal, StateVel, deltaTime);

        }

        public void Reset()
        {
            if(_pidSet!=null) _pidSet.Reset();
            _output = 0.0f;
            _bUseExternalGoal = false;
        }
        public void SetExternalGoal(float Value)
        {
            _externalGoal = Value;
            _bUseExternalGoal = true;
        }
        public void ClearExternalGoal()
        {
            // TODO: Double check using this is actually needed or if its updated during runtime
            _externalGoal = 0.0f;
            _bUseExternalGoal = false;
        }
        public float GetOutput() { return _output; }
        public float GetExternalGoal() { return _externalGoal; }
        public QuadPIDController GetPID()
        {
            return _pidSet;
        }
    }
}
```

Runtime/Scripts/DroneCore/Controls/ControlAllocator.cs

SHA-256 dac8f763e2b62593c5d523a069c67862bb044837e7c516f5900569b07a21227e | 621 lines | 26,351 bytes

```
using System;
using UnityEngine;
using MathUtil;
using Yaml.Drone;

namespace DroneCore.Controls
{
    // ─────────────────────────────────────────────────────────────
    //  Effectiveness matrix  (mirrors UE FEffectivenessMatrix)
    // ─────────────────────────────────────────────────────────────

    /// <summary>
    /// 6-axis effectiveness matrix and its pseudo-inverse.
    /// Axes: [0]=Roll, [1]=Pitch, [2]=Yaw, [3]=Fx, [4]=Fy, [5]=Fz(thrust).
    /// Supports up to 8 motors.
    /// </summary>
    [Serializable]
    public sealed class EffectivenessMatrix
    {
        public const int MaxMotors = 8;
        public const int NumAxes   = 6;
```

```
    public int NumMotors;

    /// <summary>Axes × Motors  [NumAxes, MaxMotors]</summary>
    public readonly float[,] Data         = new float[NumAxes, MaxMotors];

    /// <summary>Motors × Axes  [MaxMotors, NumAxes]</summary>
    public readonly float[,] PseudoInverse = new float[MaxMotors, NumAxes];

    public void Clear()
    {
        NumMotors = 0;
        Array.Clear(Data,          0, Data.Length);
        Array.Clear(PseudoInverse, 0, PseudoInverse.Length);
    }
}

[Serializable]
public sealed class AllocationDiagnostics
{
    public Vector4 Requested;
    public Vector4 Achieved;
    public float RollPitchScale = 1f;
    public float YawScale = 1f;
    public float Residual;
    public bool[] SaturatedMotors = Array.Empty<bool>();
}

// ─────────────────────────────────────────────────────────────────
//  Control allocator  (mirrors UE FControlAllocator)
// ─────────────────────────────────────────────────────────────────

/// <summary>
/// Allocates (roll, pitch, yaw, thrustZ) control demands into motor outputs [0..1]
/// using an effectiveness-matrix-based approach.
///
/// Unity body frame convention:
///   +X forward (nose)   = transform.right
///   +Y up      (lift)   = transform.up
///   +Z left             = transform.forward
///
/// Control axes:
///   Roll  = torque about +X (forward)
///   Pitch = torque about -Z (right, since +Z is left)
///   Yaw   = torque about +Y (up)
/// </summary>
public sealed class ControlAllocator
{

    public float MinAuthorityRatio = 0.01f;

    // ── Internal state ──────────────────────────────────────────
    private readonly EffectivenessMatrix _effectiveness = new EffectivenessMatrix();
    private AllocationDiagnostics _lastAllocation = new AllocationDiagnostics();

    public ControlAllocator()
    {
        Reset();
    }

    // ── Public accessors ────────────────────────────────────────
    public int                      NumMotors       => _effectiveness.NumMotors;
    public EffectivenessMatrix      Effectiveness   => _effectiveness;
    public AllocationDiagnostics    LastAllocation  => _lastAllocation;
    public int                      Rank            { get; private set; }
    public float                    ConditionNumber { get; private set; }
    public bool IsFullRank => Rank == Mathf.Min(ControlAxes.Length, NumMotors);
    // ── Reset ───────────────────────────────────────────────────
    public void Reset()
    {
        _effectiveness.Clear();
        Rank = 0;
        ConditionNumber = float.PositiveInfinity;
        ResetDiagnostics();
```

```
        }

        public void ResetDiagnostics() => _lastAllocation = new AllocationDiagnostics();

        public float AllocateWrench(
            float thrustN, float torqueXNm, float torqueYNm, float torqueZNm,
            float maxThrust, float minMotorOutput, float[] outMotor01)
        {
            int numMotors = _effectiveness.NumMotors;
            if (numMotors <= 0 || maxThrust <= 1e-6f)
            {
                if (outMotor01 != null) Array.Clear(outMotor01, 0, outMotor01.Length);
                ResetDiagnostics();
                return 0f;
            }

            // Total-thrust row makes this the fraction of all-motors-full.
            float thrust01 = thrustN / (numMotors * maxThrust);

            // Per-axis round-trip gains: g_a = maxThrust * Σ Data[a,m]*PInv[m,a].
            float gRoll  = AxisGain(0, maxThrust);
            float gPitch = AxisGain(1, maxThrust);
            float gYaw   = AxisGain(2, maxThrust);

            float roll01  = (Mathf.Abs(gRoll)  > 1e-9f) ? torqueXNm / gRoll  : 0f;
            float pitch01 = (Mathf.Abs(gPitch) > 1e-9f) ? torqueYNm / gPitch : 0f;
            float yaw01   = (Mathf.Abs(gYaw)   > 1e-9f) ? torqueZNm / gYaw   : 0f;


            return AllocateYawLimited(roll01, pitch01, yaw01, thrust01, minMotorOutput, outMotor01);
        }

        /// <summary>
        /// Physical torque produced on `axis` per unit normalized demand, by
        /// probing the live pseudo-inverse and effectiveness rows. Auto-tracks
        /// geometry and the authority clamp in ComputePseudoInverse.
        /// </summary>
        private float AxisGain(int axis, float maxThrust)
        {
            int n = _effectiveness.NumMotors;
            float s = 0f;
            for (int m = 0; m < n; m++)
                s += _effectiveness.Data[axis, m] * _effectiveness.PseudoInverse[m, axis];
            return maxThrust * s;
        }

        public void RebuildEffectiveness(RotorInstance[] rotors,RotorPhysicsDerived  rotorPhysics, SimFr
ame frame)
        {
            if (rotors == null)
            {
                Reset();
                return;
            }

            int numMotors = Mathf.Min(rotors.Length, EffectivenessMatrix.MaxMotors);
            _effectiveness.NumMotors = numMotors;

            if (numMotors <= 0)
            {
                Reset();
                return;
            }

            // Torque-to-thrust ratio  (mirrors UE Kq)
            float kq = (rotorPhysics.MaxThrust > 1e-6f)? (rotorPhysics.MaxTorque / rotorPhysics.MaxThrus
t): 0f;

            // Clear data rows
            Array.Clear(_effectiveness.Data, 0, _effectiveness.Data.Length);

            for (int m = 0; m < numMotors; m++)
            {
```

```
                ref readonly RotorInstance rotor = ref rotors[m];

                Vector3 pos       = Frames.TransformLinear(rotor.Location_Body,frame);
                float   spinDir   = Frames.TransformBinary(rotor.Spin_Dir,frame);
                Vector3 thrustDir = Frames.TransformLinear(new Vector3(0f, 1f, 0f), frame);

                // Reactive yaw torque per unit motor command
                float reactiveTorque = spinDir * kq;

                Vector3 torqueArm = Vector3.Cross(pos, thrustDir);

                // Direct FLU axis mapping — no manual negations needed
                _effectiveness.Data[0, m] = torqueArm.x;                    // Roll  about FLU +X
                _effectiveness.Data[1, m] = torqueArm.y;                    // Pitch about FLU +Y (was:
-torqueArm.z)
                _effectiveness.Data[2, m] = torqueArm.z + reactiveTorque;   // Yaw   about FLU +Z (was:
torqueArm.y + reactive)
                _effectiveness.Data[3, m] = 0f;
                _effectiveness.Data[4, m] = 0f;
                _effectiveness.Data[5, m] = thrustDir.z / numMotors;        // Thrust along FLU +Z (was:
 thrustDir.y)

            }

            ComputePseudoInverse();
            AnalyzeEffectiveness();
        }
        // Control axes routed through the allocator: Roll, Pitch, Yaw, Thrust.
        // (Fx/Fy rows are unused for a standard quad and stay zero.)
        private static readonly int[] ControlAxes = { 0, 1, 2, 5 };

        /// <summary>
        /// Build the true minimum-norm pseudo-inverse  M = Aᵀ (A Aᵀ)⁻¹  of the
        /// control-axis rows, where A = Data[ControlAxes, :].
        ///
        /// This solves  A · motor = demand  exactly while minimising the motor
        /// norm. Unlike the previous normalized-transpose, it cross-couples the
        /// axes, so a pure-thrust demand on an *unbalanced* frame (e.g. rotors
        /// not symmetric about the CoM) still yields zero residual roll/pitch/yaw
        /// — no thrust→torque bleed. Falls back to the normalized transpose if the
        /// geometry Gram matrix is singular.
        /// </summary>
        private void ComputePseudoInverse()
        {
            int numMotors = _effectiveness.NumMotors;
            Array.Clear(_effectiveness.PseudoInverse, 0, _effectiveness.PseudoInverse.Length);

            if (numMotors <= 0) return;

            int nC = ControlAxes.Length;

            // Gram = A Aᵀ   (nC × nC),  A[i,m] = Data[ControlAxes[i], m]
            double[,] gram = new double[nC, nC];
            for (int i = 0; i < nC; i++)
            {
                for (int j = 0; j < nC; j++)
                {
                    double s = 0.0;
                    for (int m = 0; m < numMotors; m++)
                        s += (double)_effectiveness.Data[ControlAxes[i], m] *
                                     _effectiveness.Data[ControlAxes[j], m];
                    gram[i, j] = s;
                }
            }

            if (!TryInvertSquare(gram, nC, out double[,] gramInv))
            {
                // Degenerate geometry — fall back to the normalized transpose.
                ComputePseudoInverseTranspose();
                return;
            }

            // M[m, ControlAxes[i]] = Σ_j Data[ControlAxes[j], m] · gramInv[j, i]
```

```
            for (int m = 0; m < numMotors; m++)
            {
                for (int i = 0; i < nC; i++)
                {
                    double s = 0.0;
                    for (int j = 0; j < nC; j++)
                        s += (double)_effectiveness.Data[ControlAxes[j], m] * gramInv[j, i];
                    _effectiveness.PseudoInverse[m, ControlAxes[i]] = (float)s;
                }
            }
        }

        /// <summary>
        /// Legacy normalized-transpose with authority clamping. Retained as a
        /// fallback for degenerate geometries where the Gram matrix is singular.
        /// </summary>
        private void ComputePseudoInverseTranspose()
        {
            int numMotors = _effectiveness.NumMotors;
            int numAxes = EffectivenessMatrix.NumAxes;
            Array.Clear(_effectiveness.PseudoInverse, 0, _effectiveness.PseudoInverse.Length);

            if (numMotors <= 0) return;

            // Row norm squared for each axis
            float[] rowNormSq = new float[numAxes];
            for (int axis = 0; axis < numAxes; axis++)
            {
                for (int m = 0; m < numMotors; m++)
                    rowNormSq[axis] += _effectiveness.Data[axis, m] * _effectiveness.Data[axis, m];
            }

            // Max authority reference
            float maxAuthority = 0f;
            for (int axis = 0; axis < numAxes; axis++)
                maxAuthority = Mathf.Max(maxAuthority, rowNormSq[axis]);

            // Build pseudo-inverse with authority clamping
            for (int m = 0; m < numMotors; m++)
            {
                for (int axis = 0; axis < numAxes; axis++)
                {
                    if (rowNormSq[axis] > 1e-10f && maxAuthority > 1e-10f)
                    {
                        float effectiveAuthority = Mathf.Max(rowNormSq[axis],maxAuthority * MinAuthority
Ratio);
                        _effectiveness.PseudoInverse[m, axis] = _effectiveness.Data[axis, m] / effective
Authority;
                    }
                    else
                    {
                        _effectiveness.PseudoInverse[m, axis] = 0f;
                    }
                }
            }
        }

        /// <summary>
        /// In-place Gauss-Jordan inverse of an n×n matrix with partial pivoting.
        /// Returns false (leaving <paramref name="inv"/> null) if singular.
        /// </summary>
        private static bool TryInvertSquare(double[,] a, int n, out double[,] inv)
        {
            inv = null;

            // Augmented [A | I]
            double[,] m = new double[n, 2 * n];
            for (int r = 0; r < n; r++)
            {
                for (int c = 0; c < n; c++) m[r, c] = a[r, c];
                m[r, n + r] = 1.0;
            }
```

```
        for (int col = 0; col < n; col++)
        {
            // Partial pivot: largest magnitude in this column.
            int piv = col;
            double best = Math.Abs(m[col, col]);
            for (int r = col + 1; r < n; r++)
            {
                double v = Math.Abs(m[r, col]);
                if (v > best) { best = v; piv = r; }
            }
            if (best < 1e-12) return false;   // singular

            if (piv != col)
            {
                for (int c = 0; c < 2 * n; c++)
                {
                    double t = m[col, c]; m[col, c] = m[piv, c]; m[piv, c] = t;
                }
            }

            double diag = m[col, col];
            for (int c = 0; c < 2 * n; c++) m[col, c] /= diag;

            for (int r = 0; r < n; r++)
            {
                if (r == col) continue;
                double f = m[r, col];
                if (f == 0.0) continue;
                for (int c = 0; c < 2 * n; c++) m[r, c] -= f * m[col, c];
            }
        }

        inv = new double[n, n];
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++) inv[r, c] = m[r, n + c];
        return true;
    }

    private void AnalyzeEffectiveness()
    {
        int rows = ControlAxes.Length;
        int cols = _effectiveness.NumMotors;
        double[,] work = new double[rows, cols];
        double maxMagnitude = 0.0;
        for (int r = 0; r < rows; r++)
        {
            for (int c = 0; c < cols; c++)
            {
                double value = _effectiveness.Data[ControlAxes[r], c];
                work[r, c] = value;
                maxMagnitude = Math.Max(maxMagnitude, Math.Abs(value));
            }
        }

        Rank = 0;
        double tolerance = Math.Max(1e-10, maxMagnitude * 1e-7);
        for (int col = 0; col < cols && Rank < rows; col++)
        {
            int pivot = Rank;
            for (int r = Rank + 1; r < rows; r++)
                if (Math.Abs(work[r, col]) > Math.Abs(work[pivot, col])) pivot = r;
            if (Math.Abs(work[pivot, col]) <= tolerance) continue;

            for (int c = col; c < cols; c++)
            {
                double temp = work[Rank, c];
                work[Rank, c] = work[pivot, c];
                work[pivot, c] = temp;
            }
            double divisor = work[Rank, col];
            for (int c = col; c < cols; c++) work[Rank, c] /= divisor;
            for (int r = 0; r < rows; r++)
            {
```

```
                    if (r == Rank) continue;
                    double factor = work[r, col];
                    for (int c = col; c < cols; c++) work[r, c] -= factor * work[Rank, c];
                }
                Rank++;
            }

            if (!IsFullRank)
            {
                ConditionNumber = float.PositiveInfinity;
                return;
            }

            double aNorm = 0.0;
            for (int r = 0; r < rows; r++)
            {
                double sum = 0.0;
                for (int c = 0; c < cols; c++)
                    sum += Math.Abs(_effectiveness.Data[ControlAxes[r], c]);
                aNorm = Math.Max(aNorm, sum);
            }

            double inverseNorm = 0.0;
            for (int m = 0; m < cols; m++)
            {
                double sum = 0.0;
                for (int r = 0; r < rows; r++)
                    sum += Math.Abs(_effectiveness.PseudoInverse[m, ControlAxes[r]]);
                inverseNorm = Math.Max(inverseNorm, sum);
            }
            ConditionNumber = (float)(aNorm * inverseNorm);
        }

        public float AllocateYawLimited(float rollTorque01,float pitchTorque01,float yawTorque01,float t
hrustZ01,float minMotorOutput, float[] outMotor01)
        {
            int numMotors = _effectiveness.NumMotors;

            if (outMotor01 == null || outMotor01.Length < numMotors)
                throw new ArgumentException(
                    $"outMotor01 must be at least length {numMotors}.");

            if (numMotors <= 0)
            {
                if (outMotor01 != null) Array.Clear(outMotor01, 0, outMotor01.Length);
                ResetDiagnostics();
                return 0f;
            }

            // ── 1) Thrust base, then roll/pitch within remaining headroom ──
            // Keep collective thrust authoritative. The former final "shift down"
            // preserved torque differences by removing collective, causing altitude
            // loss during aggressive horizontal commands.
            float[] uThrust = { 0f, 0f, 0f, 0f, 0f, thrustZ01 };
            float[] uRollPitch = { rollTorque01, pitchTorque01, 0f, 0f, 0f, 0f };

            float[] baseMotor = new float[numMotors];
            float[] rollPitchDirection = new float[numMotors];
            for (int m = 0; m < numMotors; m++)
            {
                float thrust = 0f;
                float rollPitch = 0f;
                for (int axis = 0; axis < EffectivenessMatrix.NumAxes; axis++)
                {
                    thrust += _effectiveness.PseudoInverse[m, axis] * uThrust[axis];
                    rollPitch += _effectiveness.PseudoInverse[m, axis] * uRollPitch[axis];
                }

                baseMotor[m] = thrust;
                rollPitchDirection[m] = rollPitch;
            }

            float rollPitchScale = ComputeDemandScaleFromHeadroom(
```

```
            baseMotor, rollPitchDirection, numMotors, minMotorOutput, 1f);

        for (int m = 0; m < numMotors; m++)
            baseMotor[m] += rollPitchDirection[m] * rollPitchScale;

        // ── 2) Per-motor direction for +1.0 yaw command ──
        //   (column for yaw axis = axis 2 of pseudo-inverse)

        // TODO: Missing Num 2 from ControlAllocator.cpp
        float yawSign = (yawTorque01 >= 0f) ? 1f : -1f;
        float[] yawDirSigned = new float[numMotors];
        for (int m = 0; m < numMotors; m++)
            yawDirSigned[m] = _effectiveness.PseudoInverse[m, 2] * yawSign; // TODO: Change here

        // ── 3) Compute max yaw scale based on headroom ──

        float sMax = ComputeYawScaleFromHeadroom(
            baseMotor,
            yawDirSigned,
            numMotors,
            minMotorOutput,
            1f,
            0f
        );

        // sMax is an absolute allowed yaw magnitude in allocator units,
        // not a fractional multiplier.
        float yawAbs = Mathf.Abs(yawTorque01);
        float yawApplied01 = Mathf.Sign(yawTorque01) * Mathf.Min(yawAbs, sMax);

        // ── 4) Final allocation with applied yaw ──
        for (int m = 0; m < numMotors; m++)
        {
            outMotor01[m] = baseMotor[m]
                          + _effectiveness.PseudoInverse[m, 2] * yawApplied01;
        }

        // Numerical guard only. Feasible torque demands were scaled above;
        // do not translate all motors and thereby alter collective thrust.
        for (int m = 0; m < numMotors; m++)
            outMotor01[m] = Mathf.Clamp(outMotor01[m], minMotorOutput, 1f);

        float yawScale = (yawAbs > 1e-6f) ? Mathf.Min(1f, sMax / yawAbs) : 1f;
        UpdateDiagnostics(
            rollTorque01, pitchTorque01, yawTorque01, thrustZ01,
            rollPitchScale, yawScale, minMotorOutput, outMotor01);
        return yawScale;

    }

    private void UpdateDiagnostics(
        float roll, float pitch, float yaw, float thrust,
        float rollPitchScale, float yawScale, float minMotorOutput,
        float[] motor01)
    {
        int n = _effectiveness.NumMotors;
        var achieved = new Vector4();
        for (int m = 0; m < n; m++)
        {
            achieved.x += _effectiveness.Data[0, m] * motor01[m];
            achieved.y += _effectiveness.Data[1, m] * motor01[m];
            achieved.z += _effectiveness.Data[2, m] * motor01[m];
            achieved.w += _effectiveness.Data[5, m] * motor01[m];
        }

        Vector4 requested = new Vector4(roll, pitch, yaw, thrust);
        bool[] saturated = new bool[n];
        for (int m = 0; m < n; m++)
            saturated[m] = motor01[m] <= minMotorOutput + 1e-6f || motor01[m] >= 1f - 1e-6f;

        _lastAllocation = new AllocationDiagnostics
        {
            Requested = requested,
```

```
            Achieved = achieved,
            RollPitchScale = rollPitchScale,
            YawScale = yawScale,
            Residual = (requested - achieved).magnitude,
            SaturatedMotors = saturated,
        };
    }

    private static float ComputeDemandScaleFromHeadroom(
        float[] baseMotor, float[] direction, int numMotors,
        float minMotorOutput, float maxMotorOutput)
    {
        float scale = 1f;

        for (int m = 0; m < numMotors; m++)
        {
            float d = direction[m];

            if (d > 1e-8f)
                scale = Mathf.Min(scale, (maxMotorOutput - baseMotor[m]) / d);
            else if (d < -1e-8f)
                scale = Mathf.Min(scale, (baseMotor[m] - minMotorOutput) / -d);
        }

        return Mathf.Clamp01(scale);
    }

    // ─────────────────────────────────────────────────────────────
    //  PostProcessMotorOutputs  (mirrors UE static function)
    // ─────────────────────────────────────────────────────────────
    private static void PostProcessMotorOutputs(float[] motor01,int count,float minMotorOutput)
    {
        if (count <= 0) return;

        float minOut = motor01[0];
        float maxOut = motor01[0];
        for (int i = 1; i < count; i++)
        {
            minOut = Mathf.Min(minOut, motor01[i]);
            maxOut = Mathf.Max(maxOut, motor01[i]);
        }

        // Shift up if below minimum
        if (minOut < minMotorOutput)
        {
            float shift = minMotorOutput - minOut;
            for (int i = 0; i < count; i++) motor01[i] += shift;
            maxOut += shift;
        }

        // Shift down if above 1
        if (maxOut > 1f)
        {
            float shift = maxOut - 1f;
            for (int i = 0; i < count; i++) motor01[i] -= shift;
        }

        // Final clamp
        for (int i = 0; i < count; i++)
            motor01[i] = Mathf.Clamp(motor01[i], minMotorOutput, 1f);
    }

    // ─────────────────────────────────────────────────────────────
    //  ComputeYawScaleFromHeadroom  (mirrors UE static function)
    // ─────────────────────────────────────────────────────────────
    private static float ComputeYawScaleFromHeadroom(float[] baseMotor, float[] yawDirPerUnit,int nu
mMotors,float minMotorOutput,float maxMotorOutput = 1f,float safetyMargin   = 0f)
    {
        float sMax = 1f;

        for (int m = 0; m < numMotors; m++)
        {
            float b = baseMotor[m];
```

```
                float d = yawDirPerUnit[m];

                if (Mathf.Abs(d) < 1e-8f)
                    continue;

                // We want: min <= b + s*d <= max
                // Solve for s bounds depending on sign of d.

                if (d > 0f)
                {
                    float sUp = (maxMotorOutput - safetyMargin - b) / d;
                    sMax = Mathf.Min(sMax, sUp);
                }
                else // d < 0
                {
                    float sDown = (b - (minMotorOutput + safetyMargin)) / (-d);
                    sMax = Mathf.Min(sMax, sDown);
                }
            }

            return Mathf.Clamp(sMax, 0f, 1f);
        }
    }
}
```

Runtime/Scripts/DroneCore/Controls/Quadactuationmodel.cs

SHA-256 ddd1d6dd51294d86d770d7b813d07b4fbb21fc262d650cdd4e80625c48b1768e | 75 lines | 2,654 bytes

```
using UnityEngine;
using MathUtil;
using Yaml.Drone;

namespace DroneCore.Controls
{

    public sealed class MotorLagFilter
    {
        public bool  Enabled        = false;
        public float TauUpSeconds   = 0.020f;
        public float TauDownSeconds = 0.020f;

        private float[] _state;

        public void Reset(int n) => _state = new float[n];

        /// In-place: overwrites motor01 with the lagged actual output.
        public void Apply(float[] motor01, float dt)
        {
            if (!Enabled || motor01 == null) return;
            if (_state == null || _state.Length != motor01.Length)
                _state = (float[])motor01.Clone();   // first tick: start at command

            for (int i = 0; i < motor01.Length; i++)
            {
                _state[i] = RotorMath.FirstOrderStep(
                    _state[i], motor01[i], dt, TauUpSeconds, TauDownSeconds);
                motor01[i] = _state[i];
            }
        }
    }

    public static class QuadActuationModel
    {
        private const int SanityCap = 64;

        public static void ComputeBodyWrench(float[] motor01, RotorInstance[] rotors, RotorPhysicsDerive
d rotorPhysics, out Vector3 outForceBody, out Vector3 outTorqueBody)
        {
            outForceBody  = Vector3.zero;
            outTorqueBody = Vector3.zero;

            if (motor01 == null || rotors == null) return;
```

```
            int n = Mathf.Min(motor01.Length, rotors.Length);
            n = Mathf.Min(n, SanityCap);

            for (int i = 0; i < n; i++)
            {
                float cs = Mathf.Clamp01(motor01[i]);

                float thrustN  = rotorPhysics.ControlToThrust(cs, 1f);
                float torqueNm = rotorPhysics.ControlToYawTorque(cs, rotors[i].Spin_Dir, 1f);

                Vector3 rotorLoc = rotors[i].Location_Body;

                // Body thrust points +Y (up) — equivalent to UE's +Z body thrust
                Vector3 forceVec = new Vector3(0f, thrustN, 0f);

                Vector3 thrustMoment = Vector3.Cross(rotorLoc, forceVec);
                Vector3 dragTorque   = new Vector3(0f, torqueNm, 0f);
                outForceBody  += forceVec;
                outTorqueBody += thrustMoment + dragTorque;
            }
        }

        public static void BodyToWorldWrench(
            Quaternion bodyToWorld, Vector3 forceBody, Vector3 torqueBody,
            out Vector3 forceWorld, out Vector3 torqueWorld)
        {
            forceWorld = bodyToWorld * forceBody;
            torqueWorld = bodyToWorld * torqueBody;
        }
    }
}
```

8. Sensors and Georeferencing

Generates simulation-time IMU, GPS, barometer, and magnetometer observations with configurable rates and deterministic noise. The supporting georeference and magnetic-field utilities connect local ENU motion to WGS84 and geographic field values.

Files in this section

- Runtime/Scripts/RobotCore/RobotCore.cs
- Runtime/Scripts/RobotCore/SensorData.cs
- Runtime/Scripts/RobotCore/SensorManager.cs
- Runtime/Scripts/RobotCore/Sensors/BaroSensor.cs
- Runtime/Scripts/RobotCore/Sensors/GPSSensor.cs
- Runtime/Scripts/RobotCore/Sensors/IMUSensor.cs
- Runtime/Scripts/RobotCore/Sensors/MagSensor.cs
- Runtime/Scripts/RobotCore/Core Utils/GaussianRng.cs
- Runtime/Scripts/RobotCore/Core Utils/GeoMagDeclination.cs
- Runtime/Scripts/RobotCore/Core Utils/GeoMagneticTables.cs
- Runtime/Scripts/RobotCore/Core Utils/GeoReferencingSystem.cs

Runtime/Scripts/RobotCore/RobotCore.cs

SHA-256 bc020efd6a5db89f298b151e560b78ba36072c675675b15f62c7a92c0c942f43 | 20 lines | 491 bytes

```
using DroneCore;
using UnityEngine;

namespace RobotCore
{
    [RequireComponent(typeof(QuadPawn))]
    [DisallowMultipleComponent]
    public sealed class RobotCore : MonoBehaviour
    {
        [SerializeField] private SensorManager sensors;

        public SensorManager Sensors => sensors;

        private void Awake()
        {
            if (sensors == null)
                sensors = GetComponentInChildren<SensorManager>(includeInactive: true);
        }
    }
}
```

Runtime/Scripts/RobotCore/SensorData.cs

SHA-256 4036484a37da1ad85e16a269d999425e0d00eb94b42a3ebe4b49804d0dd1ba62 | 68 lines | 3,164 bytes

```
using MathUtil;
using UnityEngine;

namespace RobotCore
{
    // ============================================================================
    // FRAME CONVENTIONS (see MathUtil/Frames.cs header for the full policy)
    //
    //   WORLD data  → always ENU (x=East, y=North, z=Up).
    //                 NED appears only in explicitly named fields (GpsVelNED).
    //   BODY data   → BodyFrame convention: FLU (default) or FRD (PX4 mode).
    // ============================================================================
    public struct SensorData
    {
        public long EpisodeId;
        public ulong StepIndex;
        public long SimTimeNanos;
```

```
        public ulong Sequence;
        public string SourceFrame;

        /// <summary>Body-axis convention of all body-frame fields below
        /// (ImuAngVel/ImuAttitude/ImuAccel/ImuVel/ImuOrientation/MagField).
        /// FLU by default; FRD when the sim is in PX4 mode. World-frame
        /// fields are unaffected — always ENU.</summary>
        public SimFrame BodyFrame;

        // IMU — BODY frame (BodyFrame convention)
        public Vector3 ImuAngVel;        // rad/s, body
        public Vector3 ImuAttitude;      // RPY deg in BodyFrame convention
        public Vector3 ImuAccel;         // m/s², body specific force (gravity included)
        public Vector3 ImuVel;           // m/s, BODY-frame velocity (not world!)
        public Quaternion ImuOrientation; // body(BodyFrame) → world(ENU)
        public double ImuTimestampSec;
        public bool ImuValid;

        // Controller feedback — yaw/heading frame, independent of roll and pitch.
        // X=forward, Y=left, Z=up. This is intentionally not the IMU body velocity:
        // using a fully tilted body frame would project horizontal flight onto Z.
        public Vector3 HeadingVel;

        // GPS — WORLD frame, always ENU (x=E, y=N, z=Up/altitude)
        public Vector3 GpsPosition;
        public double GpsTimestampSec;
        public bool GpsValid;

        // Baro (pressure in Pa, altitudes in m, temp in degC) — scalars
        public float  BaroPressurePa;
        public float  BaroPressureHPa;       // HIL_SENSOR.abs_pressure
        public float  BaroTemperatureC;      // HIL_SENSOR.temperature
        public float  BaroAltitudeMSL;       // true input altitude (ground truth)
        public float  BaroPressureAltitude;  // HIL_SENSOR.pressure_alt (derived from pressure)
        public double BaroTimestampSec;
        public bool   BaroValid;

        // Mag — BODY frame (BodyFrame convention), Gauss
        public Vector3 MagField;
        public float   MagHeadingDeg;
        public float   MagDeclinationDeg;
        public double  MagTimestampSec;
        public bool    MagValid;

        // GPS geodetic (WGS84) + WORLD-frame NED velocity (PX4 HIL_GPS wire quantity)
        public double  GpsLatDeg;
        public double  GpsLonDeg;
        public double  GpsAltM;        // ellipsoidal
        public Vector3 GpsVelNED;      // (vn, ve, vd) m/s — world NED, explicitly named
    }
}
```

Runtime/Scripts/RobotCore/SensorManager.cs

SHA-256 e3f37ce17f2c09eaefcbf1f1d49f4a0b1320405e6d84dfc9b029ed1d2ab62296 | 319 lines | 12,035 bytes

```
using DroneCore;
using MathUtil;
using RobotCore.Sensors;
using SimCore;
using UnityEngine;
using RobotCore.Core_Utils;

namespace RobotCore
{
    [RequireComponent(typeof(QuadPawn))]
    [DisallowMultipleComponent]
    public sealed class SensorManager : MonoBehaviour
    {
        [Header("Refs")]
        [SerializeField] private Rigidbody rb;
        [SerializeField] private Transform bodyTransform;

        [Header("Output Frame")]
```

```
public SimFrame outputFrame = SimFrame.FLU;

[Header("Rates (Hz)")]
public double imuHz = 250.0;
public double gpsHz = 10.0;
public double baroHz = 50.0;
public double magHz = 50.0;

[Header("Noise")]
// Keep enabled for PX4 SITL/HIL: PX4's sensor validator flags more
// than 100 bit-identical samples as STALE even when timestamps advance.
public bool enableNoise = false;

[Header("Georeferencing (PX4 origin)")]
public double originLatDeg  = 47.397742;  // PX4 SITL default home (Zurich)
public double originLonDeg  = 8.545594;
public double originHeightM = 488.0;

[Header("Debug Logging")]
public bool logSensors = true;
public float logHz = 2f;

private double _logAccum;

public SimFrame TargetFrame = SimFrame.FLU;
private FrequencyLimiter _imuLimiter;
private FrequencyLimiter _gpsLimiter;
private FrequencyLimiter _baroLimiter;
private FrequencyLimiter _magLimiter;
private FrequencyLimiter _logLimiter;

public GeoReferencingSystem Geo { get; private set; }
public IMUSensor IMU { get; private set; }
public GPSSensor GPS { get; private set; }
public BaroSensor Baro { get; private set; }
public MagSensor Mag { get; private set; }


public SensorData Latest { get; private set; }
public int noiseSeed = 12345;
private long _episodeId;
private ulong _sequence;
public int ExecutionOrder => 50;
public string DebugName => name;

[ContextMenu("Toggle Output Frame  FLU <-> FRD")]
public void ToggleOutputFrame()
{
    SetOutputFrame((outputFrame == SimFrame.FRD) ? SimFrame.FLU : SimFrame.FRD);
}

/// <summary>
/// Select the body-axis convention exposed to external clients.
/// World-frame fields remain ENU. UnityBody is intentionally not exposed
/// because vectors and pseudovectors require different chirality maps.
/// </summary>
public bool SetOutputFrame(SimFrame frame)
{
    if (frame != SimFrame.FLU && frame != SimFrame.FRD)
        return false;

    outputFrame = frame;
    if (IMU != null) IMU.OutputFrame = frame;
    if (Mag != null) Mag.OutputFrame = frame;
    Debug.Log($"[SensorManager] outputFrame → {outputFrame}");
    return true;
}

private void ResolveRefsOrThrow()
{
    if (rb == null) rb = GetComponentInParent<Rigidbody>();
    if (rb == null)
        throw new System.Exception($"SensorManager on '{name}' cannot find Rigidbody in parents.
```

```
 Put it under DroneRoot.");

            if (bodyTransform == null) bodyTransform = rb.transform;
        }

        private void Awake()
        {
            ResolveRefsOrThrow();
        }

        /// <summary>Apply scene-level env BEFORE OnStart(). Single writer for seed/noise/origin.
        /// The +1/+2/+3/+4 per-sensor offsets are still applied from noiseSeed inside OnStart().</summa
ry>
        public void Configure(int seed, bool noise, double latDeg, double lonDeg, double heightM,SimFram
e frame)
        {
            noiseSeed     = seed;
            enableNoise   = noise;
            originLatDeg  = latDeg;
            originLonDeg  = lonDeg;
            originHeightM = heightM;
            outputFrame = frame;

        }

        public void ReseedForEpisode(int episodeSeed, int agentIndex)
        {
            noiseSeed = DeriveSeed(episodeSeed, agentIndex, 0);
            IMU?.Reseed(DeriveSeed(episodeSeed, agentIndex, 1));
            GPS?.Reseed(DeriveSeed(episodeSeed, agentIndex, 2));
            Baro?.Reseed(DeriveSeed(episodeSeed, agentIndex, 3));
            Mag?.Reseed(DeriveSeed(episodeSeed, agentIndex, 4));
        }

        public void BeginEpisode(long episodeId, int episodeSeed, int agentIndex)
        {
            _episodeId = episodeId;
            _sequence = 0;
            ReseedForEpisode(episodeSeed, agentIndex);
        }

        private static int DeriveSeed(int episodeSeed, int agentIndex, int streamId)
        {
            unchecked
            {
                uint value = (uint)episodeSeed;
                value ^= 0x9E3779B9u * (uint)(agentIndex + 1);
                value ^= 0x85EBCA6Bu * (uint)(streamId + 1);
                value ^= value >> 16;
                value *= 0x7FEB352Du;
                value ^= value >> 15;
                value *= 0x846CA68Bu;
                value ^= value >> 16;
                return (int)value;
            }
        }
        public void OnStart()
        {
            ResolveRefsOrThrow();

            IMU  = new IMUSensor(rb, bodyTransform, noiseSeed + 1);
            GPS  = new GPSSensor(rb, noiseSeed + 2);

            Geo = SimCoreApi.Get().Geo;
            Geo.SetOrigin(originLatDeg, originLonDeg,originHeightM);

            Baro = new BaroSensor(rb, bodyTransform, noiseSeed + 3);

            Mag = new MagSensor(rb, bodyTransform, noiseSeed + 4);


            // outputFrame selects the BODY-axis convention only (FLU/FRD).
            // World-frame data (GPS local position) is always ENU.
```

```
        IMU.OutputFrame = outputFrame;
        Mag.OutputFrame = outputFrame;

        IMU.Reset();
        IMU.Initialize(rb);

        GPS.Reset();
        GPS.Initialize(Geo);

        Baro.Reset();
        Baro.Initialize(Geo);

        Mag.Reset();
        Mag.Initialize(Geo);

        long now = ClockFactory.Clock.NowNanos;
        _imuLimiter  = new FrequencyLimiter(imuHz, now);
        _gpsLimiter  = new FrequencyLimiter(gpsHz, now);
        _baroLimiter = new FrequencyLimiter(baroHz, now);
        _magLimiter = new FrequencyLimiter(magHz, now);

        _logLimiter  = new FrequencyLimiter(logHz <= 0 ? 1e-6 : logHz, now);

        Latest = default;
        PrimeSnapshot(now);
    }

    public void OnReset()
    {
        _sequence = 0;
        IMU.Reset();
        IMU.Initialize(rb);

        GPS.Reset();
        GPS.Initialize(Geo);

        Baro.Reset();
        Baro.Initialize(Geo);

        Mag.Reset();
        Mag.Initialize(Geo);

        long now = ClockFactory.Clock.NowNanos;

        _imuLimiter?.Reset(now);
        _gpsLimiter?.Reset(now);
        _baroLimiter?.Reset(now);
        _magLimiter?.Reset(now);
        _logLimiter?.Reset(now);

        Latest = default;
        PrimeSnapshot(now);
    }

    /// <summary>
    /// Populate the cached snapshot immediately without advancing physics.
    /// This guarantees that the first SDK get_sensors() call is valid even
    /// when the simulation starts or remains paused.
    /// </summary>
    private void PrimeSnapshot(long nowNanos)
    {
        double dtSec = ClockFactory.BaseDtSec;
        if (dtSec <= 0.0)
            dtSec = 1.0 / 250.0;

        UpdateSensors(dtSec, nowNanos);
    }

    public void UpdateSensors(double dtSec, long nowNanos)
    {
        double nowSec = nowNanos * 1e-9;

        // Keep BODY output frame synced (IMU + mag). GPS position is ENU always.
```

```
            IMU.OutputFrame = outputFrame;
            Mag.OutputFrame = outputFrame;

            var latest = Latest;
            latest.BodyFrame = outputFrame;   // self-description for the RPC payload
            latest.EpisodeId = _episodeId;
            latest.StepIndex = ClockFactory.Clock.StepCount;
            latest.SimTimeNanos = nowNanos;
            latest.Sequence = ++_sequence;
            latest.SourceFrame = outputFrame == SimFrame.FRD
                ? "body_frd/world_enu"
                : "body_flu/world_enu";

            // Translational control uses a yaw-only heading frame. Roll and pitch
            // must not rotate horizontal motion into the vertical velocity loop.
            // Unity local axes are X=forward, Y=up, Z=left; expose FLU ordering.
            // Keep this explicit transform identical to the setpoint conversion
            // in CascadedController.OutputFrameVelToHeadingVel.
            float yawRad = bodyTransform.eulerAngles.y * Mathf.Deg2Rad;
            float cosYaw = Mathf.Cos(yawRad);
            float sinYaw = Mathf.Sin(yawRad);
            Vector3 velocityWorld = rb.linearVelocity;
            latest.HeadingVel = new Vector3(
                cosYaw * velocityWorld.x + sinYaw * velocityWorld.z,
                -sinYaw * velocityWorld.x + cosYaw * velocityWorld.z,
                velocityWorld.y);

            // IMU
            if (_imuLimiter.ShouldRunAndConsume(nowNanos))
            {
                IMU.Sample(nowSec, (float)dtSec, enableNoise);   // was: IMU.Sample(nowSec, (float)dtSec
);

                latest.ImuAngVel = IMU.LastAngVel;
                latest.ImuAttitude = IMU.LastAtt;
                latest.ImuOrientation = IMU.LastOrientation;
                latest.ImuAccel = IMU.LastLinAcc;
                latest.ImuVel = IMU.LastVel;
                latest.ImuTimestampSec = IMU.LastTimestampSec;
                latest.ImuValid = IMU.IsValid;
            }

            // GPS — local position is always world-frame ENU from GPSSensor
            if (_gpsLimiter.ShouldRunAndConsume(nowNanos))
            {
                GPS.Sample(nowSec, enableNoise);

                latest.GpsPosition     = GPS.LastPosition;
                latest.GpsTimestampSec = GPS.LastTimestampSec;
                latest.GpsValid        = GPS.HasFix;

                latest.GpsLatDeg = GPS.LastLatDeg;
                latest.GpsLonDeg = GPS.LastLonDeg;
                latest.GpsAltM   = GPS.LastAltM;
                latest.GpsVelNED = GPS.LastVelNED;
            }

            // Baro
            if (_baroLimiter.ShouldRunAndConsume(nowNanos))
            {
                Baro.Sample(nowSec, enableNoise);

                latest.BaroPressurePa       = Baro.LastPressure;
                latest.BaroPressureHPa      = Baro.LastPressureHPa;
                latest.BaroTemperatureC     = Baro.LastTemperature;
                latest.BaroAltitudeMSL      = Baro.EstimatedAltitude;
                latest.BaroPressureAltitude = Baro.PressureAltitude;
                latest.BaroTimestampSec     = Baro.LastTimestampSec;
                latest.BaroValid            = Baro.IsValid;
            }
            // Mag
            if (_magLimiter.ShouldRunAndConsume(nowNanos))
            {
```

```
                Mag.Sample(nowSec, (float)dtSec, enableNoise);

                latest.MagField          = Mag.LastMagField;
                latest.MagHeadingDeg     = Mag.MagneticHeading;
                latest.MagDeclinationDeg = Mag.MagneticDeclination;
                latest.MagTimestampSec   = Mag.LastTimestampSec;
                latest.MagValid          = Mag.IsValid;
            }

            Latest = latest;
        }
    }
}
```

Runtime/Scripts/RobotCore/Sensors/BaroSensor.cs

SHA-256 fb332b2419125402d42a596d77f0cf08fb97aea7cacf96bad34554b6ec1344f9 | 124 lines | 4,399 bytes

```
using SimCore;
using MathUtil;
using UnityEngine;
using RobotCore.Core_Utils;

namespace RobotCore.Sensors
{
    public sealed class BaroSensor
    {
        private float BaroDriftPa = 0.0f;
        private float PressureNoiseStdDev = 0.5f;
        private float DriftPaPerSec = 0.005f;

        private readonly GaussianRng _rng;

        private float PressureOffset = 0.0f;
        private float TemperatureOffset = 0.0f;

        private const float ISA_LAPSE_RATE = 0.0065f;
        private const float ISA_TEMPERATURE_MSL = 288.15f;
        private const float ISA_PRESSURE_MSL = 101325.0f;
        private const float ISA_EXPONENT = 5.256f; // g*M/(R*L)

        private Rigidbody _rb;
        private Transform _bodyTransform;
        private GeoReferencingSystem _geo;

        private double _lastSampleSec;
        private bool _hasPrevSample;

        public float LastPressure { get; private set; }
        public float LastPressureHPa => LastPressure / 100.0f; // HIL_SENSOR.abs_pressure units
        public float LastTemperature { get; private set; } // degC
        public float EstimatedAltitude { get; private set; } // true geometric MSL altitude (input)

        // Altitude derived back from the noisy pressure — this is PX4 HIL_SENSOR.pressure_alt,
        // and the "altitude the baro believes" once noise/drift/offset are applied.
        public float PressureAltitude
        {
            get
            {
                float ratio = Mathf.Pow(LastPressure / ISA_PRESSURE_MSL, 1.0f / ISA_EXPONENT);
                return (ISA_TEMPERATURE_MSL / ISA_LAPSE_RATE) * (1.0f - ratio);
            }
        }

        public double LastTimestampSec { get; private set; }
        public bool IsValid { get; private set; }

        public BaroSensor(Rigidbody rb, Transform bodyTransform, int noiseSeed = 3)
        {
            _rb = rb;
            _bodyTransform = bodyTransform;
            _rng = new GaussianRng(noiseSeed);
        }
```

```
        public void Reseed(int seed) => _rng.Reset(seed);

        public void Reset()
        {
            BaroDriftPa = 0.0f;
            _hasPrevSample = false;
            _lastSampleSec = 0.0;
            LastPressure = ISA_PRESSURE_MSL;
            LastTemperature = ISA_TEMPERATURE_MSL - 273.15f;
            EstimatedAltitude = 0.0f;
            LastTimestampSec = 0.0;
            IsValid = false;
        }

        public void Initialize(GeoReferencingSystem geo)
        {
            _geo = geo;
            IsValid = (_geo != null && _rb != null);
        }

        // Called by SensorManager at baro rate. dt is computed internally so drift stays
        // correct regardless of how the manager schedules us.
        public void Sample(double nowSec, bool addNoise)
        {
            if (_geo == null || _rb == null)
            {
                IsValid = false;
                return;
            }

            float altMsl = GetAltitudeMSL();
            EstimatedAltitude = altMsl;

            CalculatePressureFromAltitude(altMsl); // sets clean LastPressure + LastTemperature

            if (addNoise)
            {
                double dt = _hasPrevSample ? (nowSec - _lastSampleSec) : 0.0;
                BaroDriftPa += DriftPaPerSec * (float)dt;
                LastPressure += BaroDriftPa;
                LastPressure += PressureNoiseStdDev * _rng.Next();
            }

            LastPressure += PressureOffset;
            LastTemperature += TemperatureOffset;

            _lastSampleSec = nowSec;
            _hasPrevSample = true;
            LastTimestampSec = nowSec;
            IsValid = true;
        }

        private float GetAltitudeMSL()
        {
            // Single frame boundary: world -> FLU, Z is up. Origin height + up = MSL altitude.
            float up = Frames.TransformWorldPosition(_rb.position, SimFrame.FLU).z;
            return (float)(_geo.OriginHeight + up);
        }

        private void CalculatePressureFromAltitude(float altitudeMsl)
        {
            float tLocal = ISA_TEMPERATURE_MSL - ISA_LAPSE_RATE * altitudeMsl;
            float ratio = Mathf.Pow(ISA_TEMPERATURE_MSL / tLocal, ISA_EXPONENT);
            LastPressure = ISA_PRESSURE_MSL / ratio;
            LastTemperature = tLocal - 273.15f;
        }
    }
}
```

Runtime/Scripts/RobotCore/Sensors/GPSSensor.cs

SHA-256 580895b1417960b6974fc6665bfa34dc5503134e17cd85d65dc42cd627c1bb09 | 98 lines | 3,654 bytes

```
using MathUtil;
```

```
using RobotCore.Core_Utils;
using UnityEngine;

namespace RobotCore.Sensors
{
    public sealed class GPSSensor
    {
        private Rigidbody _rb;

        /// <summary>
        /// Last sampled local position, ALWAYS world-frame ENU
        /// (x=East, y=North, z=Up/altitude), independent of the body-frame
        /// output convention (FLU/FRD) used by the IMU and magnetometer.
        /// NED is a client-side relabel: (n, e, d) = (y, x, -z).
        /// Policy: world I/O is ENU everywhere inside QuadSim; NED exists
        /// only at the MAVLink wire boundary and in LastVelNED.
        /// </summary>
        public Vector3 LastPosition { get; private set; }

        public double LastTimestampSec { get; private set; }
        public bool HasFix { get; private set; }

        public float LatLonNoiseStdDev = 0.5f;  // m, horizontal (X,Y)
        public float AltNoiseStdDev    = 1.0f;  // m, vertical (Z)
        private readonly GaussianRng _rng;
        GeoReferencingSystem _geo;

        public double  LastLatDeg { get; private set; }
        public double  LastLonDeg { get; private set; }
        public double  LastAltM   { get; private set; }   // ellipsoidal
        public Vector3 LastVelNED { get; private set; }
        public bool    IsValid          { get; private set; }


        public GPSSensor(Rigidbody inRb, int noiseSeed = 2)
        {
            _rb = inRb;
            _rng = new GaussianRng(noiseSeed);
        }

        public void Reseed(int seed) => _rng.Reset(seed);

        public void Reset()
        {
            LastPosition = Vector3.zero;
            LastTimestampSec = 0;
            HasFix = false;
        }

        public void Initialize(GeoReferencingSystem inGeo )
        {
            _geo = inGeo;
            IsValid = (_geo != null && _rb != null);
            HasFix = true;
        }

        public void Sample(double nowSec, bool addNoise = false)
        {
            if (_rb == null)
            {
                HasFix = false;
                return;
            }

            // World-frame local position is always ENU (see LastPosition doc).
            Vector3 pos = Frames.TransformWorldPosition(_rb.position, SimFrame.FLU);

            if (addNoise)
            {
                pos.x += _rng.Next() * LatLonNoiseStdDev;  // East  (horizontal)
                pos.y += _rng.Next() * LatLonNoiseStdDev;  // North (horizontal)
                pos.z += _rng.Next() * AltNoiseStdDev;     // Up    (vertical)
            }
```

```
            if (_geo != null)
            {
                // Noise-free ENU for the geodetic conversion (ground truth).
                Vector3 enu = Frames.TransformWorldPosition(_rb.position, SimFrame.FLU);
                _geo.EnuToGeodetic(enu.x, enu.y, enu.z,
                    out double lat, out double lon, out double h);
                LastLatDeg = lat;
                LastLonDeg = lon;
                LastAltM   = h;

                // Unity world is X=East, Y=Up, Z=North (Mag's own convention comment),
                // so NED = (N, E, D) = (vz, vx, -vy). No Frames call needed, and this
                // avoids any origin-offset ambiguity in transforming a velocity vector.
                Vector3 v = _rb.linearVelocity;          // _rb.linearVelocity on Unity 6+
                LastVelNED = new Vector3(v.z, v.x, -v.y);
            }

            LastPosition     = pos;
            LastTimestampSec = nowSec;
            HasFix = true;
        }
    }
}
```

Runtime/Scripts/RobotCore/Sensors/IMUSensor.cs

SHA-256 6050284042f66177755d888e0ad1d7be63ade508a7ed0f86114766f984a8d336 | 161 lines | 6,096 bytes

```
using MathUtil;
using RobotCore.Core_Utils;
using UnityEngine;

namespace RobotCore.Sensors
{
    // ReSharper disable once InconsistentNaming
    public sealed class IMUSensor
    {
        private Rigidbody _rb;
        private Transform _bodyTransform;

        // State
        private Vector3 _prevVelWorld;
        private bool _hasPrev;

        public SimFrame OutputFrame = SimFrame.FLU;
        public bool RemoveGravity = false;

        public Vector3 LastAngVel { get; private set; }      // rad/s in OutputFrame
        public Vector3 LastAtt { get; private set; }         // degrees (Euler) in OutputFrame
        public Quaternion LastOrientation { get; private set; } // orientation in OutputFrame
        public Vector3 LastLinAcc { get; private set; }      // m/s^2 in OutputFrame
        public Vector3 LastVel { get; private set; }

        public double LastTimestampSec { get; private set; }
        public bool IsValid { get; private set; }

        public float AccelNoiseStdDev = 0.05f;   // m/s^2
        public float GyroNoiseStdDev  = 0.002f;  // rad/s (also drives attitude noise, converted to deg)
        public float VelNoiseStdDev   = 0.02f;   // m/s
        private readonly GaussianRng _rng;

        public IMUSensor(Rigidbody rb, Transform bodyTransform, int noiseSeed = 1)
        {
            _rb = rb;
            _bodyTransform = bodyTransform;
            _rng = new GaussianRng(noiseSeed);
        }

        public void Reseed(int seed) => _rng.Reset(seed);

        public void Reset()
        {
            _hasPrev = false;
```

```
    _prevVelWorld = Vector3.zero;
    LastAngVel = Vector3.zero;
    LastAtt = Vector3.zero;
    LastOrientation = Quaternion.identity;
    LastLinAcc = Vector3.zero;
    LastVel = Vector3.zero;
    LastTimestampSec = 0;
    IsValid = false;
}
public void Initialize(Rigidbody rb)
{
    if (rb == null)
    {
        _rb = null;
        _bodyTransform = null;
        IsValid = false;
        return;
    }

    _rb = rb;
    _bodyTransform = rb.transform;

    _prevVelWorld = rb.linearVelocity;
    _hasPrev = true;
    IsValid = true;
}
public void SampleAngularVelocity(double nowSec, float deltaTime)
{
    Vector3 wWorld = _rb.angularVelocity;
    Vector3 wBody = _bodyTransform.InverseTransformDirection(wWorld);

    LastAngVel = Frames.TransformAngularVelocity(wBody, OutputFrame); // Output is in Rad/s
}
public void SampleAttitude(double nowSec, float deltaTime)
{
    Vector3 euler = _bodyTransform.rotation.eulerAngles;

    // Reorder to (roll, pitch, yaw) — NO sign changes here,
    // TransformAttitude handles LH→RH
    Vector3 rpyDeg = new Vector3(euler.x, euler.z, euler.y);

    Vector3 att = Frames.TransformAttitude(rpyDeg, OutputFrame);

    // Wrap to ±180 (Unity eulerAngles are [0,360), UE FRotator is [-180,180))
    att.x = Wrap180(att.x);
    att.y = Wrap180(att.y);
    att.z = Wrap180(att.z);

    LastAtt = att;
    LastOrientation = Frames.TransformQuaternion(_bodyTransform.rotation, OutputFrame);
}
private static float Wrap180(float a)
{
    a %= 360f;
    if (a > 180f) a -= 360f;
    if (a < -180f) a += 360f;
    return a;
}
public void SampleAcceleration(double nowSec, float deltaTime)
{
    Vector3 vWorld = _rb.linearVelocity;
    Vector3 aWorld = (vWorld - _prevVelWorld) / deltaTime;
    _prevVelWorld = vWorld;

    // Specific force (what a real accelerometer / PX4 HIL_SENSOR wants):
    //   f = a_coordinate - g    (g = Physics.gravity = (0,-9.81,0))
    // At rest a=0  -> f = -g = (0,+9.81,0) world (up) -> FRD (0,0,-9.81)
    // Free-fall    -> f = 0
    if (!RemoveGravity)
        aWorld -= Physics.gravity;

    Vector3 aBody = _bodyTransform.InverseTransformDirection(aWorld);
    LastLinAcc = Frames.TransformAcceleration(aBody, OutputFrame);
```

```
        }
        public void SampleVelocity(double nowSec, float deltaTime)
        {
            Vector3 vWorld = _rb.linearVelocity;
            Vector3 vBody = _bodyTransform.InverseTransformDirection(vWorld);

            LastVel = Frames.TransformLinear(vBody, OutputFrame);
        }
        // Call from SensorManager's post-physics sample phase using deterministic dt.
        public void Sample(double nowSec, float deltaTime, bool addNoise = false)
        {
            if (_rb == null || _bodyTransform == null || deltaTime <= 0f)
            {
                IsValid = false;
                return;
            }

            if (!_hasPrev)
            {
                _prevVelWorld = _rb.linearVelocity;
                _hasPrev = true;
            }

            SampleAcceleration(nowSec, deltaTime);
            SampleAngularVelocity(nowSec, deltaTime);
            SampleAttitude(nowSec, deltaTime);
            SampleVelocity(nowSec, deltaTime);

            if (addNoise)
            {
                LastLinAcc += new Vector3(_rng.Next(), _rng.Next(), _rng.Next()) * AccelNoiseStdDev;
                LastAngVel += new Vector3(_rng.Next(), _rng.Next(), _rng.Next()) * GyroNoiseStdDev;
                LastVel    += new Vector3(_rng.Next(), _rng.Next(), _rng.Next()) * VelNoiseStdDev;

                float attStdDeg = GyroNoiseStdDev * Mathf.Rad2Deg;
                LastAtt += new Vector3(_rng.Next(), _rng.Next(), _rng.Next()) * attStdDeg;
            }

            LastTimestampSec = nowSec;
            IsValid = true;
        }
    }
}
```

Runtime/Scripts/RobotCore/Sensors/MagSensor.cs

SHA-256 0dcf87255a211eca6b9e492ea346a28e805795e8182a941bad935f519ed7c4b3 | 134 lines | 5,355 bytes

```
using SimCore;
using MathUtil;
using UnityEngine;
using RobotCore.Core_Utils;

namespace RobotCore.Sensors
{
    public sealed class MagSensor
    {
        public float MagNoiseStdDev = 0.005f;                // Gauss
        public float MagOffsetX = 0f, MagOffsetY = 0f, MagOffsetZ = 0f;  // Gauss bias

        public const float CALIBRATION_DURATION = 1.0f;       // vestigial; mirrors Unreal, gates nothin
g

        private readonly GaussianRng _rng;
        private Rigidbody _rb;
        private Transform _bodyTransform;
        private GeoReferencingSystem _geo;

        public SimFrame OutputFrame = SimFrame.FLU;

        public Vector3 LastMagField     { get; private set; } // body, OutputFrame, Gauss  (PX4 wants FR
D)
        public Vector3 EarthMagFieldNED { get; private set; } // geographic NED, Gauss (debug)
        public float   MagneticHeading      { get; private set; } // deg, debug (≈ declination — see not
```

e)

```
        public float   MagneticDeclination  { get; private set; } // deg, from WMM
        public double  LastTimestampSec { get; private set; }
        public bool    IsValid          { get; private set; }

        public bool  IsCalibrating { get; private set; }
        private float _calibrationTime;
        private bool  _earthFieldValid;
        private bool  _declinationLogged;

        public MagSensor(Rigidbody rb, Transform bodyTransform, int noiseSeed = 4)
        {
            _rb = rb;
            _bodyTransform = bodyTransform;
            _rng = new GaussianRng(noiseSeed);
        }

        public void Reseed(int seed) => _rng.Reset(seed);

        public void Reset()
        {
            LastMagField = Vector3.zero;
            EarthMagFieldNED = Vector3.zero;
            MagneticHeading = 0f;
            MagneticDeclination = 0f;
            LastTimestampSec = 0.0;
            IsValid = false;
            IsCalibrating = true;
            _calibrationTime = 0f;
            _earthFieldValid = false;
            _declinationLogged = false;
        }

        public void Initialize(GeoReferencingSystem geo)
        {
            _geo = geo;
            IsValid = (_geo != null && _rb != null && _bodyTransform != null);
        }

        public void Sample(double nowSec, float dtSec, bool addNoise)
        {
            if (_geo == null || _rb == null || _bodyTransform == null)
            {
                IsValid = false;
                return;
            }

            if (IsCalibrating)
            {
                _calibrationTime += dtSec;
                if (_calibrationTime >= CALIBRATION_DURATION) IsCalibrating = false;
            }

            UpdateEarthMagField();
            if (!_earthFieldValid) { IsValid = false; return; }

            // Earth field NED (Gauss) -> Unity world (X=East, Y=Up, Z=North):
            //   N -> +Z, E -> +X, D -> -Y
            float n = EarthMagFieldNED.x, e = EarthMagFieldNED.y, d = EarthMagFieldNED.z;
            Vector3 fieldWorldUnity = new Vector3(e, -d, n);

            // World -> body (Unity), then -> OutputFrame. Same machinery as the IMU.
            Vector3 fieldBodyUnity = _bodyTransform.InverseTransformDirection(fieldWorldUnity);
            Vector3 field = Frames.TransformLinear(fieldBodyUnity, OutputFrame);

            if (addNoise)
                field += new Vector3(_rng.Next(), _rng.Next(), _rng.Next()) * MagNoiseStdDev;

            field.x += MagOffsetX;
            field.y += MagOffsetY;
            field.z += MagOffsetZ;

            LastMagField = field;
```

```
            CalculateMagneticHeading();

            LastTimestampSec = nowSec;
            IsValid = true;
        }

        private void UpdateEarthMagField()
        {
            // World position in ENU (East, North, Up) — TransformWorldPosition(FLU) is the ENU map.
            Vector3 flu = Frames.TransformWorldPosition(_rb.position, SimFrame.FLU);
            _geo.EnuToGeodetic(flu.x, flu.y, flu.z, out double lat, out double lon, out _);

            EarthMagFieldNED    = GeoMagneticTables.GetMagFieldVectorNED((float)lat, (float)lon);
            MagneticDeclination = GeoMagneticTables.GetMagDeclinationDegrees((float)lat, (float)lon);

            if (!_declinationLogged)
            {
                Debug.Log($"[Mag] WMM declination={MagneticDeclination:F2}deg  lat={lat:F5} lon={lon:F5}
");
                _declinationLogged = true;
            }
            _earthFieldValid = true;
        }

        private void CalculateMagneticHeading()
        {
            // Rotate measured body field back to Unity world and read the horizontal bearing.
            // This points at *magnetic* North regardless of vehicle yaw, so the value tracks
            // declination — it's a transform self-consistency check, not vehicle heading.
            Vector3 bodyUnity  = Frames.InverseTransformLinear(LastMagField, OutputFrame);
            Vector3 worldUnity = _bodyTransform.TransformDirection(bodyUnity);

            float headingRad = Mathf.Atan2(worldUnity.x /*East*/, worldUnity.z /*North*/);
            float headingDeg = Mathf.Rad2Deg * headingRad;
            if (headingDeg < 0f) headingDeg += 360f;
            MagneticHeading = headingDeg;
        }
    }
}
```

Runtime/Scripts/RobotCore/Core Utils/GaussianRng.cs

SHA-256 17d73475a7411dbc0dbaecd25efa52dae8b9ff7d614a51b439faba13dd051132 | 35 lines | 985 bytes

```
namespace RobotCore.Core_Utils
{
    public sealed class GaussianRng
    {
        private System.Random _rng;
        private bool   _hasSpare;
        private double _spare;

        public GaussianRng(int seed) { _rng = new System.Random(seed); }

        public void Reset(int seed)
        {
            _rng = new System.Random(seed);
            _hasSpare = false;
            _spare = 0.0;
        }

        public float Next()
        {
            if (_hasSpare) { _hasSpare = false; return (float)_spare; }

            double u1, u2, s;
            do {
                u1 = 2.0 * _rng.NextDouble() - 1.0;
                u2 = 2.0 * _rng.NextDouble() - 1.0;
                s  = u1 * u1 + u2 * u2;
            } while (s >= 1.0 || s == 0.0);

            double mul = System.Math.Sqrt(-2.0 * System.Math.Log(s) / s);
            _spare    = u2 * mul;
```

```
            _hasSpare = true;
            return (float)(u1 * mul);
        }
    }
}
```

Runtime/Scripts/RobotCore/Core Utils/GeoMagDeclination.cs

SHA-256 3305702e9cbe1b2ee90ba83a8d1c8d74c72163da92af95cfc7775ce65f4e10e1 | 6 lines | 88 bytes

```
namespace RobotCore.Core_Utils
{
    public class GeoMagDeclination
    {
    }
}
```

Runtime/Scripts/RobotCore/Core Utils/GeoMagneticTables.cs

SHA-256 3a17606baa6cee131c6c65491093e7d7629a9e174c535c4664d18bd56be86ac8 | 156 lines | 21,317 bytes

```
using UnityEngine;

namespace RobotCore.Core_Utils
{
    public class GeoMagneticTables
    {
        // ---- WMM lookup (ported from FGeoMagDeclination) ----
        private const float SAMPLING_RES     = 10.0f;
        private const float SAMPLING_MIN_LAT = -90.0f;
        private const float SAMPLING_MAX_LAT =  90.0f;
        private const float SAMPLING_MIN_LON = -180.0f;
        private const float SAMPLING_MAX_LON =  180.0f;

        private static uint GetLookupTableIndex(ref float val, float min, float max)
        {
            val = Mathf.Clamp(val, min, max - SAMPLING_RES);   // -RES so (index+1) is always valid
            return (uint)((val - min) / SAMPLING_RES);
        }

        private static float GetTableData(float latDeg, float lonDeg, short[,] table, float scale)
        {
            latDeg = Mathf.Clamp(latDeg, SAMPLING_MIN_LAT, SAMPLING_MAX_LAT);
            if (lonDeg > SAMPLING_MAX_LON) lonDeg -= 360.0f;
            if (lonDeg < SAMPLING_MIN_LON) lonDeg += 360.0f;

            float minLat = Mathf.Floor(latDeg / SAMPLING_RES) * SAMPLING_RES;
            float minLon = Mathf.Floor(lonDeg / SAMPLING_RES) * SAMPLING_RES;

            uint la = GetLookupTableIndex(ref minLat, SAMPLING_MIN_LAT, SAMPLING_MAX_LAT);
            uint lo = GetLookupTableIndex(ref minLon, SAMPLING_MIN_LON, SAMPLING_MAX_LON);

            float sw = table[la,     lo];
            float se = table[la,     lo + 1];
            float ne = table[la + 1, lo + 1];
            float nw = table[la + 1, lo];

            float latScale = Mathf.Clamp01((latDeg - minLat) / SAMPLING_RES);
            float lonScale = Mathf.Clamp01((lonDeg - minLon) / SAMPLING_RES);

            float dMin = lonScale * (se - sw) + sw;
            float dMax = lonScale * (ne - nw) + nw;
            return (latScale * (dMax - dMin) + dMin) * scale;
        }

        public static float GetMagDeclinationDegrees(float lat, float lon)
            => GetTableData(lat, lon, DeclinationTable, WMM_DECLINATION_SCALE_TO_DEGREES);

        public static float GetMagInclinationDegrees(float lat, float lon)
            => GetTableData(lat, lon, InclinationTable, WMM_INCLINATION_SCALE_TO_DEGREES);

        public static float GetMagStrengthTesla(float lat, float lon)
            => GetTableData(lat, lon, TotalIntensityTable, WMM_TOTALINTENSITY_SCALE_TO_NANOTESLA) * 1e-9
f;
```

```
        public static float GetMagStrengthGauss(float lat, float lon)
            => GetMagStrengthTesla(lat, lon) * 1e4f;

        // Earth field as NED (North, East, Down) in Gauss, declination baked in (PX4 corrects it).
        public static Vector3 GetMagFieldVectorNED(float lat, float lon)
        {
            float decl = GetMagDeclinationDegrees(lat, lon) * Mathf.Deg2Rad;
            float incl = GetMagInclinationDegrees(lat, lon) * Mathf.Deg2Rad;
            float s    = GetMagStrengthGauss(lat, lon);
            return new Vector3(
                s * Mathf.Cos(incl) * Mathf.Cos(decl),   // N
                s * Mathf.Cos(incl) * Mathf.Sin(decl),   // E
                s * Mathf.Sin(incl));                    // D
        }

        private static readonly short[,] DeclinationTable =
        {
    //    LONGITUDE:   -180,  -170,  -160,  -150,  -140,  -130,  -120,  -110,  -100,   -90,   -80,   -70
,   -60,   -50,   -40,   -30,   -20,   -10,     0,    10,    20,    30,    40,    50,    60,    70,    8
0,    90,   100,   110,   120,   130,   140,   150,   160,   170,   180,
    /* LAT: -90 */ {  27264, 25429, 23595, 21761, 19926, 18092, 16258, 14423, 12589, 10754,  8920,  7086
,  5251,  3417,  1583,  -252, -2086, -3921, -5755, -7589, -9424,-11258,-13092,-14927,-16761,-18596,-2043
0,-22264,-24099,-25933,-27768,-29602,-31436, 32767, 30933, 29098, 27264, },
    /* LAT: -80 */ {  23650, 21416, 19382, 17521, 15799, 14184, 12647, 11165,  9721,  8303,  6904,  5519
,  4143,  2772,  1397,     9, -1403, -2848, -4333, -5863, -7437, -9056,-10718,-12423,-14175,-15984,-1786
4,-19835,-21922,-24152,-26545,-29105,-31803, 31464, 28718, 26092, 23650, },
    /* LAT: -70 */ {  15755, 14282, 13094, 12077, 11159, 10281,  9392,  8457,  7454,  6381,  5257,  4109
,  2971,  1865,   794,  -267, -1359, -2527, -3797, -5172, -6632, -8142, -9669,-11188,-12690,-14181,-1568
6,-17253,-18969,-20997,-23675,-27707, 32110, 25303, 20617, 17720, 15755, }, // WARNING! black out zone
    /* LAT: -60 */ {   8903,  8634,  8328,  8030,  7755,  7481,  7150,  6689,  6037,  5171,  4116,  2948
,  1775,   704,  -211,  -998, -1763, -2635, -3706, -4982, -6396, -7846, -9240,-10513,-11630,-12572,-1332
0,-13839,-14019,-13549,-11322, -3523,  5353,  8167,  8943,  9057,  8903, }, // WARNING! black out zone
    /* LAT: -50 */ {   5806,  5839,  5774,  5672,  5587,  5541,  5499,  5359,  4991,  4286,  3216,  1875
,   470,  -754, -1647, -2218, -2623, -3091, -3835, -4934, -6270, -7627, -8830, -9771,-10388,-10623,-1040
0, -9587, -7986, -5492, -2431,   460,  2679,  4178,  5097,  5594,  5806, },
    /* LAT: -40 */ {   4186,  4282,  4284,  4229,  4159,  4118,  4119,  4103,  3914,  3342,  2253,   728
,  -912, -2273, -3151, -3597, -3767, -3819, -4016, -4665, -5740, -6891, -7817, -8365, -8453, -8028, -706
9, -5603, -3811, -2034,  -509,   772,  1872,  2790,  3486,  3942,  4186, },
    /* LAT: -30 */ {   3161,  3250,  3275,  3251,  3182,  3094,  3026,  2988,  2851,  2337,  1229,  -384
, -2070, -3364, -4111, -4446, -4503, -4252, -3790, -3609, -4059, -4874, -5597, -5938, -5777, -5133, -410
9, -2845, -1594,  -619,    83,   710,  1372,  2016,  2561,  2948,  3161, },
    /* LAT: -20 */ {   2485,  2532,  2542,  2534,  2481,  2375,  2255,  2172,  2018,  1486,   353, -1228
, -2777, -3865, -4390, -4481, -4240, -3623, -2707, -1902, -1669, -2082, -2768, -3241, -3254, -2856, -217
9, -1328,  -531,   -35,   229,   542,  1013,  1529,  1985,  2318,  2485, },
    /* LAT: -10 */ {   2072,  2064,  2032,  2018,  1980,  1883,  1758,  1657,  1461,   874,  -261, -1719
, -3046, -3887, -4129, -3855, -3229, -2397, -1510,  -734,  -274,  -342,  -854, -1379, -1586, -1478, -114
3,  -624,  -116,   121,   159,   321,   720,  1197,  1625,  1939,  2072, },
    /* LAT:   0 */ {   1847,  1810,  1742,  1723,  1703,  1621,  1500,  1371,  1102,   444,  -660, -1952
, -3043, -3628, -3589, -3035, -2219, -1401,  -724,  -169,   254,   350,    38,  -401,  -658,  -711,  -60
8,  -338,   -43,    36,   -42,    45,   413,   896,  1354,  1703,  1847, },
    /* LAT:  10 */ {   1699,  1705,  1653,  1662,  1681,  1620,  1480,  1275,   878,   122,  -954, -2080
, -2929, -3256, -3012, -2356, -1536,  -796,  -265,   129,   466,   610,   425,    85,  -155,  -268,  -29
9,  -222,  -121,  -175,  -335,  -312,    14,   509,  1030,  1470,  1699, },
    /* LAT:  20 */ {   1494,  1649,  1708,  1799,  1884,  1855,  1681,  1355,   781,  -117, -1209, -2197
, -2803, -2893, -2528, -1885, -1137,  -461,    13,   332,   601,   746,   640,   383,   178,    51,   -5
5,  -135,  -228,  -437,  -694,  -758,  -503,   -19,   561,  1114,  1494, },
    /* LAT:  30 */ {   1160,  1544,  1816,  2050,  2215,  2221,  2017,  1567,   802,  -275, -1438, -2341
, -2756, -2673, -2242, -1629,  -939,  -294,   184,   499,   738,   885,   855,   696,   542,   408,   23
0,    -8,  -317,  -718, -1109, -1275, -1099,  -642,   -30,   613,  1160, },
    /* LAT:  40 */ {    765,  1379,  1900,  2314,  2577,  2622,  2393,  1831,   876,  -403, -1685, -2574
, -2892, -2722, -2248, -1623,  -936,  -278,   252,   629,   908,  1109,  1199,  1184,  1104,   944,   64
7,   195,  -387, -1030, -1576, -1826, -1696, -1253,  -625,    77,   765, },
    /* LAT:  50 */ {    437,  1218,  1936,  2528,  2919,  3036,  2799,  2116,   928,  -623, -2091, -3030
, -3327, -3121, -2604, -1925, -1180,  -449,   193,   719,  1153,  1519,  1805,  1977,  1991,  1784,  129
7,   527,  -430, -1381, -2085, -2373, -2234, -1771, -1112,  -355,   437, },
    /* LAT:  60 */ {    198,  1087,  1936,  2674,  3216,  3452,  3243,  2411,   855, -1158, -2926, -3941
, -4206, -3936, -3339, -2560, -1696,  -817,    27,   812,  1534,  2191,  2754,  3160,  3315,  3098,  239
7,  1190,  -317, -1699, -2593, -2902, -2725, -2214, -1502,  -680,   198, },
    /* LAT:  70 */ {   -111,   863,  1803,  2648,  3305,  3634,  3405,  2285,    58, -2678, -4700, -5589
, -5627, -5143, -4352, -3383, -2317, -1205,   -83,  1026,  2098,  3108,  4014,  4748,  5204,  5214,  453
3,  2927,   575, -1644, -2988, -3441, -3276, -2730, -1962, -1068,  -111, }, // WARNING! black out zone
    /* LAT:  80 */ {  -1054,   -90,   801,  1527,  1946,  1812,   731, -1632, -4654, -6842, -7732, -7698
, -7114, -6204, -5096, -3864, -2554, -1197,   184,  1572,  2947,  4292,  5581,  6779,  7828,  8630,  898
```

```
8,  8486,  6313,  2026, -1910, -3677, -4009, -3636, -2914, -2019, -1054, }, // WARNING! black out zone
    /* LAT:  90 */ { -30607,-28773,-26938,-25104,-23269,-21435,-19601,-17766,-15932,-14097,-12263,-10429
, -8594, -6760, -4926, -3091, -1257,   578,  2412,  4246,  6081,  7915,  9749, 11584, 13418, 15253, 1708
7, 18921, 20756, 22590, 24424, 26259, 28093, 29928, 31762,-32441,-30607, }, // WARNING! black out zone
        };

        private static readonly float WMM_DECLINATION_SCALE_TO_DEGREES = 0.00545143529f;

    // Magnetic inclination data in 0.002699 degrees
    // Model: WMM-2020,
    // Version: 0.5.1.11,
    // Date: 2024.41257,
        private static readonly short[,] InclinationTable ={
    //    LONGITUDE:   -180,  -170,  -160,  -150,  -140,  -130,  -120,  -110,  -100,   -90,   -80,   -70
,   -60,   -50,   -40,   -30,   -20,   -10,     0,    10,    20,    30,    40,    50,    60,    70,    8
0,    90,   100,   110,   120,   130,   140,   150,   160,   170,   180,
    /* LAT: -90 */ { -26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663
,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-2666
3,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663,-26663, },
    /* LAT: -80 */ { -28959,-28675,-28333,-27951,-27540,-27113,-26682,-26258,-25854,-25482,-25150,-24867
,-24633,-24450,-24315,-24226,-24183,-24188,-24245,-24361,-24539,-24783,-25092,-25462,-25884,-26346,-2683
3,-27328,-27812,-28262,-28656,-28973,-29191,-29297,-29287,-29170,-28959, },
    /* LAT: -70 */ { -29906,-29230,-28552,-27867,-27164,-26436,-25685,-24930,-24205,-23555,-23023,-22638
,-22403,-22293,-22261,-22264,-22272,-22285,-22330,-22447,-22678,-23053,-23582,-24256,-25053,-25944,-2689
8,-27885,-28874,-29833,-30717,-31441,-31829,-31704,-31211,-30578,-29906, }, // WARNING! black out zone
    /* LAT: -60 */ { -28675,-27922,-27202,-26495,-25766,-24977,-24097,-23135,-22147,-21240,-20551,-20191
,-20196,-20492,-20922,-21317,-21564,-21635,-21592,-21554,-21657,-22006,-22635,-23517,-24587,-25775,-2702
3,-28290,-29540,-30732,-31788,-32398,-32004,-31178,-30310,-29471,-28675, }, // WARNING! black out zone
    /* LAT: -50 */ { -26518,-25787,-25081,-24395,-23709,-22973,-22127,-21129,-20009,-18913,-18103,-17862
,-18313,-19305,-20497,-21563,-22295,-22602,-22503,-22157,-21856,-21888,-22387,-23298,-24466,-25730,-2697
4,-28112,-29052,-29680,-29904,-29747,-29313,-28709,-28008,-27265,-26518, },
    /* LAT: -40 */ { -23859,-23111,-22367,-21631,-20911,-20195,-19433,-18535,-17434,-16245,-15366,-15320
,-16340,-18110,-20062,-21801,-23168,-24048,-24290,-23883,-23136,-22578,-22605,-23240,-24242,-25316,-2625
1,-26918,-27243,-27250,-27062,-26776,-26401,-25910,-25300,-24601,-23859, },
    /* LAT: -30 */ { -20386,-19567,-18750,-17917,-17085,-16293,-15544,-14724,-13640,-12351,-11424,-11666
,-13370,-15956,-18605,-20904,-22812,-24277,-25061,-24972,-24136,-23057,-22385,-22426,-22983,-23667,-2419
6,-24414,-24269,-23896,-23536,-23265,-22968,-22536,-21939,-21200,-20386, },
    /* LAT: -20 */ { -15655,-14695,-13789,-12872,-11930,-11023,-10206, -9346, -8156, -6711, -5803, -6449
, -8904,-12373,-15815,-18676,-20887,-22480,-23348,-23356,-22530,-21203,-20034,-19551,-19678,-20026,-2031
3,-20331,-19954,-19380,-18986,-18828,-18633,-18206,-17535,-16650,-15655, },
    /* LAT: -10 */ {  -9385, -8209, -7218, -6278, -5306, -4355, -3491, -2554, -1249,   222,   935,   -75
, -3005, -7132,-11304,-14645,-16891,-18169,-18676,-18490,-17568,-16077,-14672,-13969,-13905,-14103,-1434
2,-14377,-13970,-13348,-13045,-13109,-13066,-12627,-11813,-10677, -9385, },
    /* LAT:   0 */ {  -1941,  -572,   447,  1316,  2208,  3089,  3901,  4808,  6012,  7204,  7577,  6434
,  3584,  -534, -4855, -8273,-10317,-11152,-11228,-10827, -9842, -8280, -6784, -6023, -5903, -6042, -629
9, -6446, -6167, -5676, -5592, -5955, -6145, -5775, -4872, -3514, -1941, },
    /* LAT:  10 */ {   5421,  6791,  7742,  8478,  9236, 10013, 10743, 11532, 12453, 13205, 13235, 12131
,  9759,  6384,  2805,   -45, -1666, -2121, -1883, -1354,  -444,   946,  2291,  2989,  3125,  3048,  284
4,  2651,  2737,  2945,  2751,  2128,  1664,  1792,  2543,  3843,  5421, },
    /* LAT:  20 */ {  11488, 12637, 13462, 14098, 14766, 15492, 16206, 16917, 17594, 17991, 17771, 16744
, 14925, 12573, 10194,  8309,  7245,  7046,  7400,  7937,  8662,  9672, 10651, 11186, 11314, 11296, 1120
3, 11079, 11045, 10991, 10595,  9848,  9176,  8967,  9335, 10249, 11488, },
    /* LAT:  30 */ {  16064, 16869, 17552, 18157, 18813, 19547, 20295, 21001, 21567, 21785, 21452, 20532
, 19178, 17663, 16272, 15213, 14627, 14573, 14900, 15364, 15901, 16550, 17167, 17534, 17662, 17704, 1771
9, 17697, 17639, 17449, 16959, 16185, 15409, 14931, 14899, 15320, 16064, },
    /* LAT:  40 */ {  19670, 20142, 20688, 21294, 21986, 22747, 23519, 24226, 24746, 24903, 24570, 23795
, 22787, 21787, 20958, 20373, 20075, 20088, 20340, 20695, 21077, 21477, 21845, 22111, 22280, 22419, 2254
4, 22615, 22571, 22323, 21795, 21038, 20247, 19638, 19335, 19364, 19670, },
    /* LAT:  50 */ {  22933, 23187, 23612, 24182, 24864, 25609, 26354, 27016, 27477, 27593, 27291, 26658
, 25888, 25162, 24586, 24201, 24016, 24023, 24176, 24404, 24654, 24906, 25156, 25404, 25658, 25925, 2617
1, 26324, 26295, 26017, 25483, 24780, 24056, 23451, 23049, 22878, 22933, },
    /* LAT:  60 */ {  26155, 26303, 26615, 27072, 27639, 28270, 28905, 29465, 29833, 29890, 29605, 29081
, 28474, 27904, 27442, 27117, 26932, 26874, 26915, 27023, 27173, 27359, 27588, 27873, 28215, 28592, 2894
3, 29169, 29170, 28904, 28420, 27827, 27237, 26736, 26376, 26179, 26155, },
    /* LAT:  70 */ {  29204, 29286, 29480, 29773, 30146, 30572, 31008, 31388, 31612, 31580, 31301, 30883
, 30427, 29998, 29634, 29351, 29155, 29043, 29006, 29037, 29130, 29287, 29511, 29805, 30163, 30562, 3095
3, 31246, 31335, 31173, 30826, 30404, 29992, 29643, 29386, 29237, 29204, }, // WARNING! black out zone
    /* LAT:  80 */ {  31820, 31841, 31914, 32033, 32188, 32362, 32524, 32622, 32598, 32449, 32224, 31969
, 31713, 31474, 31265, 31093, 30963, 30878, 30840, 30850, 30908, 31015, 31169, 31368, 31607, 31878, 3216
7, 32450, 32679, 32767, 32672, 32479, 32271, 32087, 31945, 31855, 31820, }, // WARNING! black out zone
    /* LAT:  90 */ {  32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694
, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 3269
4, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, 32694, }, // WARNING! black out zone
```

```
        };

         private static readonly float WMM_INCLINATION_SCALE_TO_DEGREES = 0.00269892697f;


        // Magnetic totalintensity data in 2.042 nanoTesla
        // Model: WMM-2020,
        // Version: 0.5.1.11,
        // Date: 2024.41257,
        private static readonly short[,] TotalIntensityTable = {
            //    LONGITUDE:   -180,  -170,  -160,  -150,  -140,  -130,  -120,  -110,  -100,   -90,   -8
0,   -70,   -60,   -50,   -40,   -30,   -20,   -10,     0,    10,    20,    30,    40,    50,    60,
70,    80,    90,   100,   110,   120,   130,   140,   150,   160,   170,   180,
            /* LAT: -90 */ {  26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 2664
4, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 266
44, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, 26644, },
            /* LAT: -80 */ {  29615, 29299, 28908, 28453, 27945, 27395, 26815, 26218, 25619, 25032, 2447
4, 23959, 23502, 23115, 22808, 22592, 22476, 22469, 22578, 22807, 23157, 23621, 24187, 24836, 25543, 262
79, 27014, 27716, 28358, 28914, 29365, 29700, 29913, 30004, 29979, 29846, 29615, },
            /* LAT: -70 */ {  30814, 30152, 29407, 28592, 27711, 26765, 25759, 24706, 23635, 22584, 2159
6, 20706, 19938, 19300, 18794, 18422, 18195, 18139, 18285, 18666, 19302, 20195, 21322, 22642, 24092, 256
01, 27090, 28480, 29701, 30696, 31432, 31894, 32090, 32046, 31794, 31371, 30814, }, // WARNING! black ou
t zone
            /* LAT: -60 */ {  30256, 29303, 28305, 27273, 26195, 25041, 23783, 22419, 20989, 19575, 1827
4, 17168, 16296, 15644, 15162, 14803, 14555, 14459, 14593, 15046, 15888, 17136, 18753, 20656, 22732, 248
62, 26925, 28803, 30388, 31598, 32391, 32767, 32766, 32449, 31882, 31131, 30256, }, // WARNING! black ou
t zone
            /* LAT: -50 */ {  28585, 27448, 26301, 25162, 24017, 22813, 21484, 19998, 18389, 16768, 1529
6, 14132, 13352, 12914, 12677, 12503, 12330, 12195, 12223, 12590, 13464, 14914, 16881, 19207, 21698, 241
75, 26497, 28540, 30193, 31368, 32034, 32224, 32006, 31456, 30654, 29673, 28585, },
            /* LAT: -40 */ {  26388, 25175, 23965, 22783, 21629, 20463, 19212, 17819, 16281, 14687, 1323
1, 12153, 11593, 11476, 11568, 11662, 11671, 11598, 11530, 11680, 12349, 13743, 15828, 18371, 21056, 236
15, 25886, 27774, 29196, 30107, 30531, 30536, 30188, 29538, 28642, 27566, 26388, },
            /* LAT: -30 */ {  23875, 22687, 21506, 20350, 19238, 18163, 17088, 15953, 14699, 13348, 1209
0, 11211, 10886, 11019, 11346, 11688, 11997, 12225, 12312, 12367, 12723, 13749, 15575, 17999, 20611, 230
34, 25059, 26587, 27567, 28051, 28176, 28039, 27646, 26992, 26106, 25040, 23875, },
            /* LAT: -20 */ {  21154, 20100, 19069, 18060, 17097, 16201, 15373, 14569, 13693, 12716, 1178
2, 11141, 10963, 11194, 11640, 12182, 12805, 13413, 13818, 13974, 14106, 14617, 15845, 17765, 19985, 220
68, 23750, 24872, 25370, 25402, 25252, 25023, 24621, 23994, 23174, 22204, 21154, },
            /* LAT: -10 */ {  18558, 17760, 17002, 16280, 15615, 15025, 14516, 14058, 13572, 13001, 1240
3, 11927, 11727, 11873, 12314, 12952, 13712, 14475, 15055, 15339, 15405, 15556, 16199, 17471, 19086, 206
62, 21941, 22713, 22869, 22606, 22275, 21968, 21540, 20930, 20196, 19388, 18558, },
            /* LAT:   0 */ {  16709, 16246, 15828, 15466, 15193, 15003, 14869, 14765, 14634, 14389, 1400
0, 13543, 13167, 13043, 13274, 13791, 14440, 15095, 15645, 15999, 16143, 16257, 16647, 17443, 18484, 195
34, 20407, 20910, 20925, 20587, 20154, 19705, 19162, 18518, 17858, 17244, 16709, },
            /* LAT:  10 */ {  16075, 15916, 15810, 15786, 15900, 16127, 16390, 16638, 16796, 16740, 1639
9, 15842, 15234, 14793, 14691, 14911, 15314, 15798, 16286, 16698, 17008, 17318, 17762, 18355, 19031, 197
18, 20303, 20638, 20637, 20325, 19783, 19078, 18284, 17502, 16834, 16355, 16075, },
            /* LAT:  20 */ {  16646, 16650, 16769, 17024, 17466, 18051, 18672, 19233, 19618, 19672, 1931
0, 18619, 17813, 17152, 16804, 16764, 16949, 17315, 17793, 18272, 18719, 19208, 19759, 20311, 20853, 214
11, 21916, 22235, 22275, 21976, 21295, 20297, 19178, 18148, 17345, 16845, 16646, },
            /* LAT:  30 */ {  18228, 18245, 18498, 18973, 19662, 20491, 21341, 22097, 22620, 22746, 2238
8, 21634, 20724, 19941, 19447, 19240, 19272, 19535, 19972, 20460, 20944, 21470, 22044, 22622, 23213, 238
45, 24444, 24864, 24980, 24684, 23906, 22724, 21376, 20127, 19146, 18510, 18228, },
            /* LAT:  40 */ {  20676, 20646, 20945, 21535, 22339, 23247, 24138, 24900, 25411, 25538, 2520
8, 24491, 23592, 22759, 22150, 21796, 21685, 21808, 22117, 22515, 22946, 23429, 23995, 24649, 25392, 261
92, 26939, 27472, 27650, 27367, 26593, 25423, 24081, 22812, 21778, 21055, 20676, },
            /* LAT:  50 */ {  23666, 23603, 23852, 24371, 25074, 25848, 26580, 27175, 27541, 27593, 2729
1, 26681, 25898, 25111, 24452, 23986, 23733, 23688, 23819, 24073, 24420, 24871, 25459, 26197, 27059, 279
64, 28776, 29343, 29542, 29310, 28661, 27703, 26610, 25561, 24680, 24034, 23666, },
            /* LAT:  60 */ {  26412, 26324, 26436, 26723, 27132, 27589, 28019, 28351, 28524, 28493, 2824
0, 27789, 27209, 26590, 26018, 25556, 25239, 25081, 25078, 25221, 25503, 25932, 26515, 27239, 28055, 288
77, 29590, 30087, 30289, 30171, 29764, 29152, 28446, 27753, 27153, 26695, 26412, },
            /* LAT:  70 */ {  28043, 27927, 27896, 27940, 28040, 28166, 28285, 28364, 28372, 28290, 2810
9, 27838, 27503, 27138, 26784, 26477, 26247, 26116, 26097, 26198, 26422, 26766, 27217, 27750, 28323, 288
79, 29358, 29708, 29894, 29908, 29765, 29507, 29181, 28836, 28513, 28242, 28043, }, // WARNING! black ou
t zone
            /* LAT:  80 */ {  28358, 28266, 28189, 28125, 28072, 28025, 27977, 27922, 27854, 27769, 2766
7, 27549, 27422, 27295, 27178, 27083, 27020, 26999, 27025, 27102, 27229, 27402, 27613, 27847, 28090, 283
24, 28531, 28697, 28813, 28875, 28885, 28848, 28776, 28680, 28572, 28462, 28358, }, // WARNING! black ou
t zone
            /* LAT:  90 */ {  27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 2784
9, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 278
```

```
49, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, 27849, }, // WARNING! black ou
t zone
        };
        private static readonly float WMM_TOTALINTENSITY_SCALE_TO_NANOTESLA = 2.042f;



    }
}
```

Runtime/Scripts/RobotCore/Core Utils/GeoReferencingSystem.cs

SHA-256 81809e27cf64ca5c78d59a233d68a17b51f5af6fe310b8be82afa2bec2af8631 | 128 lines | 5,893 bytes

```
using UnityEngine;
using System;
using System.Collections;

namespace RobotCore.Core_Utils
{
    /// <summary>
    /// WGS84 georeferencing. Converts between a local ENU tangent frame (meters,
    /// relative to a configured geodetic origin) and geodetic coordinates
    /// (latitude/longitude in degrees, ellipsoidal height in meters).
    ///
    /// Mirrors the role of Unreal's AGeoReferencingSystem (flat-planet / ENU mode).
    /// Local "up" (U) is along the ellipsoid normal at the origin; for local-area
    /// flight that is effectively true vertical.
    ///
    /// All math is double precision — ECEF magnitudes are ~6.4e6 m and float would
    /// discard ~0.5 m of position.
    /// </summary>
    public class GeoReferencingSystem
    {
        // WGS84 ellipsoid
        private const double A  = 6378137.0;              // semi-major axis [m]
        private const double F  = 1.0 / 298.257223563;    // flattening
        private const double E2 = F * (2.0 - F);          // first eccentricity squared
        private const double B  = A * (1.0 - F);          // semi-minor axis [m]

        public double OriginLatDeg { get; private set; }
        public double OriginLonDeg { get; private set; }
        public double OriginHeight { get; private set; }    // ellipsoidal height [m]

        private double _x0, _y0, _z0;       // origin in ECEF
        private double _sinLat, _cosLat, _sinLon, _cosLon;

        public GeoReferencingSystem(double originLatDeg, double originLonDeg, double originHeightM)
            => SetOrigin(originLatDeg, originLonDeg, originHeightM);

        public void SetOrigin(double latDeg, double lonDeg, double heightM)
        {
            OriginLatDeg = latDeg;
            OriginLonDeg = lonDeg;
            OriginHeight = heightM;

            double lat = latDeg * Math.PI / 180.0;
            double lon = lonDeg * Math.PI / 180.0;
            _sinLat = Math.Sin(lat); _cosLat = Math.Cos(lat);
            _sinLon = Math.Sin(lon); _cosLon = Math.Cos(lon);

            GeodeticToEcef(latDeg, lonDeg, heightM, out _x0, out _y0, out _z0);
        }

        // --- Public conversions -------------------------------------------------

        /// <summary>Local ENU (m) -> geodetic (lat°, lon°, ellipsoidal height m).</summary>
        public void EnuToGeodetic(double e, double n, double u,
                                  out double latDeg, out double lonDeg, out double heightM)
        {
            EnuToEcef(e, n, u, out double x, out double y, out double z);
            EcefToGeodetic(x, y, z, out latDeg, out lonDeg, out heightM);
        }
```

```
        /// <summary>Geodetic -> local ENU (m).</summary>
        public void GeodeticToEnu(double latDeg, double lonDeg, double heightM,
                                  out double e, out double n, out double u)
        {
            GeodeticToEcef(latDeg, lonDeg, heightM, out double x, out double y, out double z);
            EcefToEnu(x, y, z, out e, out n, out u);
        }

        /// <summary>Exact ellipsoidal height [m] of a local ENU point (for the baro).</summary>
        public double EnuToEllipsoidalHeight(double e, double n, double u)
        {
            EnuToGeodetic(e, n, u, out _, out _, out double h);
            return h;
        }

        // --- ECEF <-> ENU ----------------------------------------------------

        private void EnuToEcef(double e, double n, double u, out double x, out double y, out double z)
        {
            x = _x0 + (-_sinLon * e) + (-_sinLat * _cosLon * n) + (_cosLat * _cosLon * u);
            y = _y0 + ( _cosLon * e) + (-_sinLat * _sinLon * n) + (_cosLat * _sinLon * u);
            z = _z0 +                  ( _cosLat          * n) + (_sinLat          * u);
        }

        private void EcefToEnu(double x, double y, double z, out double e, out double n, out double u)
        {
            double dx = x - _x0, dy = y - _y0, dz = z - _z0;
            e = (-_sinLon * dx) + (_cosLon * dy);
            n = (-_sinLat * _cosLon * dx) + (-_sinLat * _sinLon * dy) + (_cosLat * dz);
            u = ( _cosLat * _cosLon * dx) + ( _cosLat * _sinLon * dy) + (_sinLat * dz);
        }

        // --- Geodetic <-> ECEF -----------------------------------------------

        private static void GeodeticToEcef(double latDeg, double lonDeg, double h,
                                           out double x, out double y, out double z)
        {
            double lat = latDeg * Math.PI / 180.0, lon = lonDeg * Math.PI / 180.0;
            double sLat = Math.Sin(lat), cLat = Math.Cos(lat);
            double sLon = Math.Sin(lon), cLon = Math.Cos(lon);

            double N = A / Math.Sqrt(1.0 - E2 * sLat * sLat);
            x = (N + h) * cLat * cLon;
            y = (N + h) * cLat * sLon;
            z = (N * (1.0 - E2) + h) * sLat;
        }

        private static void EcefToGeodetic(double x, double y, double z,
                                           out double latDeg, out double lonDeg, out double h)
        {
            // Bowring closed-form (sub-mm near the surface, no iteration)
            double lon = Math.Atan2(y, x);
            double p   = Math.Sqrt(x * x + y * y);
            double th  = Math.Atan2(z * A, p * B);
            double sT  = Math.Sin(th), cT = Math.Cos(th);
            double ep2 = (A * A - B * B) / (B * B);   // second eccentricity squared

            double lat = Math.Atan2(z + ep2 * B * sT * sT * sT,
                                    p - E2 * A * cT * cT * cT);
            double sLat = Math.Sin(lat);
            double N = A / Math.Sqrt(1.0 - E2 * sLat * sLat);
            h = p / Math.Cos(lat) - N;

            latDeg = lat * 180.0 / Math.PI;
            lonDeg = lon * 180.0 / Math.PI;
        }
    }
}
```

9. Coordinate and Rotor Mathematics

Centralizes the frame-conversion policy for Unity, ENU, NED, FLU, and FRD coordinates and supplies the static rotor thrust, yaw-torque, and first-order motor-response equations used by actuation and tests.

Files in this section

Runtime/Scripts/MathUtil/Frames.cs
Runtime/Scripts/MathUtil/RotorMath.cs

Runtime/Scripts/MathUtil/Frames.cs

SHA-256 6e4d82079936e0266c55091ce379d7a0f0c64d34c2f074ca4b1d82d9273174b9 | 362 lines | 15,171 bytes

```
using UnityEngine;

namespace MathUtil
{
    // ============================================================================
    // COORDINATE CONVENTIONS — single source of truth
    // ============================================================================
    //
    // Unity WORLD (left-handed):   X = East,    Y = Up,   Z = North
    // Unity BODY  (left-handed):   X = forward, Y = up,   Z = left
    //
    // WORLD frames (right-handed):
    //   ENU (E, N, U) = ( u.x,  u.z,  u.y )      <- canonical world frame.
    //   NED (N, E, D) = ( u.z,  u.x, -u.y )      <- MAVLink wire / PX4 only.
    //
    // BODY frames (right-handed):
    //   FLU (F, L, U) = ( b.x,  b.z,  b.y )      <- canonical body frame (ROS).
    //   FRD (F, R, D) = ( b.x, -b.z, -b.y )      <- PX4 HIL body frame.
    //
    // Pseudovectors (angular velocity, torque): relabel + negate-all,
    // correcting Unity's left-handed chirality (see TransformAngularVelocity).
    //
    // POLICY:
    //   * All world-frame I/O (gps_position, position commands, reset_pose,
    //     viz) is ALWAYS ENU. NED exists only at the MAVLink wire boundary
    //     (Px4Link) and in the explicitly named gps_vel_ned field.
    //   * SimFrame selects the BODY-axis convention for sensor outputs
    //     (FLU default; FRD for the PX4 HIL bridge). The world-position
    //     FRD branches below map to true NED and are kept correct for
    //     completeness, but no internal system routes world data through
    //     them.
    // ============================================================================
    public enum SimFrame
    {
        UnityBody, // drone local: +X forward, +Y up, +Z left
        FLU,       // body: +X forward, +Y left, +Z up   (ROS)  | world pairing: ENU
        FRD        // body: +X forward, +Y right, +Z down (PX4) | world pairing: NED
    }

    public static class Frames
    {
        // ========================================================================
        // Public API — Outbound (Unity → Client Frame)
        // ========================================================================

        public static void GetTargetBasisInUnity(SimFrame target,
            out Vector3 xAxis, out Vector3 yAxis, out Vector3 zAxis)
        {
            switch (target)
            {
                case SimFrame.FLU: // X=forward, Y=left(+Z), Z=up(+Y)
                    xAxis = new Vector3(1f, 0f, 0f);
                    yAxis = new Vector3(0f, 0f, 1f);
                    zAxis = new Vector3(0f, 1f, 0f);
                    break;
```

```
            case SimFrame.FRD: // X=forward, Y=right(-Z), Z=down(-Y)
                xAxis = new Vector3(1f, 0f, 0f);
                yAxis = new Vector3(0f, 0f, -1f);
                zAxis = new Vector3(0f, -1f, 0f);
                break;
            default: // UnityBody
                xAxis = new Vector3(1f, 0f, 0f);
                yAxis = new Vector3(0f, 1f, 0f);
                zAxis = new Vector3(0f, 0f, 1f);
                break;
        }
    }

    public static Vector3 TransformLinear(Vector3 vBodyUnity, SimFrame target)
    {
        return target switch
        {
            SimFrame.FRD => UnityBodyToFRD_Linear(vBodyUnity),
            SimFrame.FLU => UnityBodyToFLU_Linear(vBodyUnity),
            _ => vBodyUnity
        };
    }

    public static float TransformBinary(float Unity, SimFrame target)
    {
        if (target == SimFrame.FRD || target == SimFrame.FLU)
            return -Unity;
        return Unity;
    }

    public static Vector3 TransformFlip(Vector3 Unity, SimFrame target)
    {
        if (target == SimFrame.FRD)
        {
            return new Vector3(0, 0, Unity.z);
        }
        return Unity;
    }

    public static Vector3 TransformAcceleration(Vector3 aBodyUnity, SimFrame target)
    {
        return target switch
        {
            SimFrame.FRD => UnityBodyToFRD_Accel(aBodyUnity),
            SimFrame.FLU => UnityBodyToFLU_Accel(aBodyUnity),
            _ => aBodyUnity
        };
    }

    public static Vector3 TransformAngularVelocity(Vector3 wBodyUnity, SimFrame target)
    {
        return target switch
        {
            SimFrame.FRD => UnityBodyToFRD_AngularRate(wBodyUnity),
            SimFrame.FLU => UnityBodyToFLU_AngularRate(wBodyUnity),
            _ => wBodyUnity
        };
    }

    public static Vector3 TransformAttitude(Vector3 rpyDegUnityBody, SimFrame target)
    {
        return target switch
        {
            SimFrame.FLU => new Vector3(
                -rpyDegUnityBody.x,   // roll
                -rpyDegUnityBody.y,   // pitch
                rpyDegUnityBody.z     // yaw
            ),
            SimFrame.FRD => new Vector3(
                rpyDegUnityBody.x,
                rpyDegUnityBody.y,
                -rpyDegUnityBody.z
            ),
```

```
        _ => rpyDegUnityBody
    };
}

/// <summary>
/// Convert a world-space position/vector from Unity (X=E, Y=U, Z=N)
/// to the target world frame.
///
/// FLU pairing → ENU: (E, N, U) = (x, z, y)
/// FRD pairing → NED: (N, E, D) = (z, x, -y)
///
/// NOTE: NED is an X/Y swap relative to ENU, not a sign flip of it.
/// The previous FRD branch returned (x, -z, -y) = (East, South, Down),
/// which is not NED. Nothing internal consumes world-frame FRD
/// (world I/O is pinned to ENU — see header policy), but the mapping
/// is now correct for any future NED client.
/// </summary>
public static Vector3 TransformWorldPosition(Vector3 posUnityWorld, SimFrame target)
{
    return target switch
    {
        SimFrame.FLU => new Vector3(posUnityWorld.x, posUnityWorld.z, posUnityWorld.y),
        SimFrame.FRD => new Vector3(posUnityWorld.z, posUnityWorld.x, -posUnityWorld.y),
        _ => posUnityWorld
    };
}

public static Vector3 TransformWorldVector(Vector3 vectorUnityWorld, SimFrame target)
{
    return TransformWorldPosition(vectorUnityWorld, target);
}

public static Quaternion TransformQuaternion(Quaternion q, SimFrame target)
{
    return target switch
    {
        // Negate all spatial components to flip chirality, swap Y and Z.
        // Maps Unity body->world rotation to body-FLU -> world-ENU.
        SimFrame.FLU => new Quaternion(-q.x, -q.z, -q.y, q.w),
        // KNOWN GAP: no FRD branch — in FRD (PX4) mode the raw Unity
        // quaternion passes through. Unconsumed today (HIL_SENSOR has
        // no attitude; the bridge never reads imu_orientation). The
        // FRD->NED remap needs q_ned = Q_wned^-1 * q_unity * Q_bfrd,
        // with DIFFERENT world/body basis quaternions — derive AND
        // verify empirically (yaw-sweep invariance test) before
        // adding it. Do not guess a component remap here; that is
        // how the original det=-1 reflection bug happened.
        _ => q
    };
}

// ================================================================
// Public API — Inbound (Client Frame → Unity)
// ================================================================

/// <summary>
/// Convert a world-space position from client world frame back to
/// Unity (X=E, Y=U, Z=N). Inverse of TransformWorldPosition.
///
/// ENU (e,n,u) → Unity (e, u, n)   = (x, z, y)   [self-inverse relabel]
/// NED (n,e,d) → Unity (e, -d, n)  = (y, -z, x)
/// </summary>
public static Vector3 InverseTransformWorldPosition(Vector3 posClient, SimFrame source)
{
    return source switch
    {
        SimFrame.FLU => new Vector3(posClient.x, posClient.z, posClient.y),
        SimFrame.FRD => new Vector3(posClient.y, -posClient.z, posClient.x),
        _ => posClient
    };
}

/// <summary>
```

```
/// Convert a torque/angular-velocity vector from client frame back to Unity body.
/// Inverse of TransformAngularVelocity.
///
/// This is the critical chirality fix: FLU is right-handed, Unity is left-handed.
/// TransformAngularVelocity (out) does: axis swap + negate all (LH→RH).
/// The inverse (in) is the same operation: negate all + axis swap (RH→LH).
/// Since negate-all commutes with axis swap, the formula is identical.
///
/// FLU (x,y,z) → Unity (-x, -z, -y)   [negate all + swap Y⇔Z]
/// FRD (x,y,z) → Unity (x, -z, -y)     [swap Y⇔Z, negate swapped pair]
/// </summary>
public static Vector3 InverseTransformAngularVelocity(Vector3 wClient, SimFrame source)
{
    return source switch
    {
        SimFrame.FLU => new Vector3(-wClient.x, -wClient.z, -wClient.y),
        SimFrame.FRD => new Vector3(-wClient.x, wClient.z, wClient.y),
        _ => wClient
    };
}

/// <summary>
/// Convert a quaternion from client frame back to Unity body frame.
/// Inverse of TransformQuaternion.
///
/// TransformQuaternion does:   q_target = Q_basis^-1 * q_unity * Q_basis
/// Inverse is:                 q_unity  = Q_basis * q_target * Q_basis^-1
/// </summary>
public static Quaternion InverseTransformQuaternion(Quaternion qClient,
    SimFrame source)
{
    return source switch
    {
        SimFrame.FLU => new Quaternion(-qClient.x, -qClient.z, -qClient.y,
            qClient.w),
        _ => qClient
    };
}

/// <summary>
/// Convert a body-frame linear vector from client frame back to Unity body.
/// Inverse of TransformLinear.
///
/// The axis swap (x,z,y) is its own inverse for FLU.
/// FRD uses sign flips that are also self-inverse.
/// </summary>
public static Vector3 InverseTransformLinear(Vector3 vClient, SimFrame source)
{
    return source switch
    {
        SimFrame.FLU => new Vector3(vClient.x, vClient.z, vClient.y),
        SimFrame.FRD => new Vector3(vClient.x, -vClient.z, -vClient.y),
        _ => vClient
    };
}

// ====================================================================
// Implementation — UnityBody → FLU/FRD
// ====================================================================
//
// UnityBody basis:
//   X = forward, Y = up, Z = left
//
// FLU:
//   X = forward, Y = left, Z = up
//   Pure vectors: (x,y,z)_body → (x, z, y)_flu
//
// FRD:
//   X = forward, Y = right = -left, Z = down = -up
//   Pure vectors: (x,y,z)_body → (x, -z, -y)_frd

// Linear / Acceleration: pure axis relabel
private static Vector3 UnityBodyToFLU_Linear(Vector3 v)
```

```
            => new Vector3(v.x, v.z, v.y);
        private static Vector3 UnityBodyToFRD_Linear(Vector3 v)
            => new Vector3(v.x, -v.z, -v.y);


        // Angular velocity: relabel + negate all (LH → RH chirality correction)
        private static Vector3 UnityBodyToFLU_AngularRate(Vector3 w)
            => new Vector3(-w.x, -w.z, -w.y);

        private static Vector3 UnityBodyToFRD_AngularRate(Vector3 w)
            => new Vector3(-w.x, w.z, w.y);


        private static Vector3 UnityBodyToFLU_Accel(Vector3 a)
            => new Vector3(a.x, a.z, a.y);
        private static Vector3 UnityBodyToFRD_Accel(Vector3 a)
            => new Vector3(a.x, -a.z, -a.y);

        // ====================================================================
        // Quaternion basis-change helpers
        // ====================================================================

        private static Quaternion GetUnityFromTargetBasis(SimFrame target)
        {
            GetTargetBasisInUnity(target, out var x, out var y, out var z);
            return QuaternionFromBasis(x, y, z);
        }

        private static Quaternion QuaternionFromBasis(Vector3 xAxisUnityBody, Vector3 yAxisUnityBody, Ve
ctor3 zAxisUnityBody)
        {
            var m = new Matrix4x4();
            m.SetColumn(0, new Vector4(xAxisUnityBody.x, xAxisUnityBody.y, xAxisUnityBody.z, 0f));
            m.SetColumn(1, new Vector4(yAxisUnityBody.x, yAxisUnityBody.y, yAxisUnityBody.z, 0f));
            m.SetColumn(2, new Vector4(zAxisUnityBody.x, zAxisUnityBody.y, zAxisUnityBody.z, 0f));
            m.SetColumn(3, new Vector4(0f, 0f, 0f, 1f));
            return QuaternionFromMatrix(m);
        }

        private static Quaternion QuaternionFromMatrix(Matrix4x4 m)
        {
            float trace = m.m00 + m.m11 + m.m22;
            if (trace > 0f)
            {
                float s = Mathf.Sqrt(trace + 1f) * 2f;
                float invS = 1f / s;
                return new Quaternion(
                    (m.m21 - m.m12) * invS,
                    (m.m02 - m.m20) * invS,
                    (m.m10 - m.m01) * invS,
                    0.25f * s
                );
            }

            if (m.m00 > m.m11 && m.m00 > m.m22)
            {
                float s = Mathf.Sqrt(1f + m.m00 - m.m11 - m.m22) * 2f;
                float invS = 1f / s;
                return new Quaternion(
                    0.25f * s,
                    (m.m01 + m.m10) * invS,
                    (m.m02 + m.m20) * invS,
                    (m.m21 - m.m12) * invS
                );
            }

            if (m.m11 > m.m22)
            {
                float s = Mathf.Sqrt(1f + m.m11 - m.m00 - m.m22) * 2f;
                float invS = 1f / s;
                return new Quaternion(
                    (m.m01 + m.m10) * invS,
                    0.25f * s,
```

```
                    (m.m12 + m.m21) * invS,
                    (m.m02 - m.m20) * invS
                );
            }

            float sZ = Mathf.Sqrt(1f + m.m22 - m.m00 - m.m11) * 2f;
            float invSZ = 1f / sZ;
            return new Quaternion(
                (m.m02 + m.m20) * invSZ,
                (m.m12 + m.m21) * invSZ,
                0.25f * sZ,
                (m.m10 - m.m01) * invSZ
            );
        }
    }
}
```

Runtime/Scripts/MathUtil/RotorMath.cs

SHA-256 aaea01e181a0230554d16f9ea285a8fa1e180ddd46c975b71159484be44348fd | 65 lines | 2,371 bytes

```
using System;

namespace MathUtil
{
    public enum RotorTorqueConvention
    {
        Legacy2Point5Pi = 0,
        PowerCoefficient2Pi = 1,
    }

    public static class RotorMath
    {
        public const float SeaLevelAirDensity = 1.225f;

        public static float StaticThrust(
            float thrustCoefficient, float airDensity, float revolutionsPerSecond,
            float propellerDiameter)
        {
            double n2 = revolutionsPerSecond * revolutionsPerSecond;
            double d4 = Math.Pow(propellerDiameter, 4);
            return (float)(thrustCoefficient * airDensity * n2 * d4);
        }

        public static float StaticYawTorque(
            float torqueCoefficient, float airDensity, float revolutionsPerSecond,
            float propellerDiameter, RotorTorqueConvention convention)
        {
            double denominator = convention == RotorTorqueConvention.PowerCoefficient2Pi
                ? 2.0 * Math.PI
                : 2.5 * Math.PI;
            double n2 = revolutionsPerSecond * revolutionsPerSecond;
            double d5 = Math.Pow(propellerDiameter, 5);
            return (float)(torqueCoefficient * airDensity * n2 * d5 / denominator);
        }

        public static float CommandToThrust(float command01, float maxThrust, float airDensityRatio = 1f
)
        {
            return Clamp01(command01) * maxThrust * airDensityRatio;
        }

        public static float CommandToYawTorque(
            float command01, float maxTorque, int turningDirection, float airDensityRatio = 1f)
        {
            return Clamp01(command01) * maxTorque * turningDirection * airDensityRatio;
        }

        public static float FirstOrderStep(
            float current, float target, float dtSeconds,
            float tauUpSeconds, float tauDownSeconds)
        {
            if (dtSeconds <= 0f) return current;
            float tau = target >= current ? tauUpSeconds : tauDownSeconds;
```

```
            if (tau <= 0f) return target;
            double alpha = 1.0 - Math.Exp(-dtSeconds / tau);
            return current + (target - current) * (float)alpha;
        }

        private static float Clamp01(float value)
        {
            if (value < 0f) return 0f;
            if (value > 1f) return 1f;
            return value;
        }
    }
}
```

10. YAML Configuration and Drone Profiles

Loads project overrides or packaged defaults, parses schema-versioned drone configuration, and stores physical, propulsion, controller, camera, and limit parameters. The sample profiles mirror the packaged defaults so imported samples can be edited independently.

Files in this section

- Runtime/Scripts/Yaml/Drone/DroneConfigLoader.cs
- Runtime/Scripts/Yaml/Drone/DroneConfigStructs.cs
- Runtime/Scripts/Yaml/Drone/YamlParser.cs
- Runtime/Resources/QuadSim/Configs/Drones/crazyflie2_quad.yaml
- Runtime/Resources/QuadSim/Configs/Drones/flightmare_quad.yaml
- Runtime/Resources/QuadSim/Configs/Drones/sentinel_quad.yaml
- Runtime/Resources/QuadSim/Configs/Drones/standard_quad.yaml
- Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/crazyflie2_quad.yaml
- Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/flightmare_quad.yaml
- Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/sentinel_quad.yaml
- Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/standard_quad.yaml

Runtime/Scripts/Yaml/Drone/DroneConfigLoader.cs

SHA-256 b5165731f307d8dcdf0a7c2cc315cd869f30b7e64139d0e8e789624e86d5f4ed | 158 lines | 6,155 bytes

```
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

namespace Yaml.Drone
{
    public static class DroneConfigLoader
    {
        // User/project override location:
        //   Assets/StreamingAssets/Configs/Drones/<config>.yaml
        //
        // Package default location:
        //   Runtime/Resources/QuadSim/Configs/Drones/<config>.yaml
        //
        // Resources.Load paths intentionally omit the extension, so
        // Runtime/Resources/QuadSim/Configs/Drones/flightmare_quad.yaml is loaded as:
        //   Resources.Load<TextAsset>("QuadSim/Configs/Drones/flightmare_quad")
        private const string StreamingConfigFolder = "Configs/Drones";
        private const string PackageResourceFolder = "QuadSim/Configs/Drones";

        public static string GetConfigDirectory()
        {
            return Path.Combine(Application.streamingAssetsPath, "Configs", "Drones");
        }

        public static List<string> GetAvailableConfigs()
        {
            var results = new List<string>();
            var seen = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

            // 1) User/project overrides from StreamingAssets.
            string dir = GetConfigDirectory();
            if (Directory.Exists(dir))
            {
                string[] files = Directory.GetFiles(dir, "*.yaml", SearchOption.TopDirectoryOnly);
                foreach (string file in files)
                {
                    string name = NormalizeConfigName(file);
                    if (seen.Add(name))
                        results.Add(name);
                }
```

```
            }

            // 2) Package defaults from Resources.
            TextAsset[] defaults = Resources.LoadAll<TextAsset>(PackageResourceFolder);
            foreach (TextAsset asset in defaults)
            {
                if (asset == null) continue;

                string name = NormalizeConfigName(asset.name);
                if (seen.Add(name))
                    results.Add(name);
            }

            if (results.Count == 0)
            {
                Debug.LogWarning(
                    "[DroneConfigLoader] No drone configs found.\n" +
                    $"  Checked project overrides: {dir}\n" +
                    $"  Checked package resources: Resources/{PackageResourceFolder}/*.yaml"
                );
            }

            return results;
        }

        public static bool LoadConfig(string configName, out DroneConfig outConfig)
        {
            outConfig = new DroneConfig();

            if (string.IsNullOrWhiteSpace(configName))
            {
                Debug.LogError("[DroneConfigLoader] Empty config name.");
                return false;
            }

            string cleanName = NormalizeConfigName(configName);

            // 1) User/project override. Checked first so users can tune or replace
            // package configs without modifying the installed package.
            string streamingPath = Path.Combine(GetConfigDirectory(), cleanName + ".yaml");
            if (File.Exists(streamingPath))
            {
                string yamlText;
                try
                {
                    yamlText = File.ReadAllText(streamingPath);
                }
                catch (Exception e)
                {
                    Debug.LogError($"[DroneConfigLoader] Failed to read config override: {streamingPath}
\n{e}");
                    return false;
                }

                bool success = ParseYamlText(yamlText, streamingPath, cleanName, ref outConfig);
                if (success)
                    Debug.Log($"[DroneConfigLoader] Loaded project override config: {outConfig.ModelPara
ms.Name} ({streamingPath})");

                return success;
            }

            // 2) Package default fallback. These live inside the package under:
            // Runtime/Resources/QuadSim/Configs/Drones/<name>.yaml
            string resourcePath = $"{PackageResourceFolder}/{cleanName}";
            TextAsset packageAsset = Resources.Load<TextAsset>(resourcePath);
            if (packageAsset != null)
            {
                string sourceLabel = $"Resources/{resourcePath}.yaml";
                bool success = ParseYamlText(packageAsset.text, sourceLabel, cleanName, ref outConfig);
                if (success)
                    Debug.Log($"[DroneConfigLoader] Loaded package default config: {outConfig.ModelParam
s.Name} ({sourceLabel})");
```

```
                return success;
            }

            Debug.LogError(
                "[DroneConfigLoader] Config file not found.\n" +
                $"  Requested: {configName}\n" +
                $"  Checked project override: {streamingPath}\n" +
                $"  Checked package default: Resources/{resourcePath}.yaml\n" +
                "  To override package defaults, create:\n" +
                $"  Assets/StreamingAssets/{StreamingConfigFolder}/{cleanName}.yaml"
            );

            return false;
        }

        private static bool ParseYamlText(string yamlText, string sourceLabel, string cleanName, ref Dro
neConfig outConfig)
        {
            bool success = YamlParser.LoadDroneConfigFromString(
                yamlText,
                sourceLabel,
                out outConfig.ModelParams,
                out outConfig.DroneParams,
                out outConfig.RotorParams,
                out outConfig.CamParams,
                out outConfig.FlightParams,
                out outConfig.Angle,
                out outConfig.Acro,
                out outConfig.Velocity,
                out outConfig.Position
            );

            if (success)
                outConfig.SourceFile = cleanName;

            return success;
        }

        private static string NormalizeConfigName(string configName)
        {
            string name = configName.Trim();
            name = Path.GetFileNameWithoutExtension(name);
            return name;
        }
    }
}
```

Runtime/Scripts/Yaml/Drone/DroneConfigStructs.cs

SHA-256 de674d9a681136ee8da469f775b721713b0aa3eabc47d4bdc16c9ff5b3c625dc | 241 lines | 7,014 bytes

```
//using UnityEngine.Addres; // Required

using System;
using MathUtil;
using UnityEngine;
using Mathf = UnityEngine.Mathf;

namespace Yaml.Drone
{
    public struct ModelType
    {
        public int SchemaVersion;
        public string Name;
        //public AssetReference BodyMesh;

    }

    public struct DroneParameters
    {
        public float Mass;
        public Vector3 CenterOfMass;
        public Vector3 Inertia;
```

```
    public float Arm_Length;
    public float LinearDrag_Coeff;
    public float AngularDrag_Coeff;
}

public struct RotorModel
{
    public float Prop_Diameter;
    public float MaxRPM;
    public float ThrustCoeff;
    public float TorqueCoeff;
    public RotorTorqueConvention TorqueConvention;
    public float MotorTimeConstantUp;
    public float MotorTimeConstantDown;

}

public struct RotorInstance
{
    // FL, FR, BL, BR
    public int ID;
    public Vector3 Location_Body;
    // +1 = CW, -1 = CCW
    public int Spin_Dir;
}

public struct RotorModelInstance
{
    public RotorModel Model;
    public RotorInstance[] Rotors;

    public RotorModelInstance(int numRotors, RotorModel model)
    {
        Model = model;
        Rotors = new RotorInstance[numRotors];
    }
}

public struct CamParameters
{
    public float CamArm_Length;
    public float FOV;
    public float CamPitch;
    public Vector3 CamSocket_Offset;
}

public struct PID
{
    public float P;
    public float I;
    public float D;
}
public struct AcroPID
{
    public PID AcroRoll {get;set;}
    public PID AcroPitch {get;set;}
    public PID AcroYaw {get;set;}

    public Vector4 GetPGains()
    {
        Vector4 PGains = new Vector4(AcroRoll.P, AcroPitch.P, AcroYaw.P, 0.0f);
        return PGains;
    }
    public Vector4 GetIGains()
    {
        Vector4 IGains = new Vector4(AcroRoll.I, AcroPitch.I, AcroYaw.I, 0.0f);
        return IGains;
    }
    public Vector4 GetDGains()
    {
        Vector4 DGains = new Vector4(AcroRoll.D, AcroPitch.D, AcroYaw.D, 0.0f);
        return DGains;
    }
```

```
}
public struct AnglePID
{
    public PID AngleRoll {get;set;}
    public PID AnglePitch {get;set;}
    public PID AngleYaw {get;set;}

    public Vector4 GetPGains()
    {
        Vector4 PGains = new Vector4(AngleRoll.P, AnglePitch.P, AngleYaw.P, 0.0f);
        return PGains;
    }
    public Vector4 GetIGains()
    {
        Vector4 IGains = new Vector4(AngleRoll.I, AnglePitch.I, AngleYaw.I, 0.0f);
        return IGains;
    }
    public Vector4 GetDGains()
    {
        Vector4 DGains = new Vector4(AngleRoll.D, AnglePitch.D, AngleYaw.D, 0.0f);
        return DGains;
    }
}
public struct VelocityPID
{
    public PID VelX {get;set;}
    public PID VelY {get;set;}
    public PID VelZ {get;set;}

    public Vector4 GetPGains()
    {
        Vector4 PGains = new Vector4(VelX.P, VelY.P,0.0f, VelZ.P);
        return PGains;
    }
    public Vector4 GetIGains()
    {
        Vector4 IGains = new Vector4(VelX.I, VelY.I,0.0f, VelZ.I);
        return IGains;
    }
    public Vector4 GetDGains()
    {
        Vector4 DGains = new Vector4(VelX.D, VelY.D, 0.0f, VelZ.D);
        return DGains;
    }
}
public struct PositionPID
{
    public PID PosX {get;set;}
    public PID PosY {get;set;}
    public PID PosZ {get;set;}

    public Vector4 GetPGains()
    {
        Vector4 PGains = new Vector4(PosX.P, PosY.P,0.0f, PosZ.P);
        return PGains;
    }
    public Vector4 GetIGains()
    {
        Vector4 IGains = new Vector4(PosX.I, PosY.I,0.0f, PosZ.I);
        return IGains;
    }
    public Vector4 GetDGains()
    {
        Vector4 DGains = new Vector4(PosX.D, PosY.D, 0.0f, PosZ.D);
        return DGains;
    }
}

public struct FlightParameters
{
    public float MaxVelXY;
    public float MaxVelZ;
    public float MaxAngle;
```

```
        public float MaxRateRollPitch;
        public float MaxRateYaw;
        public float MaxThrust;
        public float MaxPID;

        // Nav
        public float AcceptableDistance;
    }

    // TODO: Not in use at the moment, make sure to implement down the line
    public struct ObstacleParameters
    {
        public Vector3 OuterBoundary;
        public Vector3 InnerBoundary;
        public Vector3 SpawnLocation;
    }

    public struct DroneConfig
    {
        public ModelType ModelParams;
        public DroneParameters DroneParams;
        public RotorModelInstance RotorParams;
        public CamParameters CamParams;
        public FlightParameters FlightParams;

        public AcroPID Acro;
        public AnglePID Angle;
        public VelocityPID Velocity;
        public PositionPID Position;

        // Source File TODO: Lets try and implement this
        public string SourceFile;

        // Should add isValid or unnecessary?
    }

    // Math Stuff for Rotors
    public struct RotorPhysicsDerived
    {
        public float MaxThrust;
        public float MaxTorque;

        public void ComputeRotor(RotorModel InModel,float InArmLengthMeters)
        {
            float revPerSec = InModel.MaxRPM / 60.0f;
            MaxThrust = RotorMath.StaticThrust(
                InModel.ThrustCoeff, RotorMath.SeaLevelAirDensity,
                revPerSec, InModel.Prop_Diameter);
            MaxTorque = RotorMath.StaticYawTorque(
                InModel.TorqueCoeff, RotorMath.SeaLevelAirDensity, revPerSec,
                InModel.Prop_Diameter, InModel.TorqueConvention);
        }

        public float ComputeHoverThrottle(float VehicleMassKg, int MotorCount = 4)
        {
            if (MotorCount <= 0 || MaxThrust <= 0.0f) return 0.5f;
            float perMotorHoverN = (VehicleMassKg * 9.81f) / (float)MotorCount;
            return Mathf.Clamp(perMotorHoverN / MaxThrust, 0.0f, 1.0f);
        }

        public float ControlToThrust(float Control01, float AirDensityRatio = 1.0f)
        {
            return RotorMath.CommandToThrust(Control01, MaxThrust, AirDensityRatio);
        }

        public float ControlToYawTorque(float Control01, int TurningDir, float AirDensityRatio = 1.0f)
        {
            return RotorMath.CommandToYawTorque(Control01, MaxTorque, TurningDir, AirDensityRatio);
        }

    }

}
```

Runtime/Scripts/Yaml/Drone/YamlParser.cs

SHA-256 47a566ff5e1d4fbf11f1624e0986470ae0dba6092e680ee99fd1000213bf496d | 449 lines | 19,061 bytes

```
using System;
using UnityEngine;
using YamlDotNet.RepresentationModel;
using System.Globalization;
using System.IO;
using System.Collections.Generic;
using MathUtil;

namespace Yaml.Drone
{
    public class YamlParser
    {
        // ------------------------------------------------------------
        // Helpers
        // ------------------------------------------------------------

        public static bool ParseVector(YamlNode node, out Vector3 outVec)
        {
            outVec = Vector3.zero;
            if (node == null || !(node is YamlMappingNode mappingNode)) return false;

            if (TryGetFloat(mappingNode, "x", out float x)) outVec.x = x;
            if (TryGetFloat(mappingNode, "y", out float y)) outVec.y = y;
            if (TryGetFloat(mappingNode, "z", out float z)) outVec.z = z;
            return true;
        }

        public static bool ParsePID(YamlNode node, out PID outPID)
        {
            outPID = new PID();
            if (node == null || !(node is YamlMappingNode mappingNode)) return false;

            if (TryGetFloat(mappingNode, "p", out float p)) outPID.P = p;
            if (TryGetFloat(mappingNode, "i", out float i)) outPID.I = i;
            if (TryGetFloat(mappingNode, "d", out float d)) outPID.D = d;
            return true;
        }

        private static bool TryGetFloat(YamlMappingNode map, string key, out float result)
        {
            result = 0f;
            if (map.Children.TryGetValue(new YamlScalarNode(key), out YamlNode valueNode))
            {
                if (valueNode is YamlScalarNode scalar)
                    return float.TryParse(scalar.Value, NumberStyles.Float, CultureInfo.InvariantCulture
, out result);
            }
            return false;
        }

        private static bool TryGetString(YamlMappingNode map, string key, out string result)
        {
            result = "";
            if (map.Children.TryGetValue(new YamlScalarNode(key), out YamlNode valueNode))
            {
                if (valueNode is YamlScalarNode scalar)
                {
                    result = scalar.Value;
                    return true;
                }
            }
            return false;
        }

        private static bool TryGetNode(YamlMappingNode map, string key, out YamlNode result)
        {
            return map.Children.TryGetValue(new YamlScalarNode(key), out result);
        }

        private static void InitDefaults(
```

```
    out ModelType outModel,
    out DroneParameters outPhys,
    out RotorModelInstance outProp,
    out CamParameters outCam,
    out FlightParameters outLimits,
    out AnglePID outAngle,
    out AcroPID outAcro,
    out VelocityPID outVel,
    out PositionPID outPos
)
{
    outModel = new ModelType();
    outPhys = new DroneParameters();
    outProp = new RotorModelInstance();
    outCam = new CamParameters();
    outLimits = new FlightParameters();
    outAngle = new AnglePID();
    outAcro = new AcroPID();
    outVel = new VelocityPID();
    outPos = new PositionPID();
}

// ------------------------------------------------------------
// Main Load Functions
// ------------------------------------------------------------

public static bool LoadDroneConfig(
    string fullPath,
    out ModelType outModel,
    out DroneParameters outPhys,
    out RotorModelInstance outProp,
    out CamParameters outCam,
    out FlightParameters outLimits,
    out AnglePID outAngle,
    out AcroPID outAcro,
    out VelocityPID outVel,
    out PositionPID outPos
)
{
    InitDefaults(
        out outModel,
        out outPhys,
        out outProp,
        out outCam,
        out outLimits,
        out outAngle,
        out outAcro,
        out outVel,
        out outPos
    );

    if (!File.Exists(fullPath))
    {
        Debug.LogError($"[Drone.YamlParser] File does not exist: {fullPath}");
        return false;
    }

    string yamlText;
    try
    {
        yamlText = File.ReadAllText(fullPath);
    }
    catch (Exception e)
    {
        Debug.LogError($"[Drone.YamlParser] Failed to read file: {fullPath}\n{e}");
        return false;
    }

    return LoadDroneConfigFromString(
        yamlText,
        fullPath,
        out outModel,
        out outPhys,
```

```
                out outProp,
                out outCam,
                out outLimits,
                out outAngle,
                out outAcro,
                out outVel,
                out outPos
            );
        }

        public static bool LoadDroneConfigFromString(
            string yamlText,
            string sourceLabel,
            out ModelType outModel,
            out DroneParameters outPhys,
            out RotorModelInstance outProp,
            out CamParameters outCam,
            out FlightParameters outLimits,
            out AnglePID outAngle,
            out AcroPID outAcro,
            out VelocityPID outVel,
            out PositionPID outPos
        )
        {
            InitDefaults(
                out outModel,
                out outPhys,
                out outProp,
                out outCam,
                out outLimits,
                out outAngle,
                out outAcro,
                out outVel,
                out outPos
            );

            if (string.IsNullOrWhiteSpace(yamlText))
            {
                Debug.LogError($"[Drone.YamlParser] YAML text is empty: {sourceLabel}");
                return false;
            }

            YamlMappingNode rootNode;

            try
            {
                using (var reader = new StringReader(yamlText))
                {
                    var yaml = new YamlStream();
                    yaml.Load(reader);

                    if (yaml.Documents.Count == 0)
                    {
                        Debug.LogError($"[Drone.YamlParser] YAML has no documents: {sourceLabel}");
                        return false;
                    }

                    if (!(yaml.Documents[0].RootNode is YamlMappingNode mapping))
                    {
                        Debug.LogError($"[Drone.YamlParser] YAML root is not a mapping node: {sourceLabe
l}");
                        return false;
                    }

                    rootNode = mapping;
                }
            }
            catch (Exception e)
            {
                Debug.LogError($"[Drone.YamlParser] YAML load exception in {sourceLabel}: {e.Message}");
                return false;
            }
```

```
            return ParseDroneConfigNode(
                rootNode,
                sourceLabel,
                out outModel,
                out outPhys,
                out outProp,
                out outCam,
                out outLimits,
                out outAngle,
                out outAcro,
                out outVel,
                out outPos
            );
        }

        private static bool ParseDroneConfigNode(
            YamlMappingNode rootNode,
            string sourceLabel,
            out ModelType outModel,
            out DroneParameters outPhys,
            out RotorModelInstance outProp,
            out CamParameters outCam,
            out FlightParameters outLimits,
            out AnglePID outAngle,
            out AcroPID outAcro,
            out VelocityPID outVel,
            out PositionPID outPos
        )
        {
            InitDefaults(
                out outModel,
                out outPhys,
                out outProp,
                out outCam,
                out outLimits,
                out outAngle,
                out outAcro,
                out outVel,
                out outPos
            );

            try
            {
                // 1. Model Name & Mesh
                if (TryGetFloat(rootNode, "schema_version", out float schemaVersion))
                {
                    if (schemaVersion != 1f)
                    {
                        Debug.LogError($"[Drone.YamlParser] Unsupported schema_version {schemaVersion}:
{sourceLabel}");
                        return false;
                    }
                    outModel.SchemaVersion = 1;
                }
                else
                {
                    Debug.LogWarning($"[Drone.YamlParser] Legacy unversioned drone config: {sourceLabel}
");
                }

                if (TryGetString(rootNode, "name", out string name)) outModel.Name = name;

                if (TryGetString(rootNode, "body_mesh", out string meshPath))
                {
                    // Mesh path is kept in YAML for future model catalog support.
                    // Runtime prefab/model loading is handled elsewhere for now.
                }

                // 2. Physical Properties
                if (TryGetNode(rootNode, "physical_properties", out YamlNode physNode) && physNode is Ya
mlMappingNode phys)
                {
                    if (TryGetFloat(phys, "mass_kg", out float mass)) outPhys.Mass = mass;
```

```
                    if (TryGetNode(phys, "com_offset", out YamlNode comNode))
                        ParseVector(comNode, out outPhys.CenterOfMass);

                    if (TryGetNode(phys, "inertia", out YamlNode inertiaNode))
                        ParseVector(inertiaNode, out outPhys.Inertia);

                    if (TryGetFloat(phys, "arm_length", out float arm)) outPhys.Arm_Length = arm;
                    if (TryGetFloat(phys, "linear_drag_coef", out float lDrag)) outPhys.LinearDrag_Coeff
 = lDrag;
                    if (TryGetFloat(phys, "angular_drag_coef", out float aDrag)) outPhys.AngularDrag_Coe
ff = aDrag;
                }

                // 3. Propulsion
                if (TryGetNode(rootNode, "propulsion", out YamlNode propNode) && propNode is YamlMapping
Node prop)
                {
                    if (TryGetFloat(prop, "prop_diameter", out float diam)) outProp.Model.Prop_Diameter
= diam;
                    if (TryGetFloat(prop, "max_rpm", out float rpm)) outProp.Model.MaxRPM = rpm;
                    if (TryGetFloat(prop, "thrust_coef", out float thCo)) outProp.Model.ThrustCoeff = th
Co;
                    if (TryGetFloat(prop, "torque_coef", out float toCo)) outProp.Model.TorqueCoeff = to
Co;
                    if (TryGetString(prop, "torque_coefficient_convention", out string torqueConvention)
)
                    {
                        if (torqueConvention == "power_coefficient_2_pi")
                            outProp.Model.TorqueConvention = RotorTorqueConvention.PowerCoefficient2Pi;
                        else if (torqueConvention == "legacy_2_5_pi")
                            outProp.Model.TorqueConvention = RotorTorqueConvention.Legacy2Point5Pi;
                        else
                        {
                            Debug.LogError($"[Drone.YamlParser] Unknown torque coefficient convention '{
torqueConvention}': {sourceLabel}");
                            return false;
                        }
                    }
                    if (TryGetFloat(prop, "motor_time_constant_up_s", out float tauUp))
                        outProp.Model.MotorTimeConstantUp = tauUp;
                    if (TryGetFloat(prop, "motor_time_constant_down_s", out float tauDown))
                        outProp.Model.MotorTimeConstantDown = tauDown;

                    if (TryGetNode(prop, "propellers", out YamlNode rotorsNode) && rotorsNode is YamlSeq
uenceNode rotorSeq)
                    {
                        var rotorList = new List<RotorInstance>(rotorSeq.Children.Count);

                        foreach (YamlNode child in rotorSeq)
                        {
                            if (child is YamlMappingNode p)
                            {
                                RotorInstance r = new RotorInstance();

                                if (TryGetFloat(p, "id", out float id)) r.ID = (int)id;
                                if (TryGetFloat(p, "dir", out float dir)) r.Spin_Dir = (int)dir;

                                float rx = 0, ry = 0, rz = 0;
                                TryGetFloat(p, "x", out rx);
                                TryGetFloat(p, "y", out ry);
                                TryGetFloat(p, "z", out rz);
                                r.Location_Body = new Vector3(rx, ry, rz);

                                rotorList.Add(r);
                            }
                        }

                        rotorList.Sort((a, b) => a.ID.CompareTo(b.ID));
                        outProp.Rotors = rotorList.ToArray();
                    }
                }
```

```
                if (!ValidateRotors(outProp.Rotors, sourceLabel))
                    return false;

                // 4. Controller Gains
                if (TryGetNode(rootNode, "controller", out YamlNode ctrlNode) && ctrlNode is YamlMapping
Node ctrl)
                {
                    if (TryGetNode(ctrl, "position_gains", out YamlNode posNode) && posNode is YamlMappi
ngNode posMap)
                    {
                        if (TryGetNode(posMap, "x", out YamlNode px) && ParsePID(px, out PID tmpX)) outP
os.PosX = tmpX;
                        if (TryGetNode(posMap, "y", out YamlNode py) && ParsePID(py, out PID tmpY)) outP
os.PosY = tmpY;
                        if (TryGetNode(posMap, "z", out YamlNode pz) && ParsePID(pz, out PID tmpZ)) outP
os.PosZ = tmpZ;
                    }

                    if (TryGetNode(ctrl, "velocity_gains", out YamlNode velNode) && velNode is YamlMappi
ngNode velMap)
                    {
                        if (TryGetNode(velMap, "x", out YamlNode vx) && ParsePID(vx, out PID tmpX)) outV
el.VelX = tmpX;
                        if (TryGetNode(velMap, "y", out YamlNode vy) && ParsePID(vy, out PID tmpY)) outV
el.VelY = tmpY;
                        if (TryGetNode(velMap, "z", out YamlNode vz) && ParsePID(vz, out PID tmpZ)) outV
el.VelZ = tmpZ;
                    }

                    if (TryGetNode(ctrl, "attitude_gains", out YamlNode attNode) && attNode is YamlMappi
ngNode attMap)
                    {
                        if (TryGetNode(attMap, "roll", out YamlNode ar) && ParsePID(ar, out PID tmpR)) o
utAngle.AngleRoll = tmpR;
                        if (TryGetNode(attMap, "pitch", out YamlNode ap) && ParsePID(ap, out PID tmpP))
outAngle.AnglePitch = tmpP;
                        if (TryGetNode(attMap, "yaw", out YamlNode ay) && ParsePID(ay, out PID tmpY)) ou
tAngle.AngleYaw = tmpY;
                    }

                    if (TryGetNode(ctrl, "rate_gains", out YamlNode rateNode) && rateNode is YamlMapping
Node rateMap)
                    {
                        if (TryGetNode(rateMap, "roll", out YamlNode rr) && ParsePID(rr, out PID tmpR))
outAcro.AcroRoll = tmpR;
                        if (TryGetNode(rateMap, "pitch", out YamlNode rp) && ParsePID(rp, out PID tmpP))
 outAcro.AcroPitch = tmpP;
                        if (TryGetNode(rateMap, "yaw", out YamlNode ry) && ParsePID(ry, out PID tmpY)) o
utAcro.AcroYaw = tmpY;
                    }
                }

                // 5. Camera
                if (TryGetNode(rootNode, "camera_parameters", out YamlNode camNode) && camNode is YamlMa
ppingNode cam)
                {
                    if (TryGetFloat(cam, "Cam_Arm_Length", out float arm)) outCam.CamArm_Length = arm;
                    if (TryGetFloat(cam, "FOV", out float fov)) outCam.FOV = fov;
                    if (TryGetFloat(cam, "Cam_Pitch", out float pitch)) outCam.CamPitch = pitch;

                    if (TryGetNode(cam, "Cam_Socket_Offset", out YamlNode offsetNode))
                    {
                        ParseVector(offsetNode, out outCam.CamSocket_Offset);
                    }
                }

                // 6. Limits
                if (TryGetNode(rootNode, "limits", out YamlNode limNode) && limNode is YamlMappingNode l
im)
                {
                    if (TryGetFloat(lim, "max_velocity_xy", out float mv)) outLimits.MaxVelXY = mv;
                    if (TryGetFloat(lim, "max_velocity_z", out float mvz)) outLimits.MaxVelZ = mvz;
                    if (TryGetFloat(lim, "max_tilt_angle_deg", out float ma)) outLimits.MaxAngle = ma;
```

```
                    if (TryGetFloat(lim, "max_angular_rate_RP_deg", out float mr)) outLimits.MaxRateRoll
Pitch = mr;
                    if (TryGetFloat(lim, "max_angular_rate_Yaw_deg", out float mry)) outLimits.MaxRateYa
w = mry;
                    if (TryGetFloat(lim, "max_thrust_newtons", out float mt)) outLimits.MaxThrust = mt;
                    if (TryGetFloat(lim, "max_pid_output", out float mp)) outLimits.MaxPID = mp;
                    if (TryGetFloat(lim, "acceptable_distance", out float ad)) outLimits.AcceptableDista
nce = ad;
                }

                return true;
            }
            catch (Exception e)
            {
                Debug.LogError($"[Drone.YamlParser] Parsing logic error in {sourceLabel}: {e.Message}\n{
e.StackTrace}");
                return false;
            }
        }

        private static bool ValidateRotors(RotorInstance[] rotors, string sourceLabel)
        {
            if (rotors == null || rotors.Length == 0)
            {
                Debug.LogError($"[Drone.YamlParser] No propellers declared: {sourceLabel}");
                return false;
            }
            if (rotors.Length != 4)
            {
                Debug.LogError($"[Drone.YamlParser] Expected exactly four propellers: {sourceLabel}");
                return false;
            }

            var ids = new HashSet<int>();
            for (int i = 0; i < rotors.Length; i++)
            {
                if (!ids.Add(rotors[i].ID))
                {
                    Debug.LogError($"[Drone.YamlParser] Duplicate propeller id {rotors[i].ID}: {sourceLa
bel}");
                    return false;
                }
                if (rotors[i].Spin_Dir != -1 && rotors[i].Spin_Dir != 1)
                {
                    Debug.LogError($"[Drone.YamlParser] Propeller {rotors[i].ID} dir must be -1 or 1: {s
ourceLabel}");
                    return false;
                }
            }
            return true;
        }
    }
}
```

Runtime/Resources/QuadSim/Configs/Drones/crazyflie2_quad.yaml

SHA-256 f9552e20da18a77878798b3dcc83fccdde39a6c9846a248020d109eab32bc527 | 92 lines | 2,558 bytes

```
# Drone Configuration File - Crazyflie 2.1
schema_version: 1
name: "Crazyflie_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/CrazyFlie/cf2_assembly.cf2_assembly"

physical_properties:
  mass_kg: 0.029
  com_offset: {x: 0.0, y: 0.0, z: 0.0}
  inertia: {x: 0.00001657, y: 0.00001666, z: 0.00002926}
  arm_length: 0.046
  linear_drag_coef: 0.02
  angular_drag_coef: 0.001

propulsion:
  prop_diameter: 0.045
  max_rpm: 24000
```

```
    # Bitcraze CF2.1+ default:
    # THRUST_MAX = 0.12 N per motor
    # THRUST2TORQUE = 0.0069928948992470565 m
    max_thrust: 0.147
    thrust_coef: 0.183
    torque_coef: 0.0069928948992470565
    torque_coefficient_convention: legacy_2_5_pi

    # Firmware thrust curve, not directly represented by current QuadSim schema:
    # VMOTOR2THRUST0 = -0.02476537915958403
    # VMOTOR2THRUST1 =  0.06523793527519485
    # VMOTOR2THRUST2 = -0.026792504967750107
    # VMOTOR2THRUST3 =  0.006776789303971145
    # THRUST_MIN = 0.012817578393224994 N per motor

    # QuadSim canonical order: 0=FL, 1=FR, 2=BL, 3=BR
    # Unity body: +x forward, +y up, +z left
    # ARM_LENGTH = 0.046 m, so X/Z offsets are arm_length / sqrt(2)
    propellers:
      - {id: 0, x:  0.032527, y: 0.008, z:  0.032527, dir:  1}  # FL
      - {id: 1, x:  0.032527, y: 0.008, z: -0.032527, dir: -1}  # FR
      - {id: 2, x: -0.032527, y: 0.008, z:  0.032527, dir: -1}  # BL
      - {id: 3, x: -0.032527, y: 0.008, z: -0.032527, dir:  1}  # BR

  runtime:
    physics_rate_hz: 250
    controller_rate_hz: 250

  controller:
    position_gains:
      x: {p: 4.0, i: 0.0, d: 0.0}
      y: {p: 4.0, i: 0.0, d: 0.0}
      z: {p: 4.0, i: 0.0, d: 0.0}

    velocity_gains:
      x: {p: 3.0, i: 0.0, d: 0.0}
      y: {p: 3.0, i: 0.0, d: 0.0}
      z: {p: 3.0, i: 0.0, d: 0.0}

    attitude_gains:
      roll:  {p: 5.0, i: 0.0, d: 0.0}
      pitch: {p: 5.0, i: 0.0, d: 0.0}
      yaw:   {p: 5.0, i: 0.0, d: 0.0}

    rate_gains:
      roll:  {p: 0.025, i: 0.0, d: 0.0}
      pitch: {p: 0.025, i: 0.0, d: 0.0}
      yaw:   {p: 0.008, i: 0.0, d: 0.0}
  camera_parameters:
    Cam_Arm_Length: 0.05
    FOV: 100.0
    Cam_Pitch: -15.0
    Cam_Socket_Offset:
      x: -5.0
      y: 0.0
      z: 2.0

  limits:
    max_velocity_xy: 1.0
    max_velocity_z: 1.0
    max_tilt_angle_deg: 20.0
    max_angular_rate_RP_deg: 120.0
    max_angular_rate_Yaw_deg: 90.0

    # 0.12 N per motor * 4 motors
    max_thrust_newtons: 0.6

    max_pid_output: 1.0
    altitude_threshold: 0.1
    min_altitude_local: 0.05
    acceptable_distance: 0.2
```

```
  altitude_rate: 0.5
  yaw_rate: 90.0
  min_velocity_yaw: 0.1
```

Runtime/Resources/QuadSim/Configs/Drones/flightmare_quad.yaml

SHA-256 f740ea769500e4f904ab8cfbfefa5c511d2705633709129caa0a1374793a0e9f | 69 lines | 1,943 bytes

```
# Drone Configuration File
# Version: 1.0
schema_version: 1
name: "Standard_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/QuadRotor1/Quadrotor1.Quadrotor1"
physical_properties:
  mass_kg: 0.73
  com_offset: {x: 0.0, y: 0.0, z: 0.05}
  inertia: {x: 0.00791, y: 0.00791, z: 0.01231}
  arm_length: 0.17
  linear_drag_coef: 0.1
  angular_drag_coef: 0.05

propulsion:
  # FRotorModel fields
  prop_diameter: 0.127
  max_rpm: 28000
  max_thrust: 6.98
  thrust_coef: 0.101
  torque_coef: 0.016
  torque_coefficient_convention: legacy_2_5_pi
  # FRotorInstance array
  # FIX THESE
  propellers:
    - {id: 0, x: 0.11, y: 0.00, z: 0.12, dir: -1}
    - {id: 1, x: 0.11, y: 0.00, z: -0.14, dir: 1}
    - {id: 2, x: -0.11, y: 0.00, z: 0.12, dir: 1}
    - {id: 3, x: -0.11, y: 0.00, z: -0.14, dir: -1}

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250

controller:
  position_gains:
    x: {p: 1.0, i: 0.0, d: 0.5}
    y: {p: 1.0, i: 0.0, d: 0.5}
    z: {p: 1.0, i: 0.1, d: 1.0}

  velocity_gains:
    x: {p: 3.0, i: 0.01, d: 0.0001}
    y: {p: 3.0, i: 0.01, d: 0.0001}
    z: {p: 5.0, i: 0.0, d: 0.0}

  attitude_gains:
    roll:  {p: 6.0, i: 0.0, d: 0.0}
    pitch: {p: 6.0, i: 0.0, d: 0.0}
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.35, i: 0.02, d: 0.0001}
    pitch: {p: 0.35, i: 0.02, d: 0.0001}
    yaw:   {p: 1.0, i: 0.0, d: 0.0}

camera_parameters:
  Cam_Arm_Length: 0.9          # meters, orbit/follow distance
  FOV: 65.0                    # clamped 20..140
  Cam_Pitch: -0.0             # degrees, clamped -80..80
  Cam_Socket_Offset: {x: 0.0, y: 0.0, z: 20.0}   # NOTE: cm, and UE axis order

limits:
  max_velocity_xy: 8.0
  max_velocity_z: 10.0
  max_tilt_angle_deg: 45.0
  max_angular_rate_RP_deg: 45.0
  max_angular_rate_Yaw_deg: 90.0
```

```
  max_thrust_newtons: 7
  max_pid_output: 700.0
  acceptable_distance: 5
```

Runtime/Resources/QuadSim/Configs/Drones/sentinel_quad.yaml

SHA-256 80a36d46c9aeaf3d50af64d94658c3160286cc2a69134b5cd5cb5e26217639da | 102 lines | 3,154 bytes

```
schema_version: 1
name: "ModalAI_Sentinel_RB5_S500"

body_mesh: "/QuadSimPlugin/Assets/Drone_Models/Sentinel/Sentinel.Sentinel"

physical_properties:
  # Gazebo sentinel_base: 1.347 kg body + four 0.016076923 kg rotor links.
  mass_kg: 1.4113076923

  # QuadSim is one rigid body centered at the composed Gazebo center of mass.
  com_offset: {x: 0.0, y: 0.0, z: 0.0}

  # Gazebo composed inertia about that center is FLU
  # (0.02383614, 0.02393907, 0.04399995). Unity axes are X-forward,
  # Y-up, Z-left, hence the Y/Z permutation here.
  inertia: {x: 0.02383614, y: 0.04399995, z: 0.02393907}
  arm_length: 0.24607316
  linear_drag_coef: 0.15
  angular_drag_coef: 0.08

propulsion:
  prop_diameter: 0.254

  # Gazebo maxRotVelocity = 1095 rad/s
  max_rpm: 10456.48

  # motorConstant * maxRotVelocity^2
  # 1.075e-5 * 1095^2 = 12.8895 N per motor
  max_thrust: 12.88952

  # Converted from Gazebo's T = motorConstant * omega^2
  # into QuadSim's T = Ct * rho * n^2 * D^4 convention.
  thrust_coef: 0.0832333

  # Previously validated QuadSim yaw-torque coefficient. Keep this empirical
  # plant value until a controlled PX4 yaw-rate sweep justifies changing it.
  torque_coef: 0.0303
  torque_coefficient_convention: legacy_2_5_pi

  # Gazebo MulticopterMotorModel asymmetric response constants. QuadSim applies
  # them to its normalized motor command; the remaining motor-law difference is
  # handled by plant-specific thrust calibration and rate-response comparison.
  motor_time_constant_up_s: 0.0125
  motor_time_constant_down_s: 0.025

  # QuadSim canonical order is FL, FR, BL, BR. Its physics applies dir as the
  # reactive body-yaw torque sign (-1=CCW, +1=CW), while Gazebo names the
  # rotor's own rotation. PX4 none_iris outputs are also remapped to this order
  # by px4_bridge.MOTOR_MAP, so Gazebo motor directions cannot be copied here.
  propellers:
    - {id: 0, x:  0.174, y: 0.0, z:  0.174, dir: -1}
    - {id: 1, x:  0.174, y: 0.0, z: -0.174, dir:  1}
    - {id: 2, x: -0.174, y: 0.0, z:  0.174, dir:  1}
    - {id: 3, x: -0.174, y: 0.0, z: -0.174, dir: -1}

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250

controller:

  position_gains:
    x: {p: 0.95, i: 0.0, d: 0.0}
    y: {p: 0.95, i: 0.0, d: 0.0}
    z: {p: 1.0, i: 0.0, d: 0.0}
```

```
  velocity_gains:
    x: {p: 3.0, i: 0.1, d: 0.0}
    y: {p: 3.0, i: 0.1, d: 0.0}
    z: {p: 8.0, i: 2.0, d: 0.0}

  attitude_gains:
    roll:  {p: 5.5, i: 0.0, d: 0.0}
    pitch: {p: 5.5, i: 0.0, d: 0.0}
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.08, i: 0.4, d: 0.0}
    pitch: {p: 0.08, i: 0.4, d: 0.0}
    yaw:   {p: 0.13, i: 0.3, d: 0.0}

camera_parameters:
  Cam_Arm_Length: 2.5
  FOV: 90
  Cam_Pitch: -15.0
  Cam_Socket_Offset:
    x: -5.0
    y: 0.0
    z: 2.0

limits:
  max_velocity_xy: 3.0
  max_velocity_z: 1.12
  max_tilt_angle_deg: 30.0
  max_angular_rate_RP_deg: 150.0
  max_angular_rate_Yaw_deg: 150.0

  # Per-motor maximum, matching propulsion.max_thrust
  max_thrust_newtons: 12.88952

  max_pid_output: 700.0
  acceptable_distance: 5
```

Runtime/Resources/QuadSim/Configs/Drones/standard_quad.yaml

SHA-256 02a302d4ca9ac4514684520b4e7fd50058730e57ceb8a0176ad55759b29f5316 | 72 lines | 1,956 bytes

```
# Drone Configuration File
# Version: 1.0
schema_version: 1
name: "Standard_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/QuadRotor1/Quadrotor1.Quadrotor1"

physical_properties:
  mass_kg: 1.28
  com_offset: {x: 0.0, y: 0.05, z: 0.0} # Leave empty if not known
  inertia: {x: 0.02383614, y: 0.04399995, z: 0.02393907}
  arm_length: 0.1789 # 0.127  # Missing - FDroneParameters::Arm_Length
  linear_drag_coef: 0.1
  angular_drag_coef: 0.05

propulsion:
  # FRotorModel fields
  prop_diameter: 0.127 #0.2286
  max_rpm: 28000 #6396.667 #28000
  max_thrust: 4.179446268 #8
  thrust_coef: 0.11 #0.109919
  torque_coef: 0.040164
  torque_coefficient_convention: legacy_2_5_pi
  # FRotorInstance array
  propellers:
    - {id: 0, x: 0.211, y: 0.0117, z: 0.211, dir: -1} # FL
    - {id: 1, x: 0.211, y: 0.0117, z: -0.211, dir: 1} # FR
    - {id: 2, x: -0.211, y: 0.0117, z: 0.211, dir: 1} # BL
    - {id: 3, x: -0.211, y: 0.0117, z: -0.211, dir: -1} # BR

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250
```

```
controller:
  position_gains:
    x: {p: 1.0, i: 0.0, d: 0.5}
    y: {p: 1.0, i: 0.0, d: 0.5}
    z: {p: 1.0, i: 0.1, d: 1.0}

  velocity_gains:
    x: {p: 3.0, i: 0.01, d: 0.0001}
    y: {p: 3.0, i: 0.01, d: 0.0001}
    z: {p: 5.0, i: 0.0, d: 0.0}

  attitude_gains:
    roll:  {p: 6.0, i: 0.0, d: 0.0}
    pitch: {p: 6.0, i: 0.0, d: 0.0}
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.35, i: 0.02, d: 0.0001}
    pitch: {p: 0.35, i: 0.02, d: 0.0001}
    yaw:   {p: 1.0, i: 0.0, d: 0.0}

camera_parameters:
  Cam_Arm_Length: 1.5
  FOV: 90
  Cam_Pitch: -15.0
  Cam_Socket_Offset:
    x: -5.0
    y: 0.0
    z: 2.0

limits:
  max_velocity_xy: 8.0
  max_velocity_z: 10.0
  max_tilt_angle_deg: 45.0
  max_angular_rate_RP_deg: 45.0
  max_angular_rate_Yaw_deg: 90.0
  max_thrust_newtons: 7
  max_pid_output: 700.0
  acceptable_distance: 5
```

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/crazyflie2_quad.yaml

SHA-256 f9552e20da18a77878798b3dcc83fccdde39a6c9846a248020d109eab32bc527 | 92 lines | 2,558 bytes

```
# Drone Configuration File - Crazyflie 2.1
schema_version: 1
name: "Crazyflie_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/CrazyFlie/cf2_assembly.cf2_assembly"

physical_properties:
  mass_kg: 0.029
  com_offset: {x: 0.0, y: 0.0, z: 0.0}
  inertia: {x: 0.00001657, y: 0.00001666, z: 0.00002926}
  arm_length: 0.046
  linear_drag_coef: 0.02
  angular_drag_coef: 0.001

propulsion:
  prop_diameter: 0.045
  max_rpm: 24000

  # Bitcraze CF2.1+ default:
  # THRUST_MAX = 0.12 N per motor
  # THRUST2TORQUE = 0.0069928948992470565 m
  max_thrust: 0.147
  thrust_coef: 0.183
  torque_coef: 0.0069928948992470565
  torque_coefficient_convention: legacy_2_5_pi

  # Firmware thrust curve, not directly represented by current QuadSim schema:
  # VMOTOR2THRUST0 = -0.02476537915958403
  # VMOTOR2THRUST1 =  0.06523793527519485
  # VMOTOR2THRUST2 = -0.026792504967750107
  # VMOTOR2THRUST3 =  0.006776789303971145
```

```
    # THRUST_MIN = 0.012817578393224994 N per motor

    # QuadSim canonical order: 0=FL, 1=FR, 2=BL, 3=BR
    # Unity body: +x forward, +y up, +z left
    # ARM_LENGTH = 0.046 m, so X/Z offsets are arm_length / sqrt(2)
    propellers:
      - {id: 0, x:  0.032527, y: 0.008, z:  0.032527, dir:  1}  # FL
      - {id: 1, x:  0.032527, y: 0.008, z: -0.032527, dir: -1}  # FR
      - {id: 2, x: -0.032527, y: 0.008, z:  0.032527, dir: -1}  # BL
      - {id: 3, x: -0.032527, y: 0.008, z: -0.032527, dir:  1}  # BR

  runtime:
    physics_rate_hz: 250
    controller_rate_hz: 250

  controller:
    position_gains:
      x: {p: 4.0, i: 0.0, d: 0.0}
      y: {p: 4.0, i: 0.0, d: 0.0}
      z: {p: 4.0, i: 0.0, d: 0.0}

    velocity_gains:
      x: {p: 3.0, i: 0.0, d: 0.0}
      y: {p: 3.0, i: 0.0, d: 0.0}
      z: {p: 3.0, i: 0.0, d: 0.0}

    attitude_gains:
      roll:  {p: 5.0, i: 0.0, d: 0.0}
      pitch: {p: 5.0, i: 0.0, d: 0.0}
      yaw:   {p: 5.0, i: 0.0, d: 0.0}

    rate_gains:
      roll:  {p: 0.025, i: 0.0, d: 0.0}
      pitch: {p: 0.025, i: 0.0, d: 0.0}
      yaw:   {p: 0.008, i: 0.0, d: 0.0}
  camera_parameters:
    Cam_Arm_Length: 0.05
    FOV: 100.0
    Cam_Pitch: -15.0
    Cam_Socket_Offset:
      x: -5.0
      y: 0.0
      z: 2.0

  limits:
    max_velocity_xy: 1.0
    max_velocity_z: 1.0
    max_tilt_angle_deg: 20.0
    max_angular_rate_RP_deg: 120.0
    max_angular_rate_Yaw_deg: 90.0

    # 0.12 N per motor * 4 motors
    max_thrust_newtons: 0.6

    max_pid_output: 1.0
    altitude_threshold: 0.1
    min_altitude_local: 0.05
    acceptable_distance: 0.2

    altitude_rate: 0.5
    yaw_rate: 90.0
    min_velocity_yaw: 0.1
```

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/flightmare_quad.yaml

SHA-256 f740ea769500e4f904ab8cfbfefa5c511d2705633709129caa0a1374793a0e9f | 69 lines | 1,943 bytes

```
# Drone Configuration File
# Version: 1.0
schema_version: 1
name: "Standard_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/QuadRotor1/Quadrotor1.Quadrotor1"
physical_properties:
  mass_kg: 0.73
```

```
  com_offset: {x: 0.0, y: 0.0, z: 0.05}
  inertia: {x: 0.00791, y: 0.00791, z: 0.01231}
  arm_length: 0.17
  linear_drag_coef: 0.1
  angular_drag_coef: 0.05

propulsion:
  # FRotorModel fields
  prop_diameter: 0.127
  max_rpm: 28000
  max_thrust: 6.98
  thrust_coef: 0.101
  torque_coef: 0.016
  torque_coefficient_convention: legacy_2_5_pi
  # FRotorInstance array
  # FIX THESE
  propellers:
    - {id: 0, x: 0.11, y: 0.00, z: 0.12, dir: -1}
    - {id: 1, x: 0.11, y: 0.00, z: -0.14, dir: 1}
    - {id: 2, x: -0.11, y: 0.00, z: 0.12, dir: 1}
    - {id: 3, x: -0.11, y: 0.00, z: -0.14, dir: -1}

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250

controller:
  position_gains:
    x: {p: 1.0, i: 0.0, d: 0.5}
    y: {p: 1.0, i: 0.0, d: 0.5}
    z: {p: 1.0, i: 0.1, d: 1.0}

  velocity_gains:
    x: {p: 3.0, i: 0.01, d: 0.0001}
    y: {p: 3.0, i: 0.01, d: 0.0001}
    z: {p: 5.0, i: 0.0, d: 0.0}

  attitude_gains:
    roll:  {p: 6.0, i: 0.0, d: 0.0}
    pitch: {p: 6.0, i: 0.0, d: 0.0}
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.35, i: 0.02, d: 0.0001}
    pitch: {p: 0.35, i: 0.02, d: 0.0001}
    yaw:   {p: 1.0, i: 0.0, d: 0.0}

camera_parameters:
  Cam_Arm_Length: 0.9          # meters, orbit/follow distance
  FOV: 65.0                    # clamped 20..140
  Cam_Pitch: -0.0             # degrees, clamped -80..80
  Cam_Socket_Offset: {x: 0.0, y: 0.0, z: 20.0}   # NOTE: cm, and UE axis order

limits:
  max_velocity_xy: 8.0
  max_velocity_z: 10.0
  max_tilt_angle_deg: 45.0
  max_angular_rate_RP_deg: 45.0
  max_angular_rate_Yaw_deg: 90.0
  max_thrust_newtons: 7
  max_pid_output: 700.0
  acceptable_distance: 5
```

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/sentinel_quad.yaml

SHA-256 80a36d46c9aeaf3d50af64d94658c3160286cc2a69134b5cd5cb5e26217639da | 102 lines | 3,154 bytes

```
schema_version: 1
name: "ModalAI_Sentinel_RB5_S500"

body_mesh: "/QuadSimPlugin/Assets/Drone_Models/Sentinel/Sentinel.Sentinel"

physical_properties:
  # Gazebo sentinel_base: 1.347 kg body + four 0.016076923 kg rotor links.
```

```
  mass_kg: 1.4113076923

  # QuadSim is one rigid body centered at the composed Gazebo center of mass.
  com_offset: {x: 0.0, y: 0.0, z: 0.0}

  # Gazebo composed inertia about that center is FLU
  # (0.02383614, 0.02393907, 0.04399995). Unity axes are X-forward,
  # Y-up, Z-left, hence the Y/Z permutation here.
  inertia: {x: 0.02383614, y: 0.04399995, z: 0.02393907}
  arm_length: 0.24607316
  linear_drag_coef: 0.15
  angular_drag_coef: 0.08

propulsion:
  prop_diameter: 0.254

  # Gazebo maxRotVelocity = 1095 rad/s
  max_rpm: 10456.48

  # motorConstant * maxRotVelocity^2
  # 1.075e-5 * 1095^2 = 12.8895 N per motor
  max_thrust: 12.88952

  # Converted from Gazebo's T = motorConstant * omega^2
  # into QuadSim's T = Ct * rho * n^2 * D^4 convention.
  thrust_coef: 0.0832333

  # Previously validated QuadSim yaw-torque coefficient. Keep this empirical
  # plant value until a controlled PX4 yaw-rate sweep justifies changing it.
  torque_coef: 0.0303
  torque_coefficient_convention: legacy_2_5_pi

  # Gazebo MulticopterMotorModel asymmetric response constants. QuadSim applies
  # them to its normalized motor command; the remaining motor-law difference is
  # handled by plant-specific thrust calibration and rate-response comparison.
  motor_time_constant_up_s: 0.0125
  motor_time_constant_down_s: 0.025

  # QuadSim canonical order is FL, FR, BL, BR. Its physics applies dir as the
  # reactive body-yaw torque sign (-1=CCW, +1=CW), while Gazebo names the
  # rotor's own rotation. PX4 none_iris outputs are also remapped to this order
  # by px4_bridge.MOTOR_MAP, so Gazebo motor directions cannot be copied here.
  propellers:
    - {id: 0, x:  0.174, y: 0.0, z:  0.174, dir: -1}
    - {id: 1, x:  0.174, y: 0.0, z: -0.174, dir:  1}
    - {id: 2, x: -0.174, y: 0.0, z:  0.174, dir:  1}
    - {id: 3, x: -0.174, y: 0.0, z: -0.174, dir: -1}

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250

controller:

  position_gains:
    x: {p: 0.95, i: 0.0, d: 0.0}
    y: {p: 0.95, i: 0.0, d: 0.0}
    z: {p: 1.0, i: 0.0, d: 0.0}

  velocity_gains:
    x: {p: 3.0, i: 0.1, d: 0.0}
    y: {p: 3.0, i: 0.1, d: 0.0}
    z: {p: 8.0, i: 2.0, d: 0.0}

  attitude_gains:
    roll:  {p: 5.5, i: 0.0, d: 0.0}
    pitch: {p: 5.5, i: 0.0, d: 0.0}
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.08, i: 0.4, d: 0.0}
    pitch: {p: 0.08, i: 0.4, d: 0.0}
    yaw:   {p: 0.13, i: 0.3, d: 0.0}
```

```
camera_parameters:
  Cam_Arm_Length: 2.5
  FOV: 90
  Cam_Pitch: -15.0
  Cam_Socket_Offset:
    x: -5.0
    y: 0.0
    z: 2.0

limits:
  max_velocity_xy: 3.0
  max_velocity_z: 1.12
  max_tilt_angle_deg: 30.0
  max_angular_rate_RP_deg: 150.0
  max_angular_rate_Yaw_deg: 150.0

  # Per-motor maximum, matching propulsion.max_thrust
  max_thrust_newtons: 12.88952

  max_pid_output: 700.0
  acceptable_distance: 5
```

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/standard_quad.yaml

SHA-256 02a302d4ca9ac4514684520b4e7fd50058730e57ceb8a0176ad55759b29f5316 | 72 lines | 1,956 bytes

```
# Drone Configuration File
# Version: 1.0
schema_version: 1
name: "Standard_Quad_V1"
body_mesh: "/QuadSimPlugin/Assets/Drone_Models/QuadRotor1/Quadrotor1.Quadrotor1"

physical_properties:
  mass_kg: 1.28
  com_offset: {x: 0.0, y: 0.05, z: 0.0} # Leave empty if not known
  inertia: {x: 0.02383614, y: 0.04399995, z: 0.02393907}
  arm_length: 0.1789 # 0.127  # Missing - FDroneParameters::Arm_Length
  linear_drag_coef: 0.1
  angular_drag_coef: 0.05

propulsion:
  # FRotorModel fields
  prop_diameter: 0.127 #0.2286
  max_rpm: 28000 #6396.667 #28000
  max_thrust: 4.179446268 #8
  thrust_coef: 0.11 #0.109919
  torque_coef: 0.040164
  torque_coefficient_convention: legacy_2_5_pi
  # FRotorInstance array
  propellers:
    - {id: 0, x: 0.211, y: 0.0117, z: 0.211, dir: -1} # FL
    - {id: 1, x: 0.211, y: 0.0117, z: -0.211, dir: 1} # FR
    - {id: 2, x: -0.211, y: 0.0117, z: 0.211, dir: 1} # BL
    - {id: 3, x: -0.211, y: 0.0117, z: -0.211, dir: -1} # BR

runtime:
  physics_rate_hz: 250
  controller_rate_hz: 250

controller:
  position_gains:
    x: {p: 1.0, i: 0.0, d: 0.5}
    y: {p: 1.0, i: 0.0, d: 0.5}
    z: {p: 1.0, i: 0.1, d: 1.0}

  velocity_gains:
    x: {p: 3.0, i: 0.01, d: 0.0001}
    y: {p: 3.0, i: 0.01, d: 0.0001}
    z: {p: 5.0, i: 0.0, d: 0.0}

  attitude_gains:
    roll:  {p: 6.0, i: 0.0, d: 0.0}
    pitch: {p: 6.0, i: 0.0, d: 0.0}
```

```
    yaw:   {p: 2.5, i: 0.0, d: 0.0}

  rate_gains:
    roll:  {p: 0.35, i: 0.02, d: 0.0001}
    pitch: {p: 0.35, i: 0.02, d: 0.0001}
    yaw:   {p: 1.0, i: 0.0, d: 0.0}

camera_parameters:
  Cam_Arm_Length: 1.5
  FOV: 90
  Cam_Pitch: -15.0
  Cam_Socket_Offset:
    x: -5.0
    y: 0.0
    z: 2.0

limits:
  max_velocity_xy: 8.0
  max_velocity_z: 10.0
  max_tilt_angle_deg: 45.0
  max_angular_rate_RP_deg: 45.0
  max_angular_rate_Yaw_deg: 90.0
  max_thrust_newtons: 7
  max_pid_output: 700.0
  acceptable_distance: 5
```

11. Camera and Developer Visualization

Controls free, orbit, follow, and first-person camera views; provides image and depth capture for external clients; and supplies runtime and editor visualization of drone axes and configured rotor geometry.

Files in this section

Runtime/Scripts/DroneCore/Camera/CameraDirector.cs
Runtime/Scripts/DroneCore/Camera/DroneCameraRig.cs
Runtime/Scripts/DroneCore/Camera/DroneImageSensor.cs
Runtime/Scripts/DroneCore/Debug/DroneDebugDraw.cs
Runtime/Scripts/DroneCore/Debug/RotorGeometryGizmo.cs

Runtime/Scripts/DroneCore/Camera/CameraDirector.cs

SHA-256 86124dfa24a17648162de65cff672f83f933b90945cb7b392c0daa2ad545b855 | 532 lines | 20,382 bytes

```
using DroneCore.Core;
using UnityEngine;
using Yaml.Drone;

namespace DroneCore.CameraSystem
{
    public enum CameraViewMode
    {
        Free = 0,
        Orbit = 1,
        Follow = 2,
        FPV = 3,
    }

    [DisallowMultipleComponent]
    public sealed class CameraDirector : MonoBehaviour
    {
        [Header("View")]
        [SerializeField] private Camera viewCamera;
        [SerializeField] private CameraViewMode startupMode = CameraViewMode.Follow;

        [Header("Follow")]
        [SerializeField] private float followDistance = 0.9f;
        [SerializeField] private float followPitchDeg = -15f;
        [SerializeField] private float lookHeight = 0.15f;

        [Header("Orbit Damping")]
        [Tooltip("Orbit camera position response. Follow and FPV track exactly to avoid visual lag behin
d the Rigidbody.")]
        [SerializeField] private float positionSharpness = 10f;
        [Tooltip("Orbit camera rotation response. Follow and FPV track exactly to avoid visual lag behin
d the Rigidbody.")]
        [SerializeField] private float rotationSharpness = 14f;

        [Header("Orbit / Free")]
        [SerializeField] private float orbitYawDeg;
        [SerializeField] private float orbitPitchDeg = 20f;
        [SerializeField] private float orbitMouseSensitivity = 0.2f;
        [SerializeField] private float zoomWheelSensitivity = 2f;
        [SerializeField] private float freeLookSensitivity = 0.15f;
        [SerializeField] private float freeMoveSpeed = 10f;
        [SerializeField] private float freeBoostMultiplier = 3f;
        [SerializeField] private float freeWheelDollyDistance = 3f;

        private static CameraDirector _instance;
        private static RuntimeViewState _savedState;
        private DroneCameraRig _targetRig;
        private QuadPawn _cameraSourceDrone;
        private CameraViewMode _mode;
        private Vector3 _socketOffset;
        private Vector3 _pendingFreeMovement;
```

```
private Vector2 _pendingFreeLook;
private float _pendingFreeDolly;
private float _freeYawDeg;
private float _freePitchDeg;
private QuadPawn _freeFollowTarget;
private Vector3 _lastFreeFollowPosition;
private float _fieldOfView = 60f;
private bool _cameraSettingsInitialized;
private bool _restoreFreePose;

private struct RuntimeViewState
{
    public bool Valid;
    public CameraViewMode Mode;
    public float FollowDistance;
    public float FollowPitchDeg;
    public float OrbitYawDeg;
    public float OrbitPitchDeg;
    public float FieldOfView;
    public Vector3 SocketOffset;
    public bool HasFreePose;
    public Vector3 FreePosition;
    public Quaternion FreeRotation;
}

public Camera ViewCamera => viewCamera;
public CameraViewMode Mode => _mode;
public QuadPawn TargetDrone => _targetRig != null ? _targetRig.Owner : null;
public QuadPawn CameraSourceDrone => _cameraSourceDrone;
public QuadPawn FreeFollowTarget => _freeFollowTarget;
public float FollowDistance
{
    get => followDistance;
    set
    {
        followDistance = Mathf.Clamp(value, 0.25f, 100f);
        SaveRuntimeState();
    }
}
public float OrbitYawDeg
{
    get => orbitYawDeg;
    set
    {
        orbitYawDeg = Mathf.Repeat(value + 180f, 360f) - 180f;
        SaveRuntimeState();
    }
}
public float OrbitPitchDeg
{
    get => orbitPitchDeg;
    set
    {
        orbitPitchDeg = Mathf.Clamp(value, -80f, 80f);
        SaveRuntimeState();
    }
}
public float FieldOfView
{
    get => _fieldOfView;
    set
    {
        _fieldOfView = Mathf.Clamp(value, 20f, 140f);
        if (viewCamera != null) viewCamera.fieldOfView = _fieldOfView;
        SaveRuntimeState();
    }
}

[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.SubsystemRegistration)]
private static void ClearRuntimeState()
{
    _instance = null;
    _savedState = default;
```

```
}

private void Awake()
{
    _instance = this;
    if (_savedState.Valid)
    {
        _mode = _savedState.Mode;
        followDistance = _savedState.FollowDistance;
        followPitchDeg = _savedState.FollowPitchDeg;
        orbitYawDeg = _savedState.OrbitYawDeg;
        orbitPitchDeg = _savedState.OrbitPitchDeg;
        _fieldOfView = _savedState.FieldOfView;
        _socketOffset = _savedState.SocketOffset;
        _cameraSettingsInitialized = true;
        _restoreFreePose = _savedState.Mode == CameraViewMode.Free && _savedState.HasFreePose;
    }
    else
    {
        _mode = startupMode;
    }
}

private void OnDestroy()
{
    SaveRuntimeState();
    if (_instance == this)
        _instance = null;
}

public static CameraDirector Get()
{
    if (_instance != null) return _instance;
    _instance = FindAnyObjectByType<CameraDirector>();
    return _instance;
}

public static CameraDirector GetOrCreate(GameObject host)
{
    CameraDirector director = Get();
    if (director != null) return director;
    return host != null ? host.AddComponent<CameraDirector>() : null;
}

public void EnsureBoundToFirstDrone(DroneManager manager)
{
    if (manager == null || manager.DroneCount == 0)
    {
        if (viewCamera == null)
            AdoptCamera(Camera.main);
        return;
    }

    QuadPawn first = FindFirstSceneDrone(manager);
    if (first == null) return;

    if (viewCamera == null)
    {
        _cameraSourceDrone = first;
        AdoptCamera(first.DroneCamera != null ? first.DroneCamera : Camera.main);
    }

    if (_targetRig == null || TargetDrone == null)
        Bind(first, snap: true);

    DisableOtherDroneCameras(manager);
}

public void Bind(QuadPawn drone, bool snap = true)
{
    if (drone == null) return;

    if (viewCamera == null)
```

```
    {
        _cameraSourceDrone = drone;
        AdoptCamera(drone.DroneCamera != null ? drone.DroneCamera : Camera.main);
    }

    _targetRig = drone.CameraRig;
    if (drone.HasConfigApplied)
        ApplyCameraSettings(drone.Config.CamParams);

    if (snap)
        UpdateCameraPose(1f);
}

public void SetMode(CameraViewMode mode, bool snap = true)
{
    if (_mode == CameraViewMode.Free && mode != CameraViewMode.Free)
        _freeFollowTarget = null;

    if (mode == CameraViewMode.Free && _mode != CameraViewMode.Free)
        InitializeFreeView();

    _mode = mode;
    SaveRuntimeState();
    if (snap && mode != CameraViewMode.Free)
        UpdateCameraPose(1f);
}

public void ToggleFollowFpv()
{
    SetMode(_mode == CameraViewMode.FPV ? CameraViewMode.Follow : CameraViewMode.FPV);
}

public void AddLookInput(Vector2 delta)
{
    OrbitYawDeg += delta.x;
    OrbitPitchDeg += delta.y;
}

public void AddZoomInput(float delta)
{
    FollowDistance -= delta;
}

public void AddMouseCameraInput(Vector2 pixelDelta, float wheelDelta)
{
    if (_mode == CameraViewMode.Orbit && pixelDelta.sqrMagnitude > 0f)
        AddLookInput(new Vector2(pixelDelta.x, -pixelDelta.y) * orbitMouseSensitivity);

    if (_mode == CameraViewMode.Free && pixelDelta.sqrMagnitude > 0f)
        _pendingFreeLook += pixelDelta * freeLookSensitivity;

    if (!Mathf.Approximately(wheelDelta, 0f))
    {
        if (_mode == CameraViewMode.Follow || _mode == CameraViewMode.Orbit)
            AddZoomInput(wheelDelta * zoomWheelSensitivity);
        else if (_mode == CameraViewMode.Free)
            _pendingFreeDolly += wheelDelta * freeWheelDollyDistance;
        else if (_mode == CameraViewMode.FPV)
            FieldOfView -= wheelDelta * 5f;
    }
}

public void ResetOrbit()
{
    orbitYawDeg = 0f;
    orbitPitchDeg = 20f;
    SaveRuntimeState();
}

public void AddFreeMovementInput(Vector3 localDirection, bool boosted)
{
    _pendingFreeMovement += localDirection * (boosted ? freeBoostMultiplier : 1f);
}
```

```
public void ToggleFreeFollow(QuadPawn drone)
{
    if (_mode != CameraViewMode.Free || drone == null) return;

    if (_freeFollowTarget == drone)
    {
        _freeFollowTarget = null;
        return;
    }

    _freeFollowTarget = drone;
    _lastFreeFollowPosition = GetFollowPosition(drone);

    if (viewCamera != null)
    {
        Vector3 lookDirection = _lastFreeFollowPosition - viewCamera.transform.position;
        if (lookDirection.sqrMagnitude > 0.0001f)
        {
            viewCamera.transform.rotation = Quaternion.LookRotation(lookDirection, Vector3.up);
            InitializeFreeView();
        }
    }
}

public void ApplyCameraSettings(CamParameters parameters)
{
    Vector3 ueOffsetCm = parameters.CamSocket_Offset;
    _socketOffset = new Vector3(ueOffsetCm.x, ueOffsetCm.z, -ueOffsetCm.y) * 0.01f;

    if (!_cameraSettingsInitialized)
    {
        if (parameters.CamArm_Length > 0f)
            followDistance = parameters.CamArm_Length;
        if (parameters.FOV > 0f)
            _fieldOfView = Mathf.Clamp(parameters.FOV, 20f, 140f);

        followPitchDeg = Mathf.Clamp(parameters.CamPitch, -80f, 80f);
        _cameraSettingsInitialized = true;
    }

    if (viewCamera != null) viewCamera.fieldOfView = _fieldOfView;
    SaveRuntimeState();
}

private void LateUpdate()
{
    if (Application.isBatchMode)
    {
        if (viewCamera != null) viewCamera.enabled = false;
        return;
    }

    UpdateCameraPose(Time.unscaledDeltaTime);
}

private void UpdateCameraPose(float deltaTime)
{
    if (viewCamera == null) return;

    if (_mode == CameraViewMode.Free)
    {
        if (_freeFollowTarget != null)
        {
            Vector3 followPosition = GetFollowPosition(_freeFollowTarget);
            viewCamera.transform.position += followPosition - _lastFreeFollowPosition;
            _lastFreeFollowPosition = followPosition;
        }

        if (_pendingFreeLook.sqrMagnitude > 0f)
        {
            _freeYawDeg += _pendingFreeLook.x;
            _freePitchDeg = Mathf.Clamp(_freePitchDeg + _pendingFreeLook.y, -89f, 89f);
```

```
            viewCamera.transform.rotation = Quaternion.Euler(_freePitchDeg, _freeYawDeg, 0f);
        }

        if (_pendingFreeMovement.sqrMagnitude > 0f)
        {
            Vector3 input = Vector3.ClampMagnitude(_pendingFreeMovement, freeBoostMultiplier);
            Vector3 worldMovement = viewCamera.transform.right * input.x
                                  + Vector3.up * input.y
                                  + viewCamera.transform.forward * input.z;
            viewCamera.transform.position += worldMovement * freeMoveSpeed * deltaTime;
        }

        if (!Mathf.Approximately(_pendingFreeDolly, 0f))
            viewCamera.transform.position += viewCamera.transform.forward * _pendingFreeDolly;

        _pendingFreeMovement = Vector3.zero;
        _pendingFreeLook = Vector2.zero;
        _pendingFreeDolly = 0f;
        return;
    }

    if (_targetRig == null) return;

    Transform target = _targetRig.FollowTarget;
    Vector3 desiredPosition;
    Quaternion desiredRotation;

    if (_mode == CameraViewMode.FPV)
    {
        Transform mount = _targetRig.FpvMount;
        desiredPosition = mount.position;
        desiredRotation = mount.rotation;
    }
    else
    {
        // The chase/orbit rig follows heading, not the aircraft attitude.
        // In this project the drone's nose is its local +X (target.right).
        // Flattening it keeps pitch and roll out of the camera position,
        // while FPV above still inherits the real mount rotation.
        Vector3 nose = Vector3.ProjectOnPlane(target.right, Vector3.up);
        if (nose.sqrMagnitude < 0.0001f)
            nose = Vector3.right;
        else
            nose.Normalize();

        float targetYaw = nose.sqrMagnitude > 0f
            ? Mathf.Atan2(nose.x, nose.z) * Mathf.Rad2Deg
            : 0f;
        Quaternion heading = Quaternion.Euler(0f, targetYaw - 90f, 0f);
        Vector3 focus = target.position + heading * _socketOffset + Vector3.up * lookHeight;

        if (_mode == CameraViewMode.Orbit)
        {
            Quaternion orbit = Quaternion.Euler(orbitPitchDeg, targetYaw + orbitYawDeg, 0f);
            desiredPosition = focus + orbit * (Vector3.back * followDistance);
            desiredRotation = Quaternion.LookRotation(focus - desiredPosition, Vector3.up);
        }
        else
        {
            float height = Mathf.Tan(-followPitchDeg * Mathf.Deg2Rad) * followDistance;
            desiredPosition = focus - nose * followDistance + Vector3.up * height;
            desiredRotation = Quaternion.LookRotation(focus - desiredPosition, Vector3.up);
        }
    }

    // Follow and FPV must use the same rendered target pose as the drone.
    // Damping these modes adds a second delayed pose after Rigidbody
    // interpolation, making the aircraft visibly hitch ahead of its camera.
    // Orbit remains damped because its camera movement is user-directed and
    // does not need to keep the aircraft rigidly fixed in the frame.
    if (_mode != CameraViewMode.Orbit || deltaTime >= 1f)
    {
        viewCamera.transform.SetPositionAndRotation(desiredPosition, desiredRotation);
```

```
                return;
            }

            float positionT = 1f - Mathf.Exp(-positionSharpness * deltaTime);
            float rotationT = 1f - Mathf.Exp(-rotationSharpness * deltaTime);
            viewCamera.transform.position = Vector3.Lerp(viewCamera.transform.position, desiredPosition,
 positionT);
            viewCamera.transform.rotation = Quaternion.Slerp(viewCamera.transform.rotation, desiredRotat
ion, rotationT);
        }

        private void InitializeFreeView()
        {
            if (viewCamera == null) return;

            Vector3 forward = viewCamera.transform.forward;
            _freeYawDeg = Mathf.Atan2(forward.x, forward.z) * Mathf.Rad2Deg;
            _freePitchDeg = -Mathf.Asin(Mathf.Clamp(forward.y, -1f, 1f)) * Mathf.Rad2Deg;
            _pendingFreeMovement = Vector3.zero;
            _pendingFreeLook = Vector2.zero;
            _pendingFreeDolly = 0f;
        }

        private static Vector3 GetFollowPosition(QuadPawn drone)
        {
            if (drone == null) return Vector3.zero;
            DroneCameraRig rig = drone.CameraRig;
            return rig != null && rig.FollowTarget != null
                ? rig.FollowTarget.position
                : drone.transform.position;
        }

        private void AdoptCamera(Camera candidate)
        {
            if (candidate == null) return;

            Camera previousMain = Camera.main;
            viewCamera = candidate;
            viewCamera.transform.SetParent(transform, true);
            viewCamera.transform.localScale = Vector3.one;
            if (_restoreFreePose)
            {
                viewCamera.transform.SetPositionAndRotation(
                    _savedState.FreePosition,
                    _savedState.FreeRotation);
                _restoreFreePose = false;
                InitializeFreeView();
            }
            viewCamera.fieldOfView = _fieldOfView;
            viewCamera.enabled = !Application.isBatchMode;

            AudioListener listener = viewCamera.GetComponent<AudioListener>();
            if (listener != null)
                listener.enabled = !Application.isBatchMode;

            if (previousMain != null && previousMain != viewCamera)
            {
                previousMain.enabled = false;
                AudioListener previousListener = previousMain.GetComponent<AudioListener>();
                if (previousListener != null) previousListener.enabled = false;
            }
        }

        private void DisableOtherDroneCameras(DroneManager manager)
        {
            for (int i = 0; i < manager.RegisteredDrones.Count; i++)
            {
                QuadPawn drone = manager.RegisteredDrones[i];
                if (drone == null) continue;

                Camera camera = drone.DroneCamera;
                if (camera == null || camera == viewCamera) continue;
```

```
                camera.enabled = false;
                AudioListener listener = camera.GetComponent<AudioListener>();
                if (listener != null) listener.enabled = false;
            }
        }

        private static QuadPawn FindFirstSceneDrone(DroneManager manager)
        {
            QuadPawn first = null;
            int firstRootIndex = int.MaxValue;

            for (int i = 0; i < manager.RegisteredDrones.Count; i++)
            {
                QuadPawn drone = manager.RegisteredDrones[i];
                if (drone == null) continue;

                int rootIndex = drone.transform.root.GetSiblingIndex();
                if (first == null || rootIndex < firstRootIndex)
                {
                    first = drone;
                    firstRootIndex = rootIndex;
                }
            }

            return first;
        }

        private void SaveRuntimeState()
        {
            bool hasFreePose = _mode == CameraViewMode.Free && viewCamera != null;
            _savedState = new RuntimeViewState
            {
                Valid = true,
                Mode = _mode,
                FollowDistance = followDistance,
                FollowPitchDeg = followPitchDeg,
                OrbitYawDeg = orbitYawDeg,
                OrbitPitchDeg = orbitPitchDeg,
                FieldOfView = _fieldOfView,
                SocketOffset = _socketOffset,
                HasFreePose = hasFreePose,
                FreePosition = hasFreePose ? viewCamera.transform.position : Vector3.zero,
                FreeRotation = hasFreePose ? viewCamera.transform.rotation : Quaternion.identity,
            };
        }
    }
}
```

Runtime/Scripts/DroneCore/Camera/DroneCameraRig.cs

SHA-256 3fc35e1b7e918b45db1610ee1e3ae5a5f27e70dd74200bac19e1d8162a0ed994 | 56 lines | 2,077 bytes

```
using UnityEngine;

namespace DroneCore.CameraSystem
{
    [DisallowMultipleComponent]
    public sealed class DroneCameraRig : MonoBehaviour
    {
        [SerializeField] private Transform fpvMount;
        [SerializeField] private Transform followTarget;
        [SerializeField] private Transform orbitTarget;
        [SerializeField] private Vector3 defaultFpvOffset = new Vector3(0.15f, 0.05f, 0f);

        private QuadPawn _owner;

        public QuadPawn Owner => _owner;
        public string OwnerAgentId => _owner != null ? _owner.AgentId : string.Empty;
        public Transform FpvMount => fpvMount != null ? fpvMount : transform;
        public Transform FollowTarget => followTarget != null ? followTarget : transform;
        public Transform OrbitTarget => orbitTarget != null ? orbitTarget : FollowTarget;

        private void Awake()
        {
```

```
            Initialize(GetComponent<QuadPawn>(), null);
        }

        public void Initialize(QuadPawn owner, Transform legacyFpvMount)
        {
            if (owner != null)
                _owner = owner;

            if (_owner == null)
                _owner = GetComponentInParent<QuadPawn>();

            Transform root = _owner != null ? _owner.transform : transform;
            if (followTarget == null) followTarget = root;
            if (orbitTarget == null) orbitTarget = followTarget;

            if (fpvMount == null && IsUsableMount(legacyFpvMount))
                fpvMount = legacyFpvMount;

            if (fpvMount == null)
            {
                var mount = new GameObject("FPVMount").transform;
                mount.SetParent(root, false);
                mount.localPosition = defaultFpvOffset;
                mount.localRotation = Quaternion.LookRotation(Vector3.right, Vector3.up);
                fpvMount = mount;
            }
        }

        private static bool IsUsableMount(Transform candidate)
        {
            return candidate != null && candidate.GetComponent<Camera>() == null;
        }
    }
}
```

Runtime/Scripts/DroneCore/Camera/DroneImageSensor.cs

SHA-256 4c3617e6c3fd0829fdde7ea05bd6b8c29e51c8252f4d0c9a6d3480ea15cad991 | 247 lines | 9,178 bytes

```
using System;
using System.Collections.Generic;
using RobotCore;
using UnityEngine;

namespace DroneCore.CameraSystem
{
    [DisallowMultipleComponent]
    public sealed class DroneImageSensor : MonoBehaviour
    {
        private readonly RaycastHit[] _depthHits = new RaycastHit[16];

        private QuadPawn _owner;
        private DroneCameraRig _rig;
        private Camera _camera;
        private RenderTexture _renderTexture;
        private Texture2D _readback;
        private int _sequence;

        public void ResetSequence() => _sequence = 0;

        public void Initialize(QuadPawn owner, DroneCameraRig rig)
        {
            _owner = owner;
            _rig = rig;
        }

        public Dictionary<string, object> Capture(
            string imageType,
            int width,
            int height,
            string encoding,
            int quality,
            float fieldOfViewDeg,
            float maxDepthM)
```

```
{
    EnsureCamera();
    _camera.fieldOfView = fieldOfViewDeg;
    _camera.farClipPlane = maxDepthM;

    byte[] data;
    string pixelFormat;
    string dtype;
    int channels;

    if (imageType == "depth")
    {
        data = CaptureDepth(width, height, fieldOfViewDeg, maxDepthM);
        pixelFormat = "depth_m";
        dtype = "float32";
        channels = 1;
    }
    else
    {
        CaptureColor(width, height);
        if (imageType == "gray")
        {
            byte[] gray = ToGray(_readback.GetPixels32());
            if (encoding == "raw")
            {
                data = gray;
                pixelFormat = "gray8";
                channels = 1;
            }
            else
            {
                Color32[] pixels = new Color32[gray.Length];
                for (int i = 0; i < gray.Length; i++)
                    pixels[i] = new Color32(gray[i], gray[i], gray[i], 255);
                _readback.SetPixels32(pixels);
                _readback.Apply(false, false);
                data = Encode(_readback, encoding, quality);
                pixelFormat = "rgb8_gray";
                channels = 3;
            }
        }
        else
        {
            data = encoding == "raw"
                ? ToRgb(_readback.GetPixels32())
                : Encode(_readback, encoding, quality);
            pixelFormat = "rgb8";
            channels = 3;
        }
        dtype = "uint8";
    }

    _sequence++;
    SensorData metadata = _owner != null && _owner.Sensors != null
        ? _owner.Sensors.Latest
        : default;
    return new Dictionary<string, object>
    {
        ["agent_id"] = _owner != null ? _owner.AgentId : string.Empty,
        ["image_type"] = imageType,
        ["width"] = width,
        ["height"] = height,
        ["channels"] = channels,
        ["dtype"] = dtype,
        ["pixel_format"] = pixelFormat,
        ["encoding"] = encoding,
        ["origin"] = "bottom_left",
        ["field_of_view_deg"] = fieldOfViewDeg,
        ["max_depth_m"] = maxDepthM,
        ["episode_id"] = metadata.EpisodeId,
        ["step_index"] = metadata.StepIndex,
        ["sim_time_ns"] = metadata.SimTimeNanos,
        ["sim_time"] = metadata.SimTimeNanos * 1e-9,
```

```
                ["source_frame"] = "camera_optical_unity",
                ["sequence"] = _sequence,
                ["data"] = data,
            };
        }

        private void EnsureCamera()
        {
            if (_camera != null) return;
            if (_rig == null) _rig = GetComponent<DroneCameraRig>();

            var cameraObject = new GameObject("TrainingCamera");
            Transform parent = _rig != null ? _rig.FpvMount : transform;
            cameraObject.transform.SetParent(parent, false);
            _camera = cameraObject.AddComponent<Camera>();
            _camera.enabled = false;
            _camera.nearClipPlane = 0.05f;
            _camera.clearFlags = CameraClearFlags.Skybox;
            _camera.allowHDR = false;
            _camera.allowMSAA = false;
        }

        private void CaptureColor(int width, int height)
        {
            EnsureColorBuffers(width, height);
            RenderTexture previous = RenderTexture.active;
            RenderTexture previousTarget = _camera.targetTexture;
            _camera.targetTexture = _renderTexture;
            _camera.Render();
            RenderTexture.active = _renderTexture;
            _readback.ReadPixels(new Rect(0, 0, width, height), 0, 0, false);
            _readback.Apply(false, false);
            _camera.targetTexture = previousTarget;
            RenderTexture.active = previous;
        }

        private byte[] CaptureDepth(int width, int height, float verticalFovDeg, float maxDepthM)
        {
            float[] depth = new float[width * height];
            float verticalTan = Mathf.Tan(verticalFovDeg * Mathf.Deg2Rad * 0.5f);
            float aspect = width / (float)height;
            Transform cameraTransform = _camera.transform;
            Vector3 origin = cameraTransform.position;

            for (int y = 0; y < height; y++)
            {
                float v = ((y + 0.5f) / height * 2f - 1f) * verticalTan;
                for (int x = 0; x < width; x++)
                {
                    float u = ((x + 0.5f) / width * 2f - 1f) * verticalTan * aspect;
                    Vector3 direction = cameraTransform.TransformDirection(new Vector3(u, v, 1f).normali
zed);
                    int hitCount = Physics.RaycastNonAlloc(
                        origin, direction, _depthHits, maxDepthM, ~0, QueryTriggerInteraction.Ignore);

                    float nearest = maxDepthM;
                    for (int i = 0; i < hitCount; i++)
                    {
                        Collider collider = _depthHits[i].collider;
                        if (collider == null || collider.GetComponentInParent<QuadPawn>() == _owner)
                            continue;
                        if (_depthHits[i].distance < nearest)
                            nearest = _depthHits[i].distance;
                    }
                    depth[y * width + x] = nearest;
                }
            }

            byte[] bytes = new byte[depth.Length * sizeof(float)];
            Buffer.BlockCopy(depth, 0, bytes, 0, bytes.Length);
            return bytes;
        }
```

```
        private void EnsureColorBuffers(int width, int height)
        {
            if (_renderTexture != null
                && _renderTexture.width == width
                && _renderTexture.height == height)
                return;

            ReleaseColorBuffers();
            _renderTexture = new RenderTexture(width, height, 24, RenderTextureFormat.ARGB32)
            {
                name = $"{name}_TrainingCapture",
                antiAliasing = 1,
            };
            _renderTexture.Create();
            _readback = new Texture2D(width, height, TextureFormat.RGB24, false, false);
        }

        private static byte[] ToRgb(Color32[] pixels)
        {
            byte[] rgb = new byte[pixels.Length * 3];
            for (int i = 0; i < pixels.Length; i++)
            {
                int offset = i * 3;
                rgb[offset] = pixels[i].r;
                rgb[offset + 1] = pixels[i].g;
                rgb[offset + 2] = pixels[i].b;
            }
            return rgb;
        }

        private static byte[] ToGray(Color32[] pixels)
        {
            byte[] gray = new byte[pixels.Length];
            for (int i = 0; i < pixels.Length; i++)
                gray[i] = (byte)((77 * pixels[i].r + 150 * pixels[i].g + 29 * pixels[i].b) >> 8);
            return gray;
        }

        private static byte[] Encode(Texture2D texture, string encoding, int quality)
        {
            return encoding == "jpeg"
                ? texture.EncodeToJPG(quality)
                : texture.EncodeToPNG();
        }

        private void OnDestroy()
        {
            ReleaseColorBuffers();
            if (_camera != null) Destroy(_camera.gameObject);
        }

        private void ReleaseColorBuffers()
        {
            if (_renderTexture != null)
            {
                _renderTexture.Release();
                Destroy(_renderTexture);
                _renderTexture = null;
            }
            if (_readback != null)
            {
                Destroy(_readback);
                _readback = null;
            }
        }
    }
}
```

Runtime/Scripts/DroneCore/Debug/DroneDebugDraw.cs

SHA-256 c728577f0f143beb044c66199db3c74d987f781212e090dca8dc56a1f0a20bb1 | 91 lines | 3,605 bytes

```
using MathUtil;
using UnityEngine;
```

```
namespace DroneCore
{
    public sealed class DroneDebugDraw : MonoBehaviour
    {
        [SerializeField] private QuadPawn body;

        [Header("Draw")]
        public bool drawUnityBodyAxes = true;
        public bool drawSelectedFrameAxes = true;
        [Header("Frame Source")]
        [Tooltip("ON: highlighted triad follows the live SensorManager.outputFrame. OFF: uses frameToDra
w below (inspect-only, doesn't change output).")]
        public bool followOutputFrame = true;
        public SimFrame frameToDraw = SimFrame.FLU;

        public bool drawMotorAxes = false;
        public float axisLength = 0.25f;

        [Header("Style")]
        [Tooltip("Dim the reference (UnityBody) axes a bit so the selected frame stands out.")]
        [Range(0.1f, 1f)] public float unityBodyBrightness = 0.5f;

        private void Awake()
        {
            if (body == null) body = GetComponent<QuadPawn>();
        }

        private void Update()
        {
            if (body == null || body.Rigidbody == null) return;

            Vector3 origin = body.Rigidbody.worldCenterOfMass;

            // Drone "UnityBody" basis (per your model):
            // +X = transform.right (forward)
            // +Y = transform.up    (up)
            // +Z = transform.forward (left)
            Vector3 Xb = body.transform.right;
            Vector3 Yb = body.transform.up;
            Vector3 Zb = body.transform.forward;

            if (drawUnityBodyAxes)
            {
                DrawAxisTriplet(origin, Xb, Yb, Zb, axisLength, unityBodyBrightness);
            }

            SimFrame activeFrame = frameToDraw;
            if (followOutputFrame && body.Sensors != null)
                activeFrame = body.Sensors.outputFrame;

            if (drawSelectedFrameAxes)
            {
                GetFrameAxesWorld(activeFrame, Xb, Yb, Zb, out Vector3 Xf, out Vector3 Yf, out Vector3 Z
f);
                DrawAxisTriplet(origin, Xf, Yf, Zf, axisLength, 1.0f);
            }

            if (drawMotorAxes && body.Motors != null)
            {
                for (int i = 0; i < body.Motors.Length; i++)
                {
                    var m = body.Motors[i];
                    if (m == null) continue;

                    Vector3 p = m.position;
                    Debug.DrawRay(p, m.right * (axisLength * 0.5f), Color.red);
                    Debug.DrawRay(p, m.up * (axisLength * 0.5f), Color.green);
                    Debug.DrawRay(p, m.forward * (axisLength * 0.5f), Color.blue);
                }
            }
        }
```

```
        private static void DrawAxisTriplet(Vector3 origin, Vector3 x, Vector3 y, Vector3 z, float len,
float brightness)
        {
            // Convention: X=red, Y=green, Z=blue
            Debug.DrawRay(origin, x.normalized * len, new Color(1f * brightness, 0f, 0f));
            Debug.DrawRay(origin, y.normalized * len, new Color(0f, 1f * brightness, 0f));
            Debug.DrawRay(origin, z.normalized * len, new Color(0f, 0f, 1f * brightness));
        }

        private static void GetFrameAxesWorld(SimFrame frame, Vector3 Xb, Vector3 Yb, Vector3 Zb,
            out Vector3 Xf, out Vector3 Yf, out Vector3 Zf)
        {
            Frames.GetTargetBasisInUnity(frame, out var ax, out var ay, out var az);
            Xf = ax.x * Xb + ax.y * Yb + ax.z * Zb;
            Yf = ay.x * Xb + ay.y * Yb + ay.z * Zb;
            Zf = az.x * Xb + az.y * Yb + az.z * Zb;
        }
    }
}
```

Runtime/Scripts/DroneCore/Debug/RotorGeometryGizmo.cs

SHA-256 47d5e21f55c23b732810a9f422138e02bb4173bf5b54a362ddfb869e03cd550a | 115 lines | 4,745 bytes

```
using UnityEngine;
using Yaml.Drone;

namespace DroneCore
{
    /// <summary>
    /// Edit-time visualization of the YAML rotor geometry that actually drives
    /// the allocator. Draws rotor positions, spin directions, and the declared
    /// center of mass so mesh/config mismatches are visible in the Scene view.
    /// Runtime no-op.
    /// </summary>
    [DisallowMultipleComponent]
    public sealed class RotorGeometryGizmo : MonoBehaviour
    {
        [Tooltip("Drone YAML to preview (no extension). Editor visualization only — " +
                 "the runtime config comes from QuadSimSceneConfig.droneConfigName.")]
        public string previewConfigName = "flightmare_quad";

        public bool drawRotors = true;
        public bool drawCenterOfMass = true;
        public bool drawMountDelta = true;
        [Range(0.005f, 0.05f)] public float markerRadius = 0.018f;

#if UNITY_EDITOR
        private void OnDrawGizmosSelected()
        {
            if (string.IsNullOrWhiteSpace(previewConfigName)) return;
            if (!DroneConfigLoader.LoadConfig(previewConfigName, out DroneConfig cfg)) return;

            var rotors = cfg.RotorParams.Rotors;
            if (rotors == null) return;

            Matrix4x4 prev = Gizmos.matrix;
            Gizmos.matrix = transform.localToWorldMatrix;

            Vector3 com = cfg.DroneParams.CenterOfMass;

            if (drawRotors)
            {
                for (int i = 0; i < rotors.Length; i++)
                {
                    Vector3 p = rotors[i].Location_Body;

                    // CW (+1) warm, CCW (-1) cool.
                    Gizmos.color = rotors[i].Spin_Dir > 0
                        ? new Color(1f, 0.55f, 0.1f)
                        : new Color(0.2f, 0.7f, 1f);

                    Gizmos.DrawSphere(p, markerRadius);
                    Gizmos.DrawLine(com, p);                  // arm, measured from CoM
```

```
                    Gizmos.DrawLine(p, p + Vector3.up * 0.06f); // thrust axis
                    Gizmos.color = Color.green;
                    Gizmos.DrawRay(Vector3.zero, Vector3.right * 0.25f);   // +X = nose
                    UnityEditor.Handles.Label(Vector3.right * 0.27f, "FWD (+X)");

                    UnityEditor.Handles.matrix = transform.localToWorldMatrix;
                    UnityEditor.Handles.color = Gizmos.color;
                    UnityEditor.Handles.Label(
                        p + Vector3.up * 0.07f,
                        $"id{rotors[i].ID} {(rotors[i].Spin_Dir > 0 ? "CW" : "CCW")} " +
                        $"{Quadrant(p)}  r={(p - com).magnitude:F3}m");
                }
            }

            if (drawCenterOfMass)
            {
                Gizmos.color = Color.magenta;
                Gizmos.DrawWireSphere(com, markerRadius * 1.4f);

                // Residual centroid about the CoM — the coupling term.
                Vector3 resid = Vector3.zero;
                for (int i = 0; i < rotors.Length; i++) resid += rotors[i].Location_Body - com;
                if (rotors.Length > 0) resid /= rotors.Length;

                Gizmos.color = Color.red;
                Gizmos.DrawLine(com, com + resid * 10f);   // 10x for visibility
                UnityEditor.Handles.Label(com + resid * 10f,
                    $"residual x10  |perp|={new Vector2(resid.x, resid.z).magnitude:F4} m");
            }

            if (drawMountDelta)
            {
                var pawn = GetComponent<QuadPawn>();
                if (pawn != null)
                {
                    Gizmos.color = Color.yellow;
                    foreach (var t in new[] { "MotorFL", "MotorFR", "MotorBL", "MotorBR" })
                    {
                        Transform mt = transform.Find(t);
                        if (mt == null) continue;
                        Vector3 mp = mt.localPosition;
                        Vector3 yp = NearestRotor(rotors, mp);
                        if ((mp - yp).sqrMagnitude > 1e-6f) Gizmos.DrawLine(mp, yp);
                    }
                }
            }

            Gizmos.matrix = prev;
        }

        private static string Quadrant(Vector3 p) =>
            (p.x >= 0f ? "F" : "B") + (p.z >= 0f ? "L" : "R");
        private static Vector3 NearestRotor(RotorInstance[] r, Vector3 p)
        {
            Vector3 best = p; float bd = float.MaxValue;
            for (int i = 0; i < r.Length; i++)
            {
                float d = (r[i].Location_Body - p).sqrMagnitude;
                if (d < bd) { bd = d; best = r[i].Location_Body; }
            }
            return best;
        }
#endif
    }
}
```

12. Runtime User Interface

Builds the in-simulation ImGui taskbar, control panels, state display, PID editor, settings, and command-state stream. Mutating UI actions are deferred to Unity's main update path and commands remain subject to the global authority manager.

Files in this section

- Runtime/Scripts/RuntimeUI/ImGui/ControlPanelUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/ImGuiRuntimeBootstrap.cs
- Runtime/Scripts/RuntimeUI/ImGui/JoystickVizUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/PidSettingsUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/SettingsUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/SimImGuiStyle.cs
- Runtime/Scripts/RuntimeUI/ImGui/StateHudUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/TaskbarUI.cs
- Runtime/Scripts/RuntimeUI/ImGui/UiActions.cs
- Runtime/Scripts/RuntimeUI/ImGui/UiCommandState.cs

Runtime/Scripts/RuntimeUI/ImGui/ControlPanelUI.cs

SHA-256 2ecbe207b8cd8a3c64384ba6be48a0559c9f5df86dc77f00af4050ac7aca495c | 580 lines | 22,937 bytes

```
// Port of ControlPanelUI.cpp. 380px fixed-width, auto-height, right-anchored on
// first show. All command widgets gate on CanUICommand (authority == UI).
//
// Deviations from the old UE panel:
//  - PX4 checkbox -> "FRD" frame toggle.
//  - Angle/Acro W is Throttle01, not Z-velocity.
//  - No manual waypoint queue / no session limit sliders.
//  - Hover = Position mode holding current XY/yaw at HoverAlt.
using System.Collections.Generic;
using ImGuiNET;
using SimCore;
using SimCore.Common;
using DroneCore;
using DroneCore.Api;
using DroneCore.CameraSystem;
using DroneCore.Core;
using MathUtil;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class ControlPanelUI
    {
        private sealed class DronePanelState
        {
            public bool Gamepad;
            public bool HoverActive;
            public float HoverAlt = 3f;
            public readonly UiCommandState Cmd = new();
        }

        private const float PanelW = 380f;
        private const float RightPad = 10f;
        private const float ResetHighAltM = 5f;
        private const float PosRangeM = 50f;
        private const float AltMaxM = 50f;

        private readonly Dictionary<QuadPawn, DronePanelState> _states = new();
        private bool _open;
        public bool IsOpen => _open;
        public void ToggleOpen() => _open = !_open;
```

```
public void OnSimReset()
{
    foreach (var s in _states.Values)
    {
        s.Cmd.ZeroAll();
        s.HoverActive = false;
    }
}

public UiCommandState GetCommandState(QuadPawn pawn)
    => pawn != null ? GetState(pawn).Cmd : null;

private DronePanelState GetState(QuadPawn pawn)
{

    List<QuadPawn> dead = null;
    foreach (var k in _states.Keys)
        if (k == null) (dead ??= new List<QuadPawn>()).Add(k);
    if (dead != null)
        foreach (var k in dead) _states.Remove(k);

    if (!_states.TryGetValue(pawn, out var s))
    {
        s = new DronePanelState();
        _states[pawn] = s;
    }
    return s;
}

public void Draw(SimCoreApi sim, DroneManager dm)
{
    if (!_open || sim == null || dm == null || dm.DroneCount == 0) return;

    QuadPawn pawn = dm.SelectedDrone;
    if (pawn == null) return;

    var api = pawn.GetComponent<QuadSimApi>();
    var ctrl = pawn.Controller;
    if (api == null || ctrl == null) return;

    var state = GetState(pawn);
    bool canCommand = sim.CurrentAuthority == InputSource.UI;

    var io = ImGui.GetIO();
    ImGui.SetNextWindowPos(
        new ImVec2(Mathf.Max(0f, io.DisplaySize.X - PanelW - RightPad), 80f),
        ImGuiCond.FirstUseEver);

    ImGui.SetNextWindowSizeConstraints(
        new ImVec2(PanelW, 0f),
        new ImVec2(PanelW, float.MaxValue));

    var wFlags = ImGuiWindowFlags.NoResize | ImGuiWindowFlags.AlwaysAutoResize;

    var entityId = pawn.GetEntityId();

    if (ImGui.Begin($"Control Panel - {pawn.DroneID}##CP_{entityId}", ref _open, wFlags))
    {
        ImGui.PushID(entityId.GetHashCode());

        DrawDroneCombo(dm, pawn);
        DrawAuthorityRow(sim, canCommand);

        ImGui.Separator();

        ImGui.BeginDisabled(!canCommand);
        DrawToggleRow(pawn, api, ctrl, state, canCommand);
        ImGui.Separator();
        DrawModeButtons(pawn, api, state);
        ImGui.EndDisabled();

        ImGui.Separator();
```

```
                DrawFlightControls(pawn, api, state, canCommand);

                ImGui.Spacing();
                ImGui.PushStyleVar(ImGuiStyleVar.FramePadding, new ImVec2(16f, 9f));

                if (ImGui.Button("Follow / FPV##ControlPanelSwitchCamera"))
                    UiActions.Defer(() => pawn.SwitchCamera());

                ImGui.SameLine();

                ImGui.BeginDisabled(!canCommand);

                if (ImGui.Button("Reset High##ControlPanelResetHigh"))
                {
                    state.HoverActive = false;
                    UiActions.Defer(() => api.ResetPose(dm.SpawnOrigin + Vector3.up * ResetHighAltM, Qua
ternion.identity));
                }

                ImGui.SameLine();

                if (ImGui.Button("Reset Low##ControlPanelResetLow"))
                {
                    state.HoverActive = false;
                    UiActions.Defer(() => dm.ResetSelectedDrone());
                }

                ImGui.SameLine();

                if (ImGui.Button("Rotation##ControlPanelResetRotation"))
                    UiActions.Defer(() => api.ResetRotation());

                ImGui.EndDisabled();
                ImGui.PopStyleVar();

                DrawCameraControls(pawn);

                ImGui.PopID();
            }

            ImGui.End();
        }

        private static void DrawCameraControls(QuadPawn pawn)
        {
            CameraDirector director = CameraDirector.Get();
            if (director == null || !ImGui.CollapsingHeader("Camera##ControlPanelCamera"))
                return;

            string preview = director.Mode.ToString();
            if (ImGui.BeginCombo("View Mode##ControlPanelCameraMode", preview))
            {
                CameraViewMode[] modes =
                {
                    CameraViewMode.Follow,
                    CameraViewMode.Orbit,
                    CameraViewMode.FPV,
                    CameraViewMode.Free,
                };

                foreach (CameraViewMode mode in modes)
                {
                    bool selected = director.Mode == mode;
                    CameraViewMode selectedMode = mode;
                    if (ImGui.Selectable(mode.ToString(), selected))
                    {
                        UiActions.Defer(() =>
                        {
                            director.Bind(pawn, snap: false);
                            director.SetMode(selectedMode);
                        });
                    }
                    if (selected) ImGui.SetItemDefaultFocus();
```

```
                }
                ImGui.EndCombo();
            }

            float distance = director.FollowDistance;
            if (ImGui.SliderFloat("Distance (m)##ControlPanelCameraDistance", ref distance, 0.25f, 10f,
"%.2f"))
                director.FollowDistance = distance;

            float fov = director.FieldOfView;
            if (ImGui.SliderFloat("FOV##ControlPanelCameraFov", ref fov, 30f, 120f, "%.0f deg"))
                director.FieldOfView = fov;

            if (director.Mode == CameraViewMode.Orbit)
            {
                ImGui.TextDisabled("Right-drag viewport to orbit | Wheel to zoom");
                float yaw = director.OrbitYawDeg;
                float pitch = director.OrbitPitchDeg;
                if (ImGui.SliderFloat("Orbit Yaw##ControlPanelOrbitYaw", ref yaw, -180f, 180f, "%.1f deg
"))
                    director.OrbitYawDeg = yaw;
                if (ImGui.SliderFloat("Orbit Pitch##ControlPanelOrbitPitch", ref pitch, -80f, 80f, "%.1f
 deg"))
                    director.OrbitPitchDeg = pitch;
                if (ImGui.SmallButton("Reset Orbit##ControlPanelResetOrbit"))
                    director.ResetOrbit();
            }
            else if (director.Mode == CameraViewMode.Free)
            {
                ImGui.TextDisabled("Right-drag to look | WASD + Q/E to move | Shift boosts");
                ImGui.TextDisabled("Wheel moves forward/backward");
                ImGui.TextDisabled("Left-click a drone to select it | F toggles free follow");
                string follow = director.FreeFollowTarget != null
                    ? director.FreeFollowTarget.DroneID
                    : "Off";
                ImGui.TextDisabled($"Free follow: {follow}");
            }
            else if (director.Mode == CameraViewMode.Follow)
            {
                ImGui.TextDisabled("Wheel adjusts follow distance");
            }
            else if (director.Mode == CameraViewMode.FPV)
            {
                ImGui.TextDisabled("Wheel adjusts field of view");
            }

            string target = director.TargetDrone != null ? director.TargetDrone.DroneID : "None";
            string source = director.CameraSourceDrone != null ? director.CameraSourceDrone.DroneID : "S
cene camera";
            ImGui.TextDisabled($"Target: {target} | Camera source: {source}");
        }

        private void DrawDroneCombo(DroneManager dm, QuadPawn active)
        {
            if (ImGui.BeginCombo("Active Drone##ControlPanelActiveDrone", active.DroneID))
            {
                var drones = dm.RegisteredDrones;
                for (int i = 0; i < drones.Count; i++)
                {
                    if (drones[i] == null) continue;

                    ImGui.PushID(i);

                    bool selected = i == dm.SelectedDroneIndex;
                    int idx = i;
                    if (ImGui.Selectable($"{drones[i].DroneID}##DroneSelect", selected))
                        UiActions.Defer(() => dm.SelectDroneByIndex(idx, alsoPossess: true));

                    if (selected)
                        ImGui.SetItemDefaultFocus();

                    ImGui.PopID();
                }
```

```
                ImGui.EndCombo();
            }
        }

        private void DrawAuthorityRow(SimCoreApi sim, bool canCommand)
        {
            var auth = sim.CurrentAuthority;

            if (canCommand)
            {
                ImGui.TextColored(SimImGuiStyle.AccentGreen, $"Authority: {auth}");
            }
            else
            {
                ImGui.TextColored(SimImGuiStyle.WarnOrange, $"{auth} owns authority - UI commands disabl
ed");
                ImGui.SameLine();

                if (ImGui.SmallButton("Switch to UI##ControlPanelSwitchToUI"))
                    sim.SwitchToUI();
            }
        }

        private void DrawToggleRow(
            QuadPawn pawn,
            QuadSimApi api,
            DroneCore.Controllers.CascadedController ctrl,
            DronePanelState state,
            bool canCommand)
        {
            ImGui.Checkbox("Gamepad##ControlPanelGamepad", ref state.Gamepad);
            ImGui.SameLine();

            var sensors = pawn.Sensors;
            // NOTE: this toggles the BODY-axis convention (IMU/mag) only.
            // GPS local position stays world-frame ENU in both modes, so the
            // cascaded PositionController state is convention-independent.
            bool frd = sensors != null && sensors.outputFrame == SimFrame.FRD;

            if (ImGui.Checkbox("FRD##ControlPanelFrameFRD", ref frd) && sensors != null && canCommand)
            {
                sensors.outputFrame = frd ? SimFrame.FRD : SimFrame.FLU;
                ctrl.RebuildAllocator();
                api.ResetController();
                Debug.Log($"[ControlPanelUI] Output frame -> {sensors.outputFrame}");
            }

            if (ImGui.IsItemHovered())
            {
                ImGui.SetTooltip(
                    "Sensor output frame FLU <-> FRD (PX4 convention).\n" +
                    "Rebuilds allocator + resets controller state.");
            }

            ImGui.SameLine();

            bool dbg = ctrl.DebugVisualsEnabled;
            if (ImGui.Checkbox("Debug##ControlPanelDebug", ref dbg))
                ctrl.DebugVisualsEnabled = dbg;
        }

        private void DrawModeButtons(QuadPawn pawn, QuadSimApi api, DronePanelState state)
        {
            void EnterMode(GoalMode mode)
            {
                api.SetMode(mode);
                InitCommandForMode(pawn, state, mode);
            }

            if (!state.Gamepad)
            {
                if (ImGui.Button("Position##ModePosition")) EnterMode(GoalMode.Position);
```

```
            ImGui.SameLine();
            if (ImGui.Button("Velocity##ModeVelocity")) EnterMode(GoalMode.Velocity);
            ImGui.SameLine();
            if (ImGui.Button("Angle##ModeAngle")) EnterMode(GoalMode.Angle);
            ImGui.SameLine();
            if (ImGui.Button("Acro##ModeRate")) EnterMode(GoalMode.Rate);
        }
        else
        {
            if (ImGui.Button("Gamepad Angle##ModeGamepadAngle")) EnterMode(GoalMode.Angle);
            ImGui.SameLine();
            if (ImGui.Button("Gamepad Acro##ModeGamepadRate")) EnterMode(GoalMode.Rate);
        }
    }

    private void InitCommandForMode(QuadPawn pawn, DronePanelState state, GoalMode mode)
    {
        var cmd = state.Cmd;
        var latest = pawn.Sensors != null ? pawn.Sensors.Latest : default;
        float hoverThrottle = HoverThrottle01(pawn);

        switch (mode)
        {
            case GoalMode.Position:
                cmd.PosX = latest.GpsPosition.x;
                cmd.PosY = latest.GpsPosition.y;
                cmd.PosAlt = latest.GpsPosition.z;
                cmd.PosYawDeg = latest.ImuAttitude.z;
                break;

            case GoalMode.Velocity:
                cmd.VelX = 0f;
                cmd.VelY = 0f;
                cmd.VelZ = 0f;
                cmd.YawRateDps = 0f;
                break;

            case GoalMode.Angle:
                cmd.RollDeg = 0f;
                cmd.PitchDeg = 0f;
                cmd.YawRateDps = 0f;
                cmd.Throttle01 = hoverThrottle;
                break;

            case GoalMode.Rate:
                cmd.RollRateDps = 0f;
                cmd.PitchRateDps = 0f;
                cmd.YawRateDps = 0f;
                cmd.Throttle01 = hoverThrottle;
                break;
        }
    }

    private static float HoverThrottle01(QuadPawn pawn)
    {
        if (pawn.Rigidbody == null || !pawn.HasConfigApplied)
            return 0.4f;

        float maxThrust = pawn.Config.FlightParams.MaxThrust;
        if (maxThrust <= 0f)
            return 0.4f;

        return Mathf.Clamp01(pawn.Rigidbody.mass * 9.81f / maxThrust);
    }
    private static bool SliderFloatZero(
        string label,
        string id,
        ref float value,
        float min,
        float max,
        string format,
        float resetValue = 0f)
    {
```

```
            float buttonW = 28f;
            float spacing = ImGui.GetStyle().ItemSpacing.X;
            float sliderW = Mathf.Max(80f, ImGui.GetContentRegionAvail().X - buttonW - spacing);

            ImGui.PushID(id);

            ImGui.SetNextItemWidth(sliderW);
            bool changed = ImGui.SliderFloat(label + "##slider", ref value, min, max, format);

            ImGui.SameLine();

            if (ImGui.SmallButton("0##zero"))
            {
                value = resetValue;
                changed = true;
            }

            ImGui.PopID();
            return changed;
        }
        private void DrawFlightControls(QuadPawn pawn, QuadSimApi api, DronePanelState state, bool canCo
mmand)
        {
            if (!ImGui.CollapsingHeader("Flight Controls##ControlPanelFlightControls", ImGuiTreeNodeFlag
s.DefaultOpen))
                return;

            var fp = pawn.HasConfigApplied ? pawn.Config.FlightParams : default;

            float maxVelXY = fp.MaxVelXY > 0f ? fp.MaxVelXY : 5f;
            float maxVelZ = fp.MaxVelZ > 0f ? fp.MaxVelZ : 3f;
            float maxAngle = fp.MaxAngle > 0f ? fp.MaxAngle : 30f;
            float maxRateRP = fp.MaxRateRollPitch > 0f ? fp.MaxRateRollPitch : 180f;
            float maxRateYaw = fp.MaxRateYaw > 0f ? fp.MaxRateYaw : 90f;

            var cmd = state.Cmd;
            GoalMode mode = api.CurrentMode;

            ImGui.BeginDisabled(!canCommand);

            if (state.HoverActive)
            {
                ImGui.PushStyleColor(ImGuiCol.Button, SimImGuiStyle.AccentGreen);
                ImGui.PushStyleColor(ImGuiCol.ButtonHovered, new System.Numerics.Vector4(0.25f, 0.80f, 0
.35f, 1f));
                ImGui.PushStyleColor(ImGuiCol.ButtonActive, new System.Numerics.Vector4(0.18f, 0.60f, 0.
28f, 1f));
            }

            if (ImGui.Button("ACTIVATE HOVER MODE##ControlPanelHoverToggle"))
            {
                state.HoverActive = !state.HoverActive;

                if (state.HoverActive)
                {
                    api.SetMode(GoalMode.Position);

                    var latest = pawn.Sensors != null ? pawn.Sensors.Latest : default;
                    cmd.PosX = latest.GpsPosition.x;
                    cmd.PosY = latest.GpsPosition.y;
                    cmd.PosYawDeg = latest.ImuAttitude.z;
                    cmd.PosAlt = state.HoverAlt;
                }
            }

            if (state.HoverActive)
                ImGui.PopStyleColor(3);

            if (ImGui.SliderFloat("Hover Altitude (m)##ControlPanelHoverAlt",
                    ref state.HoverAlt, 0.5f, AltMaxM, "%.1f") && state.HoverActive)
            {
                cmd.PosAlt = state.HoverAlt;
            }
```

```
            string label = mode switch
            {
                GoalMode.Position => "Desired Position (X, Y, Z) and Yaw",
                GoalMode.Velocity => "Desired Velocities (X, Y, Z) and Yaw Rate",
                GoalMode.Angle => "Desired Angles (Roll, Pitch), Yaw Rate, Throttle",
                GoalMode.Rate => "Desired Rates (Roll, Pitch, Yaw), Throttle",
                _ => null,
            };

            if (label != null)
            {
                ImGui.Separator();
                ImGui.TextUnformatted(label);
            }

            bool yawApplicable = mode != GoalMode.Position;

            switch (mode)
            {
                case GoalMode.Position:
                    SliderFloatZero("X (m)", "ControlPanelPosX", ref cmd.PosX, -PosRangeM, PosRangeM, "%
.2f");
                    SliderFloatZero("Y (m)", "ControlPanelPosY", ref cmd.PosY, -PosRangeM, PosRangeM, "%
.2f");
                    SliderFloatZero("Z (m)", "ControlPanelPosZ", ref cmd.PosAlt, 0f, AltMaxM, "%.2f");
                    SliderFloatZero("Yaw (deg)", "ControlPanelPosYaw", ref cmd.PosYawDeg, -180f, 180f, "
%.1f");
                    break;

                case GoalMode.Velocity:
                    SliderFloatZero("Velocity X (m/s)", "ControlPanelVelX", ref cmd.VelX, -maxVelXY, max
VelXY, "%.2f");
                    SliderFloatZero("Velocity Y (m/s)", "ControlPanelVelY", ref cmd.VelY, -maxVelXY, max
VelXY, "%.2f");
                    SliderFloatZero("Velocity Z (m/s)", "ControlPanelVelZ", ref cmd.VelZ, -maxVelZ, maxV
elZ, "%.2f");
                    SliderFloatZero("Yaw Rate (deg/s)", "ControlPanelVelYawRate", ref cmd.YawRateDps, -m
axRateYaw, maxRateYaw, "%.1f");
                    break;

                case GoalMode.Angle:
                    SliderFloatZero("Roll (deg)", "ControlPanelAngleRoll", ref cmd.RollDeg, -maxAngle, m
axAngle, "%.1f");
                    SliderFloatZero("Pitch (deg)", "ControlPanelAnglePitch", ref cmd.PitchDeg, -maxAngle
, maxAngle, "%.1f");
                    SliderFloatZero("Yaw Rate (deg/s)", "ControlPanelAngleYawRate", ref cmd.YawRateDps,
-maxRateYaw, maxRateYaw, "%.1f");
                    SliderFloatZero("Throttle [0..1]", "ControlPanelAngleThrottle", ref cmd.Throttle01,
0f, 1f, "%.2f");
                    break;

                case GoalMode.Rate:
                    SliderFloatZero("Roll Rate (deg/s)", "ControlPanelRateRoll", ref cmd.RollRateDps, -m
axRateRP, maxRateRP, "%.1f");
                    SliderFloatZero("Pitch Rate (deg/s)", "ControlPanelRatePitch", ref cmd.PitchRateDps,
 -maxRateRP, maxRateRP, "%.1f");
                    SliderFloatZero("Yaw Rate (deg/s)", "ControlPanelRateYaw", ref cmd.YawRateDps, -maxR
ateYaw, maxRateYaw, "%.1f");
                    SliderFloatZero("Throttle [0..1]", "ControlPanelRateThrottle", ref cmd.Throttle01, 0
f, 1f, "%.2f");
                    break;

                default:
                    ImGui.TextDisabled($"Mode {mode} is not UI-driven (external/wrench path).");
                    break;
            }

            if (mode == GoalMode.Position || mode == GoalMode.Velocity ||
                mode == GoalMode.Angle || mode == GoalMode.Rate)
            {
                ImGui.Separator();
                ImGui.TextUnformatted("Reset to 0:");
```

```
                ImGui.SameLine();

                if (ImGui.SmallButton("X##ControlPanelResetX"))
                {
                    switch (mode)
                    {
                        case GoalMode.Position: cmd.PosX = 0f; break;
                        case GoalMode.Velocity: cmd.VelX = 0f; break;
                        case GoalMode.Angle: cmd.RollDeg = 0f; break;
                        case GoalMode.Rate: cmd.RollRateDps = 0f; break;
                    }
                }

                ImGui.SameLine();

                if (ImGui.SmallButton("Y##ControlPanelResetY"))
                {
                    switch (mode)
                    {
                        case GoalMode.Position: cmd.PosY = 0f; break;
                        case GoalMode.Velocity: cmd.VelY = 0f; break;
                        case GoalMode.Angle: cmd.PitchDeg = 0f; break;
                        case GoalMode.Rate: cmd.PitchRateDps = 0f; break;
                    }
                }

                ImGui.SameLine();

                if (ImGui.SmallButton("Z##ControlPanelResetZ"))
                {
                    switch (mode)
                    {
                        case GoalMode.Position: cmd.PosAlt = 0f; break;
                        case GoalMode.Velocity: cmd.VelZ = 0f; break;
                        default: cmd.Throttle01 = 0f; break;
                    }
                }

                if (yawApplicable)
                {
                    ImGui.SameLine();

                    if (ImGui.SmallButton("Yaw##ControlPanelResetYaw"))
                        cmd.YawRateDps = 0f;
                }
            }

            ImGui.EndDisabled();
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/ImGuiRuntimeBootstrap.cs

SHA-256 545a44dfff2ca44529b0cac6f938235b4826fa66c04b8a6afc1d09286c700819 | 389 lines | 15,098 bytes

```
using ImGuiNET;
using SimCore;
using SimCore.Common;
using DroneCore;
using DroneCore.Api;
using DroneCore.CameraSystem;
using DroneCore.Core;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    [DefaultExecutionOrder(1000)]
    public sealed class ImGuiRuntimeBootstrap : MonoBehaviour
    {
        private SimCoreApi _sim;
        private DroneManager _dm;
        private UImGui.UImGui _ui;
```

```
private Camera _boundCamera;
private CameraDirector _cameraDirector;

private TaskbarUI _taskbar;
private ControlPanelUI _controlPanel;
private StateHudUI _stateHud;
private SettingsUI _settings;
private JoystickVizUI _joystickViz;

private bool _appliedStartupPrefs;
private bool _externalPendingApproval;
private bool _layoutSeen;
private bool _subscribed;
private bool _hudVisible = true;
private bool _cameraLookDrag;
private float _noLayoutWarnTimer;

private void OnEnable()
{
    _taskbar = new TaskbarUI();
    _controlPanel = new ControlPanelUI();
    _stateHud = new StateHudUI();
    _settings = new SettingsUI();
    _joystickViz = new JoystickVizUI();
    _cameraDirector = CameraDirector.GetOrCreate(gameObject);

    _taskbar.OnSettingsClicked = () => _settings.ToggleOpen();
    _taskbar.OnSimReset = () => _controlPanel.OnSimReset();

    _ui = GetComponent<UImGui.UImGui>();
    if (_ui == null)
    {
        Debug.LogError("[ImGuiRuntimeBootstrap] Missing UImGui component on same GameObject.");
        return;
    }

    _ui.Layout += OnLayout;
    _subscribed = true;
}

private void OnDisable()
{
    if (_ui != null && _subscribed)
    {
        _ui.Layout -= OnLayout;
        _subscribed = false;
    }
    UnsubscribeSim();
}

private void ResolveRefs()
{
    if (_sim == null)
    {
        _sim = SimCoreApi.Get();
        if (_sim != null)
            _sim.OnExternalConnectionRequest += HandleExternalRequest;
    }
    if (_dm == null)
        _dm = DroneManager.Get();
    if (_cameraDirector == null)
        _cameraDirector = CameraDirector.GetOrCreate(gameObject);
    if (_cameraDirector != null)
        _cameraDirector.EnsureBoundToFirstDrone(_dm);
}

private void UnsubscribeSim()
{
    if (_sim != null)
        _sim.OnExternalConnectionRequest -= HandleExternalRequest;
}

private void HandleExternalRequest() => _externalPendingApproval = true;
```

```
        // ================================================================
        // Camera binding — keep UImGui pointed at whatever camera is live.
        // Survives drone reload/spawn (new camera object) and drone selection.
        // Switch Camera moves the SAME camera between mounts, so no re-bind.
        // ================================================================
        private void EnsureUiCamera()
        {
            if (_ui == null) return;

            _cameraDirector?.EnsureBoundToFirstDrone(_dm);
            Camera target = _cameraDirector != null ? _cameraDirector.ViewCamera : null;
            if (target == null && _dm != null && _dm.DroneCount > 0)
                target = _dm.RegisteredDrones[0]?.DroneCamera;
            if (target == null || !target.enabled || !target.gameObject.activeInHierarchy)
                target = Camera.main;

            if (target == null || target == _boundCamera) return;

            _ui.SetCamera(target);
            _boundCamera = target;
            Debug.Log($"[ImGuiRuntimeBootstrap] UImGui camera -> {target.name}");
        }

        private void Update()
        {
            ResolveRefs();

            // Drain deferred UI actions FIRST — spawn/reload/resets happen here,
            // safely outside the render pipeline.
            UiActions.Drain();

            EnsureUiCamera();

            if (_sim == null) return;

            if (!_layoutSeen)
            {
                _noLayoutWarnTimer += Time.unscaledDeltaTime;
                if (_noLayoutWarnTimer > 5f)
                {
                    Debug.LogWarning("[ImGuiRuntimeBootstrap] No ImGui layout callback after 5s. " +
                                     "Check that UImGui is enabled, has a camera, and the RenderImGui pa
ss is active.");
                    _noLayoutWarnTimer = float.NegativeInfinity;
                }
            }

            // ---- Command streaming (independent of HUD visibility) ----
            if (_sim.CurrentAuthority != InputSource.UI) return;
            if (_dm == null || _dm.DroneCount == 0) return;

            QuadPawn streamPawn = _dm.SelectedDrone;
            if (streamPawn == null) return;

            var api = streamPawn.GetComponent<QuadSimApi>();
            if (api == null || !api.IsInitialized) return;

            GoalMode mode = api.CurrentMode;
            if (mode != GoalMode.Position &&
                mode != GoalMode.Velocity &&
                mode != GoalMode.Angle &&
                mode != GoalMode.Rate)
            {
                return;
            }

            var cmd = _controlPanel.GetCommandState(streamPawn);
            if (cmd == null) return;

            api.SendCommand(InputSource.UI, cmd.Build(mode));
        }
```

```
private void OnLayout(UImGui.UImGui _)
{
    _layoutSeen = true;

    ResolveRefs();
    if (_sim == null) return;

    SimImGuiStyle.Apply();

    if (!_appliedStartupPrefs)
    {
        _appliedStartupPrefs = true;
        if (_settings.PersistControlPanel && !_controlPanel.IsOpen)
            _controlPanel.ToggleOpen();
        if (_settings.PersistStateHud)
            _stateHud.SetVisible(true);
    }

    if (_hudVisible)
    {
        _taskbar.Draw(_sim, _dm);
        DrawFloatingButtons();
        _controlPanel.Draw(_sim, _dm);
        _stateHud.Draw(_dm);
        _settings.Draw(_dm, _controlPanel, _stateHud);
        DrawJoystickViz();
    }

    DrawExternalApprovalPopup();   // safety-relevant: always drawn
    DrawHudToggleButton();         // last = topmost, always drawn
    HandleCameraInput();
}

private void HandleCameraInput()
{
    if (_cameraDirector == null) return;

    var io = ImGui.GetIO();

    // Camera look must begin as an intentional right-click in the
    // unobstructed viewport. Do not let a drag that started on an
    // ImGui window leak through when the cursor later leaves it.
    if (ImGui.IsMouseReleased(ImGuiMouseButton.Right))
        _cameraLookDrag = false;
    if (ImGui.IsMouseClicked(ImGuiMouseButton.Right))
        _cameraLookDrag = !io.WantCaptureMouse;

    if (!io.WantCaptureMouse)
    {
        var mouseDelta = _cameraLookDrag && ImGui.IsMouseDown(ImGuiMouseButton.Right)
            ? new Vector2(io.MouseDelta.X, io.MouseDelta.Y)
            : Vector2.zero;
        _cameraDirector.AddMouseCameraInput(mouseDelta, io.MouseWheel);

        if (_cameraDirector.Mode == CameraViewMode.Free
            && ImGui.IsMouseClicked(ImGuiMouseButton.Left))
        {
            TrySelectDroneUnderMouse(io.MousePos);
        }
    }

    if (_cameraDirector.Mode != CameraViewMode.Free || io.WantCaptureKeyboard)
        return;

    Vector3 movement = Vector3.zero;
    if (ImGui.IsKeyDown(ImGuiKey.W)) movement.z += 1f;
    if (ImGui.IsKeyDown(ImGuiKey.S)) movement.z -= 1f;
    if (ImGui.IsKeyDown(ImGuiKey.D)) movement.x += 1f;
    if (ImGui.IsKeyDown(ImGuiKey.A)) movement.x -= 1f;
    if (ImGui.IsKeyDown(ImGuiKey.E)) movement.y += 1f;
    if (ImGui.IsKeyDown(ImGuiKey.Q)) movement.y -= 1f;

    bool boosted = ImGui.IsKeyDown(ImGuiKey.LeftShift)
```

```
                || ImGui.IsKeyDown(ImGuiKey.RightShift);
    _cameraDirector.AddFreeMovementInput(movement.normalized, boosted);

    if (ImGui.IsKeyPressed(ImGuiKey.F, false))
        _cameraDirector.ToggleFreeFollow(_dm != null ? _dm.SelectedDrone : null);
}

private void TrySelectDroneUnderMouse(System.Numerics.Vector2 mousePosition)
{
    Camera camera = _cameraDirector.ViewCamera;
    if (camera == null || _dm == null) return;

    Rect pixelRect = camera.pixelRect;
    Vector3 screenPoint = new Vector3(
        pixelRect.x + mousePosition.X,
        pixelRect.y + pixelRect.height - mousePosition.Y,
        0f);

    if (!Physics.Raycast(camera.ScreenPointToRay(screenPoint), out RaycastHit hit))
        return;

    QuadPawn drone = hit.collider.GetComponentInParent<QuadPawn>();
    int index = drone != null ? _dm.GetDroneIndex(drone) : -1;
    if (index >= 0)
        _dm.SelectDroneByIndex(index, alsoPossess: false);
}

// ====================================================================
// Master HUD toggle — invisible 26x26 button in the top-left corner.
// Highlights on hover. Hides/shows the entire HUD.
// ====================================================================
private void DrawHudToggleButton()
{
    const float size = 26f;

    ImGui.SetNextWindowPos(new ImVec2(0f, 0f));
    ImGui.SetNextWindowSize(new ImVec2(size + 4f, size + 4f));
    ImGui.SetNextWindowBgAlpha(0f);

    var flags = ImGuiWindowFlags.NoTitleBar | ImGuiWindowFlags.NoResize |
                ImGuiWindowFlags.NoMove | ImGuiWindowFlags.NoScrollbar |
                ImGuiWindowFlags.NoSavedSettings | ImGuiWindowFlags.NoBackground |
                ImGuiWindowFlags.NoFocusOnAppearing;

    ImGui.PushStyleVar(ImGuiStyleVar.WindowPadding, new ImVec2(2f, 2f));
    if (ImGui.Begin("##QuadSimHudToggle", flags))
    {
        if (ImGui.InvisibleButton("##HudToggleBtn", new ImVec2(size, size)))
            _hudVisible = !_hudVisible;

        bool hovered = ImGui.IsItemHovered();
        var min = ImGui.GetItemRectMin();
        var max = ImGui.GetItemRectMax();
        var dl = ImGui.GetWindowDrawList();

        // Faint outline always (so it's findable); bright on hover.
        // Set the first alpha to 0 if you want it truly invisible at rest.
        uint col = hovered ? SimImGuiStyle.Col32(120, 200, 255, 230)
                           : SimImGuiStyle.Col32(255, 255, 255, 25);
        dl.AddRect(min, max, col, 5f, ImDrawFlags.None, hovered ? 2f : 1f);

        if (hovered)
        {
            // Eye-ish marker + tooltip
            var c = new ImVec2((min.X + max.X) * 0.5f, (min.Y + max.Y) * 0.5f);
            dl.AddCircleFilled(c, 4f, col);
            ImGui.SetTooltip(_hudVisible ? "Hide HUD" : "Show HUD");
        }
    }
    ImGui.End();
    ImGui.PopStyleVar();
}
```

```
        private void DrawFloatingButtons()
        {
            if (_dm == null || _dm.DroneCount == 0) return;

            var io = ImGui.GetIO();
            const float pad = 12f;

            ImGui.SetNextWindowPos(new ImVec2(io.DisplaySize.X - 240f - pad, io.DisplaySize.Y - 120f));
            ImGui.SetNextWindowSize(new ImVec2(240f, 104f));
            ImGui.SetNextWindowBgAlpha(0f);

            var flags = ImGuiWindowFlags.NoTitleBar | ImGuiWindowFlags.NoResize |
                        ImGuiWindowFlags.NoMove | ImGuiWindowFlags.NoSavedSettings |
                        ImGuiWindowFlags.NoScrollbar | ImGuiWindowFlags.NoFocusOnAppearing |
                        ImGuiWindowFlags.NoBackground;

            if (ImGui.Begin("##QuadSimControlPanelBtn", flags))
            {
                ImGui.PushID("FloatingPanelButtons");
                ImGui.PushStyleVar(ImGuiStyleVar.FramePadding, new ImVec2(24f, 14f));

                float cx = (ImGui.GetContentRegionAvail().X
                            - ImGui.CalcTextSize("Control Panel").X
                            - ImGui.GetStyle().FramePadding.X * 2f) * 0.5f;

                ImGui.SetCursorPosX(Mathf.Max(0f, cx));
                if (ImGui.Button("Control Panel##FloatingControlPanel"))
                    _controlPanel.ToggleOpen();

                ImGui.SetCursorPosX(Mathf.Max(0f, cx));
                if (ImGui.Button("State Data##FloatingStateData"))
                    _stateHud.ToggleOpen();

                ImGui.PopStyleVar();
                ImGui.PopID();
            }
            ImGui.End();
        }

        private void DrawJoystickViz()
        {
            if (_dm == null || _dm.DroneCount == 0) return;

            QuadPawn pawn = _dm.SelectedDrone;
            if (pawn == null) return;

            var api = pawn.GetComponent<QuadSimApi>();
            _joystickViz.Draw(pawn, api, _controlPanel.GetCommandState(pawn));
        }

        private void DrawExternalApprovalPopup()
        {
            if (_externalPendingApproval)
            {
                ImGui.OpenPopup("External Connection##QuadSim");
                _externalPendingApproval = false;
            }

            bool open = true;
            if (ImGui.BeginPopupModal("External Connection##QuadSim", ref open, ImGuiWindowFlags.AlwaysA
utoResize))
            {
                ImGui.Text("An External client is requesting control authority.");
                ImGui.Separator();

                if (ImGui.Button("Accept##ExternalAccept", new ImVec2(120f, 0f)))
                {
                    _sim.AcceptExternalConnection();
                    ImGui.CloseCurrentPopup();
                }
                ImGui.SameLine();
                if (ImGui.Button("Deny##ExternalDeny", new ImVec2(120f, 0f)))
                {
```

```
                    _sim.DenyExternalConnection();
                    ImGui.CloseCurrentPopup();
                }
                ImGui.EndPopup();
            }
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/JoystickVizUI.cs

SHA-256 f3bfc5a0bc5bd37a53105be645e9cd2f2d727451dc5f055e93911a6aeed8c051 | 65 lines | 3,066 bytes

```
// Bottom-center stick visualization for Angle/Rate modes.
// Currently visualizes the UI COMMAND STATE (what's actually being streamed).
// TODO(RC injection): swap the data source to real gamepad/RC input when it lands.
using ImGuiNET;
using SimCore.Common;
using DroneCore;
using DroneCore.Api;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class JoystickVizUI
    {
        public void Draw(QuadPawn pawn, QuadSimApi api, UiCommandState cmd)
        {
            if (pawn == null || api == null || cmd == null) return;
            GoalMode mode = api.CurrentMode;
            if (mode != GoalMode.Angle && mode != GoalMode.Rate) return;

            var fp = pawn.HasConfigApplied ? pawn.Config.FlightParams : default;
            float maxYaw = fp.MaxRateYaw > 0f ? fp.MaxRateYaw : 90f;
            float maxRP = mode == GoalMode.Angle
                ? (fp.MaxAngle > 0f ? fp.MaxAngle : 30f)
                : (fp.MaxRateRollPitch > 0f ? fp.MaxRateRollPitch : 180f);

            // Left stick: (yaw, throttle). Right stick: (roll, pitch).
            float lx = Mathf.Clamp(cmd.YawRateDps / maxYaw, -1f, 1f);
            float ly = Mathf.Clamp(cmd.Throttle01 * 2f - 1f, -1f, 1f);
            float rx = Mathf.Clamp((mode == GoalMode.Angle ? cmd.RollDeg : cmd.RollRateDps) / maxRP, -1f
, 1f);
            float ry = Mathf.Clamp((mode == GoalMode.Angle ? cmd.PitchDeg : cmd.PitchRateDps) / maxRP, -
1f, 1f);

            var io = ImGui.GetIO();
            const float w = 260f, h = 160f;
            ImGui.SetNextWindowPos(new ImVec2((io.DisplaySize.X - w) * 0.5f, io.DisplaySize.Y - h - 10f)
,
                ImGuiCond.Always);
            ImGui.SetNextWindowSize(new ImVec2(w, h), ImGuiCond.Always);
            var wf = ImGuiWindowFlags.NoTitleBar | ImGuiWindowFlags.NoResize |
                     ImGuiWindowFlags.NoSavedSettings | ImGuiWindowFlags.NoBackground;
            if (ImGui.Begin("Joystick##BottomViz", wf))
            {
                DrawStick("Left Stick", "##leftStick", lx, -ly);
                ImGui.SameLine(0f, 30f);
                DrawStick("Right Stick", "##rightStick", rx, ry);
            }
            ImGui.End();
        }

        private static void DrawStick(string title, string id, float x, float y)
        {
            var boxSize = new ImVec2(100f, 100f);
            var dl = ImGui.GetWindowDrawList();
            ImGui.BeginGroup();
            ImGui.Text(title);
            ImGui.InvisibleButton(id, boxSize);
            var p = ImGui.GetItemRectMin();
            var q = ImGui.GetItemRectMax();
            dl.AddRect(p, q, SimImGuiStyle.Col32(255, 255, 255, 255));
```

```
            var center = new ImVec2((p.X + q.X) * 0.5f, (p.Y + q.Y) * 0.5f);
            var pos = new ImVec2(center.X + x * boxSize.X * 0.5f, center.Y + y * boxSize.Y * 0.5f);
            dl.AddCircleFilled(pos, 5f, SimImGuiStyle.Col32(255, 255, 0, 255));
            ImGui.EndGroup();
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/PidSettingsUI.cs

SHA-256 193acfde92b39d93b78309adfd707268217c814ab76f1c980d1d47b08fa052af | 243 lines | 11,038 bytes

```
// PID tuning UI over the existing CascadedController hooks ONLY:
//   GetPIDGains / SetPIDGainTerm / IsPIDStageActive
// No second PID model. SetPIDGainTerm called only on actual value change.
// Exposes all 4 cascade stages (old UE UI lacked Velocity + Angle-Yaw; the
// current system has them, so they're shown).
// Save/History: RuntimeUI-owned CSV at persistentDataPath — core untouched.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using ImGuiNET;
using DroneCore;
using DroneCore.Api;
using DroneCore.Controllers;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class PidSettingsUI
    {
        // Fixed CSV schema: (stage, axis, shortName) — deterministic column order
        private static readonly (CascadedController.EPIDStage Stage, byte Axis, string Name)[] Schema =
        {
            (CascadedController.EPIDStage.Position, 0, "posX"),
            (CascadedController.EPIDStage.Position, 1, "posY"),
            (CascadedController.EPIDStage.Position, 3, "posZ"),
            (CascadedController.EPIDStage.Velocity, 0, "velX"),
            (CascadedController.EPIDStage.Velocity, 1, "velY"),
            (CascadedController.EPIDStage.Velocity, 3, "velZ"),
            (CascadedController.EPIDStage.Angle,    0, "angRoll"),
            (CascadedController.EPIDStage.Angle,    1, "angPitch"),
            (CascadedController.EPIDStage.Angle,    2, "angYaw"),
            (CascadedController.EPIDStage.Rate,     0, "rateRoll"),
            (CascadedController.EPIDStage.Rate,     1, "ratePitch"),
            (CascadedController.EPIDStage.Rate,     2, "rateYaw"),
        };

        private bool _syncPosXY, _syncVelXY, _syncAngleRP, _syncRateRP;
        private bool _showHistory;
        private List<string[]> _historyRows;   // cached CSV rows
        private string[] _historyHeader;

        private static string CsvPath => Path.Combine(Application.persistentDataPath, "PIDGains.csv");

        public void Draw(QuadPawn pawn, CascadedController ctrl)
        {
            if (!ImGui.CollapsingHeader("PID Settings", ImGuiTreeNodeFlags.DefaultOpen)) return;

            DrawStageGroup(ctrl, "Position PID Gains", CascadedController.EPIDStage.Position,
                new (byte, string)[] { (0, "X"), (1, "Y"), (3, "Z") },
                ref _syncPosXY, "Synchronize X and Y Axis Gains", syncA: 0, syncB: 1,
                min: 0.0001f, max: 30f);

            ImGui.Separator();
            DrawStageGroup(ctrl, "Velocity PID Gains", CascadedController.EPIDStage.Velocity,
                new (byte, string)[] { (0, "X"), (1, "Y"), (3, "Z") },
                ref _syncVelXY, "Synchronize X and Y Velocity Gains", syncA: 0, syncB: 1,
                min: 0.0001f, max: 30f);

            ImGui.Separator();
            DrawStageGroup(ctrl, "Attitude PID Gains", CascadedController.EPIDStage.Angle,
```

```
            new (byte, string)[] { (0, "Roll"), (1, "Pitch"), (2, "Yaw") },
            ref _syncAngleRP, "Synchronize Roll and Pitch Gains", syncA: 0, syncB: 1,
            min: 0.0001f, max: 30f);

        ImGui.Separator();
        DrawStageGroup(ctrl, "Angular Rate PID Gains", CascadedController.EPIDStage.Rate,
            new (byte, string)[] { (0, "Roll Rate"), (1, "Pitch Rate"), (2, "Yaw Rate") },
            ref _syncRateRP, "Synchronize Roll and Pitch Rate Gains", syncA: 0, syncB: 1,
            min: 0.0001f, max: 2f);

        ImGui.Separator();
        if (ImGui.Button("Save PID Gains", new ImVec2(200f, 50f)))
            SaveCsv(ctrl);
        ImGui.SameLine();
        if (ImGui.Checkbox("PID Configuration History", ref _showHistory) && _showHistory)
            LoadCsvCache();
    }

    private void DrawStageGroup(CascadedController ctrl, string title,
        CascadedController.EPIDStage stage, (byte Axis, string Label)[] axes,
        ref bool sync, string syncLabel, byte syncA, byte syncB, float min, float max)
    {
        ImGui.Text(title);
        ImGui.Checkbox(syncLabel + "##" + title, ref sync);
        ImGui.Indent();

        foreach (var (axis, label) in axes)
        {
            if (!ctrl.GetPIDGains(stage, axis, out float p, out float i, out float d))
            {
                ImGui.TextDisabled($"{label} PID Unavailable");
                continue;
            }

            bool active = ctrl.IsPIDStageActive(stage, axis);
            ImGui.Text(active ? label : $"{label} (inactive in current mode)");
            ImGui.Indent();
            ImGui.PushID(title + label);

            DrawTerm(ctrl, stage, axis, 0, $"{label} P", ref p, min, max, sync, syncA, syncB);
            DrawTerm(ctrl, stage, axis, 1, $"{label} I", ref i, min, max, sync, syncA, syncB);
            DrawTerm(ctrl, stage, axis, 2, $"{label} D", ref d, min, max, sync, syncA, syncB);

            ImGui.PopID();
            ImGui.Unindent();
        }
        ImGui.Unindent();
    }

    private void DrawTerm(CascadedController ctrl, CascadedController.EPIDStage stage,
        byte axis, int term, string label, ref float value, float min, float max,
        bool sync, byte syncA, byte syncB)
    {
        float before = value;
        if (ImGuiWidgets.SliderWithInput(label, ref value, min, max) &&
            !Mathf.Approximately(before, value))
        {
            ctrl.SetPIDGainTerm(stage, axis, term, value, resetIntegral: false);
            if (sync)
            {
                byte other = axis == syncA ? syncB : (axis == syncB ? syncA : axis);
                if (other != axis)
                    ctrl.SetPIDGainTerm(stage, other, term, value, resetIntegral: false);
            }
        }
    }

    // ---- CSV save/history ----

    private void SaveCsv(CascadedController ctrl)
    {
        try
        {
```

```
        if (!File.Exists(CsvPath))
        {
            var header = new List<string> { "Timestamp" };
            foreach (var e in Schema)
                header.AddRange(new[] { e.Name + "_P", e.Name + "_I", e.Name + "_D" });
            File.WriteAllText(CsvPath, string.Join(",", header) + "\n");
        }

        var row = new List<string> { DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss") };
        foreach (var e in Schema)
        {
            ctrl.GetPIDGains(e.Stage, e.Axis, out float p, out float i, out float d);
            row.Add(p.ToString("F6", CultureInfo.InvariantCulture));
            row.Add(i.ToString("F6", CultureInfo.InvariantCulture));
            row.Add(d.ToString("F6", CultureInfo.InvariantCulture));
        }
        File.AppendAllText(CsvPath, string.Join(",", row) + "\n");
        LoadCsvCache();
        Debug.Log($"[PidSettingsUI] Gains saved -> {CsvPath}");
    }
    catch (Exception e)
    {
        Debug.LogError($"[PidSettingsUI] Save failed: {e.Message}");
    }
}

private void LoadCsvCache()
{
    _historyRows = new List<string[]>();
    _historyHeader = null;
    if (!File.Exists(CsvPath)) return;
    var lines = File.ReadAllLines(CsvPath);
    if (lines.Length < 1) return;
    _historyHeader = lines[0].Split(',');
    for (int i = 1; i < lines.Length; i++)
        if (!string.IsNullOrWhiteSpace(lines[i]))
            _historyRows.Add(lines[i].Split(','));
}

public void DrawHistoryWindow(QuadPawn pawn)
{
    if (!_showHistory) return;

    ImGui.SetNextWindowPos(new ImVec2(420f, 520f), ImGuiCond.FirstUseEver);
    ImGui.SetNextWindowSize(new ImVec2(800f, 400f), ImGuiCond.FirstUseEver);
    if (ImGui.Begin("PID Configurations History", ref _showHistory))
    {
        if (_historyHeader == null || _historyRows == null || _historyRows.Count == 0)
        {
            ImGui.TextColored(SimImGuiStyle.AccentRed, $"No PID history at: {CsvPath}");
        }
        else if (ImGui.BeginTable("PIDHistoryTable", _historyHeader.Length,
                     ImGuiTableFlags.Borders | ImGuiTableFlags.RowBg |
                     ImGuiTableFlags.ScrollY | ImGuiTableFlags.ScrollX |
                     ImGuiTableFlags.SizingFixedFit))
        {
            ImGui.TableSetupScrollFreeze(1, 1);
            foreach (var h in _historyHeader)
                ImGui.TableSetupColumn(h, ImGuiTableColumnFlags.WidthFixed);
            ImGui.TableHeadersRow();

            for (int r = 0; r < _historyRows.Count; r++)
            {
                var row = _historyRows[r];
                ImGui.TableNextRow();
                for (int c = 0; c < row.Length && c < _historyHeader.Length; c++)
                {
                    ImGui.TableSetColumnIndex(c);
                    if (c == 0)
                    {
                        ImGui.TextUnformatted(row[0]);
                        ImGui.SameLine();
                        if (ImGui.SmallButton($"Load##{r}"))
```

```
                                ApplyRow(pawn, row);
                            }
                            else
                            {
                                ImGui.TextUnformatted(row[c]);
                            }
                        }
                    }
                    ImGui.EndTable();
                }
            }
            ImGui.End();
        }

        private void ApplyRow(QuadPawn pawn, string[] row)
        {
            var ctrl = pawn.Controller;
            var api = pawn.GetComponent<QuadSimApi>();
            if (ctrl == null) return;

            int col = 1; // col 0 = timestamp
            foreach (var e in Schema)
            {
                for (int term = 0; term < 3; term++, col++)
                {
                    if (col >= row.Length) return;
                    if (float.TryParse(row[col], NumberStyles.Float, CultureInfo.InvariantCulture, out f
float v))
                        ctrl.SetPIDGainTerm(e.Stage, e.Axis, term, v, resetIntegral: false);
                }
            }
            api?.ResetController();   // clean integrators once after bulk apply
            Debug.Log($"[PidSettingsUI] Applied gains from history row ({row[0]})");
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/SettingsUI.cs

SHA-256 e7eabfb59f68285eedfd9287ad4c438eda5b4f9b7f714aa313ef798025788483 | 148 lines | 6,324 bytes

```
// Settings##QuadSim window. Config tab is READ-ONLY (YAML is ground truth;
// edit files, not the UI). Reload goes through DroneManager.ReloadCurrentDrone.
using System.Collections.Generic;
using ImGuiNET;
using DroneCore;
using DroneCore.Core;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class SettingsUI
    {
        private const string PrefPersistCP = "QuadSim.UI.PersistControlPanel";
        private const string PrefPersistState = "QuadSim.UI.PersistStateHUD";

        private bool _open;
        private List<string> _configCache;
        public bool IsOpen => _open;
        public void ToggleOpen() => _open = !_open;

        public bool PersistControlPanel
        {
            get => PlayerPrefs.GetInt(PrefPersistCP, 0) == 1;
            set { PlayerPrefs.SetInt(PrefPersistCP, value ? 1 : 0); PlayerPrefs.Save(); }
        }
        public bool PersistStateHud
        {
            get => PlayerPrefs.GetInt(PrefPersistState, 0) == 1;
            set { PlayerPrefs.SetInt(PrefPersistState, value ? 1 : 0); PlayerPrefs.Save(); }
        }

        public void Draw(DroneManager dm, ControlPanelUI controlPanel, StateHudUI stateHud)
```

```
        {
            if (!_open) return;

            ImGui.SetNextWindowSize(new ImVec2(620f, 520f), ImGuiCond.FirstUseEver);
            if (ImGui.Begin("Settings##QuadSim", ref _open))
            {
                if (ImGui.BeginTabBar("##SettingsTabs"))
                {
                    if (ImGui.BeginTabItem("Config")) { DrawConfigTab(dm); ImGui.EndTabItem(); }
                    if (ImGui.BeginTabItem("Toggables")) { DrawTogglablesTab(dm, controlPanel, stateHud)
; ImGui.EndTabItem(); }
                    ImGui.EndTabBar();
                }
            }
            ImGui.End();
        }

        private void DrawConfigTab(DroneManager dm)
        {
            if (dm == null) { ImGui.TextDisabled("No DroneManager."); return; }

            // ---- Config selection (for NEXT spawn/reload) ----
            if (_configCache == null) RefreshConfigs(dm);
            ImGui.Text("Drone Config (YAML)"); ImGui.Separator();
            string current = dm.SelectedConfigName;
            if (ImGui.BeginCombo("Selected Config", current))
            {
                for (int i = 0; i < _configCache.Count; i++)
                {
                    bool sel = _configCache[i] == current;
                    if (ImGui.Selectable(_configCache[i], sel))
                        dm.SetSelectedConfig(_configCache[i]);
                    if (sel) ImGui.SetItemDefaultFocus();
                }
                ImGui.EndCombo();
            }
            ImGui.SameLine();
            if (ImGui.SmallButton("Refresh")) RefreshConfigs(dm);

            if (ImGui.Button("Reload Drone (re-read YAML)"))
                UiActions.Defer(() => dm.ReloadCurrentDrone(resetPosition:false));

            ImGui.SameLine();
            ImGui.TextDisabled("Save: edit YAML directly (StreamingAssets/Configs/Drones)");

            // ---- Read-only view of the SELECTED drone's applied config ----
            QuadPawn pawn = dm.SelectedDrone;
            if (pawn == null || !pawn.HasConfigApplied)
            {
                ImGui.TextDisabled("No drone with applied config.");
                return;
            }
            var cfg = pawn.Config;

            ImGui.Separator();
            ImGui.Text("Model"); ImGui.Separator();
            ImGui.LabelText("Name", cfg.ModelParams.Name ?? "(unnamed)");

            ImGui.Separator();
            ImGui.Text("Drone Parameters"); ImGui.Separator();
            ImGui.LabelText("Mass (kg)", $"{cfg.DroneParams.Mass:F3}");
            ImGui.LabelText("Arm Length (m)", $"{cfg.DroneParams.Arm_Length:F3}");
            ImGui.LabelText("Inertia (kg m2)", $"{cfg.DroneParams.Inertia}");

            ImGui.Separator();
            ImGui.Text("Flight Parameters (limits)"); ImGui.Separator();
            var fp = cfg.FlightParams;
            ImGui.LabelText("Max Velocity XY (m/s)", $"{fp.MaxVelXY:F2}");
            ImGui.LabelText("Max Velocity Z (m/s)", $"{fp.MaxVelZ:F2}");
            ImGui.LabelText("Max Tilt Angle (deg)", $"{fp.MaxAngle:F1}");
            ImGui.LabelText("Max Rate R/P (deg/s)", $"{fp.MaxRateRollPitch:F1}");
            ImGui.LabelText("Max Rate Yaw (deg/s)", $"{fp.MaxRateYaw:F1}");
            ImGui.LabelText("Max Thrust total (N)", $"{fp.MaxThrust:F2}");
```

```
            ImGui.LabelText("Max PID Output", $"{fp.MaxPID:F2}");
            ImGui.LabelText("Acceptable Distance (m)", $"{fp.AcceptableDistance:F2}");
        }

        private void RefreshConfigs(DroneManager dm)
        {
            _configCache = dm.GetAvailableConfigs() ?? new List<string>();
        }

        private void DrawTogglablesTab(DroneManager dm, ControlPanelUI cp, StateHudUI hud)
        {
            if (ImGui.Button("Spawn Drone Now") && dm != null)
                UiActions.Defer(() => dm.SpawnDrone(Vector3.zero, Quaternion.identity));
            ImGui.SameLine();
            ImGui.TextDisabled(dm != null && dm.SpawnPolicy == StartupSpawnPolicy.SceneDronesOnly
                ? "(startup uses scene-placed drones only)"
                : "(startup spawns a default only when none exist)");

            ImGui.Separator();
            bool pcp = PersistControlPanel;
            if (ImGui.Checkbox("Persistent Control Panel", ref pcp))
            {
                PersistControlPanel = pcp;
                if (cp != null && cp.IsOpen != pcp) cp.ToggleOpen();
            }
            bool psh = PersistStateHud;
            if (ImGui.Checkbox("Persistent State Info", ref psh))
            {
                PersistStateHud = psh;
                hud?.SetVisible(psh);
            }

            ImGui.Separator();
            ImGui.Text("Spawning");
            ImGui.BeginDisabled(true);
            bool obstacles = false;
            ImGui.Checkbox("Spawn Obstacles", ref obstacles);   // TODO: no obstacle system in Unity yet
            ImGui.EndDisabled();
            ImGui.SameLine();
            ImGui.TextDisabled("TODO");
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/SimImGuiStyle.cs

SHA-256 8698e04c30a5aa9e9dbe258a240405c641a30805443f2d42030ff66f0b655b61 | 99 lines | 4,872 bytes

```
// Port of FSimImGuiStyle. Pure ImGuiNET — no binding-specific code.
using ImGuiNET;
using ImVec2 = System.Numerics.Vector2;
using ImVec4 = System.Numerics.Vector4;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public static class SimImGuiStyle
    {
        // Base palette (dark theme)
        public static readonly ImVec4 WindowBg     = new(0.09f, 0.10f, 0.12f, 0.98f);
        public static readonly ImVec4 ChildBg      = new(0.11f, 0.12f, 0.15f, 0.98f);
        public static readonly ImVec4 PopupBg      = new(0.08f, 0.09f, 0.11f, 0.98f);
        public static readonly ImVec4 Border       = new(0.20f, 0.22f, 0.26f, 1.00f);
        public static readonly ImVec4 Text         = new(0.93f, 0.96f, 1.00f, 1.00f);

        public static readonly ImVec4 FrameBg      = new(0.16f, 0.18f, 0.22f, 1.00f);
        public static readonly ImVec4 FrameHovered = new(0.22f, 0.24f, 0.28f, 1.00f);
        public static readonly ImVec4 FrameActive  = new(0.28f, 0.31f, 0.36f, 1.00f);

        public static readonly ImVec4 Button       = new(0.18f, 0.24f, 0.35f, 1.00f);
        public static readonly ImVec4 ButtonHover  = new(0.24f, 0.32f, 0.48f, 1.00f);
        public static readonly ImVec4 ButtonActive = new(0.16f, 0.28f, 0.46f, 1.00f);

        // Accents
        public static readonly ImVec4 AccentGreen  = new(0.20f, 0.80f, 0.35f, 1.00f);
```

```
        public static readonly ImVec4 AccentRed    = new(0.90f, 0.25f, 0.30f, 1.00f);
        public static readonly ImVec4 HeaderBlue   = new(0.60f, 0.90f, 1.00f, 1.00f);
        public static readonly ImVec4 HeaderYellow = new(1.00f, 0.90f, 0.40f, 1.00f);
        public static readonly ImVec4 HeaderPurple = new(0.80f, 0.80f, 1.00f, 1.00f);
        public static readonly ImVec4 WarnOrange   = new(0.95f, 0.60f, 0.15f, 1.00f);

        public static void Apply()
        {
            var style = ImGui.GetStyle();
            var col = style.Colors;

            col[(int)ImGuiCol.Text]           = Text;
            col[(int)ImGuiCol.WindowBg]       = WindowBg;
            col[(int)ImGuiCol.ChildBg]        = ChildBg;
            col[(int)ImGuiCol.PopupBg]        = PopupBg;
            col[(int)ImGuiCol.Border]         = Border;
            col[(int)ImGuiCol.FrameBg]        = FrameBg;
            col[(int)ImGuiCol.FrameBgHovered] = FrameHovered;
            col[(int)ImGuiCol.FrameBgActive]  = FrameActive;
            col[(int)ImGuiCol.Button]         = Button;
            col[(int)ImGuiCol.ButtonHovered]  = ButtonHover;
            col[(int)ImGuiCol.ButtonActive]   = ButtonActive;

            style.FrameRounding    = 5f;
            style.GrabRounding     = 5f;
            style.WindowBorderSize = 1f;
            style.FrameBorderSize  = 0f;
            style.ItemSpacing      = new ImVec2(7f, 5f);
            style.FramePadding     = new ImVec2(7f, 4f);
        }

        public static uint Col32(byte r, byte g, byte b, byte a)
            => (uint)((a << 24) | (b << 16) | (g << 8) | r);
    }

    /// <summary>Shared widget helpers (rounded label boxes, slider+input pattern).</summary>
    internal static class ImGuiWidgets
    {
        public static bool RoundedLabel(string id, string text, float w, float h, bool activeTint = fals
e)
        {
            ImGui.InvisibleButton(id, new ImVec2(w, h));
            var min = ImGui.GetItemRectMin();
            var max = ImGui.GetItemRectMax();
            var dl = ImGui.GetWindowDrawList();
            dl.AddRectFilled(min, max, ImGui.GetColorU32(SimImGuiStyle.ChildBg), 8f);
            dl.AddRect(min, max, ImGui.GetColorU32(SimImGuiStyle.Border), 8f, ImDrawFlags.None, 1f);
            var ts = ImGui.CalcTextSize(text);
            var tp = new ImVec2(min.X + (w - ts.X) * 0.5f, min.Y + (h - ts.Y) * 0.5f);
            uint tc = activeTint ? SimImGuiStyle.Col32(51, 230, 77, 255)
                                 : ImGui.GetColorU32(ImGuiCol.Text);
            dl.AddText(tp, tc, text);
            return ImGui.IsItemClicked();
        }

        /// <summary>Slider + editable input box (old DrawPIDGainControl pattern).</summary>
        public static bool SliderWithInput(string label, ref float v, float min, float max, string fmt =
 "%.4f")
        {
            float total = ImGui.GetContentRegionAvail().X;
            const float inputW = 80f;
            float sliderW = total > inputW + 20f ? total - inputW - 20f : 100f;

            bool changed = false;
            ImGui.PushItemWidth(sliderW);
            changed |= ImGui.SliderFloat(label, ref v, min, max, fmt);
            ImGui.PopItemWidth();
            ImGui.SameLine();
            ImGui.PushItemWidth(inputW);
            changed |= ImGui.InputFloat("##in_" + label, ref v, 0f, 0f, fmt);
            ImGui.PopItemWidth();
            return changed;
        }
```

```
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/StateHudUI.cs

SHA-256 e884408319da446f3d01414f7dd11a15cd2283b08f422bc4470dbed13eb2d2d9 | 155 lines | 6,712 bytes

```
// Port of the State Data HUD (ImGuiBootstrapSubsystem.cpp lines 458-566 + PID block).
// Pure read path: SensorManager.Latest + CascadedController.Thrusts/TelemetrySnapshot.
using ImGuiNET;
using DroneCore;
using DroneCore.Core;
using UnityEngine;
using ImVec2 = System.Numerics.Vector2;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class StateHudUI
    {
        private bool _open;
        private bool _showPidSettings;
        private readonly PidSettingsUI _pid = new();

        public bool IsOpen => _open;
        public void ToggleOpen() => _open = !_open;
        public void SetVisible(bool v) => _open = v;

        public void Draw(DroneManager dm)
        {
            if (!_open || dm == null || dm.DroneCount == 0) return;
            QuadPawn pawn = dm.SelectedDrone;
            if (pawn == null || pawn.Controller == null) return;
            var ctrl = pawn.Controller;

            ImGui.SetNextWindowPos(new ImVec2(10f, 100f), ImGuiCond.Always);
            ImGui.SetNextWindowSize(new ImVec2(340f, 460f), ImGuiCond.FirstUseEver);
            if (ImGui.Begin("State Data##HUD", ref _open, ImGuiWindowFlags.NoCollapse))
            {
                DrawThrustBars(pawn, ctrl);

                ImGui.Separator();
                ImGui.TextColored(SimImGuiStyle.HeaderBlue, "State Info");
                ImGui.Separator();

                var s = pawn.Sensors != null ? pawn.Sensors.Latest : default;
                float mass = pawn.Rigidbody != null ? pawn.Rigidbody.mass : 0f;
                ImGui.Text($"Drone Mass: {mass:F2} kg");

                var tel = ctrl.TelemetrySnapshot;

                ImGui.TextColored(SimImGuiStyle.HeaderBlue, "IMU");
                ImGui.Text($"Valid: {s.ImuValid}   t={s.ImuTimestampSec:F3}s");

                if (!s.ImuValid)
                {
                    ImGui.TextDisabled("IMU has not produced a valid sample yet.");
                }
                else
                {
                    Vector3 ratesDeg = s.ImuAngVel * Mathf.Rad2Deg;

                    ImGui.Text($"Attitude R/P/Y: {s.ImuAttitude.x:F2} / {s.ImuAttitude.y:F2} / {s.ImuAtt
itude.z:F2} deg");
                    ImGui.Text($"Angular Rates R/P/Y: {ratesDeg.x:F2} / {ratesDeg.y:F2} / {ratesDeg.z:F2
} deg/s");
                    ImGui.Text($"Body Velocity X/Y/Z: {s.ImuVel.x:F2} / {s.ImuVel.y:F2} / {s.ImuVel.z:F2
} m/s");
                    ImGui.Text($"Specific Force X/Y/Z: {s.ImuAccel.x:F2} / {s.ImuAccel.y:F2} / {s.ImuAcc
el.z:F2} m/s2");
                }

                ImGui.Spacing();
                ImGui.Text($"Desired Roll/Pitch/Yaw: {tel.DesiredAnglesDeg.x:F2} / {tel.DesiredAnglesDeg
.y:F2} / {tel.DesiredAnglesDeg.z:F2} deg");
```

```
                ImGui.Text($"Desired Rates R/P/Y: {tel.DesiredRatesDeg.x:F2} / {tel.DesiredRatesDeg.y:F2
} / {tel.DesiredRatesDeg.z:F2} deg/s");
                ImGui.Text($"Desired Velocity: {tel.DesiredVel.x:F2} / {tel.DesiredVel.y:F2} / {tel.Desi
redVel.z:F2} m/s");
                ImGui.Separator();

                ImGui.TextColored(SimImGuiStyle.HeaderYellow, "GPS");
                ImGui.Text($"Position: X {s.GpsPosition.x:F1} Y {s.GpsPosition.y:F1} Z {s.GpsPosition.z:
F1}");
                ImGui.Text($"Geo: Lat {s.GpsLatDeg:F5} Lon {s.GpsLonDeg:F5} Alt {s.GpsAltM:F1}");
                ImGui.Separator();

                ImGui.TextColored(SimImGuiStyle.HeaderYellow, "Magnetometer");
                ImGui.Text($"Field: X {s.MagField.x:F3} Y {s.MagField.y:F3} Z {s.MagField.z:F3} Gauss");
                ImGui.Text($"Heading: {s.MagHeadingDeg:F1} deg   Declination: {s.MagDeclinationDeg:F1} d
eg");
                ImGui.Separator();

                ImGui.TextColored(SimImGuiStyle.HeaderPurple, "Barometer");
                ImGui.Text($"Pressure: {s.BaroPressureHPa:F2} hPa");
                ImGui.Text($"Temperature: {s.BaroTemperatureC:F1} C");
                ImGui.Text($"Altitude: {s.BaroAltitudeMSL:F1} m");

                ImGui.Separator();
                ImGui.Checkbox("Enable PID Settings", ref _showPidSettings);
                if (_showPidSettings)
                    _pid.Draw(pawn, ctrl);
            }
            ImGui.End();

            if (_showPidSettings)
                _pid.DrawHistoryWindow(pawn);
        }

        private void DrawThrustBars(QuadPawn pawn, DroneCore.Controllers.CascadedController ctrl)
        {
            ImGui.Text("Thruster Power");

            var motor01 = ctrl.LastMotor01;
            var thrusts = ctrl.Thrusts;
            string[] labels = { "FL", "FR", "BL", "BR" };

            int count = Mathf.Min(4, Mathf.Min(motor01.Count, thrusts.Count));
            if (count <= 0)
            {
                ImGui.TextDisabled("No motor telemetry yet.");
                return;
            }

            if (ImGui.BeginTable("##ThrustBars", count, ImGuiTableFlags.SizingFixedFit))
            {
                for (int i = 0; i < count; i++)
                {
                    ImGui.TableNextColumn();
                    ImGui.PushID(i);

                    float motor = motor01[i];
                    float thrust = thrusts[i];

                    if (float.IsNaN(motor) || float.IsInfinity(motor)) motor = 0f;
                    if (float.IsNaN(thrust) || float.IsInfinity(thrust)) thrust = 0f;

                    float percent = Mathf.Clamp01(motor);

                    var p = ImGui.GetCursorScreenPos();
                    var barSize = new ImVec2(22f, 100f);
                    var q = new ImVec2(p.X + barSize.X, p.Y + barSize.Y);

                    ImGui.InvisibleButton("##bar", barSize);

                    var dl = ImGui.GetWindowDrawList();
                    dl.AddRect(p, q, SimImGuiStyle.Col32(255, 255, 255, 255));
```

```
                    float fillH = barSize.Y * percent;
                    var fillMin = new ImVec2(p.X, q.Y - fillH);
                    dl.AddRectFilled(fillMin, q, SimImGuiStyle.Col32(255, 200, 0, 255));

                    string n = $"{thrust:F1}N";
                    var ts = ImGui.CalcTextSize(n);
                    var textY = fillH > ts.Y ? fillMin.Y + (fillH - ts.Y) * 0.5f : q.Y - ts.Y - 2f;

                    dl.AddText(
                        new ImVec2(p.X + (barSize.X - ts.X) * 0.5f, textY),
                        SimImGuiStyle.Col32(255, 255, 255, 255),
                        n);

                    ImGui.Text(i < labels.Length ? labels[i] : "M");

                    ImGui.PopID();
                }

                ImGui.EndTable();
            }
        }

    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/TaskbarUI.cs

SHA-256 8f10bafc3c3da3dbc6b0feb9dfa136ec440949569953efbf1ee4e2ac8daea5e1 | 154 lines | 7,654 bytes

```
// Port of the old QuadSim Taskbar (ImGuiBootstrapSubsystem.cpp lines 99-391).
// Full-width, 58px, top-anchored. Sim controls via SimCoreApi ONLY —
// never Unity Time.timeScale (ClockFactory owns time).
using ImGuiNET;
using SimCore;
using UnityEngine;
using DroneCore.Core;
using ImVec2 = System.Numerics.Vector2;
using ImVec4 = System.Numerics.Vector4;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class TaskbarUI
    {
        public const float BarHeight = 58f;

        private float _speedScale = -1f;   // init from sim on first draw
        private bool _simMgrActive, _worldMgrActive;
        private float _fps;

        public System.Action OnSettingsClicked;
        public System.Action OnSimReset;   // lets panels clear stale UI command state

        public void Draw(SimCoreApi sim, DroneManager dm)
        {
            var io = ImGui.GetIO();
            if (_speedScale < 0f)
                _speedScale = Mathf.Max(0.05f, (float)sim.TimeScale);

            float dt = Time.unscaledDeltaTime;
            if (dt > 0f) _fps = Mathf.Lerp(_fps <= 0f ? 1f / dt : _fps, 1f / dt, 0.05f);

            bool paused = sim.IsPaused;                    // sim is the truth, not a local flag
            string pauseTxt = paused ? "Play" : "Pause";

            var flags = ImGuiWindowFlags.NoTitleBar | ImGuiWindowFlags.NoResize |
                        ImGuiWindowFlags.NoMove | ImGuiWindowFlags.NoScrollbar |
                        ImGuiWindowFlags.NoSavedSettings | ImGuiWindowFlags.NoFocusOnAppearing;

            ImGui.SetNextWindowPos(new ImVec2(0f, 0f));
            ImGui.SetNextWindowSize(new ImVec2(io.DisplaySize.X, BarHeight));
            ImGui.PushStyleVar(ImGuiStyleVar.WindowBorderSize, 1.5f);
            ImGui.PushStyleVar(ImGuiStyleVar.WindowRounding, 0f);
            ImGui.PushStyleVar(ImGuiStyleVar.WindowPadding, new ImVec2(10f, 6f));
            ImGui.PushStyleColor(ImGuiCol.Border, new ImVec4(0.06f, 0.07f, 0.09f, 1f));
```

```
if (ImGui.Begin("QuadSim Taskbar##Main", flags))
{
    const float statusW = 200f;
    if (ImGui.BeginTable("##TaskbarTable", 2,
            ImGuiTableFlags.SizingFixedFit | ImGuiTableFlags.NoPadInnerX |
            ImGuiTableFlags.BordersInnerV))
    {
        ImGui.TableSetupColumn("Controls", ImGuiTableColumnFlags.WidthStretch);
        ImGui.TableSetupColumn("Status", ImGuiTableColumnFlags.WidthFixed, statusW);
        ImGui.TableNextRow();

        // ---- Controls column ----
        ImGui.TableSetColumnIndex(0);
        var st = ImGui.GetStyle();
        float padX = st.FramePadding.X, padY = st.FramePadding.Y, itemX = st.ItemSpacing.X;
        const float labelBoxW = 180f, wmLabelW = 170f, speedBoxW = 56f;
        float labelH = BarHeight - 12f;
        float btnH = ImGui.GetTextLineHeight() + padY * 2f;
        float labelTopY = Mathf.Max(0f, (BarHeight - labelH) * 0.5f);
        float btnTopY = labelTopY + Mathf.Max(0f, (labelH - btnH) * 0.5f);

        float BtnW(string t) => ImGui.CalcTextSize(t).X + padX * 2f;

        ImGui.SetCursorPosY(btnTopY);
        ImGui.Dummy(new ImVec2(26f, 0f));   // reserve top-left corner for HUD toggle
        ImGui.SameLine();
        if (ImGui.Button("Settings")) OnSettingsClicked?.Invoke();
        ImGui.SameLine();

        // Center the main control group (excluding left-anchored Settings)
        float totalW = labelBoxW + itemX + BtnW("Slower") + itemX + BtnW(pauseTxt) +
                       itemX + BtnW("Faster>>") + itemX + speedBoxW + itemX +
                       BtnW("Step") + itemX + BtnW("Reset") + itemX + 40f +
                       wmLabelW + itemX + BtnW("Spawn Drone");
        float colStartX = ImGui.GetCursorPosX();
        float avail = ImGui.GetContentRegionAvail().X;
        float settingsW = ImGui.CalcTextSize("Settings").X + padX * 2f + itemX;
        float startXWithin = Mathf.Max(0f, (avail - settingsW - totalW) * 0.5f);
        ImGui.SetCursorPosY(labelTopY);
        ImGui.SetCursorPosX(colStartX + startXWithin);

        if (ImGuiWidgets.RoundedLabel("##SimMgrLabelBtn", "Simulation Manager",
                labelBoxW, labelH, _simMgrActive))
            _simMgrActive = !_simMgrActive;
        ImGui.SameLine();

        ImGui.SetCursorPosY(btnTopY);
        if (ImGui.Button("Slower"))
        {
            _speedScale = Mathf.Clamp(_speedScale - 0.10f, 0.05f, 100f);
            sim.SetTimeScale(_speedScale);
        }
        ImGui.SameLine();
        ImGui.SetCursorPosY(btnTopY);
        if (ImGui.Button(pauseTxt)) sim.TogglePause();
        ImGui.SameLine();
        ImGui.SetCursorPosY(btnTopY);
        if (ImGui.Button("Faster>>"))
        {
            _speedScale = Mathf.Max(1.0f, _speedScale + 0.25f);
            sim.SetTimeScale(_speedScale);
        }
        ImGui.SameLine();

        // Speed display box
        {
            const float boxH = 28f;
            ImGui.SetCursorPosY(btnTopY + (btnH - boxH) * 0.5f);
            ImGuiWidgets.RoundedLabel("##SpeedBox", $"x{_speedScale:F2}", speedBoxW, boxH);
        }
        ImGui.SameLine();
        ImGui.SetCursorPosY(btnTopY);
```

```
                    if (ImGui.Button("Step")) UiActions.Defer(() => sim.StepOnce());
ImGui.SameLine();
                    ImGui.SetCursorPosY(btnTopY);
                    if (ImGui.Button("Reset"))
                    {
                        _speedScale = 1f;
                        sim.SetTimeScale(1.0);   // flag writes — safe immediately
                        UiActions.Defer(() =>
                        {
                            sim.Resume();
                            sim.ResetSimulation();
                            OnSimReset?.Invoke();
                        });
                    }

                    ImGui.SameLine(); ImGui.Dummy(new ImVec2(40f, 0f)); ImGui.SameLine();
                    ImGui.SetCursorPosY(labelTopY);
                    if (ImGuiWidgets.RoundedLabel("##WorldMgrLabelBtn", "World Manager",
                            wmLabelW, labelH, _worldMgrActive))
                        _worldMgrActive = !_worldMgrActive;
                    ImGui.SameLine();
                    ImGui.SetCursorPosY(btnTopY);
                    if (ImGui.Button("Spawn Drone") && dm != null)
                        UiActions.Defer(() => dm.SpawnDrone(Vector3.zero, Quaternion.identity));
                    // ---- Status column ----
                    ImGui.TableSetColumnIndex(1);
                    ImGuiWidgets.RoundedLabel("##FPSBox", $"FPS: {_fps:F1}", statusW, BarHeight - 12f);

                    ImGui.EndTable();
                }
            }
            ImGui.End();
            ImGui.PopStyleColor();
            ImGui.PopStyleVar(3);
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/UiActions.cs

SHA-256 f839948dcf6f6d745988ee361f5e42417dd7a66c732625d338a4a26b6a5a297d | 28 lines | 889 bytes

```
// Deferred main-thread action queue. ImGui layout callbacks run inside the
// render pipeline — Instantiate/Destroy/physics teleports there crash Unity.
// UI code enqueues mutations; ImGuiRuntimeBootstrap drains in Update().
using System;
using System.Collections.Generic;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public static class UiActions
    {
        private static readonly Queue<Action> Pending = new();

        public static void Defer(Action action)
        {
            if (action != null) Pending.Enqueue(action);
        }

        public static void Drain()
        {
            while (Pending.Count > 0)
            {
                var a = Pending.Dequeue();
                try { a(); }
                catch (Exception e) { UnityEngine.Debug.LogError($"[UiActions] Deferred action failed: {
e}"); }
            }
        }
    }
}
```

Runtime/Scripts/RuntimeUI/ImGui/UiCommandState.cs

SHA-256 b9082a696d74d129f6c50915f918867a68c6a96efda762d7419b13547d6533ed | 48 lines | 1,802 bytes

```
// Per-drone UI command state. The UI OWNS these values and streams them each
// rendered frame while UI holds authority (FlightCommandProxy expects a stream —
// stale-command fallback otherwise). Axis4 layouts per SimCore.Common.CommonStructs:
//   Position: (X_m, Y_m, Yaw_deg,     Alt_m)
//   Velocity: (Vx,  Vy,  YawRate_dps, Vz)
//   Angle:    (Roll,Pitch,YawRate_dps,Throttle01)   <- W is THROTTLE, not Z-vel
//   Rate:     (RR,  PR,  YawRate_dps, Throttle01)
using SimCore.Common;

namespace QuadSim.RuntimeUI.ImGuiHud
{
    public sealed class UiCommandState
    {
        // Position mode (output-frame world; Z semantics follow FLU/FRD selection)
        public float PosX, PosY, PosAlt, PosYawDeg;

        // Velocity mode
        public float VelX, VelY, VelZ;

        // Shared across Velocity/Angle/Rate
        public float YawRateDps;
        public float Throttle01;

        // Angle mode
        public float RollDeg, PitchDeg;

        // Rate mode
        public float RollRateDps, PitchRateDps;

        public Axis4 Build(GoalMode mode) => mode switch
        {
            GoalMode.Position => new Axis4(PosX, PosY, PosYawDeg, PosAlt),
            GoalMode.Velocity => new Axis4(VelX, VelY, YawRateDps, VelZ),
            GoalMode.Angle    => new Axis4(RollDeg, PitchDeg, YawRateDps, Throttle01),
            GoalMode.Rate     => new Axis4(RollRateDps, PitchRateDps, YawRateDps, Throttle01),
            _                 => Axis4.ZeroValue,
        };

        public void ZeroAll()
        {
            PosX = PosY = PosAlt = PosYawDeg = 0f;
            VelX = VelY = VelZ = 0f;
            YawRateDps = Throttle01 = 0f;
            RollDeg = PitchDeg = 0f;
            RollRateDps = PitchRateDps = 0f;
        }
    }
}
```

13. Reinforcement Learning and Scenario Objects

Defines per-agent observations and events and manages simple runtime scenario objects. These types are used by the drone and RPC layers for seeded episode resets and multi-agent stepping.

Files in this section

Runtime/Scripts/RL/RLAgentState.cs
Runtime/Scripts/RL/RLSceneObject.cs
Runtime/Scripts/RL/ScenarioObjectManager.cs

Runtime/Scripts/RL/RLAgentState.cs

SHA-256 54fe2be9c394406c6d76ebcdb30cc9202de8502e37c7ee091d916954fedcc7e8 | 89 lines | 3,369 bytes

```
using DroneCore;
using MathUtil;
using UnityEngine;

namespace QuadSim.RL
{
    public readonly struct RLAgentEvent
    {
        public readonly string Type;
        public readonly string AgentId;
        public readonly string OtherAgentId;
        public readonly string ObjectId;
        public readonly double SimTime;

        public RLAgentEvent(
            string type,
            string agentId,
            string otherAgentId,
            double simTime,
            string objectId = null)
        {
            Type = type;
            AgentId = agentId;
            OtherAgentId = otherAgentId;
            SimTime = simTime;
            ObjectId = objectId;
        }
    }

    public struct RLAgentState
    {
        public string AgentId;
        public string TeamId;
        public int DroneIndex;
        public Vector3 PositionEnu;
        public Vector3 VelocityEnu;
        public Quaternion OrientationFluToEnu;
        public Vector3 BodyVelocityFlu;
        public Vector3 BodyAngularVelocityFlu;
        public Vector3 ForwardEnu;
        public Vector3 UpEnu;
        public bool IsValid;
        public bool HasCollision;
        public bool IsGrounded;

        public static RLAgentState Capture(QuadPawn pawn)
        {
            if (pawn == null || pawn.Rigidbody == null)
                return default;

            Rigidbody body = pawn.Rigidbody;
            Transform transform = pawn.transform;
            Vector3 bodyVelocity = transform.InverseTransformDirection(body.linearVelocity);
            Vector3 bodyAngularVelocity = transform.InverseTransformDirection(body.angularVelocity);
```

```
            var state = new RLAgentState
            {
                AgentId = pawn.AgentId,
                TeamId = pawn.TeamId,
                DroneIndex = pawn.DroneIndex,
                PositionEnu = Frames.TransformWorldPosition(body.position, SimFrame.FLU),
                VelocityEnu = Frames.TransformWorldVector(body.linearVelocity, SimFrame.FLU),
                OrientationFluToEnu = Frames.TransformQuaternion(body.rotation, SimFrame.FLU),
                BodyVelocityFlu = Frames.TransformLinear(bodyVelocity, SimFrame.FLU),
                BodyAngularVelocityFlu = Frames.TransformAngularVelocity(bodyAngularVelocity, SimFrame.F
LU),
                ForwardEnu = Frames.TransformWorldVector(transform.right, SimFrame.FLU),
                UpEnu = Frames.TransformWorldVector(transform.up, SimFrame.FLU),
                HasCollision = pawn.HasCollision,
                IsGrounded = pawn.IsGrounded,
            };

            state.IsValid = IsFinite(state.PositionEnu)
                && IsFinite(state.VelocityEnu)
                && IsFinite(state.OrientationFluToEnu)
                && IsFinite(state.BodyVelocityFlu)
                && IsFinite(state.BodyAngularVelocityFlu);
            return state;
        }

        private static bool IsFinite(Vector3 value) =>
            IsFinite(value.x) && IsFinite(value.y) && IsFinite(value.z);

        private static bool IsFinite(Quaternion value) =>
            IsFinite(value.x) && IsFinite(value.y) && IsFinite(value.z) && IsFinite(value.w);

        private static bool IsFinite(float value) =>
            !float.IsNaN(value) && !float.IsInfinity(value);
    }
}
```

Runtime/Scripts/RL/RLSceneObject.cs

SHA-256 6d52f271110a22bb0abb4a029a307cd9d9af05671787c1f44b45a6d810bc373c | 26 lines | 843 bytes

```
using UnityEngine;

namespace QuadSim.RL
{
    [DisallowMultipleComponent]
    public sealed class RLSceneObject : MonoBehaviour
    {
        public string ObjectId { get; private set; }
        public string Shape { get; private set; }
        public bool CollisionEnabled { get; private set; }

        public void Configure(string objectId, string shape, bool collisionEnabled, Color color)
        {
            ObjectId = objectId;
            Shape = shape;
            CollisionEnabled = collisionEnabled;
            name = objectId;

            Collider collider = GetComponent<Collider>();
            if (collider != null) collider.enabled = collisionEnabled;

            Renderer renderer = GetComponent<Renderer>();
            if (renderer != null) renderer.material.color = color;
        }
    }
}
```

Runtime/Scripts/RL/ScenarioObjectManager.cs

SHA-256 eb0e45ad3078dd578479d61047fd55e09ddc1b91a27618f77db435c452689577 | 156 lines | 5,844 bytes

```
using System.Collections.Generic;
using MathUtil;
using UnityEngine;
```

```
namespace QuadSim.RL
{
    public readonly struct ScenarioObjectSpec
    {
        public readonly string ObjectId;
        public readonly string Shape;
        public readonly Vector3 PositionEnu;
        public readonly float YawDeg;
        public readonly Vector3 SizeEnu;
        public readonly bool CollisionEnabled;
        public readonly Color Color;

        public ScenarioObjectSpec(
            string objectId,
            string shape,
            Vector3 positionEnu,
            float yawDeg,
            Vector3 sizeEnu,
            bool collisionEnabled,
            Color color)
        {
            ObjectId = objectId;
            Shape = shape;
            PositionEnu = positionEnu;
            YawDeg = yawDeg;
            SizeEnu = sizeEnu;
            CollisionEnabled = collisionEnabled;
            Color = color;
        }
    }

    [DisallowMultipleComponent]
    public sealed class ScenarioObjectManager : MonoBehaviour
    {
        private static ScenarioObjectManager _instance;
        private readonly Dictionary<string, Stack<RLSceneObject>> _pool = new();
        private readonly List<RLSceneObject> _active = new();
        private readonly Dictionary<string, ScenarioObjectSpec> _specs = new();

        public static ScenarioObjectManager GetOrCreate(GameObject host)
        {
            if (_instance != null) return _instance;
            _instance = FindAnyObjectByType<ScenarioObjectManager>();
            if (_instance == null && host != null)
                _instance = host.AddComponent<ScenarioObjectManager>();
            return _instance;
        }

        private void Awake() => _instance = this;

        private void OnDestroy()
        {
            if (_instance == this) _instance = null;
        }

        public void Apply(IReadOnlyList<ScenarioObjectSpec> specs)
        {
            ReleaseActive();
            _specs.Clear();

            for (int i = 0; i < specs.Count; i++)
            {
                ScenarioObjectSpec spec = specs[i];
                RLSceneObject sceneObject = Acquire(spec.Shape);
                Transform objectTransform = sceneObject.transform;
                objectTransform.SetPositionAndRotation(
                    Frames.InverseTransformWorldPosition(spec.PositionEnu, SimFrame.FLU),
                    Frames.InverseTransformQuaternion(
                        Quaternion.AngleAxis(spec.YawDeg, Vector3.forward), SimFrame.FLU));
                objectTransform.localScale = ScaleForShape(spec.Shape, spec.SizeEnu);
                sceneObject.Configure(
                    spec.ObjectId, spec.Shape, spec.CollisionEnabled, spec.Color);
```

```
            sceneObject.gameObject.SetActive(true);
            _active.Add(sceneObject);
            _specs[spec.ObjectId] = spec;
        }
    }

    public object[] Snapshot()
    {
        object[] result = new object[_active.Count];
        for (int i = 0; i < _active.Count; i++)
        {
            RLSceneObject sceneObject = _active[i];
            ScenarioObjectSpec spec = _specs[sceneObject.ObjectId];
            result[i] = new Dictionary<string, object>
            {
                ["object_id"] = spec.ObjectId,
                ["shape"] = spec.Shape,
                ["position_enu"] = new object[]
                {
                    spec.PositionEnu.x, spec.PositionEnu.y, spec.PositionEnu.z,
                },
                ["yaw_deg"] = spec.YawDeg,
                ["size_enu"] = new object[]
                {
                    spec.SizeEnu.x, spec.SizeEnu.y, spec.SizeEnu.z,
                },
                ["collision_enabled"] = spec.CollisionEnabled,
                ["color_rgba"] = new object[]
                {
                    spec.Color.r, spec.Color.g, spec.Color.b, spec.Color.a,
                },
            };
        }
        return result;
    }

    private RLSceneObject Acquire(string shape)
    {
        if (_pool.TryGetValue(shape, out Stack<RLSceneObject> objects)
            && objects.Count > 0)
            return objects.Pop();

        PrimitiveType primitive = shape switch
        {
            "sphere" => PrimitiveType.Sphere,
            "cylinder" => PrimitiveType.Cylinder,
            "goal" => PrimitiveType.Cylinder,
            _ => PrimitiveType.Cube,
        };
        GameObject gameObject = GameObject.CreatePrimitive(primitive);
        gameObject.transform.SetParent(transform, false);
        return gameObject.AddComponent<RLSceneObject>();
    }

    private void ReleaseActive()
    {
        for (int i = 0; i < _active.Count; i++)
        {
            RLSceneObject sceneObject = _active[i];
            if (sceneObject == null) continue;
            sceneObject.gameObject.SetActive(false);
            if (!_pool.TryGetValue(sceneObject.Shape, out Stack<RLSceneObject> objects))
            {
                objects = new Stack<RLSceneObject>();
                _pool[sceneObject.Shape] = objects;
            }
            objects.Push(sceneObject);
        }
        _active.Clear();
    }

    private static Vector3 ScaleForShape(string shape, Vector3 sizeEnu)
    {
```

```
            Vector3 unitySize = new Vector3(sizeEnu.x, sizeEnu.z, sizeEnu.y);
            if (shape == "cylinder" || shape == "goal")
                unitySize.y *= 0.5f;
            return unitySize;
        }
    }
}
```

14. Experimental Systems

Contains project-authored optional systems for a top-down follower camera, jointed payloads, RPC-driven trajectory visualization, and physics-step wind forces with optional CSV logging.

Files in this section

- Runtime/Scripts/Experimental/DroneFollower.cs
- Runtime/Scripts/Experimental/PayloadAttachment.cs
- Runtime/Scripts/Experimental/TrajectoryVisualizer.cs
- Runtime/Scripts/Experimental/windModule.cs

Runtime/Scripts/Experimental/DroneFollower.cs

SHA-256 145b0d6ef8d14f52f3aafa3c76063bc333a9ff04040711ae4a7876f54351158b | 35 lines | 857 bytes

```
using DroneCore;
using DroneCore.Core;
using UnityEngine;

namespace Experimental
{
    public class DroneFollower : MonoBehaviour
    {
        [SerializeField] private float height = 50f;
        [SerializeField] private Vector3 offset;

        private void LateUpdate()
        {
            DroneManager manager = DroneManager.Get();

            if (manager == null)
                return;

            QuadPawn drone = manager.SelectedDrone;

            if (drone == null)
                return;

            Vector3 dronePos = drone.transform.position;

            transform.position = new Vector3(
                dronePos.x + offset.x,
                height + offset.y,
                dronePos.z + offset.z
            );

            transform.rotation = Quaternion.Euler(90f, 0f, 0f);
        }
    }
}
```

Runtime/Scripts/Experimental/PayloadAttachment.cs

SHA-256 41e16768610da8ea195aaa7abee36d84332d323b533f84e5ef967cfc5370381c | 59 lines | 2,096 bytes

```
namespace Experimental
{
    using UnityEngine;

    [RequireComponent(typeof(Rigidbody))]
    [RequireComponent(typeof(ConfigurableJoint))]
    public sealed class PayloadAttachment : MonoBehaviour
    {
        private Rigidbody _body;
        private ConfigurableJoint _joint;
        [SerializeField] private LineRenderer rope;
        [SerializeField] private Transform ropeEnd;

        private Transform _droneMount;
        private void Awake()
```

```
        {
            _body = GetComponent<Rigidbody>();
            _joint = GetComponent<ConfigurableJoint>();
        }

        public void AttachTo(Rigidbody droneBody, Transform mount)
        {
            if (droneBody == null || mount == null)
            {
                Debug.LogError("[PayloadAttachment] Missing drone Rigidbody or mount.");
                return;
            }
            _droneMount = mount;
            // Position the payload attachment point at the drone mount.
            _joint.connectedBody = null;
            _body.position = mount.position;
            _body.rotation = mount.rotation;

            // Avoid an initial impulse if the drone is already moving.
            _body.linearVelocity = droneBody.GetPointVelocity(mount.position);
            _body.angularVelocity = droneBody.angularVelocity;

            // WaterBottle root origin is the payload-side attachment point.
            _joint.autoConfigureConnectedAnchor = false;
            _joint.anchor = Vector3.zero;

            // Connected anchor is expressed in the drone Rigidbody's local frame.
            _joint.connectedAnchor =
                droneBody.transform.InverseTransformPoint(mount.position);

            _joint.connectedBody = droneBody;
        }

        private void LateUpdate()
        {
            if (rope == null || _droneMount == null || ropeEnd == null)
                return;

            rope.positionCount = 2;
            rope.SetPosition(0, _droneMount.position);
            rope.SetPosition(1, ropeEnd.position);
        }
    }
}
```

Runtime/Scripts/Experimental/TrajectoryVisualizer.cs

SHA-256 1994f38eefffe4cd1b0bc1d92562ad64d3902b4d50804790fe01b9c3e002c824 | 87 lines | 3,631 bytes

```
using System;
using System.Collections.Generic;
using UnityEngine;
using MathUtil;
using SimCore.Rpc;

namespace Experimental
{
    [DisallowMultipleComponent]
    public sealed class TrajectoryVisualizer : MonoBehaviour
    {
        [SerializeField] private ExternalRpcAdapter adapter; // assign in inspector
        [SerializeField] private LineRenderer pathLine;      // static ghost path
        [SerializeField] private Transform desiredGhost;     // moving desired pos
        [SerializeField] private LineRenderer velVector;     // optional (null ok)
        [SerializeField] private LineRenderer thrustVector;  // optional (null ok)
        [SerializeField] private float vectorScale = 0.05f;

        private void Start()
        {
            if (adapter == null)
            {
                Debug.LogError("[Viz] No ExternalRpcAdapter assigned.");
                enabled = false;
                return;
```

```
            }
            adapter.RegisterHandler("draw_trajectory", HandleDrawTrajectory);
            adapter.OnVizPayload += UpdateDesired;
        }

        private void OnDestroy()
        {
            if (adapter != null) adapter.OnVizPayload -= UpdateDesired;
        }

        // One-shot during setup: full path as FLU points.
        private RpcResponse HandleDrawTrajectory(RpcRequest req)
        {
            if (!req.Params.TryGetValue("points", out var raw) || raw is not object[] pts)
                return RpcResponse.Error("invalid_param: points", req.RequestId);

            var positions = new Vector3[pts.Length];
            for (int i = 0; i < pts.Length; i++)
                positions[i] = FluWorldToUnity(ToVec3(pts[i]));

            pathLine.positionCount = positions.Length;
            pathLine.SetPositions(positions);
            return RpcResponse.Ok(req.RequestId);
        }

        // Per-tick, fired off the step_with_command round trip.
        private void UpdateDesired(Dictionary<string, object> p)
        {
            if (!p.TryGetValue("viz_pos", out var posRaw)) return;  // non-viz run

            Vector3 pos = FluWorldToUnity(ToVec3(posRaw));
            if (desiredGhost != null) desiredGhost.position = pos;

            if (velVector != null && p.TryGetValue("viz_vel", out var v))
                DrawArrow(velVector, pos, FluWorldToUnity(ToVec3(v)));
            if (thrustVector != null && p.TryGetValue("viz_bz", out var bz))
                DrawArrow(thrustVector, pos, FluWorldToUnity(ToVec3(bz)));
        }

        private void DrawArrow(LineRenderer lr, Vector3 origin, Vector3 dirUnity)
        {
            lr.positionCount = 2;
            lr.SetPosition(0, origin);
            lr.SetPosition(1, origin + dirUnity * vectorScale);
        }

        // Single frame boundary. Positions AND world-direction vectors both use
        // the pure FLU->Unity world relabel (x,y,z)->(x,z,y) from Frames.cs.
        private static Vector3 FluWorldToUnity(Vector3 flu)
            => Frames.InverseTransformWorldPosition(flu, SimFrame.FLU);

        // Robust to MessagePack boxing (object[] or IList, any numeric type).
        private static Vector3 ToVec3(object o)
        {
            if (o is object[] a && a.Length >= 3)
                return new Vector3(Convert.ToSingle(a[0]), Convert.ToSingle(a[1]), Convert.ToSingle(a[2]
));
            if (o is System.Collections.IList l && l.Count >= 3)
                return new Vector3(Convert.ToSingle(l[0]), Convert.ToSingle(l[1]), Convert.ToSingle(l[2]
));
            return Vector3.zero;
        }
    }
}
```

Runtime/Scripts/Experimental/windModule.cs

SHA-256 3afd686619250ee330b070fc289f7299aac3339d54080c3f6eddfc9d96e8d1cd | 412 lines | 17,124 bytes

```
using System.Collections.Generic;
using System.IO;
using System.Text;
using UnityEngine;
using SimCore;
```

```
namespace Experimental
{
    /// <summary>
    /// A wind/fan volume. Applies velocity-relative drag toward the zone's air
    /// velocity to any non-kinematic Rigidbody inside it, exactly ONCE per
    /// rigidbody per physics step (OnTriggerStay fires once per collider, so
    /// multi-collider rigs would otherwise multiply the force).
    ///
    /// All timing is sim-time via ClockFactory — never UnityEngine.Time.
    /// Can optionally log applied force per step to CSV for ground-truth overlay
    /// against the Python-side d_est / phi. This is disabled by default because
    /// per-step string formatting and disk I/O are expensive at simulation rates.
    /// </summary>
    [RequireComponent(typeof(BoxCollider))]
    public class WindModule : MonoBehaviour
    {
        [Header("Base Wind Settings")]
        [Tooltip("Fan jet air speed at the source face, m/s. Blow direction is local forward (blue Z axi
s).")]
        [SerializeField] private float _windSpeed = 10f;

        [Tooltip("Linear drag, N per (m/s) of relative air speed.")]
        [SerializeField] private float _dragCoefficient = 0.5f;

        [Header("Spreading & Cone Settings")]
        [Range(0f, 2f)] [SerializeField] private float _spreadFactor = 0f;

        [Header("Turbulence Settings")]
        [SerializeField] private float _turbulenceStrength = 2f;
        [SerializeField] private float _turbulenceFrequency = 5f;

        [SerializeField] private bool _toggle = false;

        [Header("Diagnostics")]
        [Tooltip("Write per-step applied-force CSV to <project>/runs/wind_log_*.csv. Expensive; enable o
nly for diagnostics.")]
        [SerializeField] private bool _logForces = false;

        private BoxCollider _boxCollider;

        // --- per-step dedupe + logging state ---
        private ulong _lastStepCount = ulong.MaxValue;
        private readonly Dictionary<Rigidbody, PendingRow> _pendingRows = new();
        private StreamWriter _log;

        private struct PendingRow
        {
            public double SimTime;
            public ulong Step;
            public string Name;
            public Vector3 Pos;
            public float TravelRatio;
            public float WindScale;
            public Vector3 AirVel;
            public Vector3 RelVel;
            public Vector3 Force;
            public int CallbackCount;   // how many trigger callbacks hit this rb this step
        }

        private void Awake()
        {
            _boxCollider = GetComponent<BoxCollider>();

            // ---- Debug logging for exact dimensions and center/origin ----
            Vector3 worldSize = Vector3.Scale(_boxCollider.size, transform.lossyScale);
            Vector3 worldCenter = transform.TransformPoint(_boxCollider.center);

            Debug.Log($"[WindModule] Awake - Windzone Initialized.\n" +
                      $"Dimensions (World Size): X:{worldSize.x:F3}, Y:{worldSize.y:F3}, Z:{worldSize.z:
F3}\n" +
                      $"Origin/Center (World Position): {worldCenter}\n" +
                      $"Collider Center (Local Offset): {_boxCollider.center}");
```

```
    if (_logForces)
    {
        string dir = Path.Combine(Application.dataPath, "../runs");
        Directory.CreateDirectory(dir);
        string path = Path.Combine(dir,
            $"wind_log_{System.DateTime.Now:yyyyMMdd_HHmmss}.csv");
        _log = new StreamWriter(path, false, Encoding.ASCII);
        _log.WriteLine("step,sim_t,rb,px,py,pz,travel_ratio,wind_scale," +
                       "air_vx,air_vy,air_vz,rel_vx,rel_vy,rel_vz," +
                       "fx,fy,fz,f_mag,callbacks");
        Debug.Log($"[WindModule] Logging applied forces -> {path}");
    }
}

private void Reset()
{
    var col = GetComponent<BoxCollider>();
    if (col != null) col.isTrigger = true;
}

private void OnDestroy()
{
    FlushPendingRows();
    _log?.Flush();
    _log?.Dispose();
}
// ============================================================================
// Runtime registry + control API
//   Self-registering so the RPC layer never scans the scene. _toggle is the
//   force gate (see OnTriggerStay); SetEnabled flips it live so one build can
//   serve the wind-OFF gain study and the wind-ON net study without a rebuild.
// ============================================================================

private static readonly List<WindModule> _active = new();
public static IReadOnlyList<WindModule> Active => _active;

private void OnEnable()  { if (!_active.Contains(this)) _active.Add(this); }
private void OnDisable() { _active.Remove(this); }

public bool  IsEnabled => _toggle;
public float WindSpeed => _windSpeed;

/// <summary>Turn the disturbance force on/off at runtime.</summary>
public void SetEnabled(bool on) => _toggle = on;

/// <summary>Override fan jet speed (m/s) at runtime. Clamped non-negative.</summary>
public void SetWindSpeed(float metersPerSecond) => _windSpeed = Mathf.Max(0f, metersPerSecond);
private void OnTriggerStay(Collider other)
{
    var rb = other.attachedRigidbody;
    if (rb == null || rb.isKinematic) return;

    // ---- per-step dedupe ----
    ulong step = ClockFactory.Clock.StepCount;
    if (step != _lastStepCount)
    {
        FlushPendingRows();          // write last step's rows (with final callback counts)
        _lastStepCount = step;
    }

    // With both force and diagnostics disabled there is nothing to do.
    // Avoid geometry, turbulence, allocations, and dictionary work for
    // every trigger callback while the wind zone is inactive.
    if (!_toggle && _log == null) return;

    if (_pendingRows.TryGetValue(rb, out var existing))
    {
        // Force already applied to this rb this step — just count the duplicate.
        existing.CallbackCount++;
        _pendingRows[rb] = existing;
        return;
    }
```

```
        // ---- geometry (world-scale aware, collider-center aware) ----
        Vector3 worldSize = Vector3.Scale(_boxCollider.size, transform.lossyScale);
        float maxRange = Mathf.Max(worldSize.z, 1e-4f);

        Vector3 boxCenter = transform.TransformPoint(_boxCollider.center);
        Vector3 sourceFacePosition = boxCenter - transform.forward * (maxRange * 0.5f);

        Vector3 toTarget = rb.position - sourceFacePosition;
        float distanceAlongFlow = Mathf.Clamp(
            Vector3.Dot(toTarget, transform.forward), 0f, maxRange);
        float travelRatio = distanceAlongFlow / maxRange;

        float distanceFalloff = 1f - travelRatio;

        Vector3 centralProjectedPoint =
            sourceFacePosition + transform.forward * distanceAlongFlow;
        float lateralDistance = Vector3.Distance(rb.position, centralProjectedPoint);
        float spreadEdgeLoss =
            Mathf.Clamp01(1f - lateralDistance * _spreadFactor * travelRatio);

        float finalWindScale = distanceFalloff * spreadEdgeLoss;
        Vector3 airVelocity = transform.forward * (_windSpeed * finalWindScale);

        // ---- turbulence: SIM time, not wall time ----
        if (_turbulenceStrength > 0f && travelRatio > 0.05f)
        {
            float simT = (float)(ClockFactory.Clock.NowNanos / 1e9);
            float timeOffset = simT * _turbulenceFrequency;

            float noiseX = Mathf.PerlinNoise(rb.position.x + timeOffset, rb.position.y);
            float noiseY = Mathf.PerlinNoise(rb.position.y + timeOffset, rb.position.z);
            float noiseZ = Mathf.PerlinNoise(rb.position.z + timeOffset, rb.position.x);

            Vector3 turbDir = new Vector3(
                noiseX * 2f - 1f, noiseY * 2f - 1f, noiseZ * 2f - 1f).normalized;

            airVelocity += turbDir * (_turbulenceStrength * travelRatio * spreadEdgeLoss);
        }

        // ---- apply (once per rb per step) ----
        Vector3 relVel = airVelocity - rb.linearVelocity;
        Vector3 force = _toggle ? _dragCoefficient * relVel : Vector3.zero;

        if (_toggle)
            rb.AddForce(force, ForceMode.Force);

        // ---- record row ----
        if (_log != null)
        {
            _pendingRows[rb] = new PendingRow
            {
                SimTime = ClockFactory.Clock.NowNanos / 1e9,
                Step = step,
                Name = rb.name,
                Pos = rb.position,
                TravelRatio = travelRatio,
                WindScale = finalWindScale,
                AirVel = airVelocity,
                RelVel = relVel,
                Force = force,
                CallbackCount = 1,
            };
        }
        else
        {
            // still need the dedupe key even when not logging
            _pendingRows[rb] = default;
        }
    }

    private void FlushPendingRows()
    {
```

```
            if (_log != null)
            {
                foreach (var kv in _pendingRows)
                {
                    var r = kv.Value;
                    _log.WriteLine(
                        $"{r.Step},{r.SimTime:F4},{r.Name}," +
                        $"{r.Pos.x:F4},{r.Pos.y:F4},{r.Pos.z:F4}," +
                        $"{r.TravelRatio:F4},{r.WindScale:F4}," +
                        $"{r.AirVel.x:F4},{r.AirVel.y:F4},{r.AirVel.z:F4}," +
                        $"{r.RelVel.x:F4},{r.RelVel.y:F4},{r.RelVel.z:F4}," +
                        $"{r.Force.x:F4},{r.Force.y:F4},{r.Force.z:F4}," +
                        $"{r.Force.magnitude:F4},{r.CallbackCount}");
                }
            }
            _pendingRows.Clear();
        }

        // ---- Always draw the windzone outline ----
        private void OnDrawGizmos()
        {
            if (_boxCollider == null) _boxCollider = GetComponent<BoxCollider>();
            if (_boxCollider == null) return;

            // Save the original matrix and color to restore later
            Matrix4x4 oldMatrix = Gizmos.matrix;
            Color oldColor = Gizmos.color;

            // Align the Gizmo drawing space with the Transform's local space (handles rotation/scale)
            Gizmos.matrix = transform.localToWorldMatrix;

            // Draw the box outline (semi-transparent green)
            Gizmos.color = new Color(0f, 1f, 0f, 0.4f);
            Gizmos.DrawWireCube(_boxCollider.center, _boxCollider.size);

            // Restore previous Gizmo states
            Gizmos.matrix = oldMatrix;
            Gizmos.color = oldColor;
        }

        private void OnDrawGizmosSelected()
        {
            if (_boxCollider == null) _boxCollider = GetComponent<BoxCollider>();
            if (_boxCollider == null) return;

            Vector3 scale = transform.lossyScale;
            Vector3 worldSize = Vector3.Scale(
                _boxCollider.size,
                new Vector3(Mathf.Abs(scale.x), Mathf.Abs(scale.y), Mathf.Abs(scale.z))
            );

            float length = Mathf.Max(worldSize.z, 0.01f);
            float width  = Mathf.Max(worldSize.x, 0.01f);
            float height = Mathf.Max(worldSize.y, 0.01f);

            Vector3 forward = transform.forward.normalized;
            Vector3 right   = transform.right.normalized;
            Vector3 up      = transform.up.normalized;

            Vector3 center = transform.TransformPoint(_boxCollider.center);
            Vector3 sourceFace = center - forward * (length * 0.5f);
            Vector3 exitFace   = center + forward * (length * 0.5f);

            Color activeColor = _toggle ? Color.cyan : new Color(0.55f, 0.55f, 0.55f, 1f);
            Color softColor   = _toggle ? new Color(0f, 1f, 1f, 0.30f) : new Color(0.55f, 0.55f, 0.55f,
0.25f);

            // Source and exit faces, so you can immediately see where wind starts/ends.
            Gizmos.color = new Color(0f, 0.7f, 1f, 0.75f);
            DrawFaceRect(sourceFace, right, up, width, height);

            Gizmos.color = softColor;
            DrawFaceRect(exitFace, right, up, width, height);
```

```
        // Big center arrow: actual wind direction.
        Gizmos.color = activeColor;
        DrawArrowGizmo(
            sourceFace,
            forward,
            length,
            Mathf.Clamp(length * 0.18f, 0.25f, 1.25f),
            Mathf.Clamp(Mathf.Min(width, height) * 0.18f, 0.15f, 0.75f)
        );

        // Smaller streamlines across the fan face. These make it read as flow, not a single axis.
        float xOffset = width * 0.25f;
        float yOffset = height * 0.25f;
        float streamlineLength = length * 0.72f;
        float streamlineHeadLen = Mathf.Clamp(length * 0.10f, 0.15f, 0.65f);
        float streamlineHeadWidth = Mathf.Clamp(Mathf.Min(width, height) * 0.08f, 0.08f, 0.30f);

        Vector3[] offsets =
        {
            right * xOffset,
            -right * xOffset,
            up * yOffset,
            -up * yOffset,
            right * xOffset + up * yOffset,
            right * xOffset - up * yOffset,
            -right * xOffset + up * yOffset,
            -right * xOffset - up * yOffset,
        };

        Gizmos.color = softColor;
        foreach (Vector3 offset in offsets)
        {
            DrawArrowGizmo(
                sourceFace + offset,
                forward,
                streamlineLength,
                streamlineHeadLen,
                streamlineHeadWidth
            );
        }

        // Show the spread cone/edge loss preview at the exit face.
        float spreadRadius = Mathf.Max(0.05f, _spreadFactor * length * 0.5f);
        Gizmos.color = new Color(0f, 1f, 1f, 0.25f);
        Gizmos.DrawWireSphere(exitFace, spreadRadius);

    #if UNITY_EDITOR
        UnityEditor.Handles.color = activeColor;
        UnityEditor.Handles.Label(
            exitFace + up * (height * 0.35f),
            $"Wind → {forward}\nSpeed: {_windSpeed:F1} m/s\n{(_toggle ? "ENABLED" : "DISABLED")}"
        );
    #endif
    }

    private static void DrawArrowGizmo(
        Vector3 start,
        Vector3 direction,
        float length,
        float headLength,
        float headWidth)
    {
        if (direction.sqrMagnitude < 1e-6f) return;

        direction.Normalize();

        Vector3 end = start + direction * length;
        Gizmos.DrawLine(start, end);

        // Build arrowhead basis.
        Vector3 side = Vector3.Cross(direction, Vector3.up);
        if (side.sqrMagnitude < 1e-6f)
```

```
                side = Vector3.Cross(direction, Vector3.right);

            side.Normalize();
            Vector3 up = Vector3.Cross(side, direction).normalized;

            Vector3 baseCenter = end - direction * headLength;

            // Four-sided pyramid arrowhead.
            Vector3 p1 = baseCenter + side * headWidth;
            Vector3 p2 = baseCenter - side * headWidth;
            Vector3 p3 = baseCenter + up * headWidth;
            Vector3 p4 = baseCenter - up * headWidth;

            Gizmos.DrawLine(end, p1);
            Gizmos.DrawLine(end, p2);
            Gizmos.DrawLine(end, p3);
            Gizmos.DrawLine(end, p4);

            Gizmos.DrawLine(p1, p3);
            Gizmos.DrawLine(p3, p2);
            Gizmos.DrawLine(p2, p4);
            Gizmos.DrawLine(p4, p1);
        }

        private static void DrawFaceRect(
            Vector3 center,
            Vector3 right,
            Vector3 up,
            float width,
            float height)
        {
            Vector3 r = right.normalized * (width * 0.5f);
            Vector3 u = up.normalized * (height * 0.5f);

            Vector3 a = center - r - u;
            Vector3 b = center + r - u;
            Vector3 c = center + r + u;
            Vector3 d = center - r + u;

            Gizmos.DrawLine(a, b);
            Gizmos.DrawLine(b, c);
            Gizmos.DrawLine(c, d);
            Gizmos.DrawLine(d, a);
        }
    }
}
```

15. Unity Editor Tools

Provides editor-only inspection and repair tools for drone YAML, rotor geometry, and missing component references. These tools operate on the same configuration and allocator code used by the runtime.

Files in this section

Editor/DroneYamlInspectorWindow.cs
Editor/FindMissingScripts.cs
Editor/Inspectors/RotorGeometryGizmoEditor.cs

Editor/DroneYamlInspectorWindow.cs

SHA-256 9e25ccd87331fb8586622486719c391182b4c6f5bada538a6138ebab48bddf47 | 137 lines | 6,362 bytes

```
using DroneCore.Controls;
using MathUtil;
using UnityEditor;
using UnityEngine;
using Yaml.Drone;

namespace Nino.QuadSim.Editor
{
    public sealed class DroneYamlInspectorWindow : EditorWindow
    {
        private TextAsset yamlAsset;
        private ModelType model;
        private DroneParameters physical;
        private RotorModelInstance propulsion;
        private RotorPhysicsDerived rotorPhysics;
        private ControlAllocator allocator;
        private string sourcePath;
        private string error;

        [MenuItem("Tools/QuadSim/Drone YAML Inspector")]
        public static void Open() => GetWindow<DroneYamlInspectorWindow>("Drone YAML");

        private void OnGUI()
        {
            EditorGUI.BeginChangeCheck();
            yamlAsset = (TextAsset)EditorGUILayout.ObjectField(
                "Drone YAML", yamlAsset, typeof(TextAsset), false);
            if (EditorGUI.EndChangeCheck()) Load();

            if (yamlAsset == null)
            {
                EditorGUILayout.HelpBox(
                    "Select a drone YAML from StreamingAssets or the package Resources folder.",
                    MessageType.Info);
                return;
            }
            if (!string.IsNullOrEmpty(error))
            {
                EditorGUILayout.HelpBox(error, MessageType.Error);
                return;
            }

            EditorGUILayout.LabelField("Source", sourcePath);
            EditorGUILayout.LabelField("Schema", model.SchemaVersion.ToString());
            EditorGUILayout.LabelField("Name", model.Name);
            EditorGUILayout.LabelField("Mass", $"{physical.Mass:F4} kg");
            EditorGUILayout.LabelField("Inertia", $"{physical.Inertia} kg m²");
            EditorGUILayout.Space();
            EditorGUILayout.LabelField("Derived plant", EditorStyles.boldLabel);
            EditorGUILayout.LabelField("Max thrust / rotor", $"{rotorPhysics.MaxThrust:F5} N");
            EditorGUILayout.LabelField("Max yaw torque / rotor", $"{rotorPhysics.MaxTorque:F6} N m");
            float hover = rotorPhysics.MaxThrust > 0f
                ? physical.Mass * 9.81f / (propulsion.Rotors.Length * rotorPhysics.MaxThrust)
                : 0f;
```

```
            EditorGUILayout.LabelField("Hover command", hover.ToString("F4"));
            EditorGUILayout.LabelField("Thrust-to-weight", hover > 0f ? (1f / hover).ToString("F3") : "n
/a");
            EditorGUILayout.LabelField("Effectiveness rank", allocator.Rank.ToString());
            EditorGUILayout.LabelField("Condition estimate", allocator.ConditionNumber.ToString("G6"));

            DrawLayout();
            DrawEffectiveness();
        }

        private void Load()
        {
            error = null;
            if (yamlAsset == null) return;
            sourcePath = AssetDatabase.GetAssetPath(yamlAsset);
            if (!YamlParser.LoadDroneConfigFromString(
                    yamlAsset.text, sourcePath, out model, out physical,
                    out propulsion, out _, out _, out _, out _, out _, out _))
            {
                error = "The selected YAML did not pass the Unity runtime parser.";
                return;
            }
            if (propulsion.Rotors == null || propulsion.Rotors.Length != 4)
            {
                error = "The selected YAML must define exactly four rotors.";
                return;
            }
            rotorPhysics = new RotorPhysicsDerived();
            rotorPhysics.ComputeRotor(propulsion.Model, physical.Arm_Length);
            var rebased = new RotorInstance[propulsion.Rotors.Length];
            for (int index = 0; index < propulsion.Rotors.Length; index++)
            {
                rebased[index] = propulsion.Rotors[index];
                rebased[index].Location_Body -= physical.CenterOfMass;
            }
            allocator = new ControlAllocator();
            allocator.RebuildEffectiveness(rebased, rotorPhysics, SimFrame.FLU);
        }

        private void DrawLayout()
        {
            EditorGUILayout.Space();
            EditorGUILayout.LabelField("Rotor layout (+x forward, +z left)", EditorStyles.boldLabel);
            Rect rect = GUILayoutUtility.GetRect(240f, 240f, GUILayout.ExpandWidth(true));
            EditorGUI.DrawRect(rect, new Color(0.12f, 0.12f, 0.12f));
            Vector2 center = rect.center;
            float radius = 0f;
            foreach (RotorInstance rotor in propulsion.Rotors)
            {
                Vector3 relative = rotor.Location_Body - physical.CenterOfMass;
                radius = Mathf.Max(radius, Mathf.Abs(relative.x), Mathf.Abs(relative.z));
            }
            float scale = radius > 1e-6f ? 0.4f * Mathf.Min(rect.width, rect.height) / radius : 1f;
            EditorGUI.DrawRect(new Rect(center.x - 1f, rect.y, 2f, rect.height), Color.gray);
            EditorGUI.DrawRect(new Rect(rect.x, center.y - 1f, rect.width, 2f), Color.gray);
            foreach (RotorInstance rotor in propulsion.Rotors)
            {
                Vector3 relative = rotor.Location_Body - physical.CenterOfMass;
                Vector2 point = center + new Vector2(relative.z, -relative.x) * scale;
                Rect marker = new Rect(point.x - 12f, point.y - 12f, 24f, 24f);
                EditorGUI.DrawRect(marker, rotor.Spin_Dir > 0
                    ? new Color(0.2f, 0.65f, 1f) : new Color(1f, 0.45f, 0.2f));
                GUI.Label(marker, rotor.ID.ToString(), EditorStyles.centeredGreyMiniLabel);
            }
        }

        private void DrawEffectiveness()
        {
            EditorGUILayout.Space();
            EditorGUILayout.LabelField("Effectiveness [roll, pitch, yaw, thrust]", EditorStyles.boldLabe
l);
            int[] rows = { 0, 1, 2, 5 };
            foreach (int row in rows)
```

```
            {
                using (new EditorGUILayout.HorizontalScope())
                {
                    for (int motor = 0; motor < allocator.NumMotors; motor++)
                        EditorGUILayout.SelectableLabel(
                            allocator.Effectiveness.Data[row, motor].ToString("G5"),
                            GUILayout.Height(EditorGUIUtility.singleLineHeight));
                }
            }
        }
    }
}
```

Editor/FindMissingScripts.cs

SHA-256 4171f8bbe2e825e06f2edf1c6d3cee7e6cbbc345fa7e7c9cb8fd979379a7c439 | 40 lines | 1,245 bytes

```
using UnityEditor;
using UnityEngine;

namespace Nino.QuadSim.Editor
{
    public static class FindMissingScripts
    {
        [MenuItem("Tools/QuadSim/Find Missing Scripts In Open Scenes")]
        private static void FindInOpenScenes()
        {
            int total = 0;
            foreach (var go in Object.FindObjectsByType<GameObject>(FindObjectsInactive.Include))
            {
                var components = go.GetComponents<Component>();
                for (int i = 0; i < components.Length; i++)
                {
                    if (components[i] == null)
                    {
                        total++;
                        Debug.LogWarning($"Missing script on GameObject: {GetPath(go)}", go);
                    }
                }
            }

            Debug.Log($"FindMissingScripts: Found {total} missing scripts in open scenes.");
        }

        private static string GetPath(GameObject go)
        {
            string path = go.name;
            Transform t = go.transform;
            while (t.parent != null)
            {
                t = t.parent;
                path = t.name + "/" + path;
            }
            return path;
        }
    }
}
```

Editor/Inspectors/RotorGeometryGizmoEditor.cs

SHA-256 9fc1694e304bab8468c05f800fbe7d16690e78ea42ad27107dcb1db19f3eb63c | 111 lines | 4,326 bytes

```
using DroneCore;
using UnityEditor;
using UnityEngine;
using Yaml.Drone;

namespace QuadSim.Scripts.Editor
{
    [CustomEditor(typeof(RotorGeometryGizmo))]
    public sealed class RotorGeometryGizmoEditor : UnityEditor.Editor
    {
        private static readonly string[] MountFields = { "motorFL", "motorFR", "motorBL", "motorBR" };
        private static readonly string[] MountNames  = { "MotorFL", "MotorFR", "MotorBL", "MotorBR" };
```

```
        public override void OnInspectorGUI()
        {
            DrawDefaultInspector();
            var gz = (RotorGeometryGizmo)target;

            EditorGUILayout.Space();

            if (GUILayout.Button("Align Motor Mounts to Config"))
                Align(gz, apply: true);

            if (GUILayout.Button("Report Only (no changes)"))
                Align(gz, apply: false);
        }

        private static void Align(RotorGeometryGizmo gz, bool apply)
        {
            if (!DroneConfigLoader.LoadConfig(gz.previewConfigName, out DroneConfig cfg))
            {
                Debug.LogError($"[Aligner] Could not load '{gz.previewConfigName}'.");
                return;
            }

            var rotors = cfg.RotorParams.Rotors;
            if (rotors == null || rotors.Length != 4)
            {
                Debug.LogError("[Aligner] Expected exactly 4 rotors.");
                return;
            }

            Transform root = gz.transform;

            // Classify by quadrant, NOT by array index — YAML id order and the
            // FL/FR/BL/BR mount order are not guaranteed to agree.
            int[] slot = new int[4]; for (int i = 0; i < 4; i++) slot[i] = -1;
            for (int i = 0; i < rotors.Length; i++)
            {
                Vector3 p = rotors[i].Location_Body;
                int idx = (p.x >= 0f)
                    ? (p.z >= 0f ? 0 : 1)     // FL, FR
                    : (p.z >= 0f ? 2 : 3);    // BL, BR


                if (slot[idx] != -1)
                {
                    Debug.LogError($"[Aligner] Two rotors in the same quadrant " +
                                   $"({MountNames[idx]}): ids {rotors[slot[idx]].ID} and {rotors[i].ID}.
" +
                                   $"Fix the config before aligning.");
                    return;
                }
                slot[idx] = i;

                if (rotors[i].ID != idx)
                    Debug.LogWarning($"[Aligner] YAML id {rotors[i].ID} sits in the " +
                                     $"{MountNames[idx]} quadrant. Index order and mount order " +
                                     $"disagree — harmless for the allocator, confusing for humans.");
            }


            var pawn = gz.GetComponent<QuadPawn>();
            if (pawn == null) { Debug.LogError("[Aligner] No QuadPawn on this object."); return; }
            var so = new SerializedObject(pawn);

            for (int m = 0; m < 4; m++)
            {
                var prop = so.FindProperty(MountFields[m]);
                var t = prop?.objectReferenceValue as Transform;

                if (t == null)
                {
                    Debug.LogWarning($"[Aligner] QuadPawn.{MountFields[m]} is unassigned.");
                    continue;
                }
```

```
                if (t.GetComponentInChildren<Renderer>() != null)
                {
                    Debug.LogError($"[Aligner] {MountFields[m]} points at '{t.name}', which renders. " +
                                   "Mounts must be empty markers — aligning would move the mesh. Skipped
."");
                    continue;
                }

                Vector3 target = rotors[slot[m]].Location_Body;
                float delta = (t.localPosition - target).magnitude;

                Debug.Log($"[Aligner] {MountNames[m]} <- rotor id{rotors[slot[m]].ID} " +
                          $"{target:F4} (was {t.localPosition:F4}, delta {delta * 1000f:F1} mm)");

                if (apply && delta > 1e-5f)
                {
                    Undo.RecordObject(t, "Align motor mount");
                    t.localPosition = target;
                    EditorUtility.SetDirty(t);
                }
            }


        }
    }
}
```

16. Examples and External Tools

Demonstrates the internal C# APIs and supplies standalone Python and shell utilities for headless visualization and PX4 offboard validation. These tools consume QuadSim interfaces but are not part of the separately distributed Python SDK implementation.

Files in this section

Samples~/QuadSim Starter/Scripts/Internalapiexample.cs
Documentation~/headless-tools/live_viewer.py
Samples~/PX4 SITL Tools/test_offboard_figure8_3d.py
Samples~/PX4 SITL Tools/scripts/run_bridge.sh
Samples~/PX4 SITL Tools/scripts/run_confirm.sh
Samples~/PX4 SITL Tools/scripts/run_figure8.sh
Samples~/PX4 SITL Tools/scripts/run_viewer.sh
Samples~/PX4 SITL Tools/requirements.txt
Samples~/PX4 SITL Tools/PX4_VERSION.md

Samples~/QuadSim Starter/Scripts/Internalapiexample.cs

SHA-256 b1013f16138c1492cddd00f8ce9eb82ed3edf2d1fdee8c55b713a3f33add1a4e | 326 lines | 12,028 bytes

```
// Assets/Scripts/Examples/InternalApiExample.cs
//
// EXAMPLE: Comprehensive Internal API usage via SimCoreApi + QuadSimApi
//
// ACCESS PATTERN:
//   SimCoreApi.Get()       → sim-wide: time control, authority
//   QuadSimApi.Selected()  → per-drone: mode, commands, sensors, telemetry, resets
//   QuadSimApi.Get(i)      → per-drone by index (multi-drone scenarios)
//
// HOW TO USE:
//   1. Drop this script onto any GameObject in your scene (NOT on the drone prefab)
//   2. Set ControlAuthorityManager default source to Internal (or leave as UI — script requests authori
ty)
//   3. Hit Play — the script exercises the full API surface
//
// WHAT THIS DEMONSTRATES:
//   - SimCoreApi: authority lifecycle, time control, sim status
//   - QuadSimApi: SetMode, SendCommand, GetSensorData, GetTelemetry, resets
//   - Static accessors: QuadSimApi.Selected(), QuadSimApi.DroneCount
//   - Event subscriptions for mode/authority changes
//   - Coroutine init pattern for Unity timing
//   - Proper cleanup on destroy

using System.Collections;
using UnityEngine;
using SimCore;
using SimCore.Common;
using DroneCore.Api;

public class InternalApiExample : MonoBehaviour
{
    // ═══════════════════════════════════════════════════════════════════════
    // Configuration (tweak in Inspector)
    // ═══════════════════════════════════════════════════════════════════════

    [Header("Rate Mode Commands")]
    [SerializeField] private float rollRate = 0f;
    [SerializeField] private float pitchRate = 0f;
    [SerializeField] private float yawRate = 30f;
    [SerializeField] private float throttle = 0.41f;

    [Header("Velocity Mode Commands")]
    [SerializeField] private float forwardVel = 1.0f;
    [SerializeField] private float lateralVel = 0f;
    [SerializeField] private float velYawRate = 0f;
```

```
    [SerializeField] private float verticalVel = 0f;

    [Header("Demo Timing")]
    [SerializeField] private float rateModeDuration = 5f;
    [SerializeField] private float velocityModeDuration = 5f;

    [Header("Behavior")]
    [SerializeField] private bool requestAuthorityOnStart = true;

    [Tooltip("Run the full demo sequence (mode switches, resets, time control). " +
             "If false, just sends Rate commands continuously.")]
    [SerializeField] private bool runFullDemo = true;

    [Header("Logging")]
    [SerializeField] private int logSensorEveryNFrames = 60;
    [SerializeField] private int logTelemetryEveryNFrames = 120;

    // ================================================================================
    // Runtime State
    // ================================================================================

    private SimCoreApi _sim;
    private QuadSimApi _drone;
    private bool _isActive;
    private int _frameCount;

    private enum DemoPhase { RateMode, VelocityMode, ResetDemo, TimeControlDemo, Done }
    private DemoPhase _phase = DemoPhase.RateMode;
    private float _phaseTimer;

    // ================================================================================
    // Lifecycle
    // ================================================================================

    private void Start()
    {
        StartCoroutine(InitializeDelayed());
    }

    private IEnumerator InitializeDelayed()
    {
        // Wait one frame for drone spawning and initialization
        yield return null;

        // ---- Step 1: Get sim-wide API ----
        _sim = SimCoreApi.Get();
        if (_sim == null)
        {
            Debug.LogError("[ApiExample] No SimCoreApi found! Add SimCoreApi to SimRoot.");
            enabled = false;
            yield break;
        }

        // ---- Step 2: Register as Internal source and request authority ----
        _sim.NotifyInternalConnected();
        Debug.Log("[ApiExample] Registered as Internal source (Connected).");

        if (requestAuthorityOnStart)
        {
            _isActive = _sim.RequestAuthority(InputSource.Internal);
            if (_isActive)
                Debug.Log("[ApiExample] Authority granted for Internal.");
            else
                Debug.LogWarning("[ApiExample] Authority request denied. Commands will be rejected.");
        }

        // ---- Step 3: Get per-drone API via static accessor ----
        _drone = QuadSimApi.Selected();
        if (_drone == null || !_drone.IsInitialized)
        {
            Debug.LogError("[ApiExample] No initialized drone found!");
            enabled = false;
            yield break;
```

```
        }
        Debug.Log($"[ApiExample] Got drone API: {_drone.DroneID} " +
                  $"(total drones: {QuadSimApi.DroneCount})");

        // ---- Step 4: Subscribe to events ----
        _drone.OnModeChanged += (oldMode, newMode) =>
            Debug.Log($"[ApiExample] Mode changed: {oldMode} -> {newMode}");

        _drone.OnControllerChanged += (oldCtrl, newCtrl) =>
            Debug.Log($"[ApiExample] Controller changed: {oldCtrl} -> {newCtrl}");

        _sim.OnAuthorityChanged += (oldAuth, newAuth) =>
            Debug.Log($"[ApiExample] Authority changed: {oldAuth} -> {newAuth}");

        // ---- Step 5: Set initial mode ----
        _drone.SetMode(GoalMode.Rate);
        Debug.Log($"[ApiExample] Set mode to Rate. Current mode: {_drone.CurrentMode}");

        // ---- Step 6: Log initial sim status ----
        var status = _sim.GetStatus();
        Debug.Log($"[ApiExample] Sim status — paused={status.IsPaused}, timeScale={status.TimeScale}, "
+
                  $"authority={status.CurrentAuthority}, dt={status.FixedDtSec:F4}s");

        _phase = DemoPhase.RateMode;
        _phaseTimer = 0f;

        Debug.Log("[ApiExample] Initialization complete. Starting command loop.");
    }

    private void Update()
    {
        if (!_isActive || _drone == null) return;

        _frameCount++;
        _phaseTimer += Time.deltaTime;

        if (runFullDemo)
            RunDemoSequence();
        else
            SendRateCommands();

        if (logSensorEveryNFrames > 0 && _frameCount % logSensorEveryNFrames == 0)
            LogSensorData();

        if (logTelemetryEveryNFrames > 0 && _frameCount % logTelemetryEveryNFrames == 0)
            LogTelemetry();
    }

    private void OnDestroy()
    {
        if (_sim != null)
        {
            _sim.NotifyInternalDisconnected();
        }
        Debug.Log("[ApiExample] Destroyed. Internal source disconnected.");
    }

    // ================================================================================
    // Demo Sequence
    // ================================================================================

    private void RunDemoSequence()
    {
        switch (_phase)
        {
            case DemoPhase.RateMode:
                SendRateCommands();
                if (_phaseTimer >= rateModeDuration)
                    TransitionTo(DemoPhase.VelocityMode);
                break;

            case DemoPhase.VelocityMode:
```

```
            SendVelocityCommands();
            if (_phaseTimer >= velocityModeDuration)
                TransitionTo(DemoPhase.ResetDemo);
            break;

        case DemoPhase.ResetDemo:
            RunResetDemo();
            TransitionTo(DemoPhase.TimeControlDemo);
            break;

        case DemoPhase.TimeControlDemo:
            RunTimeControlDemo();
            TransitionTo(DemoPhase.Done);
            break;

        case DemoPhase.Done:
            if (_drone.CurrentMode != GoalMode.Rate)
                _drone.SetMode(GoalMode.Rate);
            SendRateCommands();
            break;
    }
}

private void TransitionTo(DemoPhase next)
{
    Debug.Log($"[ApiExample] === Phase transition: {_phase} -> {next} ===");
    _phase = next;
    _phaseTimer = 0f;
}

// ============================================================================
// Command Sending
// ============================================================================

private void SendRateCommands()
{
    if (_drone.CurrentMode != GoalMode.Rate)
        _drone.SetMode(GoalMode.Rate);

    Axis4 cmd = new Axis4(rollRate, pitchRate, yawRate, throttle);
    _drone.SendCommand(InputSource.Internal, cmd);
}

private void SendVelocityCommands()
{
    // Overload that sets mode + sends command in one call
    Axis4 cmd = new Axis4(forwardVel, lateralVel, velYawRate, verticalVel);
    _drone.SendCommand(InputSource.Internal, cmd, GoalMode.Velocity);
}

// ============================================================================
// Reset Demo
// ============================================================================

private void RunResetDemo()
{
    Debug.Log("[ApiExample] --- Reset Demo ---");

    // Read pre-reset sensor state
    var sensors = _drone.GetSensorData();
    Debug.Log($"[ApiExample] Pre-reset GPS position: {sensors.GpsPosition}");

    // Reset controller only (PIDs)
    _drone.ResetController();
    Debug.Log("[ApiExample] ResetController() — PIDs cleared, position unchanged");

    // Reset to a specific pose
    Vector3 resetPos = new Vector3(0f, 3f, 0f);
    _drone.ResetPose(resetPos, Quaternion.identity);
    Debug.Log($"[ApiExample] ResetPose({resetPos}) — drone repositioned");

    // Full reset
    _drone.ResetAll();
```

```
        Debug.Log("[ApiExample] ResetAll() — full reset complete");

        // Back to Rate mode
        _drone.SetMode(GoalMode.Rate);
    }

    // ================================================================================
    // Time Control Demo
    // ================================================================================

    private void RunTimeControlDemo()
    {
        Debug.Log("[ApiExample] --- Time Control Demo ---");

        Debug.Log($"[ApiExample] Sim time: {_sim.SimTimeSec:F3}s, dt: {_sim.FixedDtSec:F4}s");

        _sim.Pause();
        Debug.Log($"[ApiExample] Paused: {_sim.IsPaused}");

        _sim.StepOnce();
        Debug.Log($"[ApiExample] Stepped once. Sim time: {_sim.SimTimeSec:F3}s");

        _sim.StepMany(10);
        Debug.Log("[ApiExample] Queued 10 steps");

        _sim.SetTimeScale(2.0);
        Debug.Log($"[ApiExample] TimeScale set to {_sim.TimeScale}");

        _sim.Resume();
        Debug.Log($"[ApiExample] Resumed: paused={_sim.IsPaused}");

        _sim.SetTimeScale(1.0);
        Debug.Log($"[ApiExample] TimeScale reset to {_sim.TimeScale}");
    }

    // ================================================================================
    // Sensor & Telemetry Logging
    // ================================================================================

    private void LogSensorData()
    {
        var s = _drone.GetSensorData();
        if (!s.ImuValid) return;

        Vector3 attDeg = s.ImuAttitude;
        Vector3 angVelDeg = s.ImuAngVel * Mathf.Rad2Deg;

        Debug.Log($"[ApiExample] SENSORS — " +
                  $"att=({attDeg.x:F1}, {attDeg.y:F1}, {attDeg.z:F1})° " +
                  $"angVel=({angVelDeg.x:F1}, {angVelDeg.y:F1}, {angVelDeg.z:F1})°/s " +
                  $"gps=({s.GpsPosition.x:F2}, {s.GpsPosition.y:F2}, {s.GpsPosition.z:F2})m " +
                  $"vel=({s.ImuVel.x:F2}, {s.ImuVel.y:F2}, {s.ImuVel.z:F2})m/s");
    }

    private void LogTelemetry()
    {
        var t = _drone.GetTelemetry();

        Debug.Log($"[ApiExample] TELEMETRY — " +
                  $"mode={t.Mode} ctrl={t.Controller} " +
                  $"motors=({t.MotorOutputs.FL:F3}, {t.MotorOutputs.FR:F3}, " +
                  $"{t.MotorOutputs.BL:F3}, {t.MotorOutputs.BR:F3})");
    }
}
```

Documentation~/headless-tools/live_viewer.py

SHA-256 524f82bcd991585b716dc1e9edfcbe2d768518300479d14f162e8c9bba6cef43 | 192 lines | 7,191 bytes

```
#!/usr/bin/env python3
"""
live_viewer.py — general live 3D viewer for QuadSim headless runs.

Listens for UDP JSON from quadsim.viz.UdpViz. Two packet types:
```

```
  {"type": "path",   "points": [[x,y,z], ...]}                # one-shot ghost
  {"type": "sample", "t_rel": t,
                     "actual": [x,y,z], "target": [x,y,z],     # target optional
                     "err": e, "speed": s, "trial": n}

FLU, z up: positions are plotted as-is — no frame flips.

Draws the flown path, the commanded-setpoint trail, current drone/setpoint
markers, and (if sent) a static planned-trajectory ghost. The path resets on a
trial change or a backwards time jump; the ghost persists.

Run:  python live_viewer.py --port 14660
Controls: left-drag rotate, scroll zoom, right-drag pan.
"""

from __future__ import annotations

import argparse
import json
import socket
import time
from collections import deque

import matplotlib.pyplot as plt
import numpy as np


def _xyz(v):
    """Accept either {'x','y','z'} or [x,y,z]."""
    if isinstance(v, dict):
        return float(v["x"]), float(v["y"]), float(v["z"])
    if isinstance(v, (list, tuple)) and len(v) >= 3:
        return float(v[0]), float(v[1]), float(v[2])
    raise ValueError(f"bad xyz payload: {v!r}")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--host", default="0.0.0.0")
    ap.add_argument("--port", type=int, default=14660)
    ap.add_argument("--max-points", type=int, default=10000)
    ap.add_argument("--draw-hz", type=float, default=20.0)
    ap.add_argument("--range", type=float, default=24.0)
    ap.add_argument("--zmin", type=float, default=0.0)
    ap.add_argument("--zmax", type=float, default=25.0)
    ap.add_argument("--autoscale", action="store_true")
    ap.add_argument("--debug", action="store_true")
    args = ap.parse_args()

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((args.host, args.port))
    sock.setblocking(False)
    print(f"[viewer] listening on udp://{args.host}:{args.port}  (FLU, z up)")
    print("[viewer] controls: left-drag rotate, scroll zoom, right-drag pan")

    ax_d = deque(maxlen=args.max_points)
    ay_d = deque(maxlen=args.max_points)
    az_d = deque(maxlen=args.max_points)
    tx_d = deque(maxlen=args.max_points)
    ty_d = deque(maxlen=args.max_points)
    tz_d = deque(maxlen=args.max_points)

    plt.ion()
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection="3d")

    actual_line, = ax.plot([], [], [], lw=2.0, label="actual")
    target_line, = ax.plot([], [], [], ls="--", lw=1.2, label="setpoint trail")
    actual_marker, = ax.plot([], [], [], marker="o", ms=7, ls="", label="drone")
    target_marker, = ax.plot([], [], [], marker="x", ms=8, ls="", label="setpoint")
    path_line, = ax.plot([], [], [], color="0.6", lw=1.0, label="planned path")
```

```
ax.set_xlabel("x [m]")
ax.set_ylabel("y [m]")
ax.set_zlabel("z up [m]")
ax.legend(loc="upper left")
ax.set_xlim(-args.range, args.range)
ax.set_ylim(-args.range, args.range)
ax.set_zlim(args.zmin, args.zmax)
fig.tight_layout()
fig.show()

def set3d(line, xs, ys, zs):
    line.set_data(xs, ys)
    line.set_3d_properties(zs)

latest = None
current_trial = None
last_t = None
last_draw = 0.0

try:
    while plt.fignum_exists(fig.number):
        # ---- drain every pending packet ----
        while True:
            try:
                data, _ = sock.recvfrom(65535)
            except BlockingIOError:
                break
            try:
                msg = json.loads(data.decode("utf-8"))
            except Exception as e:
                if args.debug:
                    print(f"[viewer] bad json: {e}")
                continue

            mtype = msg.get("type")

            if mtype == "path":
                pts = msg.get("points", [])
                if pts:
                    P = np.array([_xyz(p) for p in pts])
                    set3d(path_line, P[:, 0], P[:, 1], P[:, 2])
                continue

            if mtype != "sample":
                continue

            trial = int(msg.get("trial", -1))
            t_rel = float(msg.get("t_rel", 0.0))

            trial_changed = (current_trial is not None
                             and trial != -1 and trial != current_trial)
            time_reset = (last_t is not None and t_rel < last_t - 0.5)
            if trial_changed or time_reset:
                for d in (ax_d, ay_d, az_d, tx_d, ty_d, tz_d):
                    d.clear()
                print(f"[viewer] reset path (trial {trial})")

            current_trial = trial
            last_t = t_rel

            axx, ayy, azz = _xyz(msg["actual"])
            ax_d.append(axx); ay_d.append(ayy); az_d.append(azz)
            if "target" in msg:
                txx, tyy, tzz = _xyz(msg["target"])
                tx_d.append(txx); ty_d.append(tyy); tz_d.append(tzz)
            latest = msg

        # ---- throttled redraw ----
        now = time.perf_counter()
        if latest is not None and now - last_draw >= 1.0 / max(1.0, args.draw_hz):
            last_draw = now

            set3d(actual_line, list(ax_d), list(ay_d), list(az_d))
```

```
                set3d(target_line, list(tx_d), list(ty_d), list(tz_d))
                if ax_d:
                    set3d(actual_marker, [ax_d[-1]], [ay_d[-1]], [az_d[-1]])
                if tx_d:
                    set3d(target_marker, [tx_d[-1]], [ty_d[-1]], [tz_d[-1]])

                if args.autoscale and len(ax_d) > 5:
                    X, Y, Z = np.array(ax_d), np.array(ay_d), np.array(az_d)
                    pad = 3.0
                    ax.set_xlim(float(X.min()) - pad, float(X.max()) + pad)
                    ax.set_ylim(float(Y.min()) - pad, float(Y.max()) + pad)
                    ax.set_zlim(float(Z.min()) - pad, float(Z.max()) + pad)

                yaw_text = ""
                if "yaw_error_deg" in latest:
                    yaw_text = (
                        f" | yaw={latest.get('yaw_deg', 0.0):+.1f}°"
                        f"/{latest.get('yaw_target_deg', 0.0):+.1f}°"
                        f" eYaw={latest['yaw_error_deg']:+.1f}°"
                    )
                ax.set_title(
                    f"QuadSim live | trial={latest.get('trial', -1)} "
                    f"t={latest.get('t_rel', 0.0):.1f}s "
                    f"err={latest.get('err', 0.0):.2f}m "
                    f"speed={latest.get('speed', 0.0):.2f}m/s"
                    f"{yaw_text}"
                )

                fig.canvas.draw_idle()
                plt.pause(0.001)
            else:
                plt.pause(0.005)

    except KeyboardInterrupt:
        print("\n[viewer] interrupted")
    finally:
        sock.close()


if __name__ == "__main__":
    main()
```

Samples~/PX4 SITL Tools/test_offboard_figure8_3d.py

SHA-256 ed07835cc9815c4cdebbee7fd711be5305711d3c5253dbc1c6bed645dc48b72f | 785 lines | 26,152 bytes

```
#!/usr/bin/env python3
"""
test_offboard_figure8_3d_yaw.py

PX4 OFFBOARD 3D figure-eight validation with yaw tracking.

This extends test_offboard_figure8_3d.py:
  - reads LOCAL_POSITION_NED and ATTITUDE
  - commands position plus yaw
  - default yaw mode is tangent-rate-limited
  - logs yaw command, actual yaw, yaw error, yaw rate
  - plots 3D path, position error, speed, yaw tracking
  - broadcasts actual/target/yaw over UDP for live_udp_3d_viewer_yaw.py

Coordinates:
  PX4 LOCAL_NED:
    x = North [m]
    y = East  [m]
    z = Down  [m]

Yaw convention:
  PX4 LOCAL_NED yaw:
    yaw = atan2(East velocity, North velocity)
  so yaw=0 points North, yaw=+90deg points East.

Run:
  python test_offboard_figure8_3d_yaw.py --speed 10.0
```

```
With live viewer:
  Terminal A:
    python live_udp_3d_viewer_yaw.py --port 14660 --range 24 --zmin 0 --zmax 25

  Terminal B:
    python test_offboard_figure8_3d_yaw.py --speed 10.0 --viz-udp 127.0.0.1:14660
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import socket
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Optional

import matplotlib.pyplot as plt
import numpy as np
from pymavlink import mavutil

mav = mavutil.mavlink

TYPEMASK_POS_ONLY = (
    mav.POSITION_TARGET_TYPEMASK_VX_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_VY_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_VZ_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AX_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AY_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AZ_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_YAW_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_YAW_RATE_IGNORE
)

TYPEMASK_POS_YAW = (
    mav.POSITION_TARGET_TYPEMASK_VX_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_VY_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_VZ_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AX_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AY_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_AZ_IGNORE
    | mav.POSITION_TARGET_TYPEMASK_YAW_RATE_IGNORE
)


@dataclass
class LocalPosition:
    t_boot_s: float
    x: float
    y: float
    z: float
    vx: float
    vy: float
    vz: float


@dataclass
class Attitude:
    t_boot_s: float
    roll: float
    pitch: float
    yaw: float
    rollspeed: float
    pitchspeed: float
    yawspeed: float


@dataclass
class State:
    lp: Optional[LocalPosition] = None
```

```
    att: Optional[Attitude] = None


@dataclass
class Target:
    x: float
    y: float
    z: float
    name: str
    yaw_raw: float = 0.0
    yaw_cmd: float = 0.0


def wrap_pi(a: float) -> float:
    return math.atan2(math.sin(a), math.cos(a))


def advance_angle_limited(prev: float, desired: float, max_step: float) -> float:
    err = wrap_pi(desired - prev)
    if err > max_step:
        err = max_step
    elif err < -max_step:
        err = -max_step
    return wrap_pi(prev + err)


def yaw_error(actual: float, command: float) -> float:
    return wrap_pi(command - actual)


def param_set(m, name: str, value: float, ptype=mav.MAV_PARAM_TYPE_REAL32) -> None:
    m.mav.param_set_send(
        m.target_system,
        m.target_component,
        name.encode(),
        float(value),
        ptype,
    )


def send_position_setpoint(m, t_boot_s: float, target: Target, use_yaw: bool) -> None:
    mask = TYPEMASK_POS_YAW if use_yaw else TYPEMASK_POS_ONLY
    yaw = target.yaw_cmd if use_yaw else 0.0

    t_ms = int(max(0.0, t_boot_s) * 1000.0) & 0xFFFFFFFF

    m.mav.set_position_target_local_ned_send(
        t_ms,
        m.target_system,
        m.target_component,
        mav.MAV_FRAME_LOCAL_NED,
        mask,
        float(target.x), float(target.y), float(target.z),
        0.0, 0.0, 0.0,       # ignored velocity
        0.0, 0.0, 0.0,       # ignored accel/force
        float(yaw), 0.0,     # yaw command, yaw-rate ignored
    )


def set_offboard_and_arm(m) -> None:
    m.mav.command_long_send(
        m.target_system,
        m.target_component,
        mav.MAV_CMD_DO_SET_MODE,
        0,
        mav.MAV_MODE_FLAG_CUSTOM_MODE_ENABLED,
        6,  # PX4 custom main mode OFFBOARD
        0, 0, 0, 0, 0,
    )

    m.mav.command_long_send(
        m.target_system,
        m.target_component,
```

```
        mav.MAV_CMD_COMPONENT_ARM_DISARM,
        0,
        1, 0, 0, 0, 0, 0, 0,
    )


def disarm(m) -> None:
    m.mav.command_long_send(
        m.target_system,
        m.target_component,
        mav.MAV_CMD_COMPONENT_ARM_DISARM,
        0,
        0, 0, 0, 0, 0, 0, 0,
    )


def drain_latest_state(m, state: State) -> State:
    while True:
        msg = m.recv_match(type=["LOCAL_POSITION_NED", "ATTITUDE"], blocking=False)
        if msg is None:
            break

        typ = msg.get_type()
        if typ == "LOCAL_POSITION_NED":
            state.lp = LocalPosition(
                t_boot_s=float(msg.time_boot_ms) / 1000.0,
                x=float(msg.x),
                y=float(msg.y),
                z=float(msg.z),
                vx=float(msg.vx),
                vy=float(msg.vy),
                vz=float(msg.vz),
            )
        elif typ == "ATTITUDE":
            state.att = Attitude(
                t_boot_s=float(msg.time_boot_ms) / 1000.0,
                roll=float(msg.roll),
                pitch=float(msg.pitch),
                yaw=float(msg.yaw),
                rollspeed=float(msg.rollspeed),
                pitchspeed=float(msg.pitchspeed),
                yawspeed=float(msg.yawspeed),
            )

    return state


def wait_for_state(m, timeout_wall_s: float = 30.0) -> State:
    print("[fig8yaw] waiting for LOCAL_POSITION_NED + ATTITUDE...")
    deadline = time.perf_counter() + timeout_wall_s
    state = State()

    while time.perf_counter() < deadline:
        msg = m.recv_match(type=["LOCAL_POSITION_NED", "ATTITUDE"], blocking=True, timeout=1.0)
        if msg is None:
            continue

        typ = msg.get_type()
        if typ == "LOCAL_POSITION_NED":
            state.lp = LocalPosition(
                t_boot_s=float(msg.time_boot_ms) / 1000.0,
                x=float(msg.x),
                y=float(msg.y),
                z=float(msg.z),
                vx=float(msg.vx),
                vy=float(msg.vy),
                vz=float(msg.vz),
            )
        elif typ == "ATTITUDE":
            state.att = Attitude(
                t_boot_s=float(msg.time_boot_ms) / 1000.0,
                roll=float(msg.roll),
                pitch=float(msg.pitch),
```

```
                yaw=float(msg.yaw),
                rollspeed=float(msg.rollspeed),
                pitchspeed=float(msg.pitchspeed),
                yawspeed=float(msg.yawspeed),
            )

        if state.lp is not None and state.att is not None:
            return state

    raise RuntimeError("No complete PX4 state received. EKF/attitude telemetry not ready.")


def smoothstep01(u: float) -> float:
    u = min(1.0, max(0.0, u))
    return 6*u**5 - 15*u**4 + 10*u**3


def lerp(a: float, b: float, s: float) -> float:
    return a + s * (b - a)


def figure8_target_raw(
    t_rel: float,
    start: Target,
    altitude: float,
    climb_s: float,
    hold_s: float,
    period: float,
    laps: float,
    x_amp: float,
    y_amp: float,
    z_amp: float,
    return_s: float,
    final_hold_s: float,
) -> tuple[Target, float, bool]:
    center = Target(0.0, 0.0, -altitude, "center", 0.0, 0.0)

    fig_start = climb_s + hold_s
    fig_duration = period * laps
    return_start = fig_start + fig_duration
    final_start = return_start + return_s
    mission_duration = final_start + final_hold_s

    if t_rel < climb_s:
        s = smoothstep01(t_rel / climb_s)
        # During climb, keep heading North.
        return Target(
            lerp(start.x, center.x, s),
            lerp(start.y, center.y, s),
            lerp(start.z, center.z, s),
            "climb",
            0.0,
            0.0,
        ), mission_duration, False

    if t_rel < fig_start:
        return Target(center.x, center.y, center.z, "center_hold", 0.0, 0.0), mission_duration, False

    if t_rel < return_start:
        tau = t_rel - fig_start
        theta = 2.0 * math.pi * tau / period

        x = x_amp * math.sin(theta)
        y = y_amp * math.sin(2.0 * theta)
        z = -altitude - z_amp * math.sin(2.0 * theta + math.pi / 3.0)

        w = 2.0 * math.pi / period
        dx = x_amp * w * math.cos(theta)
        dy = 2.0 * y_amp * w * math.cos(2.0 * theta)

        # PX4 NED yaw: yaw = atan2(East velocity, North velocity).
        yaw_raw = math.atan2(dy, dx)
```

```
            lap_idx = int(tau // period) + 1
            return Target(x, y, z, f"figure8_lap_{lap_idx}", yaw_raw, yaw_raw), mission_duration, False

        theta_end = 2.0 * math.pi * laps
        end = Target(
            x_amp * math.sin(theta_end),
            y_amp * math.sin(2.0 * theta_end),
            -altitude - z_amp * math.sin(2.0 * theta_end + math.pi / 3.0),
            "fig8_end",
            0.0,
            0.0,
        )

        if t_rel < final_start:
            s = smoothstep01((t_rel - return_start) / return_s)
            return Target(
                lerp(end.x, center.x, s),
                lerp(end.y, center.y, s),
                lerp(end.z, center.z, s),
                "return_center",
                0.0,
                0.0,
            ), mission_duration, False

        if t_rel < mission_duration:
            return Target(center.x, center.y, center.z, "final_hold", 0.0, 0.0), mission_duration, False

        return Target(center.x, center.y, center.z, "done", 0.0, 0.0), mission_duration, True


    def dist3(lp: LocalPosition, t: Target) -> float:
        return math.sqrt((lp.x - t.x)**2 + (lp.y - t.y)**2 + (lp.z - t.z)**2)


    def speed_norm(lp: LocalPosition) -> float:
        return math.sqrt(lp.vx**2 + lp.vy**2 + lp.vz**2)


    def make_udp_sender(viz_udp: str | None):
        if not viz_udp:
            return None, None

        host, port_s = viz_udp.rsplit(":", 1)
        addr = (host, int(port_s))
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.setblocking(False)
        print(f"[fig8yaw] live viewer UDP enabled -> {host}:{port_s}")
        return sock, addr


    def send_viz(sock, addr, lp: LocalPosition, att: Attitude, target: Target,
                 t_rel: float, err: float, spd: float, yaw_err: float) -> None:
        if sock is None or addr is None:
            return

        msg = {
            "type": "sample",
            "source": "px4_fig8_yaw",
            "t_rel": t_rel,
            "actual": {"x": lp.x, "y": lp.y, "z": lp.z},
            "target": {"x": target.x, "y": target.y, "z": target.z},
            "attitude": {
                "roll": att.roll,
                "pitch": att.pitch,
                "yaw": att.yaw,
                "yawspeed": att.yawspeed,
            },
            "yaw_cmd": target.yaw_cmd,
            "yaw_raw": target.yaw_raw,
            "yaw_err": yaw_err,
            "err": err,
            "speed": spd,
            "seg": target.name,
```

```
        }

        try:
            sock.sendto(json.dumps(msg, separators=(",", ":")).encode("utf-8"), addr)
        except OSError:
            pass


    def set_axes_equal_3d(ax) -> None:
        xlim = ax.get_xlim3d()
        ylim = ax.get_ylim3d()
        zlim = ax.get_zlim3d()

        xmid = 0.5 * (xlim[0] + xlim[1])
        ymid = 0.5 * (ylim[0] + ylim[1])
        zmid = 0.5 * (zlim[0] + zlim[1])

        radius = 0.5 * max(
            abs(xlim[1] - xlim[0]),
            abs(ylim[1] - ylim[0]),
            abs(zlim[1] - zlim[0]),
            1.0,
        )

        ax.set_xlim3d(xmid - radius, xmid + radius)
        ax.set_ylim3d(ymid - radius, ymid + radius)
        ax.set_zlim3d(zmid - radius, zmid + radius)


    def write_csv(rows: list[dict], path: Path) -> None:
        if not rows:
            print("[fig8yaw] no rows to save")
            return

        fields = [
            "t_wall", "t_sim", "t_rel", "seg_idx", "seg_name",
            "x", "y", "z", "vx", "vy", "vz",
            "tx", "ty", "tz",
            "err", "speed",
            "roll", "pitch", "yaw",
            "rollspeed", "pitchspeed", "yawspeed",
            "yaw_cmd", "yaw_raw", "yaw_err",
        ]

        with path.open("w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=fields)
            w.writeheader()
            w.writerows(rows)

        print(f"[fig8yaw] saved {path}")


    def plot_results(rows: list[dict], out_prefix: Path, show: bool) -> None:
        if not rows:
            return

        actual_n = np.array([r["x"] for r in rows])
        actual_e = np.array([r["y"] for r in rows])
        actual_u = -np.array([r["z"] for r in rows])

        target_n = np.array([r["tx"] for r in rows])
        target_e = np.array([r["ty"] for r in rows])
        target_u = -np.array([r["tz"] for r in rows])

        t = np.array([r["t_rel"] for r in rows])
        err = np.array([r["err"] for r in rows])
        spd = np.array([r["speed"] for r in rows])
        yaw = np.unwrap(np.array([r["yaw"] for r in rows]))
        yaw_cmd = np.unwrap(np.array([r["yaw_cmd"] for r in rows]))
        yaw_err = np.array([r["yaw_err"] for r in rows])
        yawspeed = np.array([r["yawspeed"] for r in rows])

        fig = plt.figure(figsize=(10, 8))
```

```
ax = fig.add_subplot(111, projection="3d")
ax.plot(target_n, target_e, target_u, linestyle="--", linewidth=1.5, label="moving target")
ax.plot(actual_n, actual_e, actual_u, linewidth=2.0, label="PX4 actual")
ax.scatter([target_n[0]], [target_e[0]], [target_u[0]], s=45, marker="o", label="start")
ax.scatter([target_n[-1]], [target_e[-1]], [target_u[-1]], s=45, marker="^", label="end")
ax.set_title("PX4 OFFBOARD 3D Figure-Eight Tracking with Yaw")
ax.set_xlabel("North x [m]")
ax.set_ylabel("East y [m]")
ax.set_zlabel("Up -z [m]")
ax.legend()
set_axes_equal_3d(ax)
fig.tight_layout()

path3d = out_prefix.with_suffix(".png")
fig.savefig(path3d, dpi=180)
print(f"[fig8yaw] saved {path3d}")

fig2 = plt.figure(figsize=(10, 4))
ax2 = fig2.add_subplot(111)
ax2.plot(t, err)
ax2.set_title("Figure-eight position tracking error")
ax2.set_xlabel("mission sim time [s]")
ax2.set_ylabel("||pos - target|| [m]")
ax2.grid(True)
fig2.tight_layout()
err_path = out_prefix.with_name(out_prefix.stem + "_error.png")
fig2.savefig(err_path, dpi=180)
print(f"[fig8yaw] saved {err_path}")

fig3 = plt.figure(figsize=(10, 4))
ax3 = fig3.add_subplot(111)
ax3.plot(t, spd)
ax3.set_title("PX4 local velocity norm")
ax3.set_xlabel("mission sim time [s]")
ax3.set_ylabel("speed [m/s]")
ax3.grid(True)
fig3.tight_layout()
spd_path = out_prefix.with_name(out_prefix.stem + "_speed.png")
fig3.savefig(spd_path, dpi=180)
print(f"[fig8yaw] saved {spd_path}")

fig4 = plt.figure(figsize=(10, 5))
ax4 = fig4.add_subplot(111)
ax4.plot(t, np.degrees(yaw_cmd), label="yaw command")
ax4.plot(t, np.degrees(yaw), label="actual yaw")
ax4.set_title("Yaw tracking")
ax4.set_xlabel("mission sim time [s]")
ax4.set_ylabel("yaw [deg, unwrapped]")
ax4.grid(True)
ax4.legend()
fig4.tight_layout()
yaw_path = out_prefix.with_name(out_prefix.stem + "_yaw.png")
fig4.savefig(yaw_path, dpi=180)
print(f"[fig8yaw] saved {yaw_path}")

fig5 = plt.figure(figsize=(10, 4))
ax5 = fig5.add_subplot(111)
ax5.plot(t, np.degrees(yaw_err), label="yaw error")
ax5.plot(t, np.degrees(yawspeed), label="yaw rate")
ax5.set_title("Yaw error and yaw rate")
ax5.set_xlabel("mission sim time [s]")
ax5.set_ylabel("deg / deg/s")
ax5.grid(True)
ax5.legend()
fig5.tight_layout()
yerr_path = out_prefix.with_name(out_prefix.stem + "_yaw_error.png")
fig5.savefig(yerr_path, dpi=180)
print(f"[fig8yaw] saved {yerr_path}")

if show:
    plt.show()
```

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="udpin:0.0.0.0:14540")
    ap.add_argument("--speed", type=float, default=10.0)
    ap.add_argument("--sim-stream-hz", type=float, default=50.0)
    ap.add_argument("--altitude", type=float, default=15.0)
    ap.add_argument("--x-amp", type=float, default=14.0)
    ap.add_argument("--y-amp", type=float, default=10.0)
    ap.add_argument("--z-amp", type=float, default=2.5)
    ap.add_argument("--period", type=float, default=36.0)
    ap.add_argument("--laps", type=float, default=3.0)
    ap.add_argument("--climb-s", type=float, default=12.0)
    ap.add_argument("--hold-s", type=float, default=2.0)
    ap.add_argument("--return-s", type=float, default=6.0)
    ap.add_argument("--final-hold-s", type=float, default=3.0)
    ap.add_argument("--yaw-mode", choices=["ignore", "hold", "tangent"], default="tangent",
                    help="tangent tracks path heading; hold commands yaw=0; ignore does not command yaw.
")
    ap.add_argument("--max-yaw-rate-deg", type=float, default=60.0,
                    help="Rate limit applied to yaw command in tangent mode.")
    ap.add_argument("--abort-err", type=float, default=100.0)
    ap.add_argument("--abort-down-z", type=float, default=20.0)
    ap.add_argument("--abort-speed", type=float, default=45.0)
    ap.add_argument("--abort-yawrate-deg", type=float, default=720.0)
    ap.add_argument("--viz-udp", default="127.0.0.1:14660",
                    help="UDP JSON stream for live_udp_3d_viewer_yaw.py. Use '' to disable.")
    ap.add_argument("--out-prefix", default="px4_figure8_3d_yaw")
    ap.add_argument("--show", action="store_true")
    ap.add_argument("--no-disarm", action="store_true")
    args = ap.parse_args()

    wall_stream_hz = max(5.0, args.sim_stream_hz * max(1.0, args.speed))
    period_wall = 1.0 / wall_stream_hz
    max_yaw_step_per_sim_s = math.radians(args.max_yaw_rate_deg)

    viz_udp = args.viz_udp if args.viz_udp else None
    viz_sock, viz_addr = make_udp_sender(viz_udp)

    print(f"[fig8yaw] connecting {args.url}")
    m = mavutil.mavlink_connection(args.url)
    m.wait_heartbeat()
    print(f"[fig8yaw] heartbeat sys={m.target_system} comp={m.target_component}")
    print(f"[fig8yaw] wall stream rate = {wall_stream_hz:.1f} Hz "
          f"({args.sim_stream_hz:.1f} Hz sim target @ speed={args.speed:.2f}x)")
    print(f"[fig8yaw] params: A={args.x_amp:.1f}m B={args.y_amp:.1f}m "
          f"z_amp={args.z_amp:.1f}m period={args.period:.1f}s laps={args.laps:.1f} "
          f"yaw_mode={args.yaw_mode} max_yaw_rate={args.max_yaw_rate_deg:.1f}deg/s")

    print("[fig8yaw] setting headless-friendly PX4 params")
    param_set(m, "NAV_RCL_ACT", 0, mav.MAV_PARAM_TYPE_INT32)
    param_set(m, "NAV_DLL_ACT", 0, mav.MAV_PARAM_TYPE_INT32)
    param_set(m, "COM_RC_IN_MODE", 4, mav.MAV_PARAM_TYPE_INT32)

    state = wait_for_state(m)

    print("[fig8yaw] draining telemetry for 1 wall second")
    t_end = time.perf_counter() + 1.0
    while time.perf_counter() < t_end:
        state = drain_latest_state(m, state)
        time.sleep(0.02)

    if state.lp is None or state.att is None:
        raise RuntimeError("Incomplete PX4 state after drain.")

    # Prestream current position so PX4 accepts OFFBOARD.
    lp = state.lp
    att = state.att
    yaw_cmd_state = 0.0 if args.yaw_mode == "hold" else att.yaw
    pre_target = Target(lp.x, lp.y, lp.z, "pre_offboard_hold", yaw_cmd_state, yaw_cmd_state)

    print(f"[fig8yaw] pre-streaming current NED=({lp.x:+.2f},{lp.y:+.2f},{lp.z:+.2f}) "
          f"yaw={math.degrees(att.yaw):+.1f}deg")
```

```
t_pre = time.perf_counter()
while time.perf_counter() - t_pre < 2.0:
    state = drain_latest_state(m, state)
    if state.lp is not None:
        lp = state.lp
        pre_target = Target(lp.x, lp.y, lp.z, "pre_offboard_hold", yaw_cmd_state, yaw_cmd_state)
    send_position_setpoint(m, lp.t_boot_s, pre_target, use_yaw=(args.yaw_mode != "ignore"))
    time.sleep(period_wall)

print("[fig8yaw] -> OFFBOARD + ARM")
set_offboard_and_arm(m)

state = drain_latest_state(m, state)
lp = state.lp
att = state.att
if lp is None or att is None:
    raise RuntimeError("Incomplete state after arming.")

start_target = Target(lp.x, lp.y, lp.z, "start", yaw_cmd_state, yaw_cmd_state)
_, mission_duration, _ = figure8_target_raw(
    0.0, start_target, args.altitude, args.climb_s, args.hold_s,
    args.period, args.laps, args.x_amp, args.y_amp, args.z_amp,
    args.return_s, args.final_hold_s,
)
print(f"[fig8yaw] mission duration = {mission_duration:.1f}s sim")

rows: list[dict] = []
wall0 = time.perf_counter()
sim0: Optional[float] = None
last_sim_t: Optional[float] = None
last_print_bucket = -1
aborted = False

try:
    while True:
        loop_wall_start = time.perf_counter()

        state = drain_latest_state(m, state)
        lp = state.lp
        att = state.att
        if lp is None or att is None:
            time.sleep(period_wall)
            continue

        if sim0 is None:
            sim0 = lp.t_boot_s

        t_rel = lp.t_boot_s - sim0
        dt_sim = 0.0 if last_sim_t is None else max(0.0, t_rel - last_sim_t)
        last_sim_t = t_rel

        target, mission_duration, done = figure8_target_raw(
            t_rel, start_target, args.altitude, args.climb_s, args.hold_s,
            args.period, args.laps, args.x_amp, args.y_amp, args.z_amp,
            args.return_s, args.final_hold_s,
        )

        if args.yaw_mode == "ignore":
            target.yaw_cmd = 0.0
        elif args.yaw_mode == "hold":
            target.yaw_cmd = 0.0
        else:
            max_step = max_yaw_step_per_sim_s * max(dt_sim, 1.0 / args.sim_stream_hz)
            yaw_cmd_state = advance_angle_limited(yaw_cmd_state, target.yaw_raw, max_step)
            target.yaw_cmd = yaw_cmd_state

        send_position_setpoint(m, lp.t_boot_s, target, use_yaw=(args.yaw_mode != "ignore"))

        err = dist3(lp, target)
        spd = speed_norm(lp)
        ye = yaw_error(att.yaw, target.yaw_cmd) if args.yaw_mode != "ignore" else 0.0
        send_viz(viz_sock, viz_addr, lp, att, target, t_rel, err, spd, ye)
```

```
            rows.append({
                "t_wall": time.perf_counter() - wall0,
                "t_sim": lp.t_boot_s,
                "t_rel": t_rel,
                "seg_idx": 0,
                "seg_name": target.name,
                "x": lp.x,
                "y": lp.y,
                "z": lp.z,
                "vx": lp.vx,
                "vy": lp.vy,
                "vz": lp.vz,
                "tx": target.x,
                "ty": target.y,
                "tz": target.z,
                "err": err,
                "speed": spd,
                "roll": att.roll,
                "pitch": att.pitch,
                "yaw": att.yaw,
                "rollspeed": att.rollspeed,
                "pitchspeed": att.pitchspeed,
                "yawspeed": att.yawspeed,
                "yaw_cmd": target.yaw_cmd,
                "yaw_raw": target.yaw_raw,
                "yaw_err": ye,
            })

            bucket = int(t_rel)
            if bucket != last_print_bucket:
                last_print_bucket = bucket
                print(
                    f"  t={t_rel:6.1f}/{mission_duration:5.1f}s "
                    f"{target.name:14s} "
                    f"err={err:5.2f}m speed={spd:5.2f}m/s "
                    f"yaw={math.degrees(att.yaw):+7.1f}deg "
                    f"yaw_cmd={math.degrees(target.yaw_cmd):+7.1f}deg "
                    f"yaw_err={math.degrees(ye):+6.1f}deg "
                    f"yawrate={math.degrees(att.yawspeed):+6.1f}deg/s"
                )

            if err > args.abort_err:
                print(f"[fig8yaw] ABORT: err={err:.1f}m > {args.abort_err:.1f}m")
                aborted = True
                break

            if lp.z > args.abort_down_z:
                print(f"[fig8yaw] ABORT: NED z={lp.z:.1f}m means vehicle is below origin / falling")
                aborted = True
                break

            if spd > args.abort_speed:
                print(f"[fig8yaw] ABORT: speed={spd:.1f}m/s > {args.abort_speed:.1f}m/s")
                aborted = True
                break

            if abs(math.degrees(att.yawspeed)) > args.abort_yawrate_deg:
                print(f"[fig8yaw] ABORT: yawspeed={math.degrees(att.yawspeed):.1f}deg/s")
                aborted = True
                break

            if done:
                print("[fig8yaw] mission complete")
                break

            sleep_s = period_wall - (time.perf_counter() - loop_wall_start)
            if sleep_s > 0:
                time.sleep(sleep_s)

    except KeyboardInterrupt:
        print("\n[fig8yaw] interrupted")
        aborted = True
```

```
    finally:
        final_target = target if "target" in locals() else pre_target
        for _ in range(int(wall_stream_hz * 0.25)):
            state = drain_latest_state(m, state)
            if state.lp is not None:
                send_position_setpoint(m, state.lp.t_boot_s, final_target, use_yaw=(args.yaw_mode != "ig
nore"))
            time.sleep(period_wall)

        if not args.no_disarm:
            print("[fig8yaw] disarming")
            disarm(m)

        if viz_sock is not None:
            try:
                viz_sock.close()
            except OSError:
                pass

    out_prefix = Path(args.out_prefix)
    write_csv(rows, out_prefix.with_suffix(".csv"))
    plot_results(rows, out_prefix.with_suffix(".png"), show=args.show)

    if aborted:
        print("[fig8yaw] ended after abort/interruption; plots still saved for diagnosis")


if __name__ == "__main__":
    main()
```

Samples~/PX4 SITL Tools/scripts/run_bridge.sh

SHA-256 3fdf5607c207f6bd42d5eef10e2580871a53eab90a40d4ca8154a684e92e5dfc | 8 lines | 136 bytes

```
#!/usr/bin/env bash
set -e

python ../px4_bridge.py \
  --quadsim-host localhost \
  --px4-port 4560 \
  --hz 250 \
  --speed 0
```

Samples~/PX4 SITL Tools/scripts/run_confirm.sh

SHA-256 d87d533a21da887107b7b9fc9666746be39330c3aed61bcb14bfb09cb7993b0b | 4 lines | 71 bytes

```
#!/usr/bin/env bash
set -e

python ../px4_confirm.py --takeoff 3.0
```

Samples~/PX4 SITL Tools/scripts/run_figure8.sh

SHA-256 8e01b9bc5e1472bf9749f3c9f69d3b7ef24f7535bbc94135121456017de8f66e | 6 lines | 119 bytes

```
#!/usr/bin/env bash
set -e

python ../test_offboard_figure8_3d.py \
  --speed 10.0 \
  --viz-udp 127.0.0.1:14660
```

Samples~/PX4 SITL Tools/scripts/run_viewer.sh

SHA-256 1923e398f20ede1d4e2883969c3c8d81a8f3aaf7a10da7a6b5a1191d720f490e | 8 lines | 134 bytes

```
#!/usr/bin/env bash
set -e

python ..Headless-Tools/live_viewer.py \
  --port 14660 \
  --range 24 \
  --zmin 0 \
  --zmax 25
```

Samples~/PX4 SITL Tools/requirements.txt

SHA-256 667572903f2b3779ffa43f324933706695604c698eb5bdc751541f8b7f2865f6 | 4 lines | 43 bytes

```
pymavlink
numpy
matplotlib
quadsim-sdk
```

Samples~/PX4 SITL Tools/PX4_VERSION.md

SHA-256 abf25233e71239adf6a57828673321804f0d259597e6850ce53adec0494a5614 | 50 lines | 1,372 bytes

````
# Tested PX4 Version

PX4 behavior can change across commits and releases. For reproducible QuadSim experiments, pin the PX4-A
utopilot checkout used for validation.

## Current tested configuration

Fill this in for each release of the QuadSim plugin.

```text
QuadSim plugin version: v0.1.0
QuadSim SDK version:   <quadsim-sdk version or commit>
PX4-Autopilot commit:  <git rev-parse HEAD>
PX4 tag/branch:        <tag or branch>
PX4 target:            px4_sitl none_iris
QGroundControl:        optional
OS:                    <Ubuntu/WSL2/etc.>
Unity version:         Unity 6
```

Get the PX4 commit:

```bash
cd PX4-Autopilot
git rev-parse HEAD
```

Get the current branch/tag context:

```bash
git status
git describe --tags --always
```

## Recommended rule

For any published result, paper figure, or reproducible demo, record:

- QuadSim plugin git tag.
- QuadSim SDK version/commit.
- PX4-Autopilot commit.
- PX4 airframe target.
- Any changed PX4 params.
- Whether QGroundControl was attached.
- Whether the run was real time or faster than real time.

## Why this matters

PX4 SITL is an active project. EKF defaults, failsafe defaults, mixer behavior, MAVLink message handling
, and SITL startup behavior can change.

A setup that flies well on one PX4 commit may need parameter or bridge adjustments on another.
````

17. Tests and Continuous Integration

Verifies protocol limits, YAML parsing, rotor equations, allocation behavior, PID safeguards, deterministic clocks, fixed-step ordering, package contracts, supported SDK versions, Unity builds, built-player RPC behavior, logs, dependencies, and secrets.

Files in this section

Tests/Editor/ProtocolAndYamlTests.cs
Tests/Editor/RotorAndAllocatorTests.cs
Tests/Editor/SimClockTests.cs
Tests/PlayMode/SimulationContractPlayModeTests.cs
ci~/built_player_smoke.py
ci~/check_unity_log.py
.github/workflows/phase3-ci.yml
.github/workflows/security.yml
ci~/UnityTestProject/.gitignore
ci~/UnityTestProject/Packages/manifest.json
ci~/UnityTestProject/ProjectSettings/ProjectVersion.txt
ci~/UnityTestProject/ProjectSettings/EditorBuildSettings.asset

Tests/Editor/ProtocolAndYamlTests.cs

SHA-256 9601dbe06c883f9e0c3bb5f1dc6068d480d06f25e3b3bbdb109b21dcb20e858c | 106 lines | 3,569 bytes

```
using System.Collections.Generic;
using System.Text;
using System.Text.RegularExpressions;
using NUnit.Framework;
using SimCore.Rpc;
using UnityEngine;
using UnityEngine.TestTools;
using Yaml.Drone;

namespace Nino.QuadSim.Tests
{
    public sealed class ProtocolAndYamlTests
    {
        [Test]
        public void ConnectDataAdvertisesVersionedContracts()
        {
            Dictionary<string, object> data = RpcCapabilities.ConnectData(
                5555, 5556, "127.0.0.1");

            Assert.That(data["protocol_major"], Is.EqualTo(1));
            Assert.That(data["unity_package_version"], Is.EqualTo("0.3.0"));
            Assert.That(data["frame_schema_version"], Is.EqualTo(1));
            Assert.That(data["time_schema_version"], Is.EqualTo(1));
            Assert.That(data["bind_address"], Is.EqualTo("127.0.0.1"));
        }

        [Test]
        public void SchemaV1ParsesAndIsRetained()
        {
            bool parsed = Parse(MinimalYaml(1), out ModelType model);

            Assert.That(parsed, Is.True);
            Assert.That(model.SchemaVersion, Is.EqualTo(1));
        }

        [Test]
        public void UnsupportedSchemaIsRejected()
        {
            LogAssert.Expect(LogType.Error,
                "[Drone.YamlParser] Unsupported schema_version 2: test");
```

```
            Assert.That(Parse(MinimalYaml(2), out _), Is.False);
        }

        [Test]
        public void BlankMethodIsRejectedBeforeDispatch()
        {
            LogAssert.Expect(LogType.Warning,
                "[RpcProtocol] Request shape exceeds protocol limits.");
            byte[] request = { 0x81, 0xa6, 0x6d, 0x65, 0x74, 0x68, 0x6f, 0x64, 0xa0 };

            Assert.That(RpcSerializer.DeserializeRequest(request).HasValue, Is.False);
        }

        [Test]
        public void OversizedMethodIsRejectedBeforeDispatch()
        {
            LogAssert.Expect(LogType.Warning,
                "[RpcProtocol] Request shape exceeds protocol limits.");
            var bytes = new List<byte>
            {
                0x81, 0xa6, 0x6d, 0x65, 0x74, 0x68, 0x6f, 0x64, 0xd9, 65
            };
            bytes.AddRange(Encoding.ASCII.GetBytes(new string('a', 65)));

            Assert.That(RpcSerializer.DeserializeRequest(bytes.ToArray()).HasValue, Is.False);
        }

        [Test]
        public void MalformedMessagePackIsRejected()
        {
            LogAssert.Expect(LogType.Warning,
                new Regex("\\[RpcProtocol\\] Failed to deserialize request:.*"));

            Assert.That(RpcSerializer.DeserializeRequest(new byte[] { 0xc1 }).HasValue, Is.False);
        }

        private static bool Parse(string yaml, out ModelType model)
        {
            return YamlParser.LoadDroneConfigFromString(
                yaml, "test", out model, out _, out _, out _, out _,
                out _, out _, out _, out _);
        }

        private static string MinimalYaml(int schemaVersion) => $@"
schema_version: {schemaVersion}
name: Test
physical_properties:
  mass_kg: 1
  com_offset: {{x: 0, y: 0, z: 0}}
  inertia: {{x: 1, y: 1, z: 1}}
  arm_length: 0.2
propulsion:
  prop_diameter: 0.2
  max_rpm: 10000
  thrust_coef: 0.1
  torque_coef: 0.03
  torque_coefficient_convention: power_coefficient_2_pi
  propellers:
    - {{id: 0, x: 0.1, y: 0, z: 0.1, dir: -1}}
    - {{id: 1, x: 0.1, y: 0, z: -0.1, dir: 1}}
    - {{id: 2, x: -0.1, y: 0, z: 0.1, dir: 1}}
    - {{id: 3, x: -0.1, y: 0, z: -0.1, dir: -1}}
";
    }
}
```

Tests/Editor/RotorAndAllocatorTests.cs

SHA-256 dd42059bac6973951bcea161ae108d66da28373784d25dce338a44462665df45 | 316 lines | 13,012 bytes

```
using System.Linq;
using DroneCore.Common;
using DroneCore.Controllers;
```

```
using DroneCore.Controllers.AxisControllers;
using DroneCore.Controls;
using MathUtil;
using NUnit.Framework;
using UnityEngine;
using UnityEngine.TestTools;
using Yaml.Drone;

namespace Nino.QuadSim.Tests
{
    public sealed class RotorAndAllocatorTests
    {
        [Test]
        public void RotorEquationsMatchGoldenValues()
        {
            const float rho = 1.225f;
            const float n = 28000f / 60f;
            const float diameter = 0.127f;

            Assert.That(RotorMath.StaticThrust(0.11f, rho, n, diameter),
                Is.EqualTo(7.634089f).Within(1e-5f));
            Assert.That(RotorMath.StaticYawTorque(
                    0.040164f, rho, n, diameter, RotorTorqueConvention.Legacy2Point5Pi),
                Is.EqualTo(0.04507288f).Within(1e-7f));
            Assert.That(RotorMath.StaticYawTorque(
                    0.040164f, rho, n, diameter, RotorTorqueConvention.PowerCoefficient2Pi),
                Is.EqualTo(0.05634110f).Within(1e-7f));
        }

        [Test]
        public void MotorLagUsesSeparateExactUpAndDownResponses()
        {
            float up = RotorMath.FirstOrderStep(0f, 1f, 0.004f, 0.0125f, 0.025f);
            float down = RotorMath.FirstOrderStep(1f, 0f, 0.004f, 0.0125f, 0.025f);

            Assert.That(up, Is.EqualTo(0.27385096f).Within(1e-6f));
            Assert.That(down, Is.EqualTo(0.85214379f).Within(1e-6f));
        }

        [Test]
        public void ActuationUsesRotorCrossForceAndSpinTorque()
        {
            RotorInstance[] rotors = QuadX();
            var physics = new RotorPhysicsDerived { MaxThrust = 10f, MaxTorque = 0.2f };
            float[] motors = { 1f, 0f, 0f, 0f };

            QuadActuationModel.ComputeBodyWrench(
                motors, rotors, physics, out Vector3 force, out Vector3 torque);

            Assert.That(force, Is.EqualTo(new Vector3(0f, 10f, 0f)));
            Assert.That(torque.x, Is.EqualTo(-2f).Within(1e-6f));
            Assert.That(torque.y, Is.EqualTo(-0.2f).Within(1e-6f));
            Assert.That(torque.z, Is.EqualTo(2f).Within(1e-6f));
        }

        [Test]
        public void BodyWrenchTransformsToWorld()
        {
            QuadActuationModel.BodyToWorldWrench(
                Quaternion.Euler(0f, 90f, 0f), Vector3.right, Vector3.up,
                out Vector3 forceWorld, out Vector3 torqueWorld);

            Assert.That(forceWorld.x, Is.EqualTo(0f).Within(1e-6f));
            Assert.That(forceWorld.z, Is.EqualTo(-1f).Within(1e-6f));
            Assert.That(torqueWorld, Is.EqualTo(Vector3.up));
        }

        [Test]
        public void SymmetricQuadRoundTripsFeasibleDemand()
        {
            ControlAllocator allocator = BuildAllocator(QuadX());
            float[] motors = new float[4];
```

```
            allocator.AllocateYawLimited(0.05f, -0.04f, 0.03f, 0.5f, 0f, motors);

            Assert.That(allocator.Rank, Is.EqualTo(4));
            Assert.That(allocator.ConditionNumber, Is.LessThan(float.PositiveInfinity));
            Assert.That(allocator.LastAllocation.Residual, Is.LessThan(1e-4f));
            Assert.That(motors.All(value => value >= 0f && value <= 1f), Is.True);
        }

        [Test]
        public void PlusQuadRoundTripsFeasibleDemand()
        {
            RotorInstance[] rotors =
            {
                new RotorInstance { ID = 0, Location_Body = new Vector3(0.25f, 0f, 0f), Spin_Dir = -1 },
                new RotorInstance { ID = 1, Location_Body = new Vector3(0f, 0f, -0.25f), Spin_Dir = 1 },
                new RotorInstance { ID = 2, Location_Body = new Vector3(-0.25f, 0f, 0f), Spin_Dir = -1 }
,
                new RotorInstance { ID = 3, Location_Body = new Vector3(0f, 0f, 0.25f), Spin_Dir = 1 },
            };
            ControlAllocator allocator = BuildAllocator(rotors);
            float[] motors = new float[4];

            allocator.AllocateYawLimited(0.04f, 0.03f, -0.02f, 0.45f, 0f, motors);

            Assert.That(allocator.Rank, Is.EqualTo(4));
            Assert.That(allocator.LastAllocation.Residual, Is.LessThan(1e-4f));
        }

        [Test]
        public void ShiftedGeometryDoesNotBleedCollectiveIntoTorque()
        {
            RotorInstance[] rotors = QuadX();
            for (int i = 0; i < rotors.Length; i++)
                rotors[i].Location_Body += new Vector3(0.03f, 0f, -0.02f);
            ControlAllocator allocator = BuildAllocator(rotors);
            float[] motors = new float[4];

            allocator.AllocateYawLimited(0f, 0f, 0f, 0.4f, 0f, motors);

            Assert.That(allocator.Rank, Is.EqualTo(4));
            Assert.That(allocator.LastAllocation.Achieved.x, Is.EqualTo(0f).Within(1e-5f));
            Assert.That(allocator.LastAllocation.Achieved.y, Is.EqualTo(0f).Within(1e-5f));
            Assert.That(allocator.LastAllocation.Achieved.z, Is.EqualTo(0f).Within(1e-5f));
            Assert.That(allocator.LastAllocation.Achieved.w, Is.EqualTo(0.4f).Within(1e-5f));
        }

        [Test]
        public void SaturationPreservesCollectiveAndDesaturatesYawFirst()
        {
            ControlAllocator allocator = BuildAllocator(QuadX());
            float[] motors = new float[4];

            allocator.AllocateYawLimited(0.05f, 0.04f, 1f, 0.5f, 0f, motors);

            Assert.That(allocator.LastAllocation.RollPitchScale, Is.EqualTo(1f).Within(1e-6f));
            Assert.That(allocator.LastAllocation.YawScale, Is.LessThan(1f));
            Assert.That(allocator.LastAllocation.Achieved.w, Is.EqualTo(0.5f).Within(1e-5f));
            Assert.That(allocator.LastAllocation.SaturatedMotors.Any(value => value), Is.True);
        }

        [Test]
        public void DownstreamSaturationStopsPidFromWindingFurther()
        {
            var pid = new QuadPIDController
            {
                Kp = 0f,
                Ki = 1f,
                Kd = 0f,
                DecayFactor = 1f,
                IntegralLimit = 100f,
                MinOutput = -100f,
                MaxOutput = 100f,
            };
```

```
    pid.SetSaturationFromTracking(1f, 0f);
    pid.Calculate(1f, 0f, 1f);
    Assert.That(pid.IntegralState, Is.EqualTo(0f).Within(1e-6f));

    pid.ClearSaturation();
    pid.Calculate(1f, 0f, 1f);
    Assert.That(pid.IntegralState, Is.EqualTo(1f).Within(1e-6f));
}

[Test]
public void DisabledIntegralDoesNotAccumulateHiddenState()
{
    var pid = new QuadPIDController
    {
        Kp = 0f,
        Ki = 0f,
        Kd = 0f,
        DecayFactor = 1f,
    };

    pid.Calculate(10f, 0f, 1f);
    Assert.That(pid.IntegralState, Is.EqualTo(0f));
}

[Test]
public void DesiredForceCouplesVerticalAccelerationIntoTilt()
{
    Vector3 force = CascadedController.ConstrainDesiredSpecificForce(
        9.81f, 0f, 9.81f, 45f, float.PositiveInfinity);

    Assert.That(force.x, Is.EqualTo(9.81f).Within(1e-5f));
    Assert.That(force.z, Is.EqualTo(19.62f).Within(1e-5f));
    float tiltDeg = Mathf.Atan2(force.x, force.z) * Mathf.Rad2Deg;
    Assert.That(tiltDeg, Is.EqualTo(26.565f).Within(1e-3f));
}

[Test]
public void NearFreeFallCannotDemandHorizontalForce()
{
    Vector3 force = CascadedController.ConstrainDesiredSpecificForce(
        9.81f, 4f, -9.81f, 45f, float.PositiveInfinity);

    Assert.That(force, Is.EqualTo(Vector3.zero));
}

[Test]
public void VerticalVelocityBrakingRetainsHalfHoverLift()
{
    var config = new DroneConfig
    {
        FlightParams = new FlightParameters { MaxAngle = 45f },
        Velocity = new VelocityPID()
    };
    var controller = new VelocityController(config);

    controller.Initialize(3, null, null);

    float expectedMinimumAccel =
        -9.81f * (1f - VelocityController.ClosedLoopMinLiftFraction);
    Assert.That(controller.Pid.MinOutput,
        Is.EqualTo(expectedMinimumAccel).Within(1e-6f));
    Assert.That(controller.Pid.MaxOutput, Is.EqualTo(9.81f).Within(1e-6f));

    Vector3 force = CascadedController.ConstrainDesiredSpecificForce(
        0f, 0f, controller.Pid.MinOutput, 45f, float.PositiveInfinity);
    Assert.That(force.z,
        Is.EqualTo(9.81f * VelocityController.ClosedLoopMinLiftFraction)
            .Within(1e-6f));
}

[TestCase(179f, -179f, 181f)]
[TestCase(-179f, 179f, -181f)]
```

```
public void YawGoalUsesShortestEquivalentAngle(
    float measuredDeg, float goalDeg, float expectedGoalDeg)
{
    Assert.That(
        AngleController.NearestEquivalentAngle(measuredDeg, goalDeg),
        Is.EqualTo(expectedGoalDeg).Within(1e-5f));
}

[TestCase("standard_quad")]
[TestCase("sentinel_quad")]
[TestCase("crazyflie2_quad")]
[TestCase("flightmare_quad")]
public void PackagedQuadProfilesParseAndHaveFullRank(string profile)
{
    TextAsset yaml = Resources.Load<TextAsset>($"QuadSim/Configs/Drones/{profile}");
    Assert.That(yaml, Is.Not.Null);
    Assert.That(Parse(yaml.text, profile, out RotorModelInstance propulsion), Is.True);

    var physics = new RotorPhysicsDerived();
    physics.ComputeRotor(propulsion.Model, 0.2f);
    var allocator = new ControlAllocator();
    allocator.RebuildEffectiveness(propulsion.Rotors, physics, SimFrame.FLU);
    Assert.That(allocator.Rank, Is.EqualTo(4));
}

[TestCase("standard_quad")]
[TestCase("sentinel_quad")]
[TestCase("crazyflie2_quad")]
[TestCase("flightmare_quad")]
public void PackagedQuadProfilesProvideYawAngleGain(string profile)
{
    TextAsset yaml = Resources.Load<TextAsset>($"QuadSim/Configs/Drones/{profile}");
    Assert.That(yaml, Is.Not.Null);
    Assert.That(YamlParser.LoadDroneConfigFromString(
        yaml.text, profile, out _, out _, out _, out _, out _,
        out AnglePID angle, out _, out _, out _), Is.True);
    Assert.That(angle.AngleYaw.P, Is.GreaterThan(0f));
}

[Test]
public void DuplicateRotorIdsAreRejected()
{
    LogAssert.Expect(LogType.Error,
        "[Drone.YamlParser] Duplicate propeller id 0: duplicate-test");
    Assert.That(Parse(InvalidYaml(0, 0, 2, 3), "duplicate-test", out _), Is.False);
}

[Test]
public void InvalidSpinDirectionIsRejected()
{
    LogAssert.Expect(LogType.Error,
        "[Drone.YamlParser] Propeller 1 dir must be -1 or 1: spin-test");
    string yaml = InvalidYaml(0, 1, 2, 3).Replace("id: 1, dir: 1", "id: 1, dir: 0");
    Assert.That(Parse(yaml, "spin-test", out _), Is.False);
}

private static ControlAllocator BuildAllocator(RotorInstance[] rotors)
{
    var physics = new RotorPhysicsDerived { MaxThrust = 10f, MaxTorque = 0.2f };
    var allocator = new ControlAllocator();
    allocator.RebuildEffectiveness(rotors, physics, SimFrame.FLU);
    return allocator;
}

private static RotorInstance[] QuadX() => new[]
{
    new RotorInstance { ID = 0, Location_Body = new Vector3(0.2f, 0f, 0.2f), Spin_Dir = -1 },
    new RotorInstance { ID = 1, Location_Body = new Vector3(0.2f, 0f, -0.2f), Spin_Dir = 1 },
    new RotorInstance { ID = 2, Location_Body = new Vector3(-0.2f, 0f, 0.2f), Spin_Dir = 1 },
    new RotorInstance { ID = 3, Location_Body = new Vector3(-0.2f, 0f, -0.2f), Spin_Dir = -1 },
};

private static bool Parse(
```

```
            string yaml, string label, out RotorModelInstance propulsion)
        {
            return YamlParser.LoadDroneConfigFromString(
                yaml, label, out _, out _, out propulsion, out _, out _,
                out _, out _, out _, out _);
        }

        private static string InvalidYaml(int id0, int id1, int id2, int id3) => $@"
name: test
propulsion:
  prop_diameter: 0.1
  max_rpm: 10000
  thrust_coef: 0.1
  torque_coef: 0.01
  propellers:
    - {{id: {id0}, dir: -1, x: 0.1, y: 0, z: 0.1}}
    - {{id: {id1}, dir: 1, x: 0.1, y: 0, z: -0.1}}
    - {{id: {id2}, dir: 1, x: -0.1, y: 0, z: 0.1}}
    - {{id: {id3}, dir: -1, x: -0.1, y: 0, z: -0.1}}
";
    }
}
```

Tests/Editor/SimClockTests.cs

SHA-256 bc6ed147e175264d1cd5cd389be755f580e109c35e3b8e9dd7ee09ffb0184604 | 48 lines | 1,811 bytes

```
using NUnit.Framework;
using SimCore;

namespace Nino.QuadSim.Tests
{
    public sealed class SimClockTests
    {
        [Test]
        public void FixedClockTimeIsDerivedFromIntegerStepCount()
        {
            var clock = new SteppableClock(1.0 / 250.0);

            for (int i = 0; i < 100_000; i++) clock.Step();

            Assert.That(clock.StepCount, Is.EqualTo(100_000));
            Assert.That(clock.StepSizeNanos, Is.EqualTo(4_000_000));
            Assert.That(clock.NowNanos, Is.EqualTo(400_000_000_000));
        }

        [Test]
        public void ResetClearsStepAndPreservesRequestedEpoch()
        {
            var clock = new SteppableClock(0.004, 10);
            clock.StepBy(0.020);
            clock.Reset(1_000);

            Assert.That(clock.StepCount, Is.Zero);
            Assert.That(clock.NowNanos, Is.EqualTo(1_000));
            Assert.That(clock.StartNanos, Is.EqualTo(1_000));
        }

        [TestCase(10, 5, true, false, 0, StepTargetStatus.Missing)]
        [TestCase(10, 5, true, true, 10, StepTargetStatus.LateOrDuplicate)]
        [TestCase(10, 5, true, true, 12, StepTargetStatus.CountMismatch)]
        [TestCase(10, 5, true, true, 20, StepTargetStatus.Skipped)]
        [TestCase(10, 5, true, true, 15, StepTargetStatus.Accepted)]
        public void StrictStepContractClassifiesInvalidTargets(
            long current, int count, bool strict, bool hasTarget, long target,
            StepTargetStatus expectedStatus)
        {
            StepTargetStatus status = StepContract.Validate(
                current, count, strict, hasTarget, target, out long expectedStep);

            Assert.That(status, Is.EqualTo(expectedStatus));
            Assert.That(expectedStep, Is.EqualTo(15));
        }
    }
```

```
}
```

Tests/PlayMode/SimulationContractPlayModeTests.cs

SHA-256 7991935f0b65b75a6168c764f3978a7f4163086747909cc6dadf08b3a26b8403 | 60 lines | 2,261 bytes

```
using System.Collections;
using System.Collections.Generic;
using NUnit.Framework;
using SimCore;
using UnityEngine;
using UnityEngine.TestTools;

namespace Nino.QuadSim.PlayModeTests
{
    public sealed class SimulationContractPlayModeTests
    {
        [UnityTest]
        public IEnumerator ManualStepsRunPrePhysicsThenPostPhysicsAndResetClock()
        {
            var root = new GameObject("SimulationContractTest");
            var manager = root.AddComponent<SimulationManager>();
            var probe = root.AddComponent<SimulationProbe>();
            manager.RegisterManagers(probe);
            manager.SetRunMode(SimulationManager.RunMode.Manual);
            manager.SetPaused(true);

            yield return null;

            manager.StepImmediate(2);
            Assert.That(manager.StepIndex, Is.EqualTo(2));
            Assert.That(manager.SimTimeNanos, Is.EqualTo(8_000_000));
            CollectionAssert.AreEqual(
                new[] { "pre", "post", "pre", "post" }, probe.Phases);

            manager.ResetSimulation();
            Assert.That(manager.StepIndex, Is.Zero);
            Assert.That(manager.SimTimeNanos, Is.Zero);
            Assert.That(probe.ResetCount, Is.EqualTo(1));

            Object.Destroy(root);
            yield return null;
        }

        [Test]
        public void SceneConfigDefaultsToLoopbackRpc()
        {
            var config = ScriptableObject.CreateInstance<QuadSimSceneConfig>();
            Assert.That(config.rpcBindAddress, Is.EqualTo("127.0.0.1"));
            Object.Destroy(config);
        }
    }

    public sealed class SimulationProbe : MonoBehaviour, ISimulatable
    {
        public readonly List<string> Phases = new List<string>();
        public int ResetCount { get; private set; }
        public int ExecutionOrder => 0;
        public string DebugName => nameof(SimulationProbe);

        public void OnSimulationStart(SimulationManager sim) { }
        public void OnSimulationReset(SimulationManager sim) => ResetCount++;
        public void PrePhysicsStep(double dtSec, long nowNanos) => Phases.Add("pre");
        public void PostPhysicsStep(double dtSec, long nowNanos) => Phases.Add("post");
    }
}
```

ci~/built_player_smoke.py

SHA-256 0c5ae591c119d23af2033f9876fffa9ddeeae6fb301b5a21358d0a234eae7bac | 213 lines | 8,216 bytes

```
from __future__ import annotations

import argparse
import hashlib
```

```
import json
import os
import platform
import time
from dataclasses import asdict
from datetime import datetime, timezone
from pathlib import Path

import quadsim
from quadsim import QuadSim
from quadsim.replay import ZERO_DIGEST, chained_digest, step_result_payload


def connect(args):
    deadline = time.monotonic() + args.startup_timeout
    last_error = None
    while time.monotonic() < deadline:
        sim = QuadSim(
            host=args.host,
            command_port=args.command_port,
            telemetry_port=args.telemetry_port,
            timeout_ms=1000,
            client_name="Phase3BuiltPlayerSmoke",
        )
        try:
            sim.connect()
            return sim
        except Exception as exc:
            last_error = exc
            sim.disconnect()
            time.sleep(1.0)
    raise RuntimeError(f"player did not accept RPC within the timeout: {last_error}")


def source_file(path):
    if not path:
        return None
    source = Path(path).resolve()
    content = source.read_bytes()
    return {
        "path": str(source),
        "sha256": hashlib.sha256(content).hexdigest(),
    }


def first_difference(left, right, path="$", tolerance=1e-6):
    if (isinstance(left, (int, float)) and not isinstance(left, bool)
            and isinstance(right, (int, float)) and not isinstance(right, bool)):
        return None if abs(float(left) - float(right)) <= tolerance else path
    if isinstance(left, dict) and isinstance(right, dict):
        if set(left) != set(right):
            return path
        for key in sorted(left):
            difference = first_difference(
                left[key], right[key], f"{path}.{key}", tolerance)
            if difference is not None:
                return difference
        return None
    if isinstance(left, (list, tuple)) and isinstance(right, (list, tuple)):
        if len(left) != len(right):
            return path
        for index, (left_item, right_item) in enumerate(zip(left, right)):
            difference = first_difference(
                left_item, right_item, f"{path}[{index}]", tolerance)
            if difference is not None:
                return difference
        return None
    return None if left == right else path


def deterministic_sequence(sim, drones, agents, steps):
    reset = sim.reset_episode(agents, seed=12345, settle_steps=2)
    digest = ZERO_DIGEST
```

```
    last_step = None
    events_seen = 0
    records = []
    for offset in range(steps):
        target_step = reset.step_index + offset + 1
        actions = {drone.agent_id: (0.0, 0.0, 0.0, 0.0) for drone in drones}
        last_step = sim.step_multi(
            actions,
            count=1,
            target_step=target_step,
            strict_lockstep=True,
        )
        state, events = step_result_payload(last_step)
        state["episode_id"] = 0
        records.append({"state": state, "events": events})
        digest = chained_digest(
            digest,
            {
                "mode": "acceleration",
                "count": 1,
                "target_step": target_step,
                "actions": actions,
            },
            state,
            None,
            events,
        )
        events_seen += len(events)
    return reset, last_step, digest, events_seen, records


def run(args) -> dict:
    sim = connect(args)
    try:
        protocol = sim.protocol_info
        if protocol is None or not protocol.negotiated:
            raise RuntimeError("player did not negotiate protocol capabilities")
        if protocol.bind_address not in ("127.0.0.1", "localhost", "::1"):
            raise RuntimeError(f"player unexpectedly exposed RPC on {protocol.bind_address}")

        initial = sim.get_status()
        drones = sim.list_drones()
        if not drones or not all(drone.is_initialized for drone in drones):
            raise RuntimeError("player did not register an initialized drone")

        sim.pause()
        agents = [
            {
                "agent_id": drone.agent_id,
                "team_id": drone.team_id or "default",
                "drone_index": drone.drone_index,
                "position_enu": (0.0, index * 2.0, 1.0),
                "yaw_deg": 0.0,
            }
            for index, drone in enumerate(drones)
        ]
        reset, step, first_digest, first_events, first_records = deterministic_sequence(
            sim, drones, agents, args.replay_steps)
        repeated_reset, repeated_step, second_digest, second_events, second_records = \
            deterministic_sequence(sim, drones, agents, args.replay_steps)
        difference = first_difference(first_records, second_records)
        if difference is not None:
            raise RuntimeError(
                f"seeded replay exceeded 1e-6 at {difference}")
        sensors = sim.drone().get_sensors()
        final = sim.get_status()

        expected_step = reset.step_index + args.replay_steps
        if step.step_index != expected_step or repeated_step.step_index != expected_step:
            raise RuntimeError(
                f"strict sequence ended at {step.step_index}, expected {expected_step}")
        if (sensors.episode_id != repeated_step.episode_id
                or sensors.step_index < repeated_step.step_index):
```

```
            raise RuntimeError("sensor snapshot predates the completed strict step")

        return {
            "captured_utc": datetime.now(timezone.utc).isoformat(),
            "platform": platform.platform(),
            "python": platform.python_version(),
            "sdk_version": quadsim.__version__,
            "unity_revision": args.unity_revision,
            "sdk_revision": args.sdk_revision,
            "scene_config": source_file(args.scene_config),
            "vehicle_config": source_file(args.vehicle_config),
            "protocol": asdict(protocol),
            "initial_status": asdict(initial),
            "drones": [asdict(drone) for drone in drones],
            "reset": {
                "episode_id": repeated_reset.episode_id,
                "step_index": repeated_reset.step_index,
                "seed": repeated_reset.seed,
                "settle_steps": repeated_reset.settle_steps,
            },
            "strict_step": {
                "episode_id": repeated_step.episode_id,
                "step_index": repeated_step.step_index,
                "sim_time_ns": repeated_step.sim_time_ns,
                "sensor_step_index": sensors.step_index,
            },
            "deterministic_replay": {
                "steps_per_run": args.replay_steps,
                "runs": 2,
                "matches_within_absolute_tolerance": True,
                "absolute_tolerance": 1e-6,
                "digests": [first_digest, second_digest],
                "events_per_run": [first_events, second_events],
            },
            "final_status": asdict(final),
        }
    finally:
        sim.disconnect()


def main(argv=None) -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--command-port", type=int, default=5555)
    parser.add_argument("--telemetry-port", type=int, default=5556)
    parser.add_argument("--startup-timeout", type=float, default=90.0)
    parser.add_argument("--replay-steps", type=int, default=8)
    parser.add_argument("--unity-revision", default=os.environ.get("GITHUB_SHA", "unknown"))
    parser.add_argument("--sdk-revision", default="unknown")
    parser.add_argument("--scene-config")
    parser.add_argument("--vehicle-config")
    parser.add_argument("--output", required=True)
    args = parser.parse_args(argv)
    manifest = run(args)
    output = Path(args.output)
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
    print(output)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

ci~/check_unity_log.py

SHA-256 b2b66c88b982a38c08f37b4609c87341a5de5449241db2ea756d4cb995dfd81f | 53 lines | 1,435 bytes

```
from __future__ import annotations

import argparse
from pathlib import Path
```

```
REJECTED = (
    "NullReferenceException",
    "InvalidOperationException",
    "error CS",
    "Scripts have compiler errors",
    "Compilation failed",
    "[RpcProtocol] Failed",
    "invalid collider",
    "dropped_step",
    "request_too_large",
    "rate_limited",
    "incompatible_protocol",
    "server_timeout",
)


def check(paths):
    failures = []
    for supplied in paths:
        path = Path(supplied)
        if not path.exists():
            failures.append(f"{path}: log path does not exist")
            continue
        files = sorted(path.rglob("*.log")) if path.is_dir() else [path]
        if not files:
            failures.append(f"{path}: no Unity log files found")
            continue
        for log in files:
            text = log.read_text(encoding="utf-8", errors="replace")
            for pattern in REJECTED:
                if pattern.lower() in text.lower():
                    failures.append(f"{log}: unexpected {pattern}")
    return failures


def main(argv=None):
    parser = argparse.ArgumentParser()
    parser.add_argument("paths", nargs="+")
    args = parser.parse_args(argv)
    failures = check(args.paths)
    for failure in failures:
        print(failure)
    return 1 if failures else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

.github/workflows/phase3-ci.yml

SHA-256 04e5dee15070b062ee8d11f9268614c0f90f855b5d00aa1dbefc70279a45b952 | 164 lines | 6,115 bytes

```
name: Phase 3 platform gate

on:
  pull_request:
  push:
    branches: [main, test]
  workflow_dispatch:

permissions:
  contents: read
  checks: write

jobs:
  package-contract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Validate package and mirrored YAML
        run: |
```

```
        python -m json.tool package.json >/dev/null
        python - <<'PY'
        import json, pathlib, re
        root = pathlib.Path('.')
        version = json.loads((root / 'package.json').read_text())['version']
        protocol = (root / 'Runtime/Scripts/SimCore/Rpc/RpcProtocol.cs').read_text()
        advertised = re.search(r'UnityPackageVersion = "([^"]+)"', protocol).group(1)
        assert version == advertised, (version, advertised)
        for source in (root / 'Runtime/Resources/QuadSim/Configs/Drones').glob('*.yaml'):
            mirror = root / 'Samples~/QuadSim Starter/StreamingAssets/Configs/Drones' / source.name
            assert source.read_bytes() == mirror.read_bytes(), source.name
        PY

  sdk:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python: ["3.10", "3.11", "3.12", "3.13", "3.14"]
    steps:
      - uses: actions/checkout@v4
        with:
          repository: ninonick0607/QuadSimLib
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
      - run: python -m pip install -e ".[test]"
      - name: Run supported SDK matrix
        run: |
          python -m pytest -q \
            Testing/test_lockstep.py \
            Testing/test_multi_agent.py \
            Testing/test_replay.py \
            Testing/test_rl_foundation_verifier.py \
            Testing/test_drone_config.py \
            Testing/test_protocol.py
      - name: Build and inspect distributions
        if: matrix.python == '3.12'
        run: |
          python -m build
          python -m twine check dist/*
          python -m zipfile -l dist/quadsim_sdk-0.16.0-py3-none-any.whl | grep quadsim/calibration.py

  unity-tests:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        mode: [editmode, playmode]
    steps:
      - uses: actions/checkout@v4
      - name: Place package below repository root for GameCI package mode
        run: |
          mkdir package-under-test
          rsync -a --exclude .git --exclude package-under-test ./ package-under-test/
      - uses: game-ci/unity-test-runner@v4
        id: unity-tests
        env:
          UNITY_LICENSE: ${{ secrets.UNITY_LICENSE }}
          UNITY_EMAIL: ${{ secrets.UNITY_EMAIL }}
          UNITY_PASSWORD: ${{ secrets.UNITY_PASSWORD }}
        with:
          packageMode: true
          projectPath: package-under-test
          unityVersion: 6000.5.8f1
          testMode: ${{ matrix.mode }}
          githubToken: ${{ secrets.GITHUB_TOKEN }}
          checkName: Unity ${{ matrix.mode }}
      - name: Reject unexpected Unity errors
        if: always()
        run: python ci~/check_unity_log.py "${{ steps.unity-tests.outputs.artifactsPath }}"
      - uses: actions/upload-artifact@v4
        if: always()
```

```
      with:
        name: unity-${{ matrix.mode }}-results
        path: ${{ steps.unity-tests.outputs.artifactsPath }}

  player-build:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        target: [StandaloneLinux64, StandaloneWindows64]
    steps:
      - uses: actions/checkout@v4
      - name: Import the build sample into the Unity host
        run: |
          mkdir -p ci~/UnityTestProject/Assets/QuadSimStarter
          rsync -a "Samples~/QuadSim Starter/" \
            ci~/UnityTestProject/Assets/QuadSimStarter/
      - uses: game-ci/unity-builder@v4
        env:
          UNITY_LICENSE: ${{ secrets.UNITY_LICENSE }}
          UNITY_EMAIL: ${{ secrets.UNITY_EMAIL }}
          UNITY_PASSWORD: ${{ secrets.UNITY_PASSWORD }}
        with:
          projectPath: ci~/UnityTestProject
          unityVersion: 6000.5.8f1
          targetPlatform: ${{ matrix.target }}
          buildName: QuadSim
      - uses: actions/checkout@v4
        if: matrix.target == 'StandaloneLinux64'
        with:
          repository: ninonick0607/QuadSimLib
          path: ci-sdk
      - uses: actions/setup-python@v5
        if: matrix.target == 'StandaloneLinux64'
        with:
          python-version: "3.12"
      - name: Run built-player RPC smoke
        if: matrix.target == 'StandaloneLinux64'
        run: |
          python -m pip install -e ci-sdk
          chmod +x build/StandaloneLinux64/QuadSim
          build/StandaloneLinux64/QuadSim \
            -batchmode -nographics \
            -logFile "$RUNNER_TEMP/quadsim-player.log" &
          PLAYER_PID=$!
          trap 'kill "$PLAYER_PID" 2>/dev/null || true' EXIT
          SDK_REVISION=$(git -C ci-sdk rev-parse HEAD)
          python ci~/built_player_smoke.py \
            --sdk-revision "$SDK_REVISION" \
            --scene-config \
              "Samples~/QuadSim Starter/Configuration/Default_SceneConfig.asset" \
            --vehicle-config \
              Runtime/Resources/QuadSim/Configs/Drones/flightmare_quad.yaml \
            --output "$RUNNER_TEMP/quadsim-smoke-manifest.json"
          kill "$PLAYER_PID" 2>/dev/null || true
          wait "$PLAYER_PID" || true
          trap - EXIT
          python ci~/check_unity_log.py "$RUNNER_TEMP/quadsim-player.log"
      - uses: actions/upload-artifact@v4
        if: always() && matrix.target == 'StandaloneLinux64'
        with:
          name: quadsim-built-player-smoke
          path: |
            ${{ runner.temp }}/quadsim-player.log
            ${{ runner.temp }}/quadsim-smoke-manifest.json
      - uses: actions/upload-artifact@v4
        with:
          name: quadsim-${{ matrix.target }}
          path: build/${{ matrix.target }}
```

.github/workflows/security.yml

SHA-256 352bd1cadbb7a1db89aad5a3667d2698c5f068f1e7efe8f22a26a7e1931908d0 | 27 lines | 539 bytes

```
name: Security gate

on:
  pull_request:
  push:
    branches: [main, test]
  workflow_dispatch:

permissions:
  contents: read
  security-events: write

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2

  dependency-review:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/dependency-review-action@v4
```

ci~/UnityTestProject/.gitignore

SHA-256 18b0e052f8506bb85eaecdea4c60771606e511e3f4e1bffa57351076d1105281 | 9 lines | 153 bytes

```
Library/
Logs/
Temp/
UserSettings/
obj/
Build/
ProjectSettings/*
!ProjectSettings/EditorBuildSettings.asset
!ProjectSettings/ProjectVersion.txt
```

ci~/UnityTestProject/Packages/manifest.json

SHA-256 e2e4daf05fb9f47b3523d349bf7850c54649c5eabec00f6f907629fbbe77ce2b | 11 lines | 261 bytes

```
{
  "dependencies": {
    "com.nino.quadsim": "file:../..",
    "com.unity.inputsystem": "1.19.0",
    "com.unity.render-pipelines.high-definition": "17.4.0",
    "com.unity.test-framework": "1.4.6"
  },
  "testables": [
    "com.nino.quadsim"
  ]
}
```

ci~/UnityTestProject/ProjectSettings/ProjectVersion.txt

SHA-256 09292e3f32140b54a6f8b5b377812ef2a19fd855ee0a3cb30d2a5fa5f421b10a | 2 lines | 85 bytes

```
m_EditorVersion: 6000.5.8f1
m_EditorVersionWithRevision: 6000.5.8f1 (5cb7df797b7d)
```

ci~/UnityTestProject/ProjectSettings/EditorBuildSettings.asset

SHA-256 e1ad5b79448e763a69a72646f729245d49b964ba3785bf5d7175a661fe20333a | 11 lines | 274 bytes

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!1045 &1
EditorBuildSettings:
```

```
m_ObjectHideFlags: 0
serializedVersion: 2
m_Scenes:
- enabled: 1
  path: Assets/QuadSimStarter/Maps/Land.unity
  guid: 6a406fad4106414e3906da158c636cd6
m_configObjects: {}
```

18. Selected Unity Configuration Assets

Includes the short maintained ScriptableObject records that select startup behavior, RPC ports, environment values, drone profiles, and prefab mappings. Large scene, prefab, material, terrain, and Shader Graph serialization is outside this code-listing scope.

Files in this section

Runtime/Configuration/Default_SceneConfig.asset
Runtime/Prefabs/DroneCatalog.asset
Samples~/QuadSim Starter/Configuration/Default_SceneConfig.asset
Samples~/QuadSim Starter/Prefabs/DroneCatalog.asset

Runtime/Configuration/Default_SceneConfig.asset

SHA-256 ea4301808e74eefe93d7a43e51827fd467bdfb2ae0ba0b2b6fc88cede04e8a82 | 33 lines | 1,033 bytes

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_ObjectHideFlags: 0
  m_CorrespondingSourceObject: {fileID: 0}
  m_PrefabInstance: {fileID: 0}
  m_PrefabAsset: {fileID: 0}
  m_GameObject: {fileID: 0}
  m_Enabled: 1
  m_EditorHideFlags: 0
  m_Script: {fileID: 11500000, guid: ac58421b6963eb5b78405541ac5fd441, type: 3}
  m_Name: Default_SceneConfig
  m_EditorClassIdentifier: Nino.QuadSim::SimCore.QuadSimSceneConfig
  RunMode: 0
  droneConfigName: flightmare_quad
  droneCatalog: {fileID: 11400000, guid: 220584e9706cc97d885dc94be9ddfd77, type: 2}
  startupSpawnPolicy: 0
  payloadPrefab: {fileID: 463154142083670055, guid: fc1074fe6267f6e078f4cfb1f24f145b, type: 3}
  spawnPayload: 0
  payloadMountPath: Bottle
  outputFrame: 1
  startingMode: 2
  commandPort: 5555
  telemetryPort: 5556
  originLatitudeDeg: 33.49414
  originLongitudeDeg: -117.7391
  originAltitudeM: 5
  masterSeed: 12345
  enableSensorNoise: 0
  windEnabled: 0
  windSpeed: 10
  windHeadingDeg: 0
```

Runtime/Prefabs/DroneCatalog.asset

SHA-256 c1ca91eee9b3b0c2afa5d89ac985dd626c5211de24004d67515febb3f42f1767 | 22 lines | 931 bytes

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_ObjectHideFlags: 0
  m_CorrespondingSourceObject: {fileID: 0}
  m_PrefabInstance: {fileID: 0}
  m_PrefabAsset: {fileID: 0}
  m_GameObject: {fileID: 0}
  m_Enabled: 1
  m_EditorHideFlags: 0
  m_Script: {fileID: 11500000, guid: b1d1515103784ad1bcce7b2db45a7322, type: 3}
  m_Name: DroneCatalog
  m_EditorClassIdentifier: Nino.QuadSim::QuadSim.SimCore.DroneCatalog
  fallbackPawn: {fileID: 4436502927399878726, guid: 66a628493a0df52a98cfc54bdf7c2a9a, type: 3}
```

```
  entries:
  - configName: flightmare_quad
    pawn: {fileID: 5909055088624131799, guid: 31fe9db5889aeef9aaa26bc935a505d7, type: 3}
  - configName: crazyflie2_quad
    pawn: {fileID: 680589187432453512, guid: b039efeaa9db92f9198ba90644464a67, type: 3}
  - configName: standard_quad
    pawn: {fileID: 7369674030872104845, guid: 2e5ce94195e2bbeb4bd792d784027dba, type: 3}
```

Samples~/QuadSim Starter/Configuration/Default_SceneConfig.asset

SHA-256 fbe9fc30063ebdcdccf02b4d44b04032ee594c3686be0aa0af29cedc99858b29 | 33 lines | 1,033 bytes

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_ObjectHideFlags: 0
  m_CorrespondingSourceObject: {fileID: 0}
  m_PrefabInstance: {fileID: 0}
  m_PrefabAsset: {fileID: 0}
  m_GameObject: {fileID: 0}
  m_Enabled: 1
  m_EditorHideFlags: 0
  m_Script: {fileID: 11500000, guid: ac58421b6963eb5b78405541ac5fd441, type: 3}
  m_Name: Default_SceneConfig
  m_EditorClassIdentifier: Nino.QuadSim::SimCore.QuadSimSceneConfig
  RunMode: 0
  droneConfigName: flightmare_quad
  droneCatalog: {fileID: 11400000, guid: 0ad1ad09475f47b192e60336df5e5ed4, type: 2}
  startupSpawnPolicy: 0
  payloadPrefab: {fileID: 463154142083670055, guid: fc1074fe6267f6e078f4cfb1f24f145b, type: 3}
  spawnPayload: 0
  payloadMountPath: Bottle
  outputFrame: 1
  startingMode: 2
  commandPort: 5555
  telemetryPort: 5556
  originLatitudeDeg: 33.49414
  originLongitudeDeg: -117.7391
  originAltitudeM: 5
  masterSeed: 12345
  enableSensorNoise: 0
  windEnabled: 0
  windSpeed: 10
  windHeadingDeg: 0
```

Samples~/QuadSim Starter/Prefabs/DroneCatalog.asset

SHA-256 628af30547da324075df0ab862fadbb726dd6cec9684bea5ce0939fafd893cbb | 22 lines | 931 bytes

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_ObjectHideFlags: 0
  m_CorrespondingSourceObject: {fileID: 0}
  m_PrefabInstance: {fileID: 0}
  m_PrefabAsset: {fileID: 0}
  m_GameObject: {fileID: 0}
  m_Enabled: 1
  m_EditorHideFlags: 0
  m_Script: {fileID: 11500000, guid: b1d1515103784ad1bcce7b2db45a7322, type: 3}
  m_Name: DroneCatalog
  m_EditorClassIdentifier: Nino.QuadSim::QuadSim.SimCore.DroneCatalog
  fallbackPawn: {fileID: 4436502927399878726, guid: 10cae2c843c9470bbfad010cd379bfc9, type: 3}
  entries:
  - configName: flightmare_quad
    pawn: {fileID: 5909055088624131799, guid: 2da5d3f2032a481cb5e056fbf1128747, type: 3}
  - configName: crazyflie2_quad
    pawn: {fileID: 680589187432453512, guid: ac999663930b4aa686700c52db434131, type: 3}
  - configName: standard_quad
    pawn: {fileID: 7369674030872104845, guid: 7ff50b157cae4e34ab6c4e927517e060, type: 3}
```